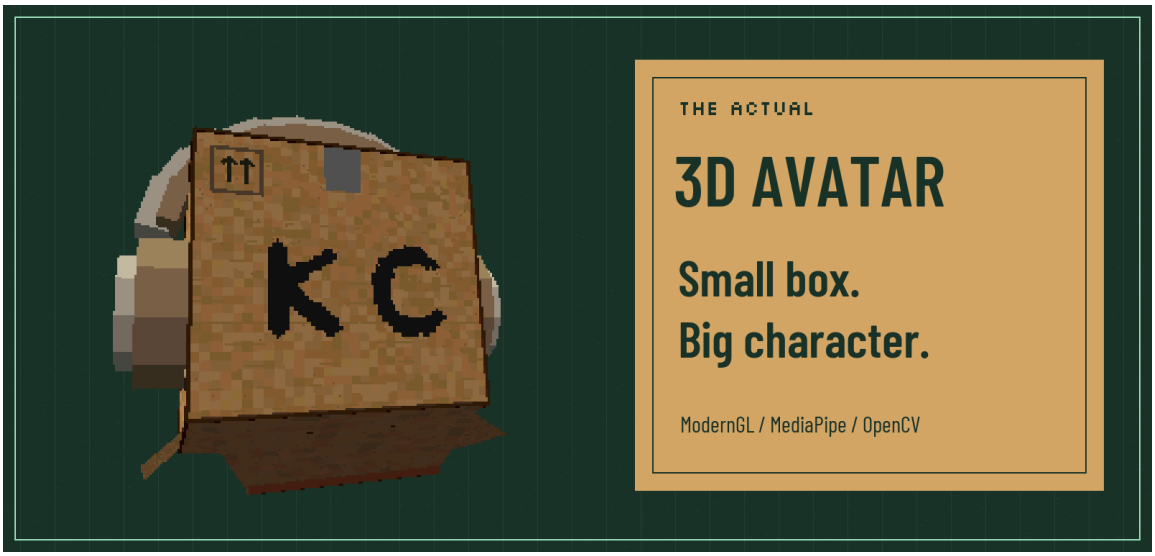
KARDBOARDCODE ENGINEERING



The architecture and implementation of

KardboardCode **VTuber**

Camera ingestion, MediaPipe tracking, real-time algorithms, privacy boundaries, low-poly GPU rendering, compositing, testing, and operations.

Debashish Mishra

Engineering Handbook
2 November 2026

About this book

A living engineering handbook compiled directly from the KardboardCode-VTuber documentation.

This edition contains **54 source documents** and **101 Mermaid diagrams**. It explains the current implementation from product constraints and visual identity through capture, tracking, algorithms, GPU rendering, privacy, testing, and roadmap.

> **GENERATED FROM SOURCE**
> The Markdown documentation remains authoritative. Rebuild this PDF after documentation changes so the printable edition never becomes a separate, manually maintained copy.

Privacy boundary

The book uses repository documentation and approved synthetic or face-safe visual assets. It must never embed camera credentials, private authenticated URLs, raw face captures, or private regression recordings.

Generated 2 November 2026 · KardboardCode VTuber Engineering Handbook.

Preface - how to read this book

Use the reading path that matches the work in front of you.

Start with Onboarding for a guided route. Read Foundations and Architecture to understand the complete system. Use the algorithm and design-principle Parts when changing concurrency, filtering, tracking, or composition. Camera Ingestion and Face Tracking are operational guides. Quality and Testing defines the evidence required before completion.

> **EVIDENCE OVER ASSUMPTION**
> Chapters distinguish implemented behavior, measured runtime evidence, and planned work. When documentation and code conflict, current code and tests win and the book must be corrected.

Table of Contents

FRONT MATTER

About this book 2
Preface - how to read this book 3

INTRODUCTION

Chapter 1 - KardboardCode VTuber Engineering Book 7

PART I · ONBOARDING

Chapter 2 - Onboarding 15
Chapter 3 - Principal engineer guide 18
Chapter 4 - Zero-to-hero learning path 23

PART II · FOUNDATIONS

Chapter 5 - Foundations 29
Chapter 6 - Product requirements and constraints 34
Chapter 7 - Visual identity and source avatar 38

PART III · ARCHITECTURE

Chapter 8 - Architecture 45
Chapter 9 - System architecture 48
Chapter 10 - Domain and runtime data model 54
Chapter 11 - Camera lifecycle 59
Chapter 12 - Textured GPU 3D renderer 63
Chapter 13 - Green-screen compositing 76
Chapter 14 - PS1 cardboard renderer 82

PART IV · ALGORITHMS AND DATA STRUCTURES

Chapter 15 - Algorithms and data structures 88
Chapter 16 - The single latest-frame slot 91
Chapter 17 - Condition-variable synchronization 94
Chapter 18 - Finite-state lifecycle and reconnect algorithm 98
Chapter 19 - Monotonic timing and FPS estimation 101
Chapter 20 - Asynchronous latest-result face inference 104
Chapter 21 - Blendshape normalization 107
Chapter 22 - Facial transformation matrix decomposition 111
Chapter 23 - Facial action state machine 114
Chapter 24 - One Euro motion filtering 118
Chapter 25 - Damped spring integration 122
Chapter 26 - Low-resolution overlay compositing 126

Chapter 27 - Bounded studio mailbox 130

PART V · DESIGN PRINCIPLES

Chapter 28 - Design principles 134
Chapter 29 - Design principle: latency over completeness 137
Chapter 30 - Design principle: dependency inversion at the capture boundary 140
Chapter 31 - Design principle: immutable observations of mutable state 143
Chapter 32 - Design principle: security and secret boundaries 146
Chapter 33 - Responsive control surfaces and progressive disclosure 149

PART VI · CAMERA INGESTION

Chapter 34 - Camera ingestion 153
Chapter 35 - Camera runtime flow 156
Chapter 36 - Android IP Webcam integration 159
Chapter 37 - Diagnostics and performance interpretation 162

PART VII · FACE TRACKING

Chapter 38 - Face tracking 166
Chapter 39 - Normalized face state 171
Chapter 40 - Live tracking debugging and validation 176

PART VIII · QUALITY AND TESTING

Chapter 41 - Quality and testing 184
Chapter 42 - Validation evidence and known limits 189

PART IX · ROADMAP

Chapter 43 - Roadmap: from working avatar to production hardening 196

PART X · DESKTOP STUDIO

Chapter 44 - Desktop Studio 201
Chapter 45 - Desktop GUI research and design 204
Chapter 46 - Interface specification and placement rationale 209
Chapter 47 - Desktop runtime architecture 214
Chapter 48 - Studio tutorial 221
Chapter 49 - Prerecorded video processing 229
Chapter 50 - Windows installation 235

APPENDICES

Appendix A - Appendix 243
Appendix B - Glossary 246
Appendix C - Repository map 250
Appendix D - Command reference 254

MEET THE PROJECT

Introduction

A map of the working KardboardCode VTuber, its privacy contract, visual identity, implemented milestone, and the reading paths through this book.

1

INTRODUCTION

CHAPTER 1

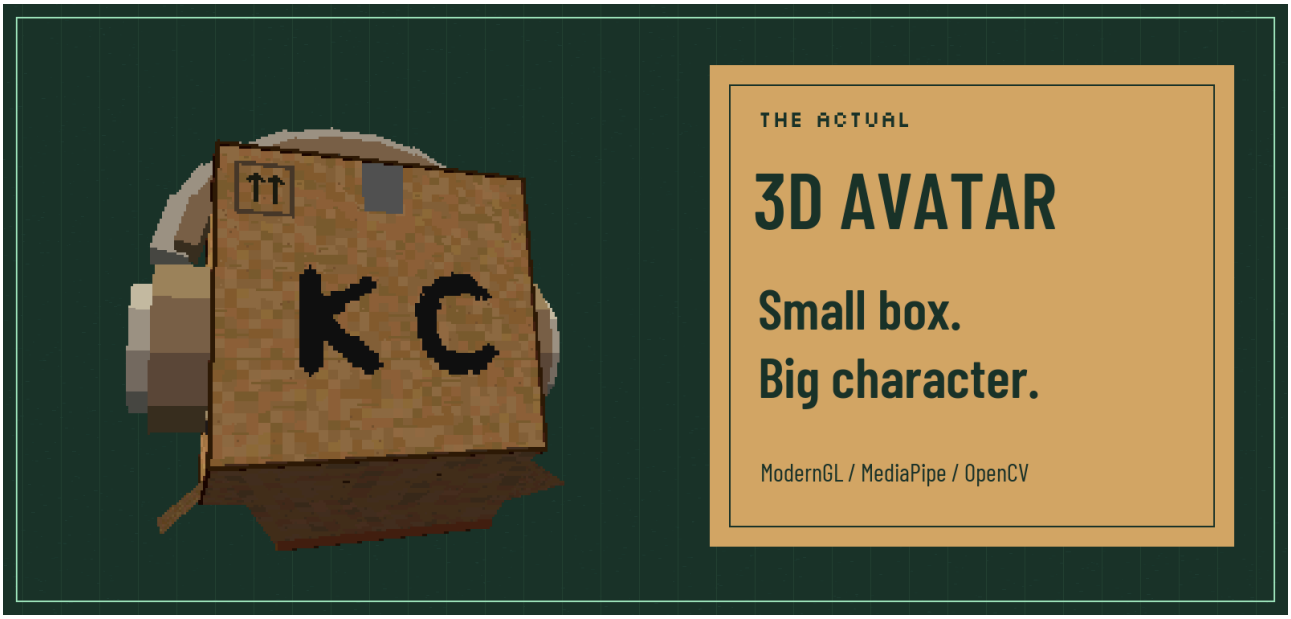
KardboardCode VTuber Engineering Book

INTRODUCTION Chapter 1

KardboardCode VTuber Engineering Book

> **TL;DR** — This is the engineering book for a lightweight Python VTuber that keeps the real camera image, tracks one face, and overlays a deliberately low-resolution PS1-style cardboard head. Camera ingestion, tracking, filtering, procedural fallback rendering, and textured GPU 3D rendering are implemented and verified.

This documentation is written to be read in sequence, used as an interview study guide, and reused as the technical backbone for a future development video. It explains not only **what** the code does, but also **why** each data structure, algorithm, boundary, and tradeoff exists.



The current default GPU-rendered avatar, generated entirely from synthetic geometry.

Printable edition

The complete documentation is also generated as `KardboardCode-VTuber-Engineering-Handbook.pdf`. Its reproducible Markdown-to-PDF pipeline and build instructions live in `book/`.

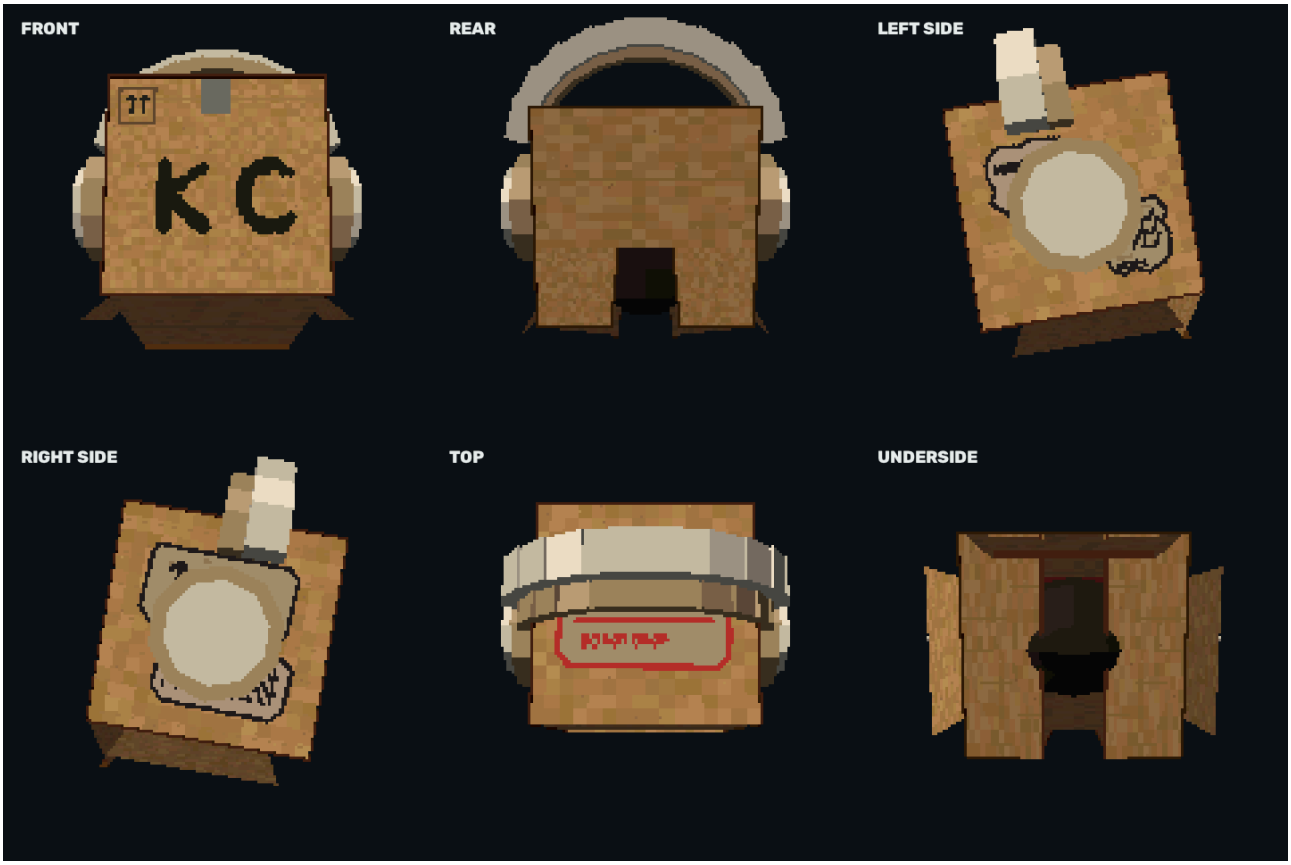


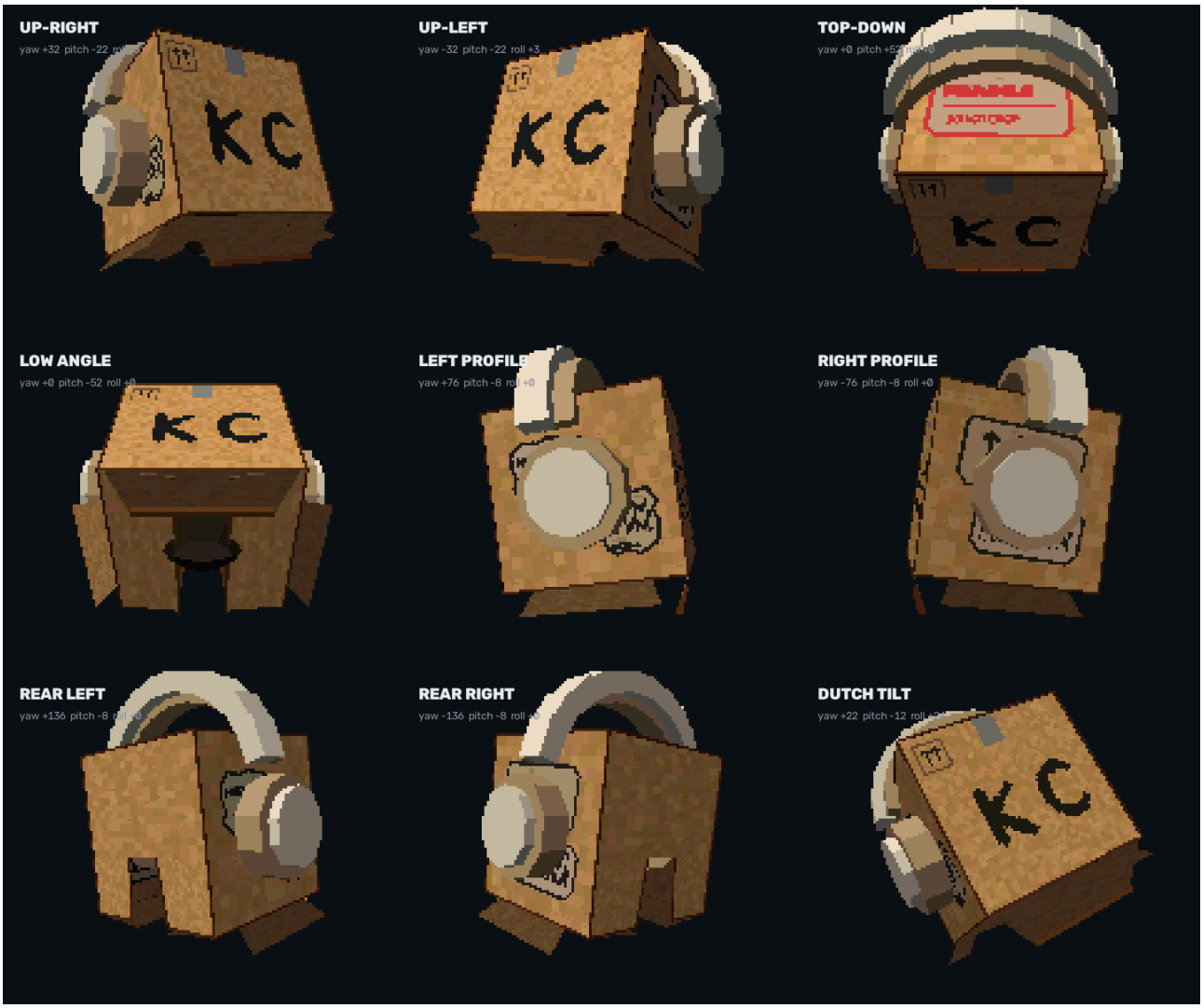
Book status

Part	Status	Evidence
Camera ingestion	Implemented and verified	`src/kardboard_vtuber/camera/`, covered by the full test suite
Android IP Webcam integration	Implemented and user-verified	1080x1920 preview at about 28-30 FPS
Face tracking	Implemented and live-validated	MediaPipe near 30 result FPS plus debounced action logs
Textured 3D renderer	Implemented as default	ModernGL mesh, cubic shell, complete front, neck-safe underside, headphones, decals, K/C eyes
Flap physics	Implemented and opt-in	Five independent shader hinges driven by bounded damped springs
Full-body diagnostics	Implemented and opt-in	33-point pose tracker and separate line-skeleton diagnostic; no filled body on output
Green-screen mode	Implemented and opt-in	Person mask preserves the body and replaces the room with chroma green
Hand occlusion	Implemented and opt-in	Hand/forearm-only foreground restoration over the avatar
Procedural renderer	Implemented fallback	Fail-closed opaque shell retained for GPU troubleshooting

	Capture	Clean preview output; chroma-key mode available with `--green-screen`
Virtual camera	Opt-in direct Windows output	`--virtual-camera` sends the clean composite to the installed OBS camera component; OBS need not run
Desktop studio	Optional native GUI	All runtime flags, per-eye live calibration, presets, recorder/model actions, logs and tutorial

Current visual reference





See Textured GPU 3D renderer for the detailed material, geometry, shader, privacy, decal, and animation specification.

Reading paths

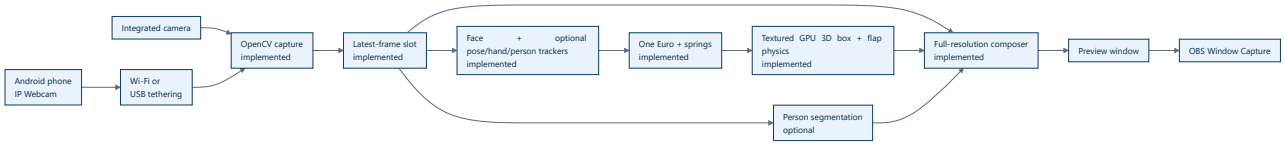
I want to...	Read this path
Understand the project in ten minutes	Foundations → System architecture
Run the working camera tool	Camera ingestion → Android IP Webcam
Explain the design in an interview	Principal engineer guide → Algorithms → Design principles
Learn every data structure	Domain and runtime data model
Study every current algorithm	Algorithms and data structures
Understand tracking	Face tracking
Understand green-screen compositing	Green-screen compositing
Inspect every current avatar surface	Textured GPU 3D renderer

Understand what comes next	Roadmap
Operate without memorizing CLI flags	Desktop studio → Tutorial
Look up a term or file	Glossary → Repository map

Chapters

#	Chapter	Question answered
00	Onboarding	How should a newcomer or principal engineer approach this system?
01	Foundations	What are we building, what does the avatar look like, and under which constraints?
02	Architecture	How do capture, tracking, rendering, and OBS fit together?
03	Algorithms and data structures	Which algorithms preserve low latency and correctness?
04	Design principles	Which engineering principles shaped the implementation?
05	Camera ingestion	How does the implemented subsystem work and how is it operated?
06	Face tracking	How are landmarks, expressions, and head pose produced?
07	Quality and testing	How do we verify behavior without depending only on hardware?
08	Roadmap	Which production-hardening tasks remain after the working avatar milestone?
09	Desktop studio	How do native controls, live calibration, privacy, and responsive worker ownership fit together?
99	Appendix	Where are commands, terms, source files, and quick references?

The system in one picture



Documentation contract

Every chapter follows these rules:

1. Start with a TL;DR and state whether behavior is implemented or planned.
2. Explain the concept at beginner level before introducing implementation detail.
3. Ground implementation claims in real repository files and line numbers.
4. Use diagrams where structure, state, flow, or tradeoffs are easier to see than to describe.
5. Separate measured facts from targets and future design.
6. Never store camera credentials, stream passwords, or private authenticated URLs.

Start with 00 · Onboarding · Look up terms in the Glossary

PART I

Onboarding

Choose the path that matches your goal: understand the architecture quickly, learn the system from first principles, or prepare to contribute safely.

2

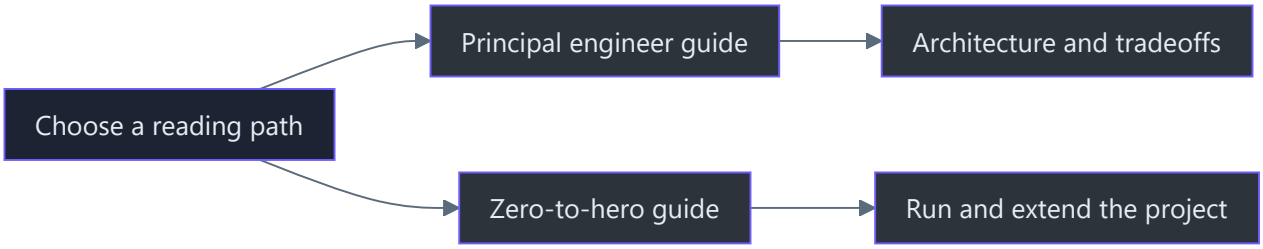
PART I · ONBOARDING

CHAPTER 2

Onboarding

PART I · ONBOARDING Chapter 2

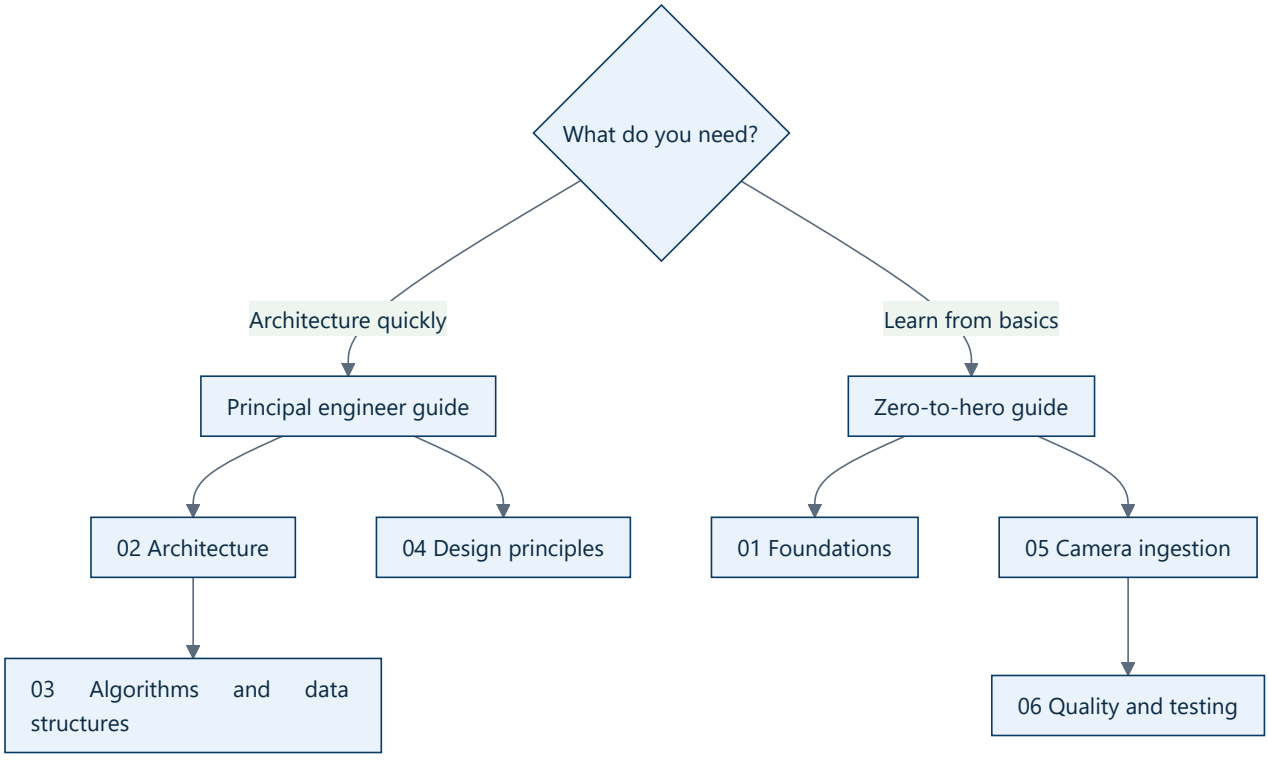
00 · Onboarding



> **TL;DR** — Choose the principal guide when you want architecture and tradeoffs quickly. Choose zero-to-hero when you want to learn the stack from first principles and run the project yourself.

Reading choices

Reader	Start here	Outcome
Principal/staff engineer	Principal engineer guide	Understand the core insight, boundaries, risks, and roadmap
New Python/OpenCV developer	Zero-to-hero guide	Build the mental model progressively and run the camera
Interview preparation	Principal guide → algorithms → principles	Explain why the system is shaped this way
Contributor	Zero-to-hero → testing → repository map	Find code and make a safe change



Source anchors

- Packaging and supported Python: `pyproject.toml:5-13`
- CLI entry point: `pyproject.toml:25-26`
- Camera models: `src/kardboard_vtuber/camera/models.py:15-157`
- Capture worker: `src/kardboard_vtuber/camera/stream.py:39-288`
- CLI loop: `src/kardboard_vtuber/cli.py:54-126`
- Tests: `tests/test_camera_models.py:6-34`, `tests/test_camera_stream.py:13-98`

🏠 Book home · ➡️ Foundations

3

PART I · ONBOARDING

CHAPTER 3

Principal engineer guide

PART I · ONBOARDING Chapter 3

Principal engineer guide

> **TL;DR** — The core architectural insight is that **video frames are expiring state, not durable work items**. The system must discard stale frames instead of faithfully processing them later. That one decision drives the single-slot buffer, asynchronous tracking plan, monotonic timing, immutable snapshots, and separation between full-resolution composition and downscaled inference.

The one idea to remember

A normal job queue values completeness:

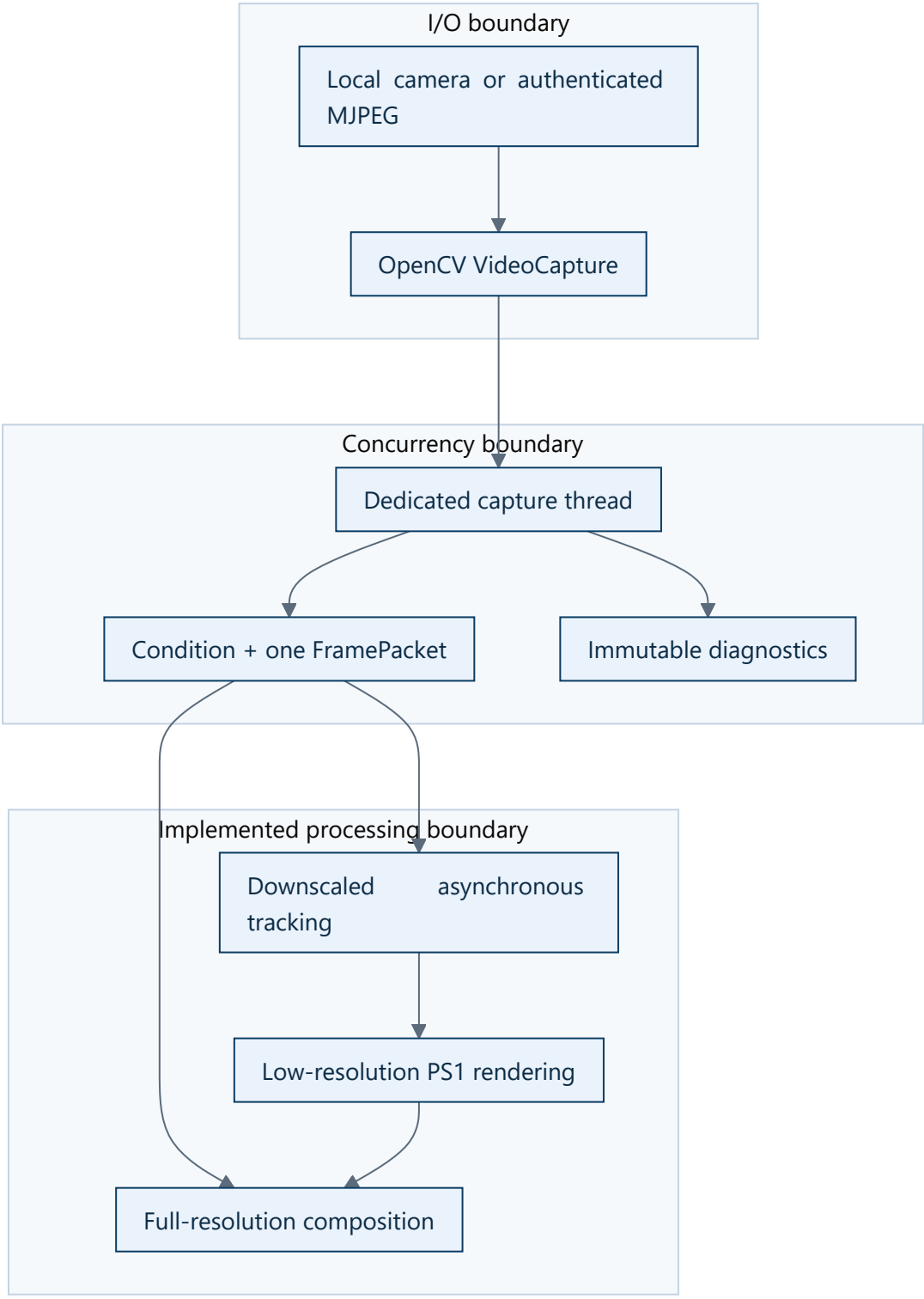
```
while (queue.length > 0) {
  process(queue.shift()); // every item matters
}
```

A live-avatar pipeline values freshness:

```
latestFrame = incomingFrame; // replace stale work
if (trackerIsReady) {
  tracker.process(latestFrame);
}
```

If a tracker can process 20 FPS while the camera produces 30 FPS, a FIFO queue adds ten frames of delay every second. A latest-value register drops work but keeps the avatar near real time.

Architecture at principal level

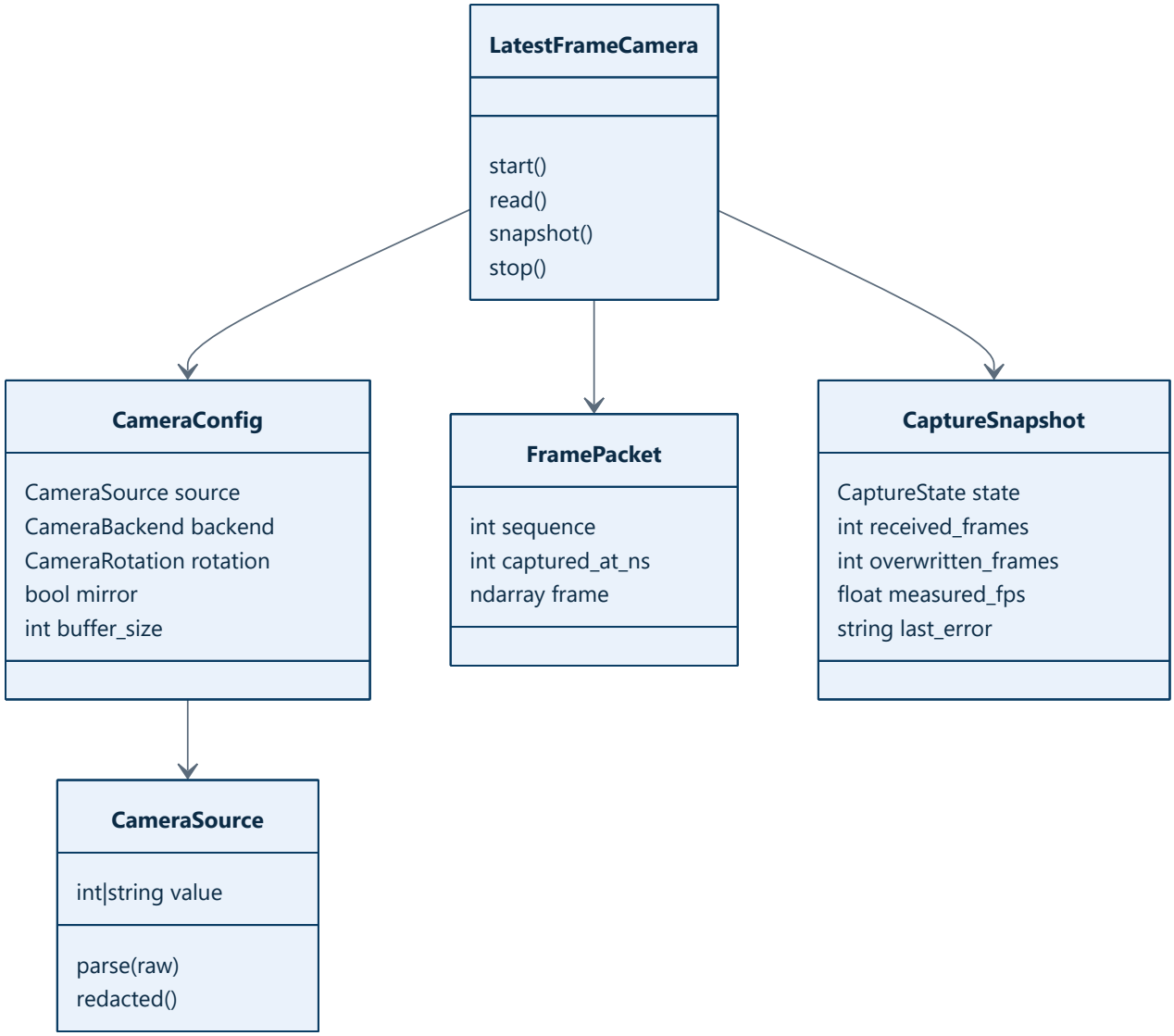


Implemented code anchors:

- The worker and shared state live in `LatestFrameCamera` (`src/kardboard_vtuber/camera/stream.py:39-288`).

- The single published value is `_latest: FramePacket | None` (`src/kardboard_vtuber/camera/stream.py:59`).
- Consumers ask for a sequence newer than the last one they processed (`src/kardboard_vtuber/camera/stream.py:114-141`).
- Runtime state is returned as an immutable `CaptureSnapshot` (`src/kardboard_vtuber/camera/models.py:141-157`).

Domain model



Strategic tradeoffs

Decision	Benefit	Cost	Why acceptable

Python prototype	Fast iteration, readable algorithms	Packaging and GIL concerns	OpenCV releases into native code; current bottleneck is not Python syntax
OpenCV capture	One API for device and URL sources	Backend behavior varies	Backend is explicit and measured
Single latest frame	Bounded latency and memory	Deliberate frame loss	Live interaction values current state
Separate tracking resolution	High-quality final video with cheap inference	Two frame representations	Tracking does not need every camera pixel
One fixed avatar	Smaller architecture and faster learning	Not a general VTuber platform	Matches the actual product requirement
Window Capture first	Simple OBS integration	No alpha-only GPU sharing	Spout2 can be added after the renderer exists

Risks to watch

1. **End-to-end latency confusion.** `frame age` starts after OpenCV returns a decoded frame; it does not measure phone exposure, encoding, network, or decoder buffering.
2. **Python version split.** Core camera/render code supports Python 3.11+, while the implemented MediaPipe tracking extra is constrained below Python 3.13 in `pyproject.toml:21-23`.
3. **Credential handling.** Authenticated URLs are convenient but can leak into shell history. Diagnostics redact them through `CameraSource.redacted()` at `src/kardboard_vtuber/camera/models.py:80-87`.
4. **Renderer scope discipline.** A single cardboard head does not justify a general scene engine or rigid-body simulation; five bounded decorative hinges are sufficient.

Where to go deep

1. System architecture
2. Latest-frame slot
3. Finite-state lifecycle
4. Latency over completeness
5. Roadmap

⬅️ Onboarding · 🏠 Book home

4

PART I · ONBOARDING

CHAPTER 4

Zero-to-hero learning path

PART I · ONBOARDING Chapter 4

Zero-to-hero learning path

> **TL;DR** — This path starts with Python and video basics, then introduces this repository's architecture, and finally shows how to run, test, and extend the application.

Part I · Foundations you need

Python objects versus JavaScript objects

This repository uses frozen, slotted dataclasses for small contracts:

```
@dataclass(frozen=True, slots=True)
class FramePacket:
    sequence: int
    captured_at_ns: int
    frame: ndarray
```

The closest JavaScript mental model is an immutable object passed between components. Python adds declared types and generated constructors, while `slots=True` prevents arbitrary attributes.

Read: `src/kardboard_vtuber/camera/models.py:124-138`.

What OpenCV contributes

OpenCV owns:

- Opening a local device or network video URL.
- Decoding frames into NumPy arrays.
- Applying rotation, mirroring, text overlays, and preview display.

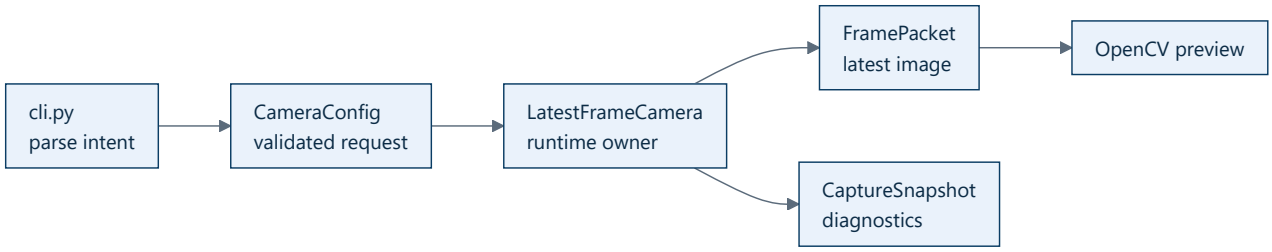
The project owns:

- Thread lifecycle.
- Stale-frame prevention.
- Reconnection policy.
- Diagnostics and security-conscious source display.

What a frame is

The current frame is a NumPy array shaped `(height, width, channels)` in BGR order. Rotation swaps width and height. A 1920x1080 landscape source therefore becomes a 1080x1920 portrait frame after a 90-degree rotation.

Part II · Learn this codebase



Read in this order:

1. `src/kardboard_vtuber/camera/models.py:15-157`
2. `src/kardboard_vtuber/camera/stream.py:39-288`
3. `src/kardboard_vtuber/cli.py:15-150`
4. `tests/test_camera_stream.py:13-98`

Part III · Set up and run

```
cd C:\devdesk\KardboardCode\KardboardCode-VTuber
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python -m ruff check .
python -m pytest
```

Run a local camera:

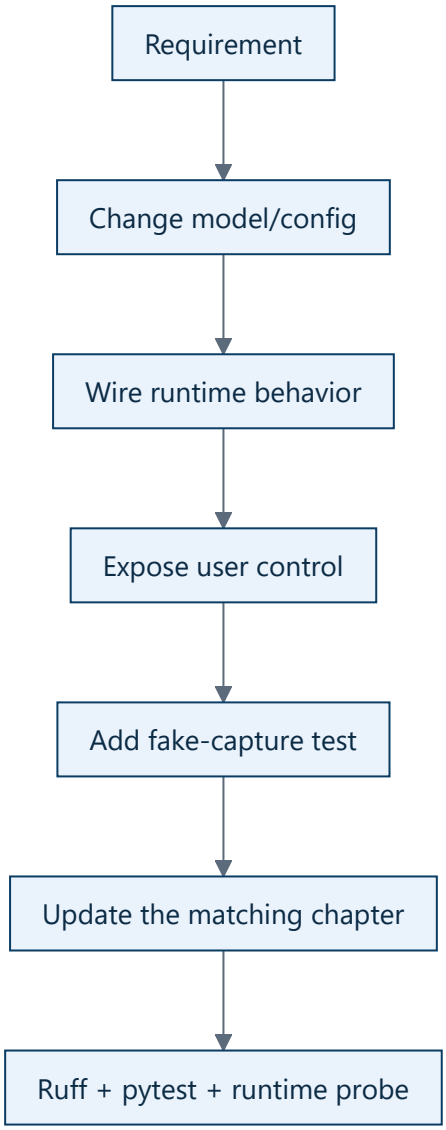
```
python -m kardboard_vtuber --source 0 --backend auto --mirror
```

Run a portrait phone stream:

```
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror
```

Never commit the authenticated URL.

Part IV · Make a safe change



Example: rotation required coordinated changes to:

- `CameraRotation` in `models.py:33-48`
- `CameraConfig.rotation` in `models.py:90-106`
- frame transformation in `stream.py:192-197`
- `--rotate` parsing in `cli.py:36-42`
- behavior test in `tests/test_camera_stream.py:83-98`

Graduation checklist

- [ ] Explain why frames are not processed with a FIFO queue.
- [ ] Explain requested versus negotiated camera properties.
- [ ] Trace one frame from OpenCV to the preview.

- [ ] Explain what the reported frame age excludes.
- [ ] Run Ruff and all tests.
- [ ] Add a behavior through model, runtime, CLI, test, and documentation layers.

⬅️ Onboarding · ➡️ Foundations

PART II

Foundations

The product boundary, privacy contract, visual identity, source avatar, and the deliberate choice to build one focused cardboard-head application.

5

PART II · FOUNDATIONS

CHAPTER 5

Foundations

PART II · FOUNDATIONS Chapter 5

01 · Foundations

> **TL;DR** — KardboardCode-VTuber is not a generic avatar engine. It is a focused application that keeps a high-quality real camera feed and replaces only the user's head with one expressive, low-poly cardboard box.

Product statement

The application:

1. Read a local or Android-phone camera.
2. Track one face and head pose.
3. Map left/right eye state to the `K` and `C` markings.
4. Tracks mouth openness for expression events and future mouth-art refinement.
5. Adds opt-in spring-driven motion to five physical flaps.
6. Render the box at low internal resolution for a PS1 aesthetic.
7. Composite it over the original high-resolution camera.
8. Present an OBS-capturable output.

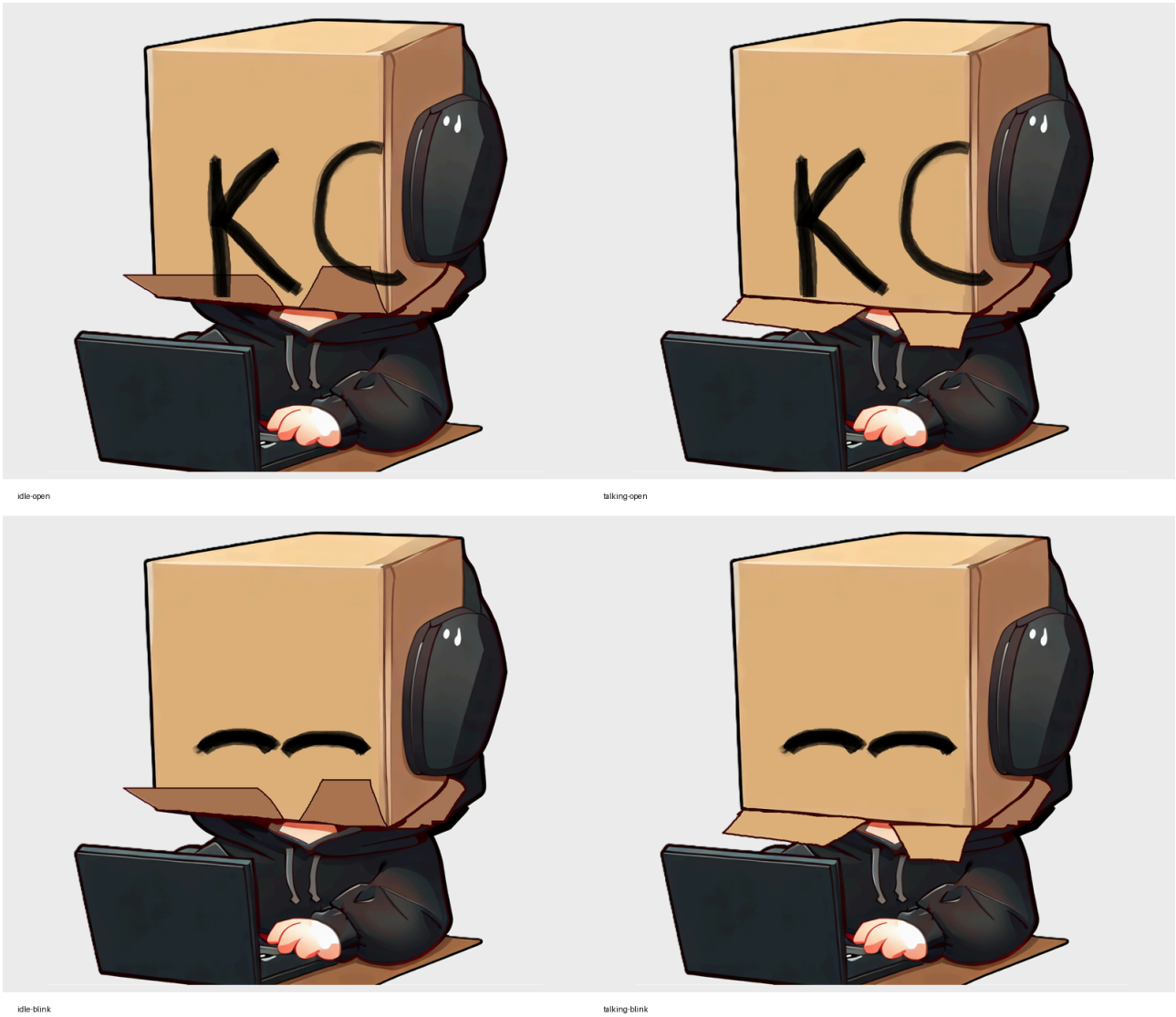
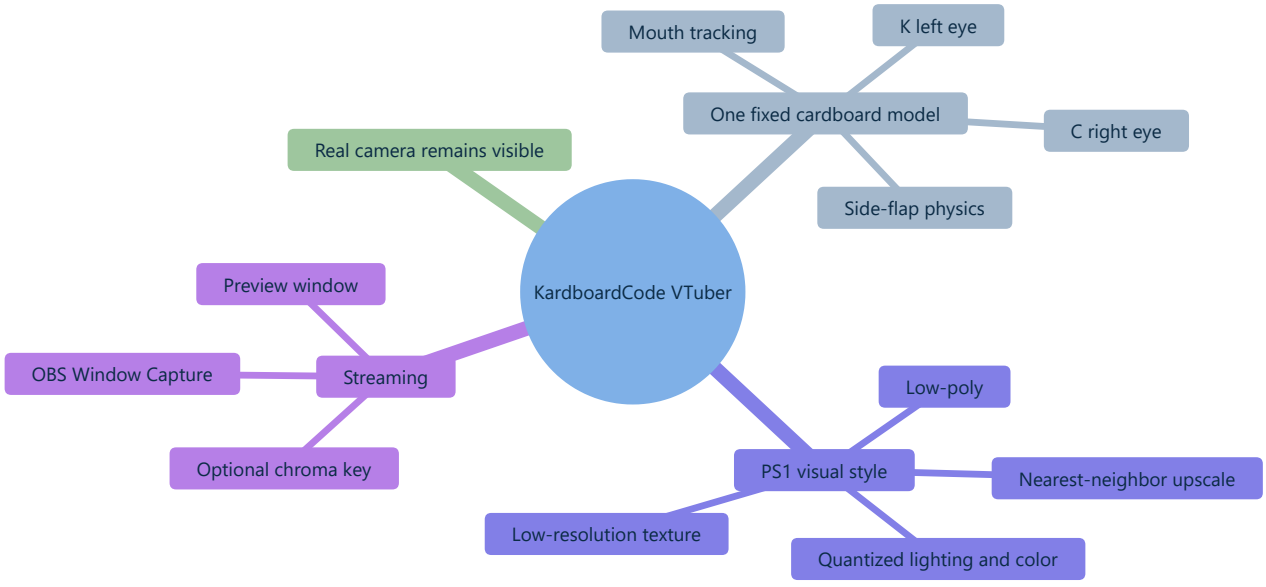


Figure 1 — The preserved source avatar and its independent talking/blinking state combinations. See the visual identity chapter for the full image-guided explanation.

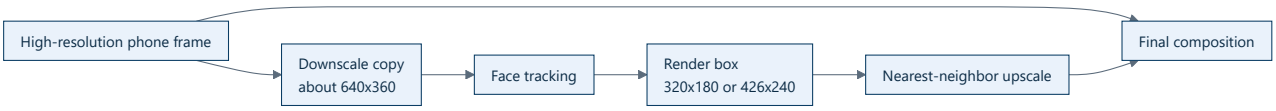


Scope boundaries

In scope	Out of scope for the prototype
One face	Multi-person tracking
One cardboard-head model	General model import
Real camera body or optional synthetic body	General character/model import
Python source-run workflow	Unity-first distribution
Window Capture for OBS	Custom OBS plugin
Relative head pose	Metric depth reconstruction

Why preserve two resolutions?

The camera image and the avatar have different jobs:



Tracking needs stable landmarks, not 2 million pixels. The final stream benefits from the full camera resolution. The box is intentionally coarse because visual imperfection is part of the art direction.

Current truth versus future design

Implemented: camera ingestion and reconnection, credential-safe diagnostics, asynchronous MediaPipe face/pose/hand/person tracking, One Euro filtering, calibrated expression events, ModernGL textured rendering, five spring hinges, procedural fallback rendering, full-resolution composition, optional full-body diagnostics, bounded hand/forearm occlusion, fail-closed green-screen composition, and OBS Window Capture output.

Opt-in diagnostics: `--tracking-debug` draws synthetic tracking evidence and performance text. `--debug-face-preview` is a separate privacy-sensitive raw camera panel.

Remaining production work: long-duration soak testing, CI/documentation validation, packaging, and optional transport beyond Window Capture.

Source anchors

- Project metadata and dependencies: `pyproject.toml:5-28`
- Preserved avatar behavior: `assets/PNGTuberV1/model-manifest.json:1`
- Working camera package: `src/kardboard_vtuber/camera/`
- Runtime orchestration: `src/kardboard_vtuber/cli.py:32-652`

- Default avatar: `src/kardboard_vtuber/renderer/textured_3d.py:111-1414`
- Green-screen path: `src/kardboard_vtuber/tracking/green_screen.py:16-152`

Related foundation chapters

- Product requirements and constraints
- Visual identity and source avatar

⬅️ Book home · ➡️ Architecture

6

PART II · FOUNDATIONS

CHAPTER 6

Product requirements and constraints

PART II · FOUNDATIONS Chapter 6

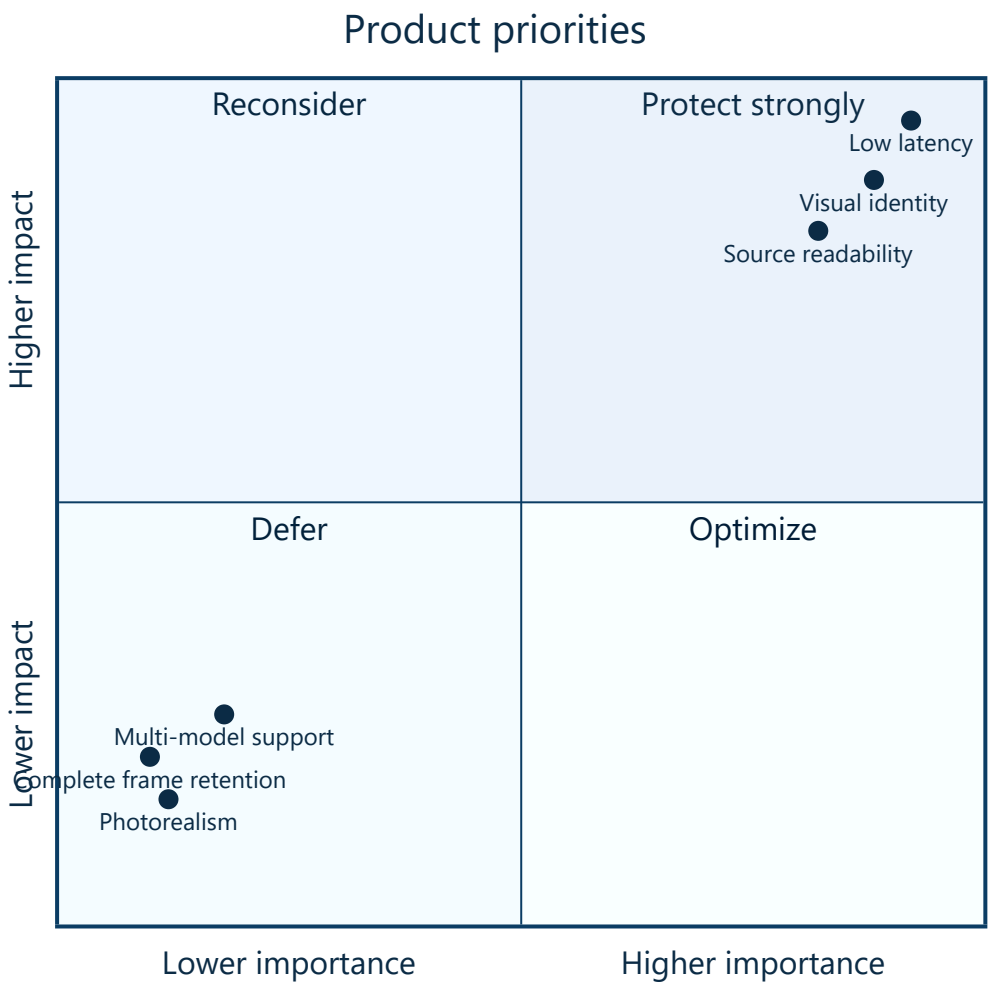
Product requirements and constraints

> **TL;DR** — The product is optimized for one creator, one recognizable cardboard character, low latency, and explainable source code. Generality is intentionally deferred.

Functional requirements

ID	Requirement	Current state
FR-01	Accept local camera indices	Implemented
FR-02	Accept authenticated MJPEG/RTSP URLs	Implemented
FR-03	Rotate and mirror frames	Implemented
FR-04	Reconnect after repeated failures	Implemented
FR-05	Track one face	Implemented
FR-06	Animate `K` and `C` independently	Implemented
FR-07	Track mouth state without exposing the real mouth	Implemented; flap redesign deferred
FR-08	Render PS1-style cardboard geometry	Implemented in ModernGL plus procedural fallback
FR-09	Produce an OBS-capturable preview	Implemented through Window Capture
FR-10	Add secondary flap motion	Implemented behind `--physics`
FR-11	Preserve the body while replacing the room with chroma green	Implemented behind `--green-screen`
FR-12	Hide tracking diagnostics by default	Implemented behind `--tracking-debug`

Non-functional requirements



- **Latency:** prefer dropping old frames over showing delayed motion.
- **Performance:** target stable 1080p30 first; 60 FPS is optional.
- **Explainability:** algorithms and tradeoffs must be teachable in an interview.
- **Security:** no credentials in source, docs, diagnostics, or commits.
- **Portability:** isolate OpenCV backend differences behind configuration.
- **Art direction:** intentional low fidelity, not accidental poor rendering.

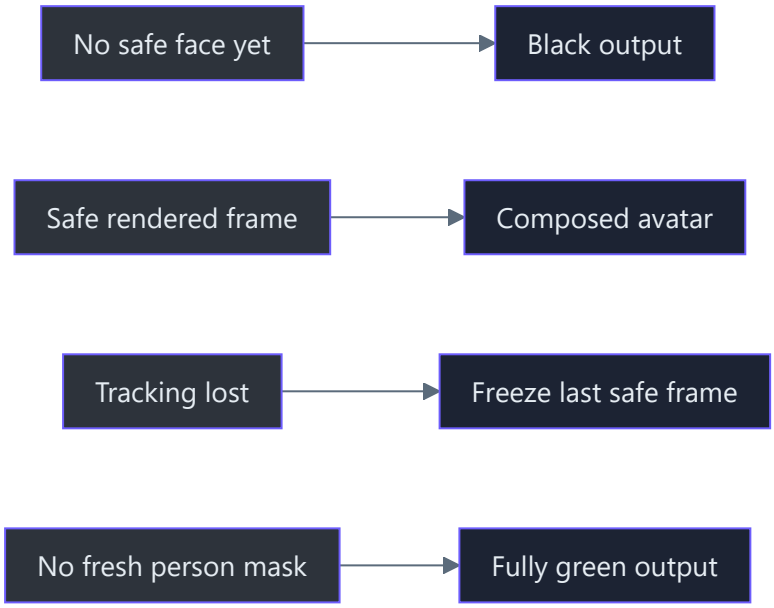
Verified camera acceptance result

The user confirmed the real preview with:

- 1080x1920 portrait output after left rotation.
- Approximately 27.9 FPS at the observed moment.
- Mirrored selfie orientation.
- No observed read failures or reconnects during the validation run.

The overlay's 0.4 ms frame age measured only post-decode in-process delay. It was not an end-to-end phone latency measurement.

Implemented privacy constraints



The textured renderer blacks output before first acquisition and freezes the last safely composed frame after face loss. Green-screen composition independently fails closed to a fully green frame before a fresh person mask exists (`src/kardboard_vtuber/renderer/textured_3d.py:180-279`, `src/kardboard_vtuber/tracking/green_screen.py:124-152`).

Sources

- CLI arguments: `src/kardboard_vtuber/cli.py:32-213`
- Capture behavior: `src/kardboard_vtuber/camera/stream.py:167-220`
- Credential redaction: `src/kardboard_vtuber/camera/models.py:80-87`
- Avatar semantics: `assets/PNGTuberV1/model-manifest.json:1`
- Textured renderer: `src/kardboard_vtuber/renderer/textured_3d.py:111-1414`
- Runtime tests: `tests/test_cli.py:1`, `tests/test_textured_3d_renderer.py:1`

⬅️ Foundations · ➡️ System architecture

7

PART II · FOUNDATIONS

CHAPTER 7

Visual identity and source avatar

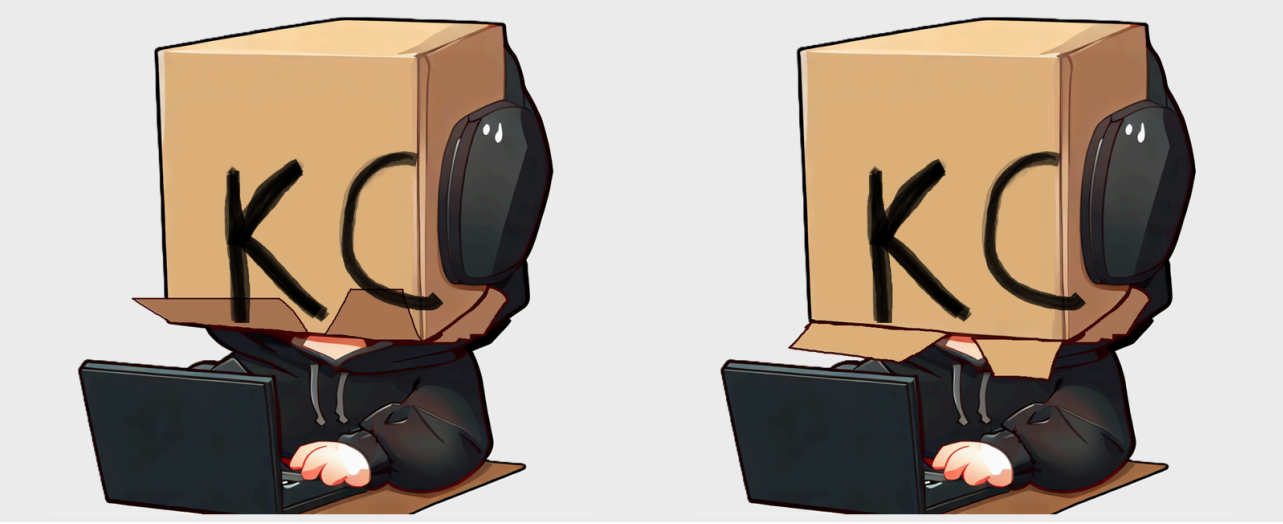
PART II · FOUNDATIONS Chapter 7

Visual identity and source avatar

> **TL;DR** — The future 3D cardboard head is not being invented without reference. Its identity, eye semantics, speaking behavior, asymmetry, and secondary motion come from the preserved PNGTuber V1 package under `assets/PNGTuberV1`.

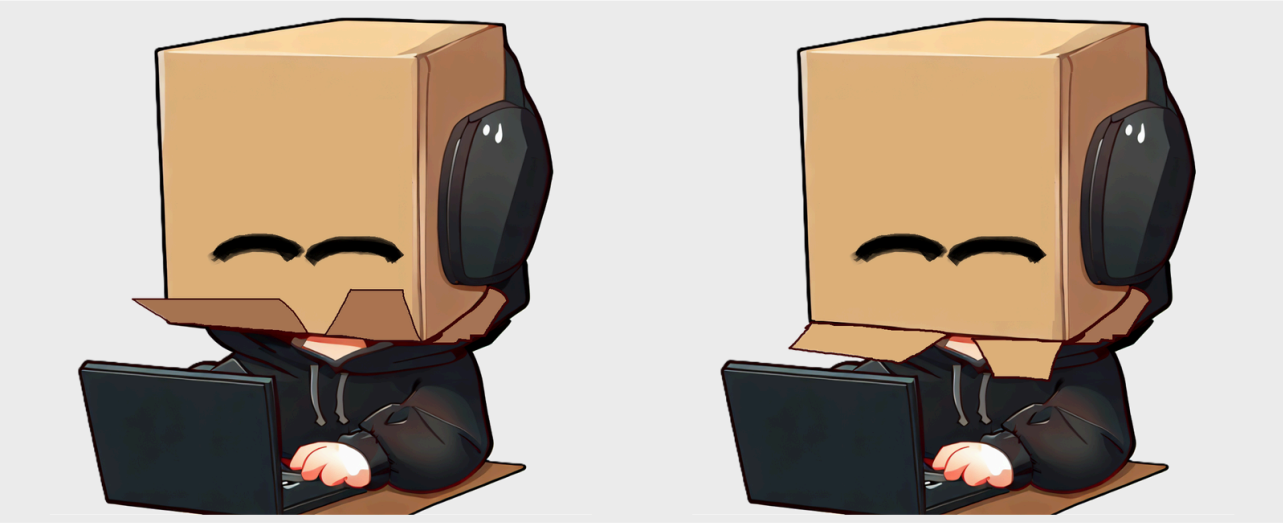
Why the source avatar matters

The camera-tracked application changes the rendering technology, not the character identity. The V1 package establishes the permanent silhouette (`assets/PNGTuberV1/Full Body.png`), independent eye states (`assets/PNGTuberV1/Eyes Open.png`, `assets/PNGTuberV1/Eyes Closed.png`), independent speaking states (`assets/PNGTuberV1/FrontFlap1.png`, `assets/PNGTuberV1/FrontFlap2.png`), and secondary side-flap motion (`assets/PNGTuberV1/RightFlap1.png`). Exact layer roles and parameters are preserved in `assets/PNGTuberV1/model-manifest.json:1`.



idle-open

talking-open

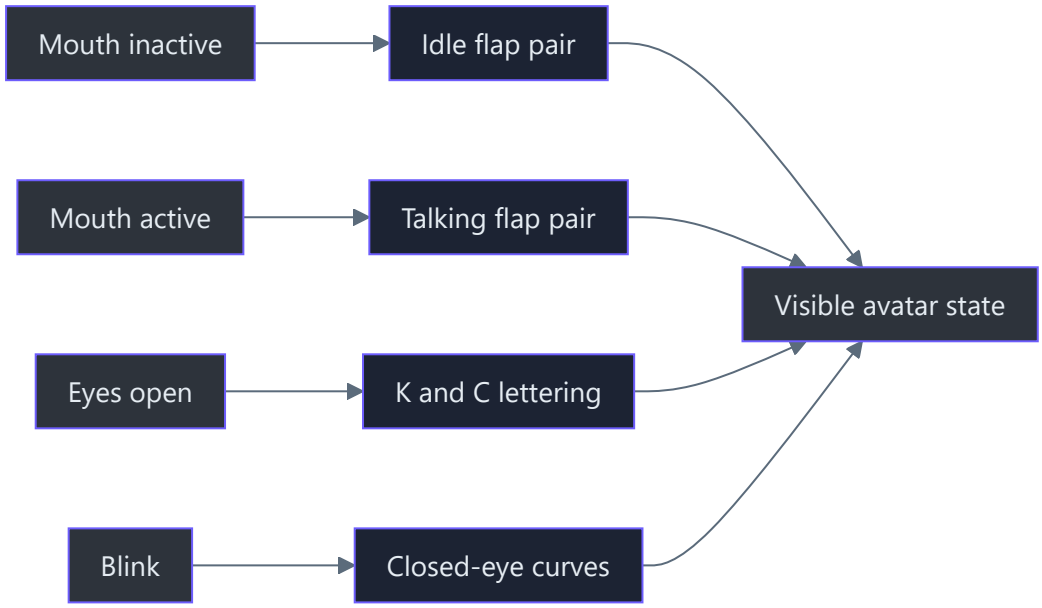


idle-blink

talking-blink

Figure 1 — The four valid V1 combinations. Horizontal movement changes idle/talking flaps; vertical movement changes open/blinking eyes.

State architecture



The two state axes are independent; the manifest lists all four combinations (`assets/PNGTuberV1/model-manifest.json:116-149`). The renderer must preserve that independence so the user can blink while talking.

Permanent silhouette



Figure 2 — The permanent V1 base. The recognizable design combines the cardboard head, right-side headphones, dark hoodie, hands, laptop, and seated pose.

The new application initially replaces only the head in the real camera feed, so the full seated body is reference material rather than geometry that must be reproduced immediately. The canonical asset guide explicitly distinguishes permanent body artwork from expression layers (`assets/PNGTuberV1/README.md:1`).

Eye identity: K and C



Figure 3 — The letters are not a logo placed near the eyes; they are the open-eye expression itself. Current tracking maps the user's left eye to K and right eye to C.



Figure 4 — Closed-eye curves replace both letters during the original binary blink state. The current renderer supports independent left/right values while preserving this visual language.

Mouth and flap identity

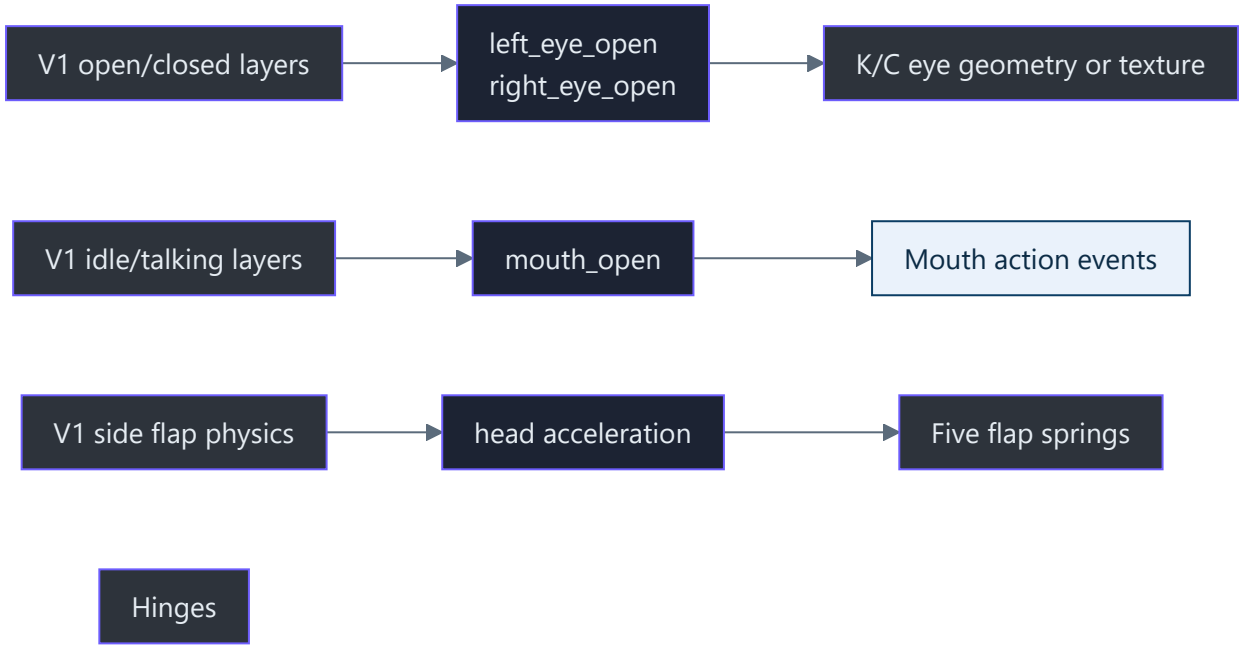


Figure 5 — Talking does not draw a conventional mouth. The front cardboard flaps change shape and motion, making the box itself speak.

The two talking flaps intentionally use unequal amplitudes in the original model (`assets/PNGTuberV1/model-manifest.json:71-111`). The 3D version should therefore avoid perf-

ectly mirrored hinge motion.

Mapping V1 semantics to the current model



Implementation guardrails

1. Preserve the cardboard-box silhouette and handmade lettering.
2. Keep K and C independently controllable.
3. Drive front flaps from mouth openness, not audio volume.
4. Keep left/right flap gains asymmetric.
5. Preserve a separately sprung side flap.
6. Treat generated composites as references, not runtime source layers.
7. Keep V1 files unchanged as historical canonical input.

References

- `assets/PNGTuberV1/README.md` — complete human-readable asset guide
- `assets/PNGTuberV1/model-manifest.json` — machine-readable hierarchy and behavior
- `assets/PNGTuberV1/Full Body.png` — permanent base artwork
- `assets/PNGTuberV1/Eyes Open.png` and `Eyes Closed.png` — eye-expression source
- `assets/PNGTuberV1/FrontFlap1.png` and `FrontFlap2.png` — talking flap source
- `assets/PNGTuberV1/RightFlap1.png` — secondary-motion source
- `assets/PNGTuberV1/reference/state-sheet.png` — four-state visual summary

⬅️ Foundations · 🏠 Book home · ➡️ System architecture

PART III

Architecture

The complete real-time pipeline: capture, immutable state, tracking, filtering, GPU rendering, privacy-safe composition, diagnostics, and OBS presentation.

8

PART III · ARCHITECTURE

CHAPTER 8

Architecture

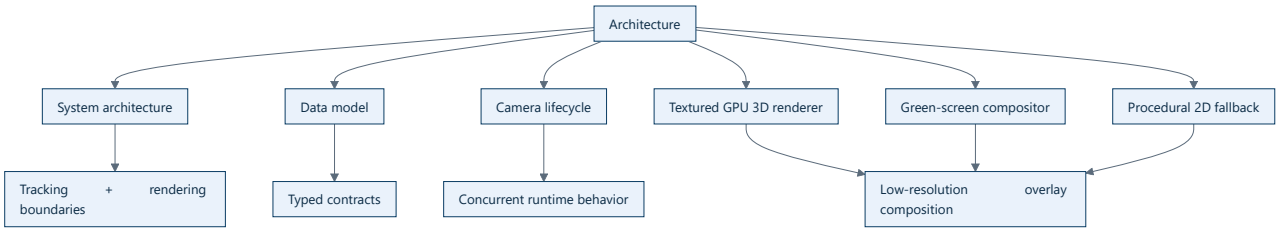
PART III · ARCHITECTURE Chapter 8

02 · Architecture

> **TL;DR** — The application is a staged real-time pipeline. Capture is already isolated behind a narrow API. Tracking, segmentation, rendering, and composition consume immutable latest-state snapshots without changing capture semantics.

Chapter map

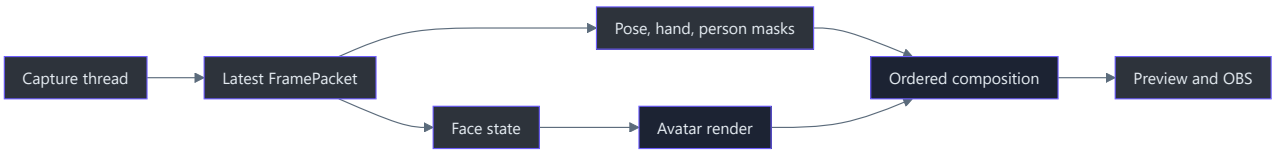
- System architecture — current components, boundaries, and frame flow
- Domain and runtime data model — every current structure and invariant
- Camera lifecycle — startup, running, failure, reconnect, and shutdown
- Textured GPU 3D renderer — default mesh, materials, lighting, pose, and compositing
- Green-screen compositing — async person masks and fail-closed chroma output
- Procedural PS1 renderer — privacy-safe 2D fallback and original prototype
- Desktop runtime architecture — native Qt host, bounded preview bridge, live per-eye tuning, cleanup, and private subprocess tasks



Architectural style

The code currently combines:

- **Pipeline architecture** for frame flow.
- **Ports and adapters** at the video-capture boundary.
- **Producer/consumer concurrency** with a single latest-value register.
- **Finite-state lifecycle management** for recovery and diagnostics.
- **Functional data contracts** through frozen dataclasses.
- **Fail-closed privacy boundaries** for face loss and stale segmentation masks.
- **Control-plane / data-plane separation** for native widgets versus the shared pipeline worker.



Sources

- `src/kardboard_vtuber/camera/models.py:15-157`
- `src/kardboard_vtuber/camera/stream.py:20-288`
- `src\kardboard_vtuber\cli.py`, `validate_options`, `main`, `_create_renderer`
- `src/kardboard_vtuber/renderer/textured_3d.py:140-327`
- `src/kardboard_vtuber/tracking/green_screen.py:16-152`
- `src\kardboard_vtuber\runtime.py`, `RuntimeBridge` and `LiveTuning`
- `src\kardboard_vtuber\gui\worker.py`, `SessionWorker`
- `src\kardboard_vtuber\gui\window.py`, `StudioWindow`

⬅️ Foundations · ➡️ Algorithms and data structures

9

PART III · ARCHITECTURE

CHAPTER 9

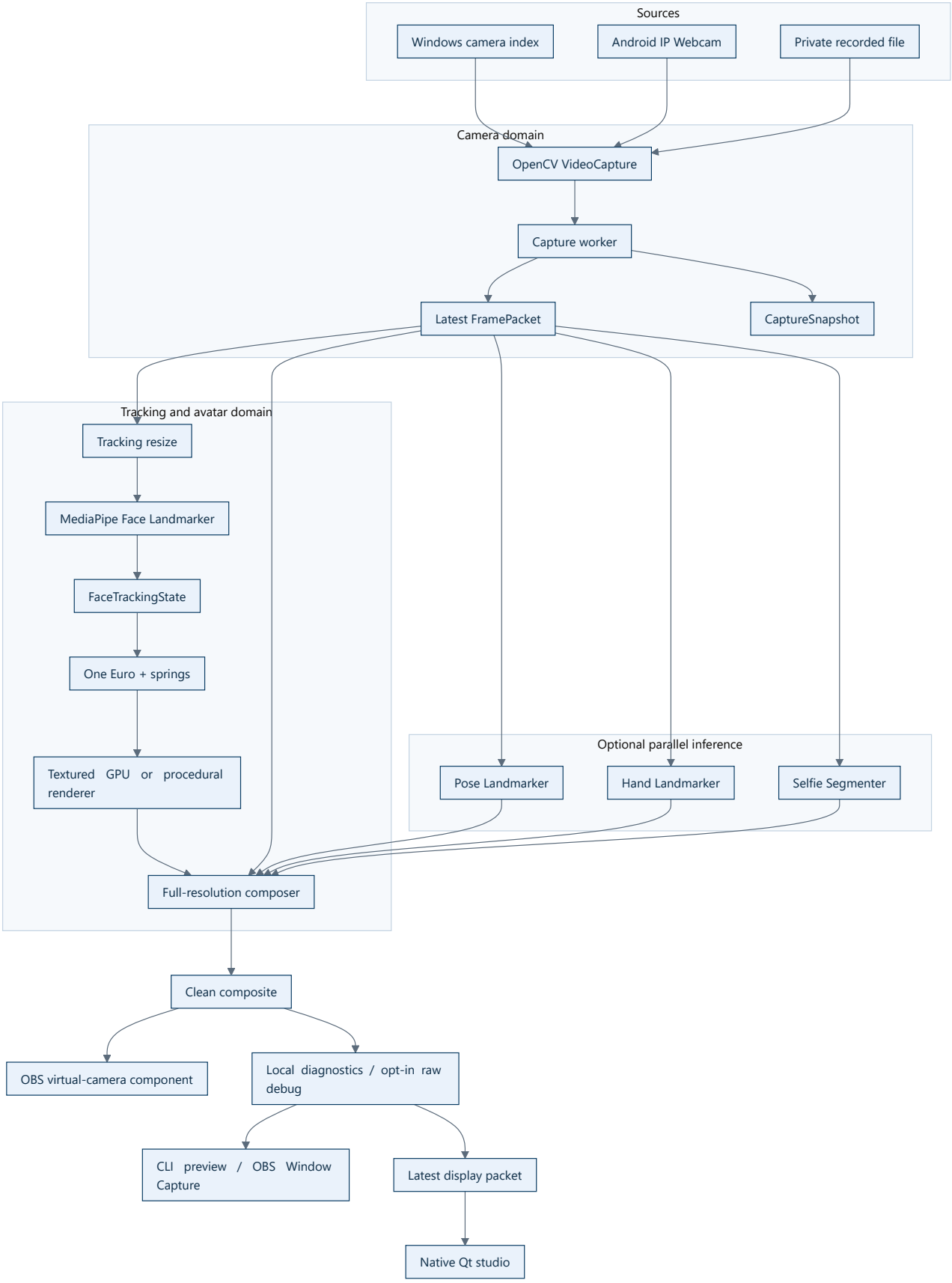
System architecture

PART III · ARCHITECTURE Chapter 9

System architecture

> **TL;DR** — Capture, tracking, rendering, composition, and presentation are separate stages with different performance characteristics. The system passes small contracts between stages rather than allowing one library to own the entire application.

End-to-end runtime



Component boundaries

Configuration boundary

`CameraConfig` contains requested behavior. It validates impossible values early but never claims that hardware accepted a request (`src/kardboard_vtuber/camera/models.py:90-121`).

Capture boundary

`VideoCaptureLike` describes the OpenCV methods the worker needs (`src/kardboard_vtuber/camera/stream.py:20-29`). This protocol is the seam used by tests.

Concurrency boundary

`LatestFrameCamera` owns the thread, capture handle, condition variable, latest packet, counters, and lifecycle (`src/kardboard_vtuber/camera/stream.py:39-73`).

Presentation boundary

`cli.main` reads packets, submits optional asynchronous inference, resolves the head mode, orders green-screen/head/hand composition and publishes clean output. The CLI then adds local overlays; Studio instead transfers diagnostics in independent side-panel channels. It does not implement capture or reconnection. Full-body diagnostics stay separate rather than adding a filled body to the output.

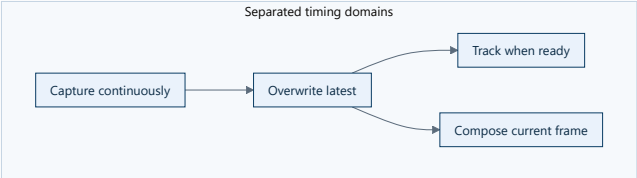
The optional native studio calls this same pipeline from `SessionWorker.run`. `RuntimeBridge` replaces HighGUI presentation with a one-slot owned preview and typed telemetry. Qt widgets and painting stay on the main thread; native capture/tracking/rendering and the ModernGL lifecycle stay off it. Desktop runtime architecture traces the complete ownership, stop, error, and live-calibration contracts.

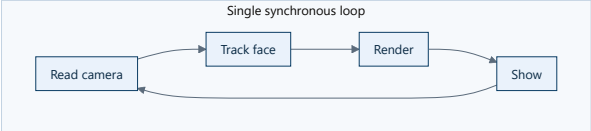
The separate VIDEO host uses PyAV and synchronous MediaPipe VIDEO tasks to process every source frame without a live camera/latest-frame dropping. Studio and the Qt-free video CLI share this offline engine, reusing the existing renderer and effects. Source size/timing and compatible audio are preserved within format precision. See video processing.

Live calibration boundary

`tracking.eyes.EyeSensitivity` owns effective anatomical left/right thresholds. `LiveTuning` atomically carries them and processing brightness to the worker. `FaceActionDetector.set_thresholds` and both renderers' `set_eye_sensitivity` use the same validated snapshot, without rebuilding the renderer or discarding its safe-frame hold. Raw eye signals now drive both action and visual closure decisions so smoothing cannot erase a blink. Geometry/pose remain filtered; telemetry also records the actual rendered state so fallback expressions are not confused with raw scores.

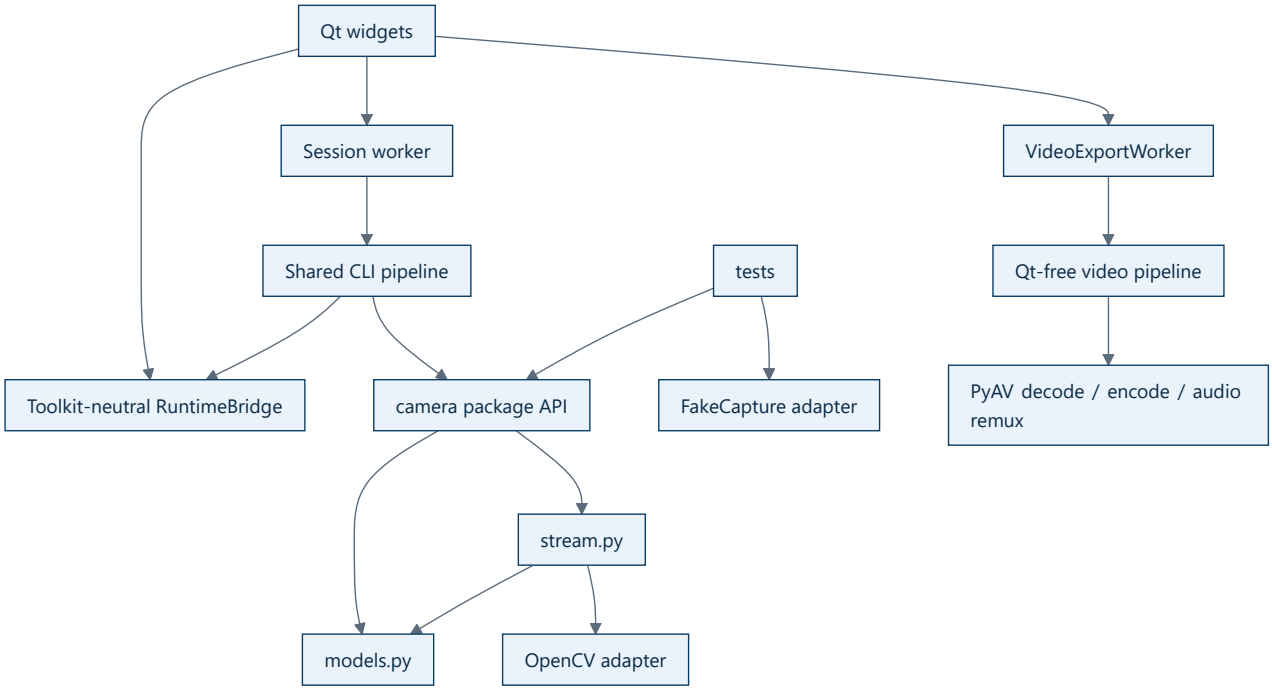
Why not one giant loop?





A synchronous loop makes camera ingestion wait for every expensive downstream stage. Separation lets capture stay current even when inference or rendering slows down.

Dependency direction



Higher-level code depends on project abstractions. OpenCV-specific constants are translated by `CameraBackend.opencv_id` and `CameraRotation.opencv_code` (`src/kardboard_vtuber/camera/models.py:15-48`).

Stable extension seams

- Trackers consume transformed frame data, not `VideoCapture`.
- Renderers consume normalized project-owned tracking state, not raw MediaPipe objects.
- Compositors consume the current full frame plus project-owned masks or rendered overlays.
- OBS transport should consume composed output and remain independent of tracking.
- Native widgets consume owned display packets and send validated control snapshots, not mutable OpenCV/MediaPipe/ModernGL objects.

These seams are implemented in the camera, tracking, renderer, and CLI packages (`src/kardboard_vtuber/tracking/models.py:15-100`, `src/kardboard_vtuber/renderer/__init__.py:1`, `src/kardboard_vtuber/tracking/green_screen.py:124-152`).

⬅️ Architecture · ➡️ Data model

10

PART III · ARCHITECTURE

CHAPTER 10

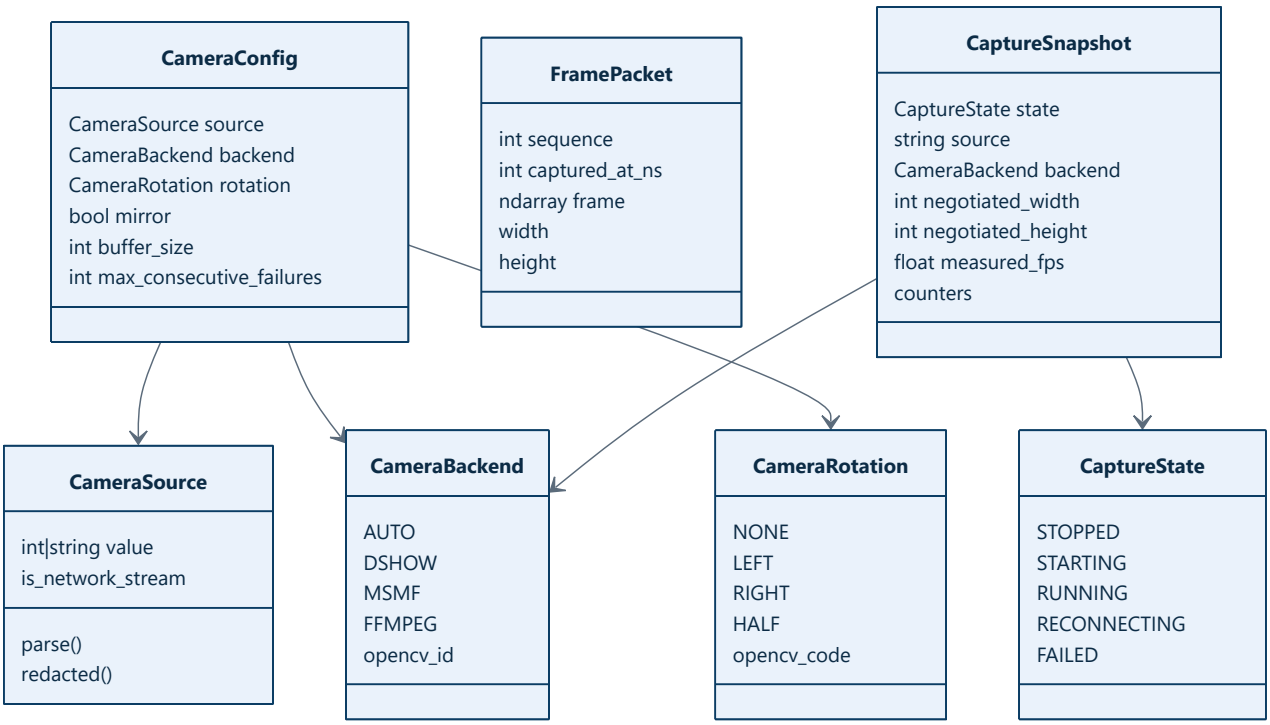
Domain and runtime data model

PART III · ARCHITECTURE Chapter 10

Domain and runtime data model

> **TL;DR** — The current model separates user intent, one captured frame, lifecycle state, and diagnostics. This prevents mutable runtime details from leaking into configuration or UI code.

Relationship diagram



`CameraSource`

Defined at `src/kardboard_vtuber/camera/models.py:62-87`.

Rule	Reason
Decimal strings become device indices	CLI input arrives as text
Other strings remain URLs/paths	OpenCV accepts string sources
Empty values fail immediately	Avoid meaningless reconnect loops
Credentials are redacted for display	Diagnostics must not leak secrets

`CameraConfig`

Defined at `src/kardboard_vtuber/camera/models.py:90-121`.

This is an immutable request, not observed truth. Width, height, FPS, and buffer size are offered to the backend. The backend may ignore them. Observed values belong in `CaptureSnapshot`.

`FramePacket`

Defined at `src/kardboard_vtuber/camera/models.py:124-138`.

Field	Invariant
`sequence`	Strictly increases for every published frame
`captured_at_ns`	Uses a monotonic clock
`frame`	OpenCV BGR NumPy array

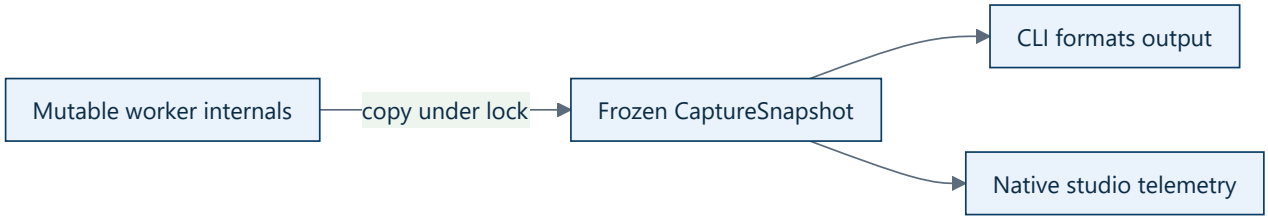
Sequence numbers answer "is this newer than what I processed?" Timestamps answer "how old is this inside the process?" They are deliberately separate.

`CaptureSnapshot`

Defined at `src/kardboard_vtuber/camera/models.py:141-157`.

A snapshot is immutable and point-in-time. It contains:

- lifecycle state;
- redacted source and selected backend;
- negotiated width, height, and FPS;
- received, overwritten, failed-read, and reconnect counters;
- measured receive rate;
- last error.



Why frozen and slotted?

- `frozen=True` makes accidental mutation fail.
- `slots=True` prevents undeclared fields and reduces per-instance overhead.
- Explicit fields make the contracts easy to test, serialize later, and explain.

Tracking models

The tracker introduces a library-neutral `FaceTrackingState` containing landmarks, face bounds, independent eye openness, mouth openness, and `HeadPose` (`src/kardboard_vtuber/tracking/models.py:58-90`).

```
class FaceTrackingState:
    head_pose: HeadPose
    left_eye_open: float
    right_eye_open: float
    mouth_open: float
    timestamp_ms: int
```

This prevents MediaPipe-specific result objects from contaminating the renderer.

Desktop control and display contracts

Contract	Ownership / invariant
`EyeSensitivity`	Immutable effective anatomical left/right blink, open, wink and asymmetry thresholds shared by actions/renderers
`LiveTuning`	One validated tuple of eye sensitivity, internal brightness and debounce intervals; atomically replaced
`RuntimeTelemetry`	Raw/filtered face state, actual rendered eye state, effective detection, observed rates, dimensions and synthetic/raw flags
`PreviewPacket`	Separate owned program preview, full-resolution clean window frame, local mesh/raw-face/skeleton buffers and matching telemetry
`RuntimeBridge`	Thread event, short lock, one latest display slot and a bounded 500-entry log ring
Studio preset	Versioned allowlisted scalar settings; excludes camera source and raw-preview consent
`VideoExportOptions`	Local input, new output and explicit lossless/MP4 choice; never an existing destination
`VideoProgress` / `VideoExportResult`	Source-time progress, frame/dimension counts and completed output/audio metadata

These are defined in `tracking\eyes.py`, `runtime.py`, and `gui\settings.py`. They separate a small control plane from the full-size compositor/output. The widget thread receives no mutable MediaPipe result or ModernGL object. The desktop architecture chapter traces the actual readers and writers. Offline file contracts live in `video.py` and the video-processing chapter.

⬅️ System architecture · ➡️ Camera lifecycle

11

PART III · ARCHITECTURE

CHAPTER 11

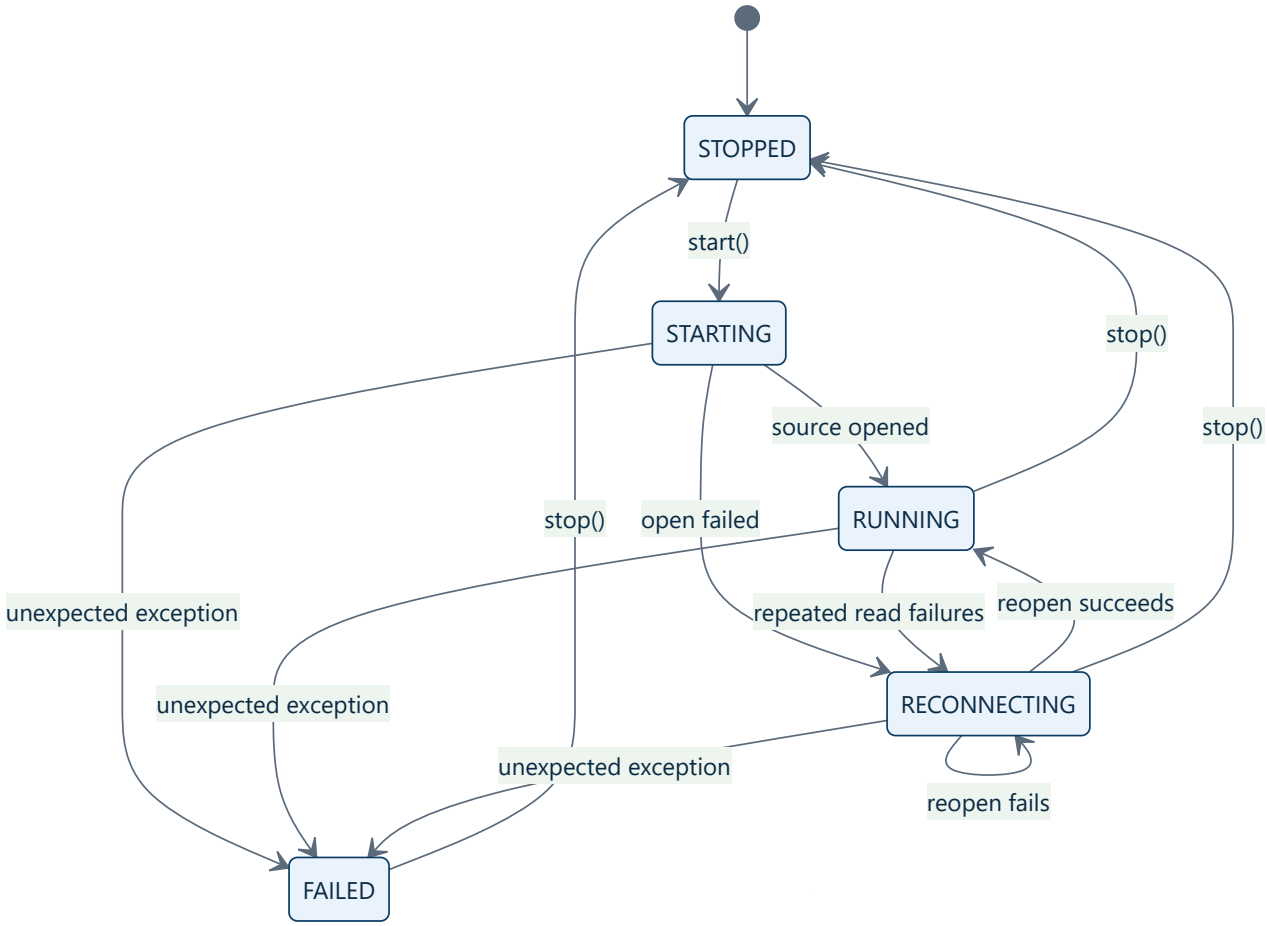
Camera lifecycle

PART III · ARCHITECTURE Chapter 11

Camera lifecycle

> **TL;DR** — Capture is modeled as a finite-state lifecycle rather than a Boolean “open” flag. Distinguishing startup, reconnection, terminal failure, and clean shutdown improves diagnostics and makes recovery behavior explicit.

State machine



`CaptureState` is defined at `src/kardboard_vtuber/camera/models.py:51-58`.

Startup

1. `start()` rejects duplicate worker creation.
2. State becomes `STARTING`.
3. A daemon thread runs `_run()`.

4. The caller waits for `RUNNING`, `FAILED`, or timeout.

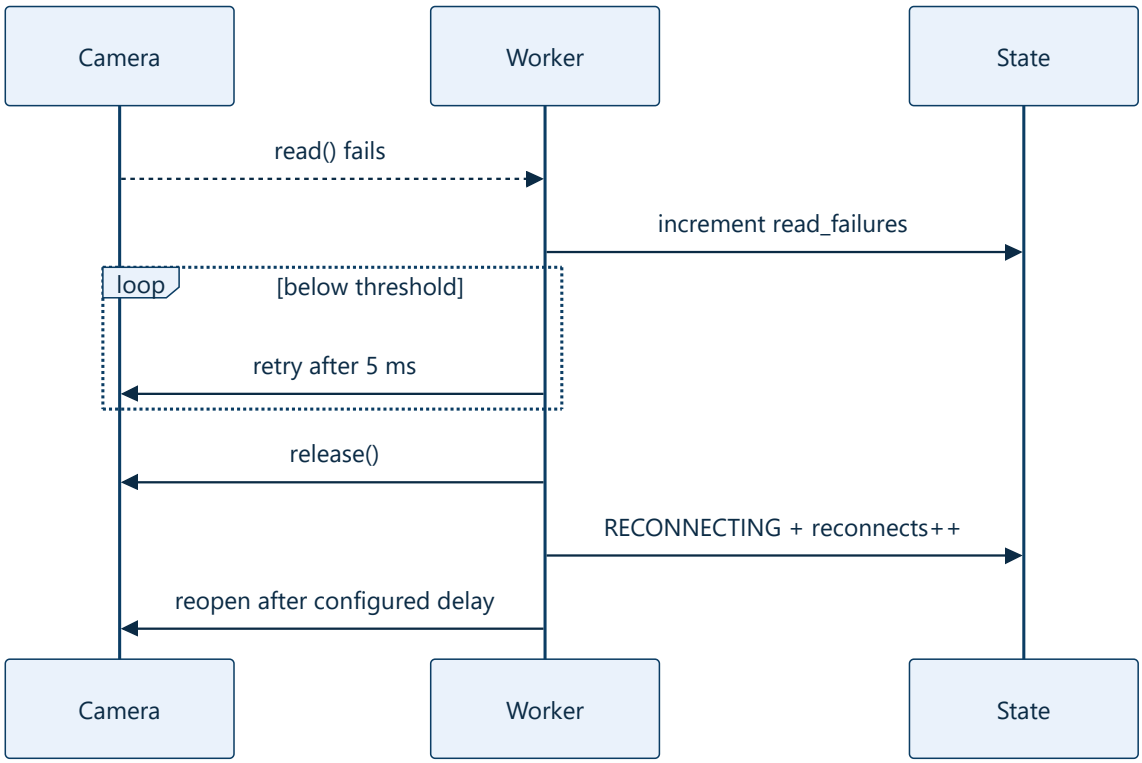
Implementation: `src/kardboard_vtuber/camera/stream.py:78-97`.

Normal operation

The worker opens the source, applies property requests, reads negotiated values, and continuously publishes frames. Successful reads restore `RUNNING`.

Implementation: `src/kardboard_vtuber/camera/stream.py:167-220`.

Temporary failure and reconnection



Repeated failures trigger `_disconnect()` at `src/kardboard_vtuber/camera/stream.py:270-279`. The threshold is configurable and validated in `models.py:104-119`.

Terminal failure

Unexpected exceptions are not swallowed. The worker records a typed message, enters `FAILED`, and wakes consumers (`stream.py:205-210`). `start()` and the CLI surface that failure.

Shutdown

`stop()` sets an event, wakes waiters, joins the thread, releases the capture object, and publishes `STOPPED` (`stream.py:99-112`). The class also supports `with LatestFrameCamera( ... )`.

⬅️ Data model · ➡️ Algorithms

12

PART III · ARCHITECTURE

CHAPTER 12

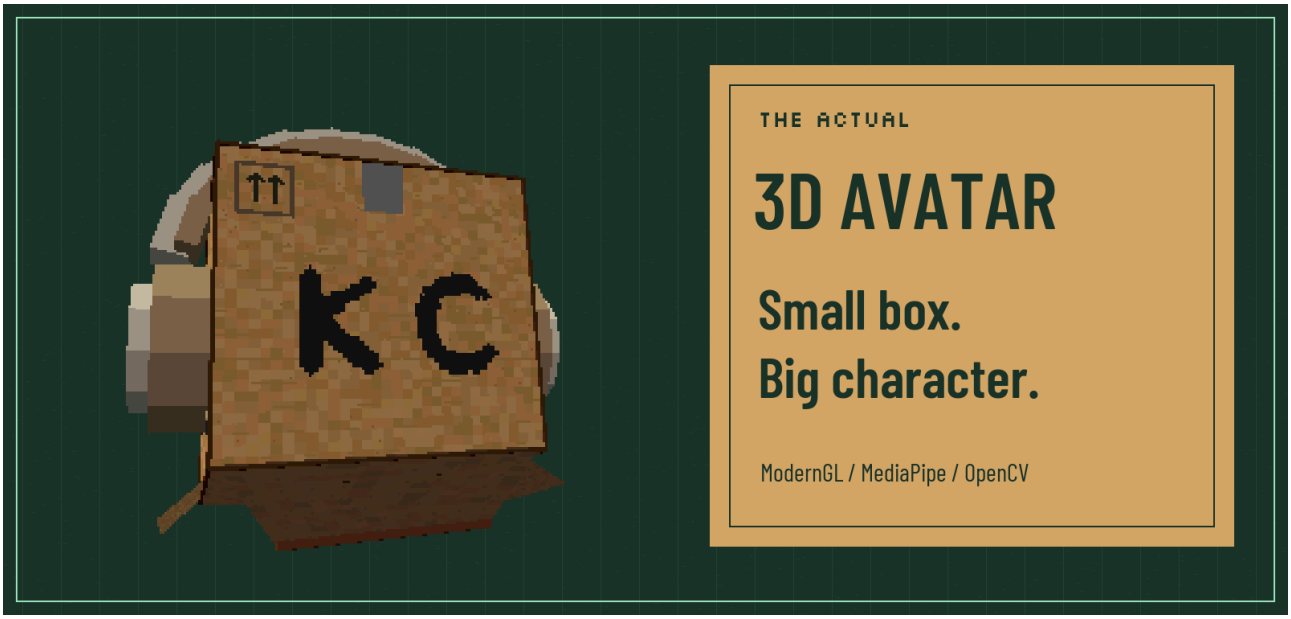
Textured GPU 3D renderer

PART III · ARCHITECTURE Chapter 12

Textured GPU 3D renderer

Status: implemented as the default cardboard renderer.

TL;DR — ModernGL renders a procedurally generated cubic cardboard character into a transparent low-resolution framebuffer. The current model includes the complete front face, neck-safe underside, headphones, aged shipping decals, independent K/C expressions, five optional spring hinges, and a configurable perspective depth offset (`src/kardboard_vtuber/renderer/textured_3d.py:111-327`).

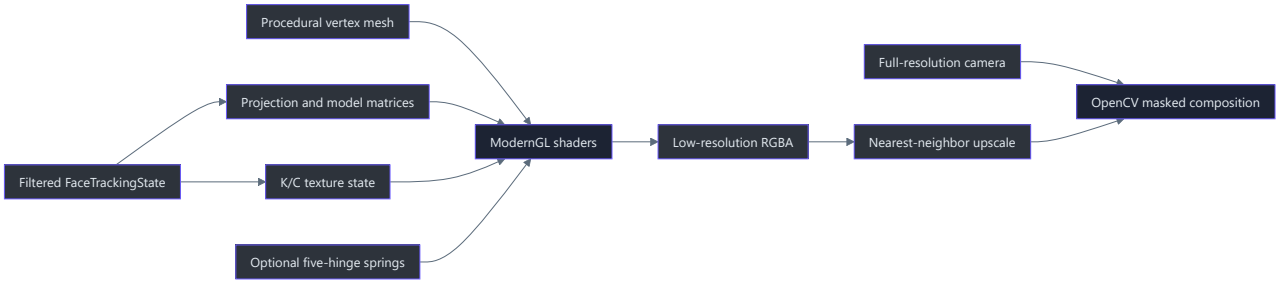




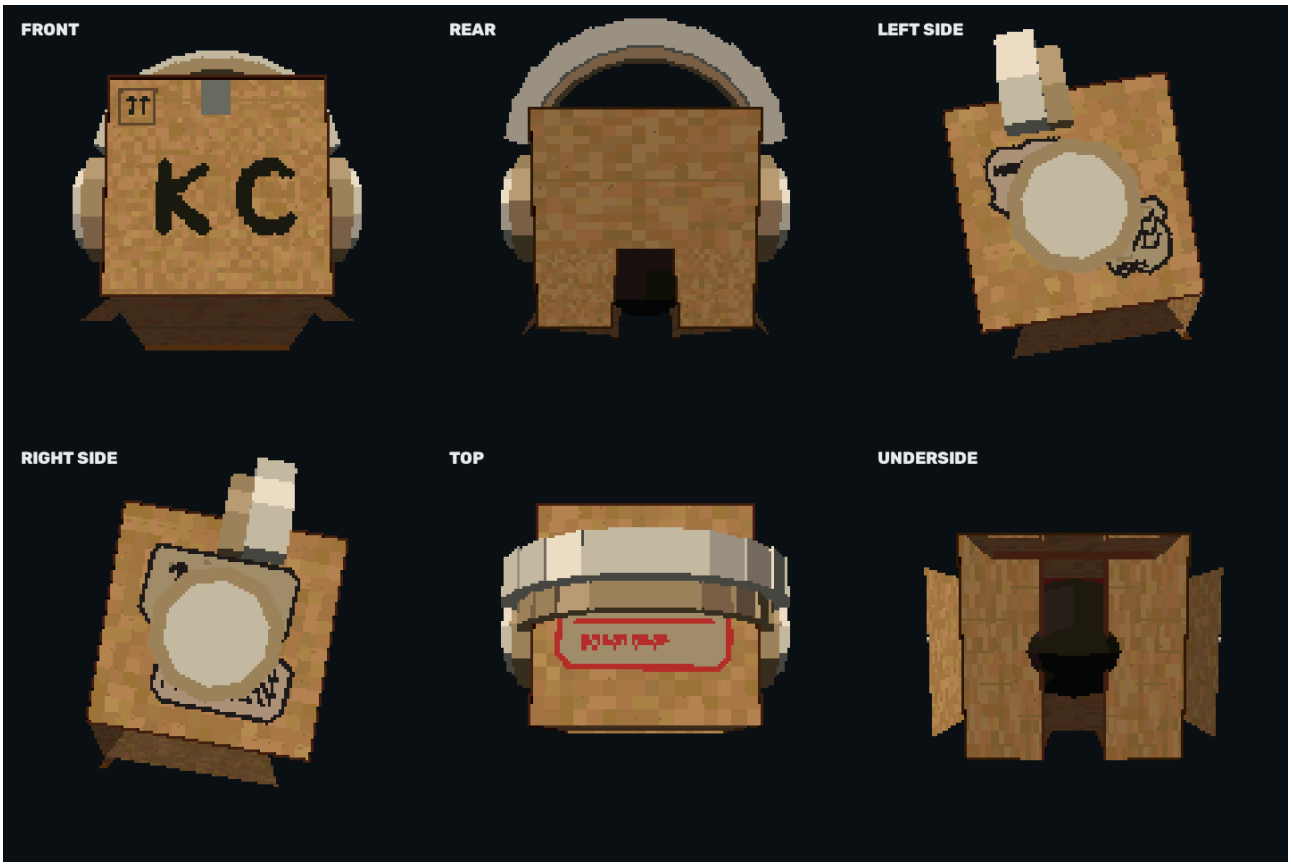
The animation uses filtered numeric pose and expression telemetry from the private regression recording, then renders every frame on a synthetic dark background with five-hinge physics enabled. No source-video pixels are decoded into or stored in the GIF (`scripts/generate_readme_animation.py:1`).

Why this renderer exists

The procedural OpenCV renderer proved tracking, pose signs, compositing, and fail-closed privacy, but it cannot provide coherent depth-tested surfaces or real 3D headphone geometry. The default renderer therefore keeps Python and OpenCV as the application shell while delegating the avatar to an offscreen OpenGL 3.3 pipeline (`src/kardboard_vtuber/renderer/ps1_cardboard.py:35-253`, `src/kardboard_vtuber/renderer/textured_3d.py:140-276`).

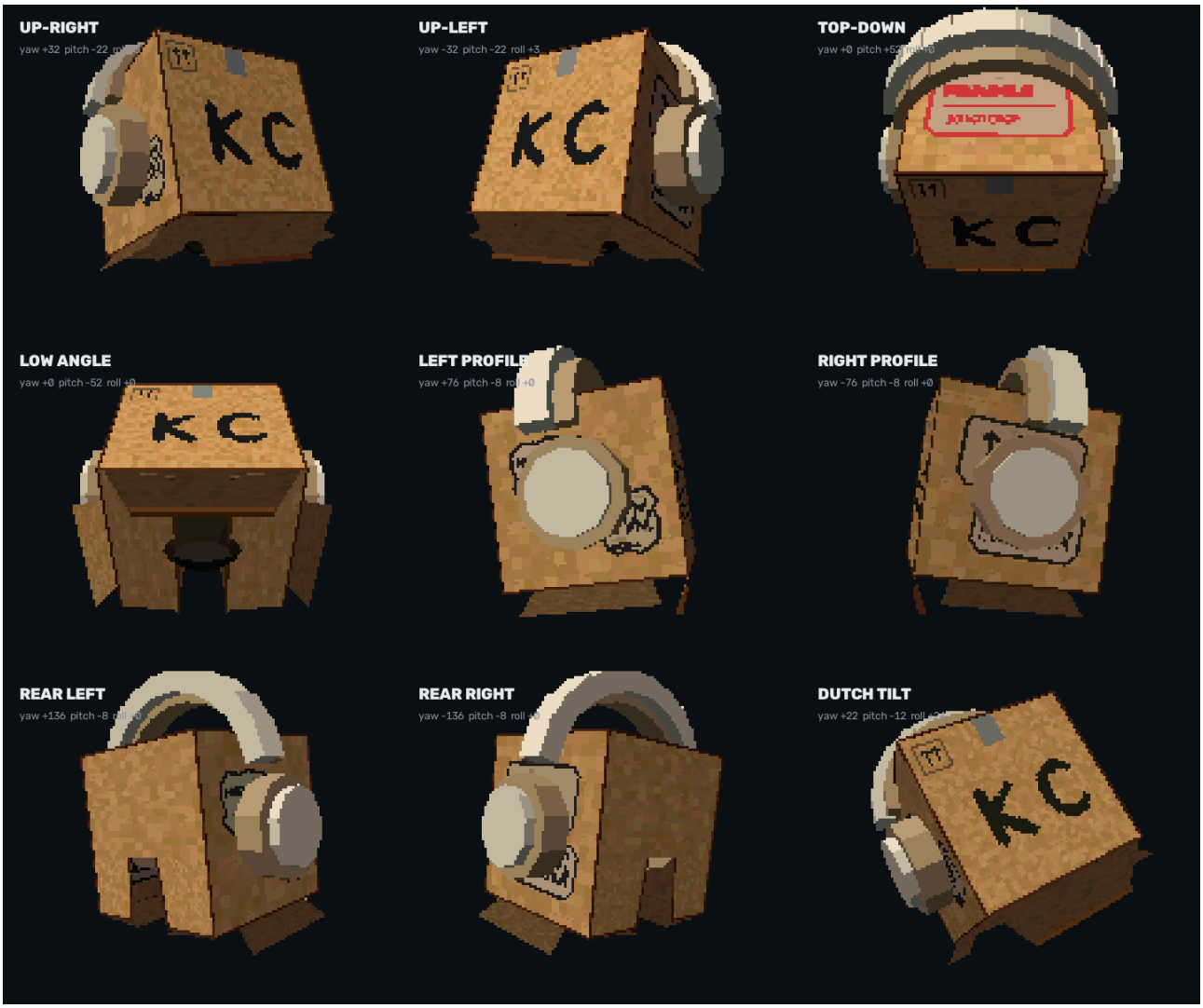


Six-sided turnaround



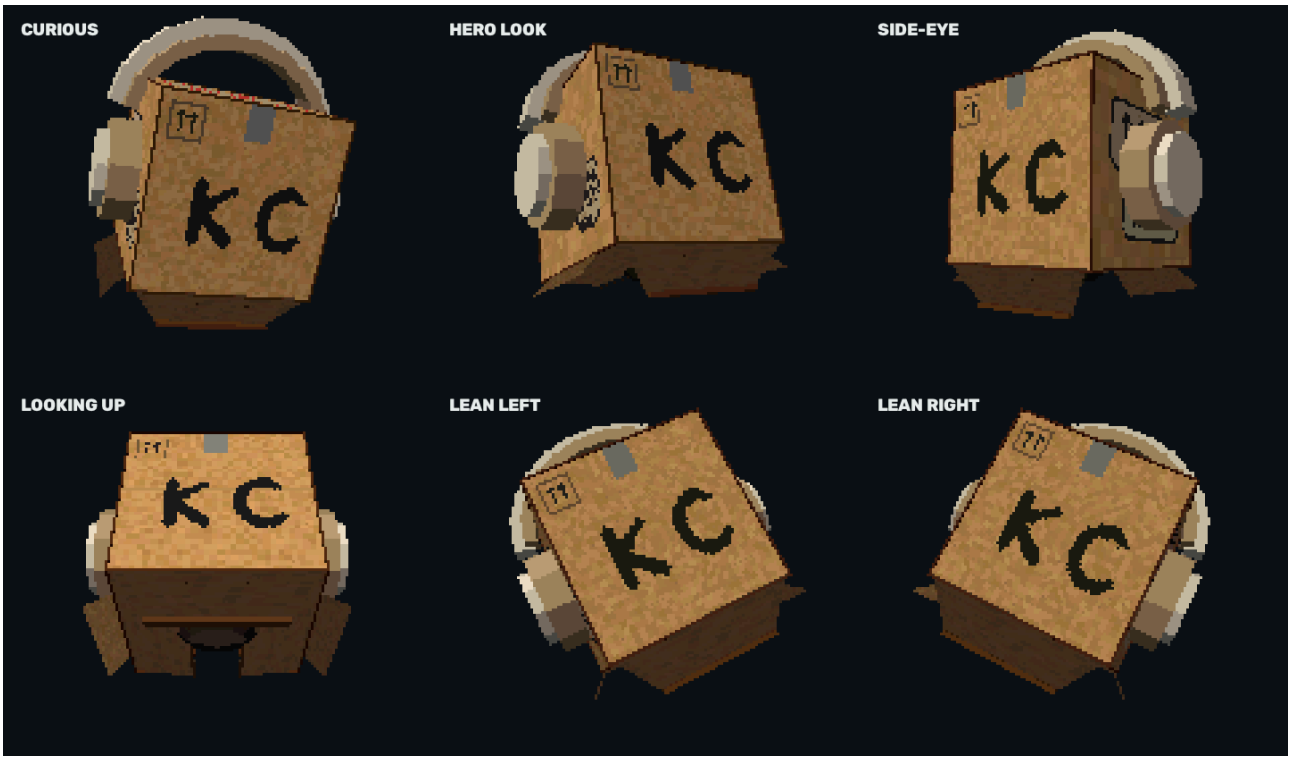
The image is generated from the same `_build_character_mesh()` and shader path used at runtime; it contains no camera pixels (`src/kardboard_vtuber/renderer/textured_3d.py:532-748`).

Expanded angle gallery



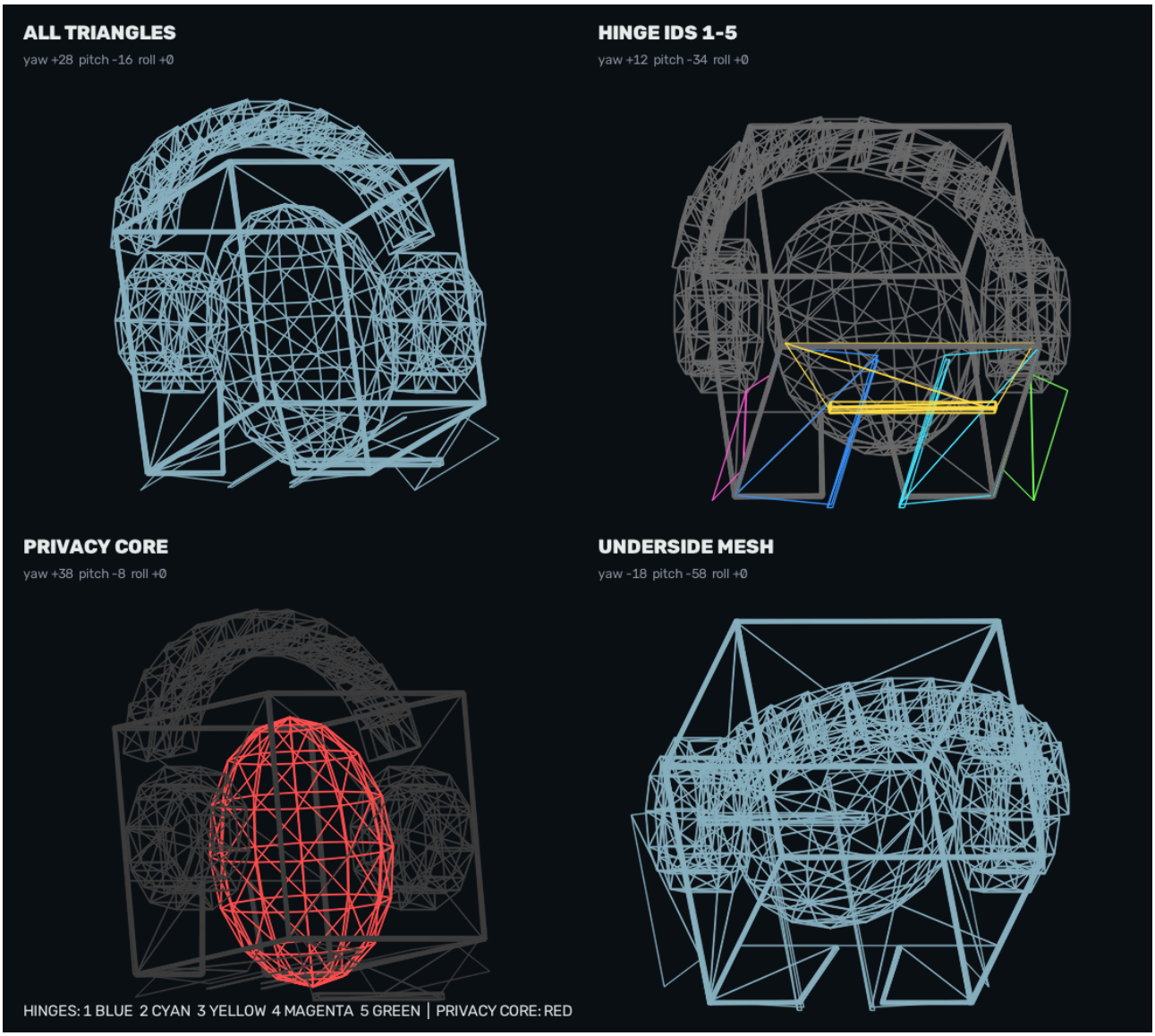
The gallery exercises positive and negative yaw, steep pitch in both directions, rear-quarter views, and roll. It makes the physical side labels, rear neck channel, top label, underside, and headphone depth visible in one face-free sheet.

Cinematic pose scenarios



These poses combine pitch, yaw, and roll instead of isolating one axis. They demonstrate how the same fixed mesh reads as curious, heroic, skeptical, upward-looking, or leaning without a separate animation rig (`src/kardboard_vtuber/renderer/textured_3d.py:280-327`).

Mesh diagnostics



The runtime CLI does not currently expose a separate OpenGL wireframe window. This documentation sheet is generated directly from `_build_character_mesh()` and therefore reflects the same vertex positions and per-vertex hinge IDs uploaded to ModernGL. Static triangles are gray in the hinge view, hinge IDs `1..5` are color-coded, and the internal privacy volume is isolated in red (`src/kardboard_vtuber/renderer/textured_3d.py:154-169`, `src/kardboard_vtuber/renderer/textured_3d.py:532-747`, `scripts/generate_documentation_gallery.py:1`).

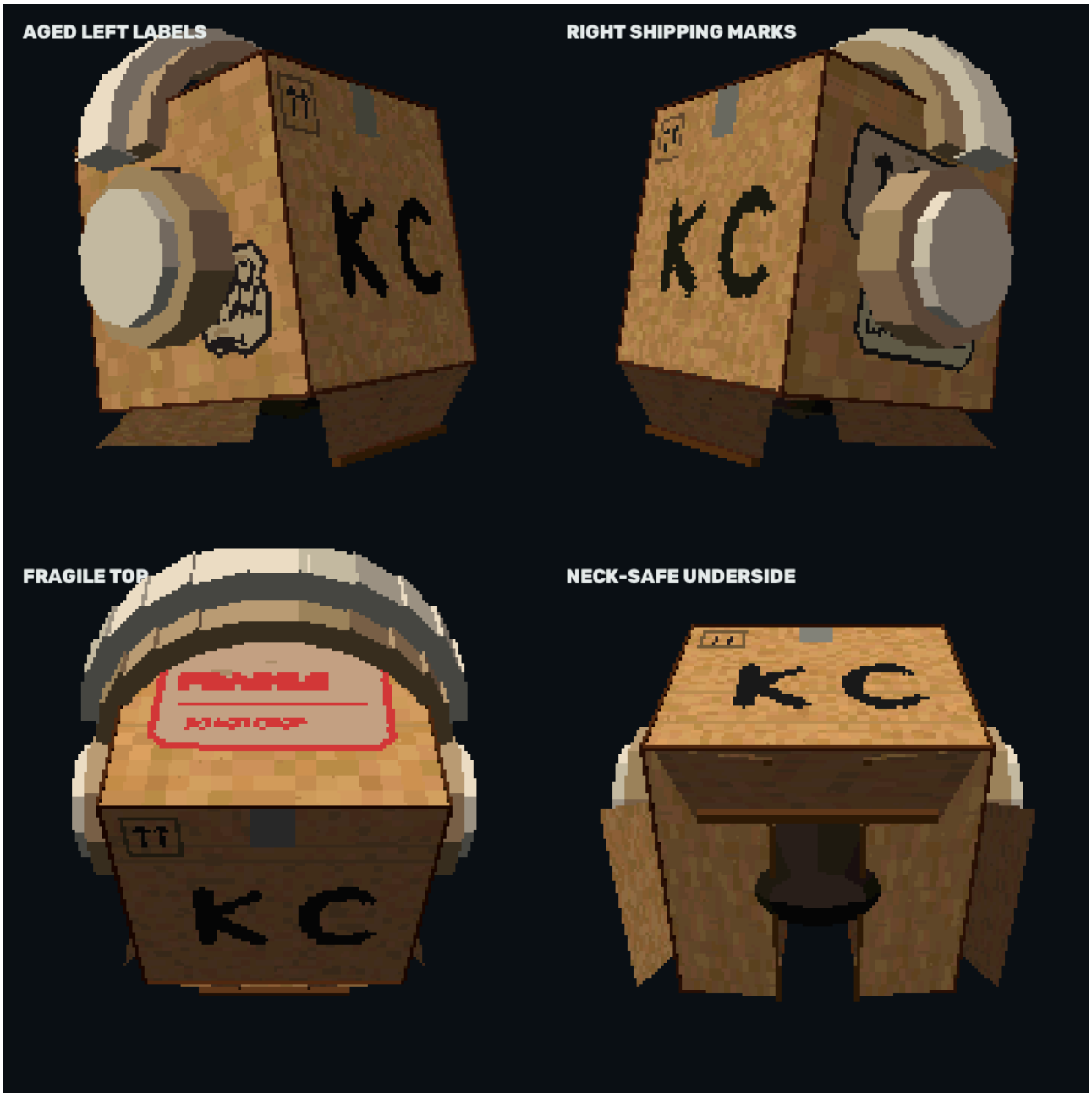
Geometry inventory

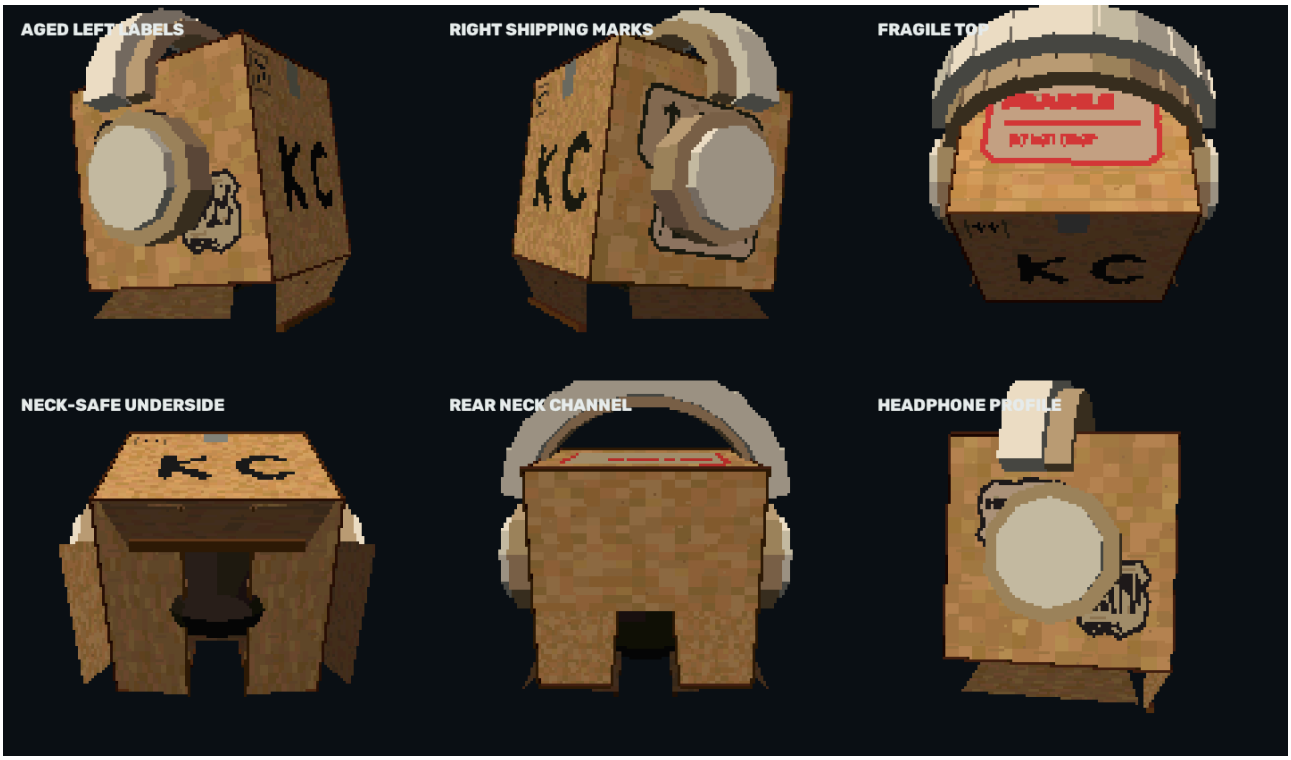
Surface or part	Current implementation
Front	One complete rectangular textured quad; the former lower V cut and red boxed-X stamp are removed

Side walls	Full left and right cardboard faces with dedicated atlas regions
Top	Full cardboard face carrying the aged `FRAGILE` label
Rear	Upper rear panel plus lower panels around an open neck channel
Underside	Two inward closure panels, one broad front-hinged flap, and a central front-to-rear neck channel
Privacy core	Opaque faceted ellipsoid covering hair, face, chin, and beard behind the visible shell
External tabs	Two cardboard tabs attached to the lower outside side edges
Headphones	Segmented cream band, inset light-beige cushion, annular ear cushions, protruding cream earcups
Edge definition	Thin low-contrast dark-brown bars on cube edges without a raised front border

The cube side length is the larger of the requested face-derived width and height and is applied uniformly to X, Y, and Z, preserving a cubic shell (`src/kardboard_vtuber/renderer/textured_3d.py:280-327`, `tests/test_textured_3d_renderer.py:351-368`).

Detailed views





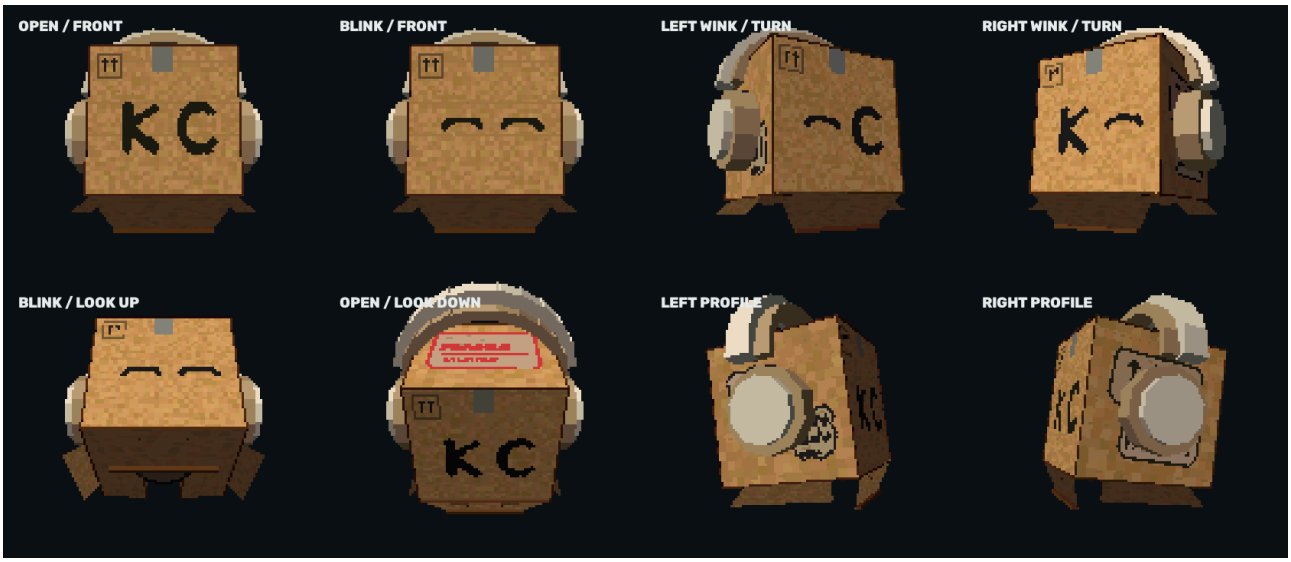
Texture atlas and aged decals

The deterministic `1024 × 512` atlas contains cardboard noise, corrugation lines, one upper-front tape strip, face markings, dedicated side regions, and a top region (`src/kardboard_vtuber/renderer/textured_3d.py:984-1029`). The current labels use:

- dirty beige paper rather than clean white;
- missing and notched corners;
- independent irregular tear silhouettes on the two left-side labels;
- non-repeating stain placement;
- hard low-resolution barcode bars;
- coarse nearest-neighbor lettering, including the red `FRAGILE` and `DO NOT DROP` text;
- no red boxed-X stamp or lower-center tape strip on the front face (`src/kardboard_vtuber/renderer/textured_3d.py:1030-1230`, `tests/test_textured_3d_renderer.py:245-322`).

K/C expression system





K and C are custom irregular painted paths, not font glyphs. They are rasterized on a seven-pixel grid using one flat ink color and then enlarged as square blocks. Wink and blink arcs use the same coarse mask, preserving independent anatomical left/right control (`src/kardboard_vtuber/renderer/textured_3d.py:1248-1350`, `tests/test_textured_3d_renderer.py:324-350`).

Five-hinge flap physics

`--physics` creates five underdamped springs and enables per-vertex hinge IDs in the vertex shader (`src/kardboard_vtuber/renderer/textured_3d.py:15-89`, `src/kardboard_vtuber/renderer/textured_3d.py:329-472`).

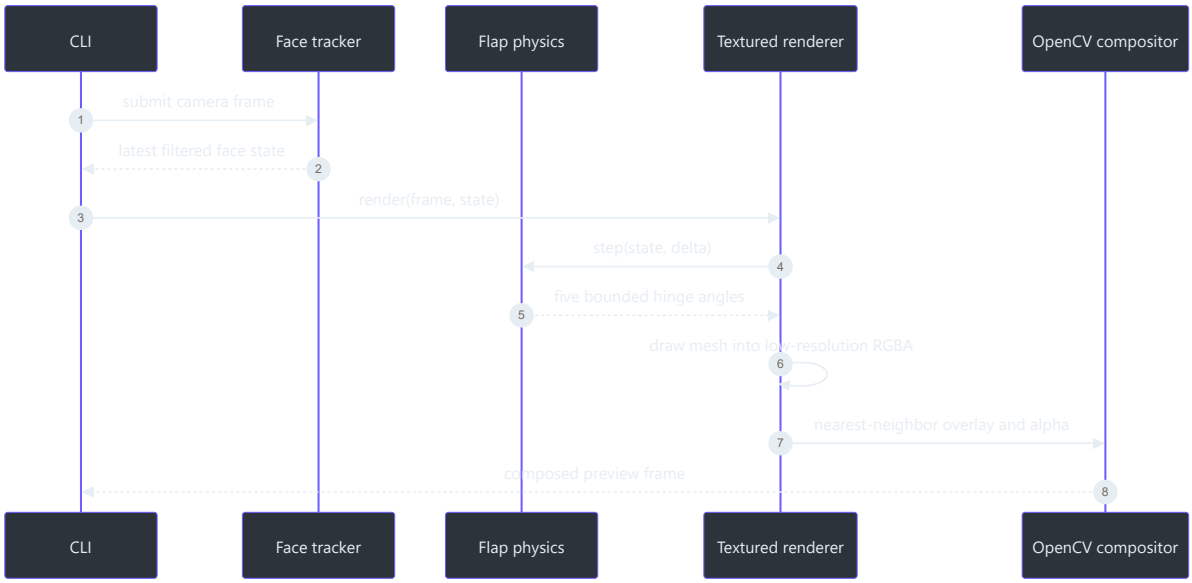
Hinge	Axis and pivot	Primary inputs	Limit
Left inward underside panel	Z axis at left lower box edge	roll, yaw, horizontal movement	26°
Right inward underside panel	Z axis at right lower box edge	roll, yaw, horizontal movement	26°
Broad front underside flap	X axis at front lower edge	pitch and vertical movement	24°
Left external side tab	Z axis at left outside edge	highly amplified yaw and horizontal movement	42°
Right external side tab	Z axis at right outside edge	highly amplified yaw and horizontal movement	42°





The springs use bounded semi-implicit Euler integration from the shared `DampedSpring` primitive. Large timestamp gaps reset motion rather than integrating stale impulses (`src/kardboard_vtuber/motion/springs.py:26-85`, `tests/test_motion_springs.py:18-71`).

Frame data flow



The CLI keeps full-body output in a separate line-skeleton diagnostic window, then restores the hand/forearm foreground after the head when hand occlusion is enabled (`src/kardboard_vtuber/cli.py`, `src/kardboard_vtuber/tracking/hand_occlusion.py:137-213`).

Perspective, pose, and depth

Pitch, yaw, and roll build the model matrix:

- positive yaw exposes the physical left side;
- negative yaw exposes the physical right side;
- positive pitch reveals the top;
- negative pitch reveals the underside;
- positive roll rotates counterclockwise on screen (`src/kardboard_vtuber/renderer/textured_3d.py:280-327`).

`perspective_depth_offset` defaults to `0.16`, moving the entire model farther from the camera. `--box-depth-offset 0` restores the previous position. Finite positive values have no upper cap, but large values reduce projected coverage and can expose the real head (`src/kardboard_vtuber/renderer/textured_3d.py:111-137`, `src/kardboard_vtuber/cli.py:122-130`).

Privacy invariants

The renderer deliberately separates decorative openness from privacy coverage:

1. Before the first detected face, output is black.
2. After a safe render, face-tracking loss freezes the last safe frame.
3. The internal opaque head volume remains static even when visible flaps move.
4. The complete front face does not contain a neck cut.
5. The real neck remains visible only through the underside channel.
6. Arbitrary RGB-only object occlusion is unsupported; only the bounded hand/forearm mask may restore camera pixels over the avatar (`src/kardboard_vtuber/renderer/textured_3d.py:180-279`, `src/kardboard_vtuber/tracking/hand_occlusion.py:137-213`, `tests/test_textured_3d_renderer.py:371-409`).

Configuration

Option	Default	Effect
`pixel_scale`	`3`	Render at one-third linear resolution
`box_width_multiplier`	`2.25`	Requested shell width from tracked face
`box_height_multiplier`	`2.05`	Requested shell height from tracked face
`upward_bias`	`0.12`	Raises the shell above landmark center

`fov_degrees`	`42`	Perspective projection field of view
`perspective_depth_offset`	`0.16`	Moves the model backward on camera Z
`physics_enabled`	`False`	Enables the five hinge springs

The CLI exposes depth as `--box-depth-offset` and physics as `--physics` (`src/kardboard_vtuber/cli.py:120-136`, `src/kardboard_vtuber/cli.py:632-652`).

Performance and validation

- Rendering occurs at one-third linear resolution and is enlarged with nearest-neighbor sampling.
- The fragment shader posterizes lighting to five levels and quantizes output to 5-bit color.
- The validated AMD Radeon 780M environment measured approximately `8.17 ms` per 1080×1920 frame.
- Automated coverage includes mesh shape, atlas regions, aged decals, expressions, hinge IDs, hinge sensitivity, depth offset, fail-closed output, and neck visibility (`tests/test_textured_3d_renderer.py:1-409`).

References

- `src/kardboard_vtuber/renderer/textured_3d.py:1-1414`
- `src/kardboard_vtuber/renderer/full_body.py:1-187`
- `src/kardboard_vtuber/motion/springs.py:1-85`
- `src/kardboard_vtuber/tracking/hand_occlusion.py:1-213`
- `src/kardboard_vtuber/tracking/models.py:15-100`
- `src/kardboard_vtuber/cli.py:32-652`
- `tests/test_textured_3d_renderer.py:1-409`
- `scripts/generate_documentation_gallery.py:1`

⬅️ Architecture · ➡️ Green-screen compositing

13

PART III · ARCHITECTURE

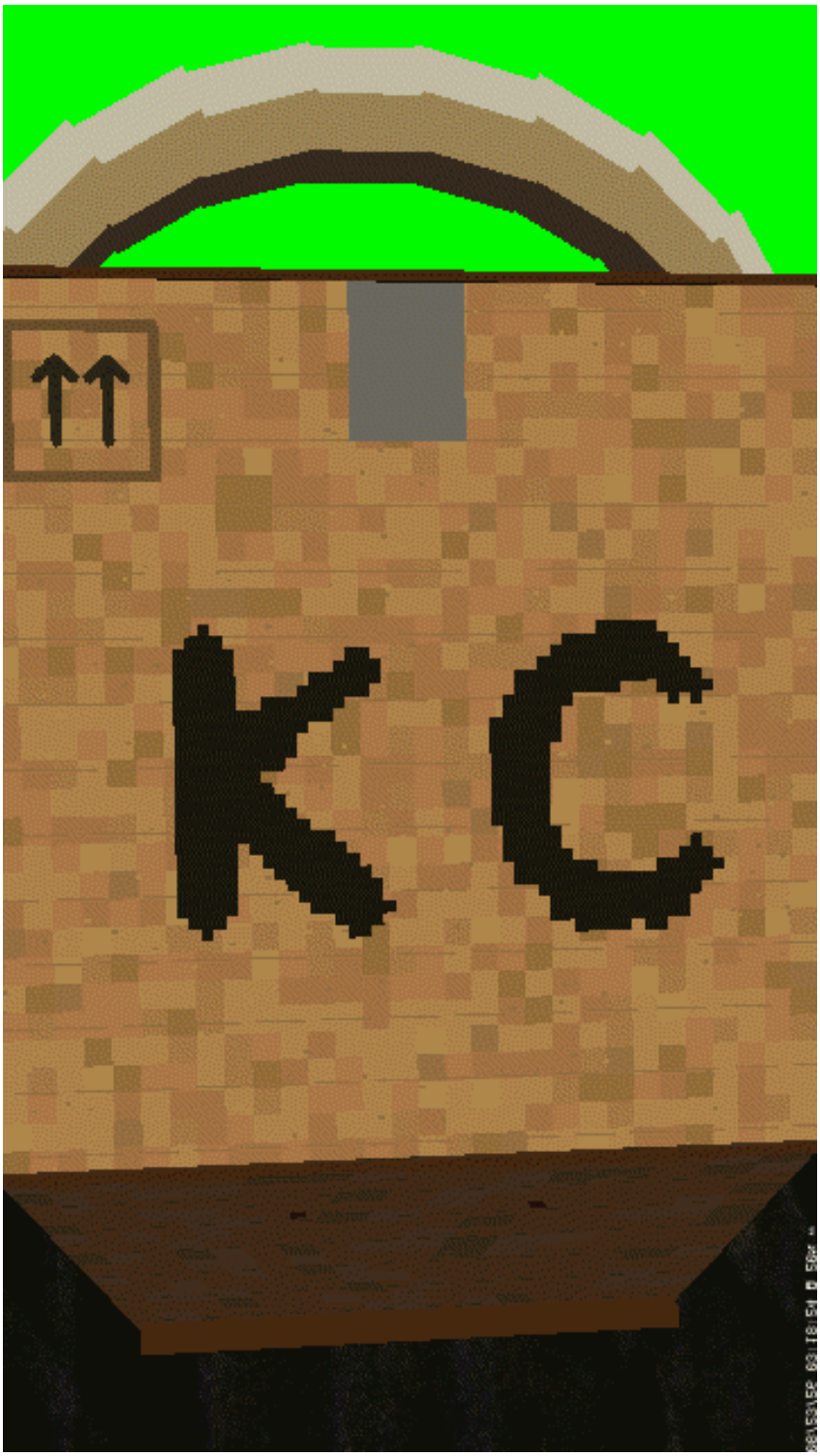
CHAPTER 13

Green-screen compositing

PART III · ARCHITECTURE Chapter 13

Green-screen compositing

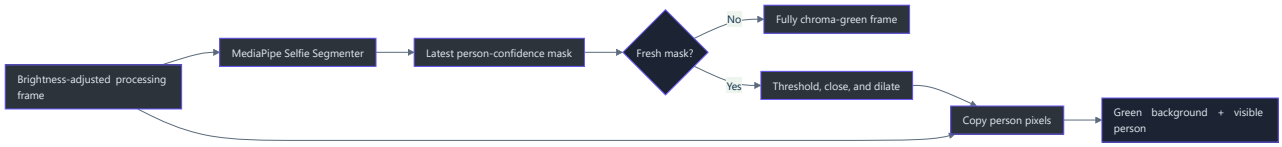
> **Status: implemented and opt-in through `--green-screen`.**
>
> **TL;DR** — The camera frame is segmented asynchronously. Detected person pixels remain visible, while every non-person pixel becomes pure OpenCV BGR green `(0, 255, 0)`. Missing or stale masks produce a fully green frame rather than exposing the room (`src/kardboard_vtuber/tracking/green_screen.py:16-152`).



User-approved camera example processed with the runtime compositing algorithm. The body remains visible; the cardboard shell and opaque backing cover the face and hairline.

Overview

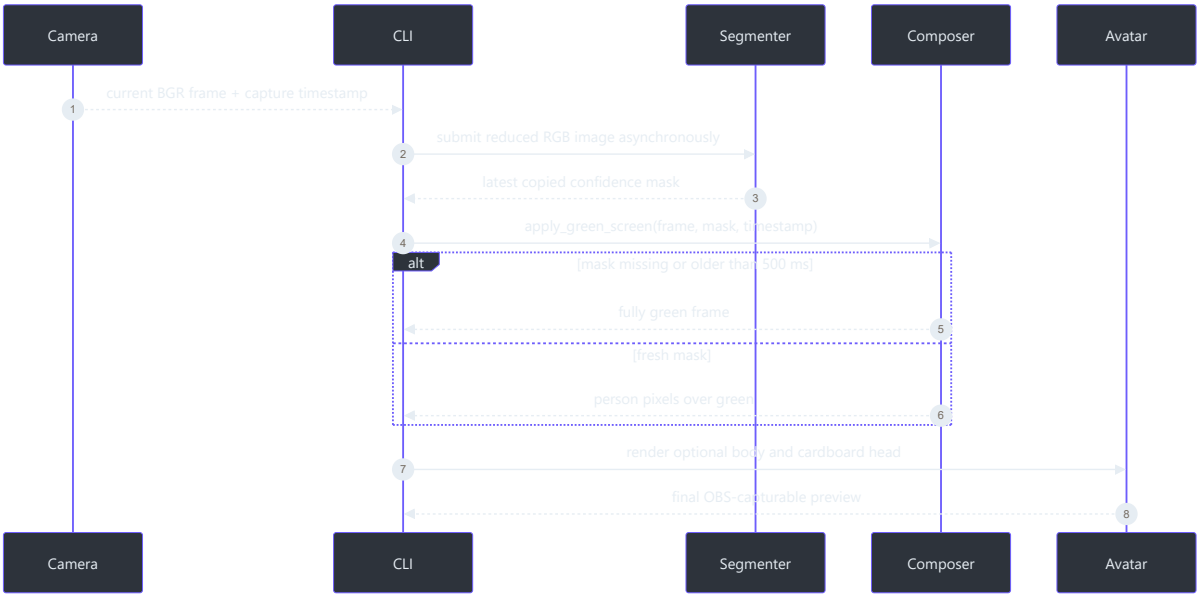
The mode exists for OBS chroma keying without requiring a physical green backdrop. It is separate from face tracking: green-screen mode can run by itself, with the cardboard head, or with flap physics. The CLI submits a brightness-adjusted processing frame to segmentation, then applies the resulting mask to an original-brightness preview copy (`src/kardboard_vtuber/cli.py`).



Components

Component	Responsibility	Evidence
`GreenScreenConfig`	Model path, input width, person threshold, stale-mask limit	`src/kardboard_vtuber/tracking/green_screen.py:16-32`
`PersonSegmentationState`	Immutable timestamp plus optional 2D confidence mask	`src/kardboard_vtuber/tracking/green_screen.py:35-42`
`MediaPipePersonSegmenter`	Reduced-resolution asynchronous LIVE_STREAM inference	`src/kardboard_vtuber/tracking/green_screen.py:45-121`
`apply_green_screen()`	Fail-closed chroma composition and mask cleanup	`src/kardboard_vtuber/tracking/green_screen.py:124-152`
CLI wiring	Submission before composition and deterministic cleanup	`src/kardboard_vtuber/cli.py:215-414`
Downloader	Official model URL plus pinned SHA-256 verification	`scripts/download_selfie_segmenter_model.py:1-52`

Runtime data flow



Mask processing

The official Selfie Segmenter returns one floating-point person-confidence mask. The callback copies and squeezes it to a stable 2D NumPy array because MediaPipe owns the callback image memory (`src/kardboard_vtuber/tracking/green_screen.py:105-121`). Composition then:

1. copies the source frame;
2. fills output with exact chroma green;
3. rejects missing, future-dated, or stale masks;
4. resizes confidence to the source frame;
5. thresholds at `0.35` ;
6. closes small holes and dilates once with a `5 × 5` elliptical kernel;
7. copies only accepted person pixels over green
   (`src/kardboard_vtuber/tracking/green_screen.py:124-152`).

The small dilation favors keeping hair, shoulders, and clothing over aggressively trimming the person silhouette. This is a deliberate visual tradeoff, not metric depth reconstruction.

Privacy behavior

Green-screen mode never uses an uninitialized mask as permission to show the background. Before the first callback and after a mask becomes older than `500 ms` , output is entirely green (`tests/test_green_screen.py:27-41`). This fail-closed rule is independent from the cardboard renderer, whose own no-face behavior remains black-before-acquisition and last-safe-frame freezing (`src/kardboard_vtuber/renderer/textured_3d.py:180-279`).

Setup and operation

```
python scripts\download_selfie_segmenter_model.py

python -m kardboard_vtuber `
  --source "YOUR_CAMERA_URL" `
  --rotate left `
  --mirror `
  --physics `
  --green-screen
```

The downloader verifies SHA-256 `191ac9529ae506ee0beefa6b2c945a172dab9d07d1e802a290a4e4038226658b` before installing `models/selfie_segmenter.tflite` (`scripts/download_selfie_segmenter_model.py:10-52`). MediaPipe remains an optional dependency restricted to Python versions supported by the project tracking extra (`pyproject.toml:19-23`).

Validation

Tests prove person preservation, exact green background replacement, stale-mask rejection, and configuration validation (`tests/test_green_screen.py:1-47`). The documented animated example uses the user-approved black-hoodie recording, retains the segmented body over pure green, and covers the face and hairline with the avatar plus an opaque privacy backing (`scripts/generate_real_life_animation.py`).

References

- `src/kardboard_vtuber/tracking/green_screen.py:1-152`
- `src/kardboard_vtuber/cli.py:32-652`
- `src/kardboard_vtuber/tracking/__init__.py:1-48`
- `scripts/download_selfie_segmenter_model.py:1-52`
- `tests/test_green_screen.py:1-47`
- `pyproject.toml:5-28`

⬅️ Textured renderer · 🏠 Architecture

14

PART III · ARCHITECTURE

CHAPTER 14

PS1 cardboard renderer

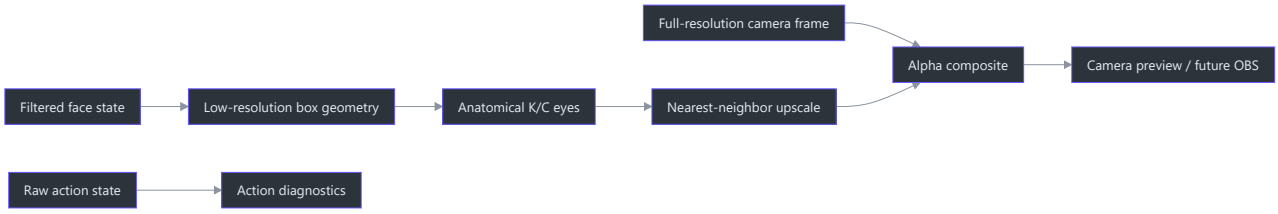
PART III · ARCHITECTURE Chapter 14

PS1 cardboard renderer

> **Status: implemented and retained as the `procedural-2d` fallback.**
>
> **TL;DR** — This legacy fallback creates one fixed opaque hollow cardboard shell around the head at low resolution, leaves the neck visible through a central bottom opening, drives anatomical K/C eyes from tracking, adds pose-dependent planes and spring motion, then composites only the avatar over the sharp camera frame (`src/kardboard_vtuber/renderer/ps1_cardboard.py:17-370`). Its V-shaped bottom opening is specific to this fallback and is not the geometry of the default textured renderer.

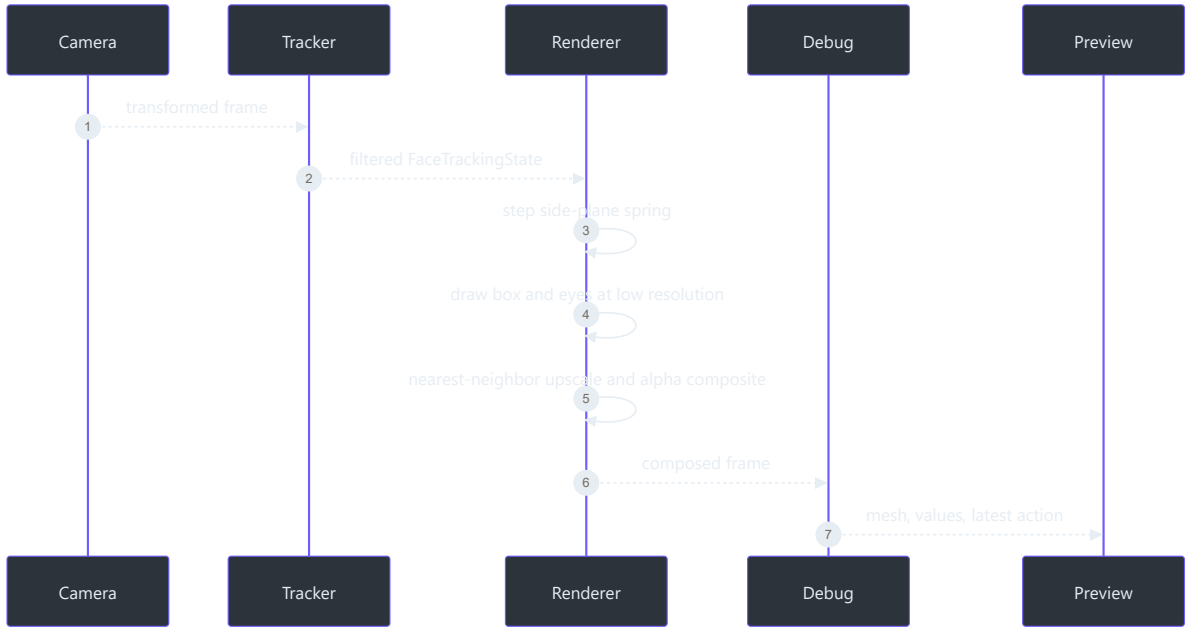
Why the renderer is deliberately fixed

The product needs one recognizable KardboardCode head, not a general avatar importer. A procedural renderer therefore encodes the current character directly and avoids premature model formats, scene graphs, asset pipelines, and user-facing rig editors (`src/kardboard_vtuber/renderer/ps1_cardboard.py:35-272` , `assets/PNGTuberV1/README.md`).



Runtime sequence

The CLI automatically creates tracking when `--render-cardboard` is supplied. Rendering occurs before the debug overlay so bounds, mesh, and action text remain visible over the prototype (`src/kardboard_vtuber/cli.py:82-167` , `src/kardboard_vtuber/cli.py:253-256`).



Visual construction

Part	Implementation
Front panel	Skewed quadrilateral anchored to face center and bounds
Top plane	Lighter polygon that grows while looking down
Underside	Two dark lower polygons around the neck opening that grow while looking up
Side plane	Darker polygon shown opposite the screen direction of the face turn
Bottom opening	Central V-shaped cutout exposing the real neck; fallback-specific
Interior rim	Dark lower-corner and opening surfaces that communicate hollow depth
K/C eyes	Text when open, upward happy-eye arcs when closed or winking
Surface	Deterministic light/dark fiber pattern over fully opaque cardboard
Pixel style	Overlay rendered at one-quarter linear resolution
Composition	Upscaled alpha mask blends avatar without pixelating camera

MediaPipe landmarks bound the face, not the full hairstyle. The shell therefore uses additional clearance in all three dimensions: `2.25x` tracked face width, `2.05x` tracked face height with a

`12%` upward center bias, and a `1.55x` depth multiplier. This larger envelope keeps hair, chin, and side-profile pixels inside the opaque silhouette during combined pitch, yaw, and roll.

The visible logo order is always `K C` from screen-left to screen-right. In mirrored preview, each letter retains its anatomical meaning: `K` follows the user's left eye and `C` follows the user's right eye. Closure uses the calibrated wink rule (`≤ 0.70` with an open opposite eye and at least `0.15` asymmetry), plus the bilateral `≤ 0.35` closed threshold (`src/kardboard_vtuber/renderer/ps1_cardboard.py:284-339` , `src/kardboard_vtuber/tracking/models.py:103-137`).

Motion

Filtered bounds and pose control the box, while a damped spring adds intentional follow-through to the side plane (`src/kardboard_vtuber/motion/springs.py:26-85` , `src/kardboard_vtuber/renderer/ps1_cardboard.py:39-88`). Positive/rightward yaw reveals the screen-left side; negative/leftward yaw reveals the screen-right side. The guided calibration established a pitch baseline near `-10` degrees: more-negative pitch means looking up and reveals the underside, while more-positive pitch means looking down and reveals the top. This preserves the distinction between measurement smoothing and animation dynamics.

After all low-resolution planes, eyes, and openings are drawn, the complete color and alpha canvases rotate together around the tracked center using filtered roll. Positive roll produces the same counterclockwise screen tilt shown by the face mesh; roll is bounded to `+/-60` degrees.

Mouth-driven front flaps are intentionally deferred after visual review. This fallback renders no mouth flap geometry; mouth tracking and action diagnostics remain available for a later redesign.

MediaPipe can lose the face when a full left or right profile hides too many frontal landmarks. Privacy is fail-closed: before the first valid detection the output is black, and after tracking loss the renderer freezes the last safely composited frame instead of emitting a new raw camera frame. In this fallback only, the narrow V opening apex is constrained below the tracked lower-face bound plus a `16%` face-height chin/beard safety margin. The shell extends downward when necessary so the opening exposes only the neck at every pitch.

Validation

- The full 64-test suite passes under Python 3.12 and 3.13.
- Tests cover black output before initial acquisition, bounded overlay region, mirrored visible K/C placement, full lower-face and chin-margin opacity, and fail-closed tracking-loss freezing, crown/hair coverage, calibrated anatomical winks, roll, yaw-side perspective, pitch-driven top/underside visibility, enlarged XYZ dimensions, and an extreme combined-pose privacy silhouette. Default opaque composition uses a masked-copy fast path, while partial opacity retains tested alpha blending (`tests/test_ps1_cardboard_renderer.py:1-219`).
- The private guided recording produced a 1,254-frame prototype video.

- Contact-sheet inspection confirmed face coverage, pose following, K/C placement, and closed-eye arcs.

The private camera footage and rendered video are not committed.

The initial flat face-sized rectangle was rejected during user review. The corrected design is approximately twice the tracked face width, extends around the head, remains fully opaque, and reveals camera pixels only through the intentional neck opening (`tests/test_ps1_cardboard_renderer.py:88-117`).

Run it

```
python -m kardboard_vtuber `
  --source "******PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --render-cardboard
```

References

- `src/kardboard_vtuber/renderer/ps1_cardboard.py:1-272`
- `src/kardboard_vtuber/renderer/__init__.py:1-8`
- `src/kardboard_vtuber/tracking/models.py:55-137`
- `src/kardboard_vtuber/motion/springs.py:1-85`
- `src/kardboard_vtuber/cli.py:82-256`
- `tests/test_ps1_cardboard_renderer.py:1-86`

⬅️ System architecture · 🏠 Architecture

PART IV

Algorithms and Data Structures

The mechanisms that keep the avatar fresh, stable, responsive, testable, and visually coherent under concurrent camera and inference workloads.

15

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 15

Algorithms and data structures

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 15

03 · Algorithms and data structures



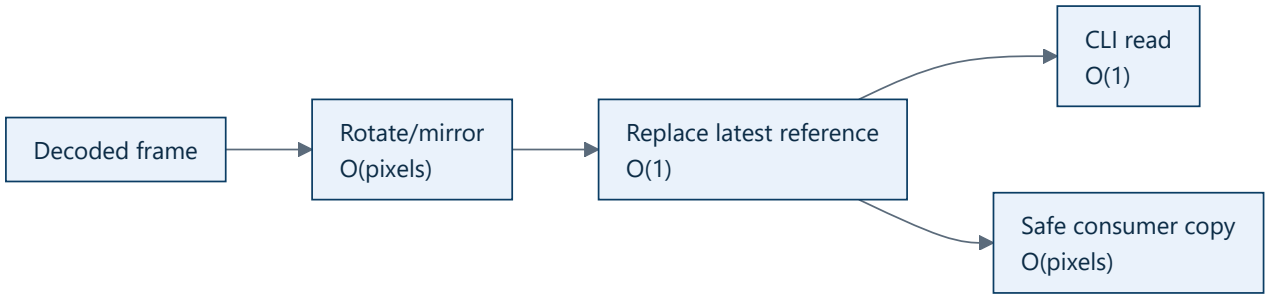
TL;DR — The current subsystem uses four important ideas: a single latest-value slot, a condition variable, a finite-state machine, and monotonic sliding-window timing. Each has its own chapter because each solves a different real-time systems problem.

Current algorithm catalogue

Chapter	Data structure / algorithm	Problem solved
Latest-frame slot	One mutable reference + sequence	Prevent latency-producing queues
Condition-variable synchronization	`threading.Condition`	Coordinate producer, consumers, lifecycle, and timeouts
Finite-state lifecycle	Enum state machine + retry threshold	Make recovery behavior explicit
Monotonic timing and FPS	Monotonic timestamp + time window	Measure age and throughput safely
Asynchronous live inference	Non-blocking submission + latest result	Keep inference out of the preview critical path
Blendshape normalization	Lookup, inversion, and clamping	Produce stable eye and mouth controls
Transformation matrix decomposition	4x4 validation + 3x3 RQ decomposition	Expose renderer-friendly pose diagnostics
Facial action state machine	Hysteresis + debounced finite-state channels	Convert continuous controls into blink, wink, and mouth events
One Euro motion filtering	Adaptive low-pass filter bank	Reduce jitter while retaining fast motion
Damped spring integration	Bounded-step harmonic oscillator	Produce controlled head/flap secondary motion
Low-resolution overlay compositing	Separate color/alpha buffers + nearest-neighbor upscale	Pixelate only the avatar
Bounded studio mailbox	One owned display slot + bounded log deque + atomic tuning	Keep a slow GUI from queuing stale video or partial calibration

Complexity summary

Operation	Time	Additional space
Publish frame	O(1) reference replacement	O(1) frames
Read latest without copy	O(1)	O(1)
Read latest with copy	O(width × height)	One frame copy
Snapshot diagnostics	O(1)	O(1)
Update FPS window	O(1)	O(1)



Future algorithm chapters

When implemented, the book will add dedicated chapters for:

- Quaternion smoothing.
- Ordered dithering and optional vertex snapping.

These are not yet implemented and should not be presented as current behavior.

⬅️ Architecture · ➡️ Design principles

16

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 16

The single latest-frame slot

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 16

The single latest-frame slot

> **TL;DR** — The application stores one current frame, not a queue. New frames overwrite unread frames so slow consumers see fresh video instead of increasingly old video.

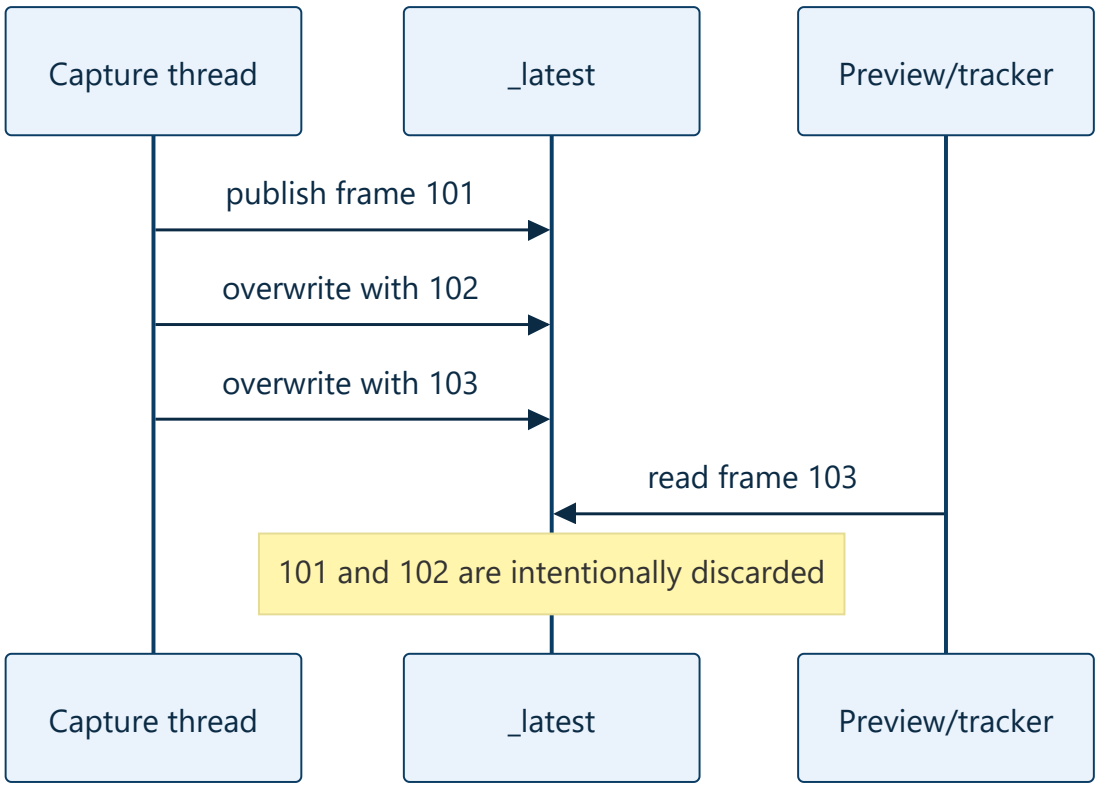
The queue failure mode

Suppose capture produces 30 FPS and tracking consumes 20 FPS:



The system may process every frame correctly while producing a visibly incorrect live experience.

The chosen structure



The slot is `_latest: FramePacket | None` at `src/kardboard_vtuber/camera/stream.py:59`. Publication occurs at `stream.py:198–204`.

Sequence-number protocol

The consumer remembers its last sequence and asks for something newer:

```
packet = camera.read(after_sequence=last_sequence, timeout=2.0)
```

The comparison is implemented at `stream.py:126-129`. This avoids processing the same current frame repeatedly while still allowing the producer to skip arbitrary sequence values from the consumer's perspective.

Overwrite metric

Whenever a previous packet exists, publication increments `overwritten_frames`. A rising value is not automatically a defect. It proves the system is discarding stale intermediate states instead of buffering them.

Complexity

- Publish: O(1), excluding rotation/mirroring.
- Stored video memory: one frame.
- Consumer copy: O(pixels) when `copy=True`.
- No queue growth and no queue-drain phase.

Tradeoff

This structure is wrong for recording, auditing, or offline batch processing where every frame matters. It is right for interactive pose tracking where old observations have almost no value.

Test evidence

`test_latest_frame_camera_overwrites_unread_frames()` deliberately lets the producer outrun the consumer and asserts that overwrites occur (`tests/test_camera_stream.py:67-80`).

⬅️ Algorithm catalogue · ➡️ Condition-variable synchronization

17

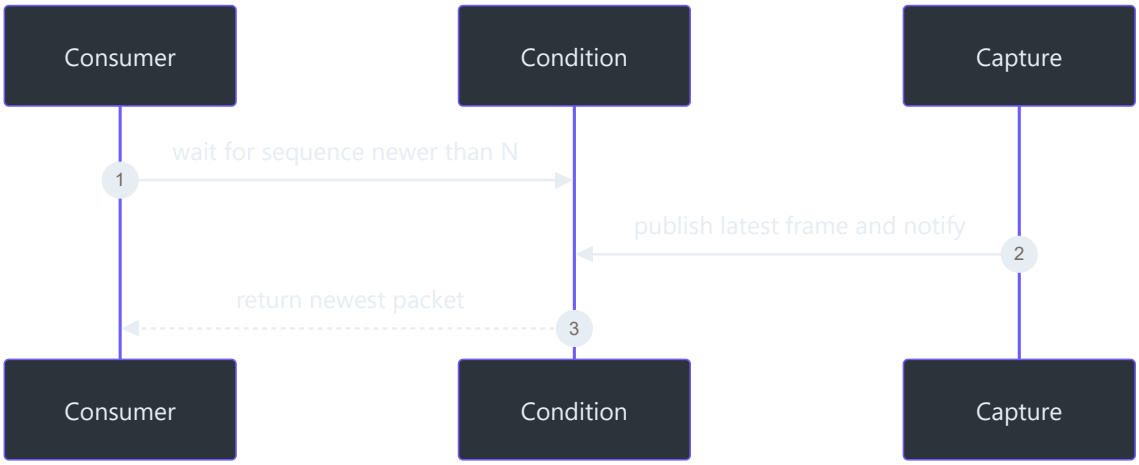
PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 17

Condition-variable synchronization

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 17

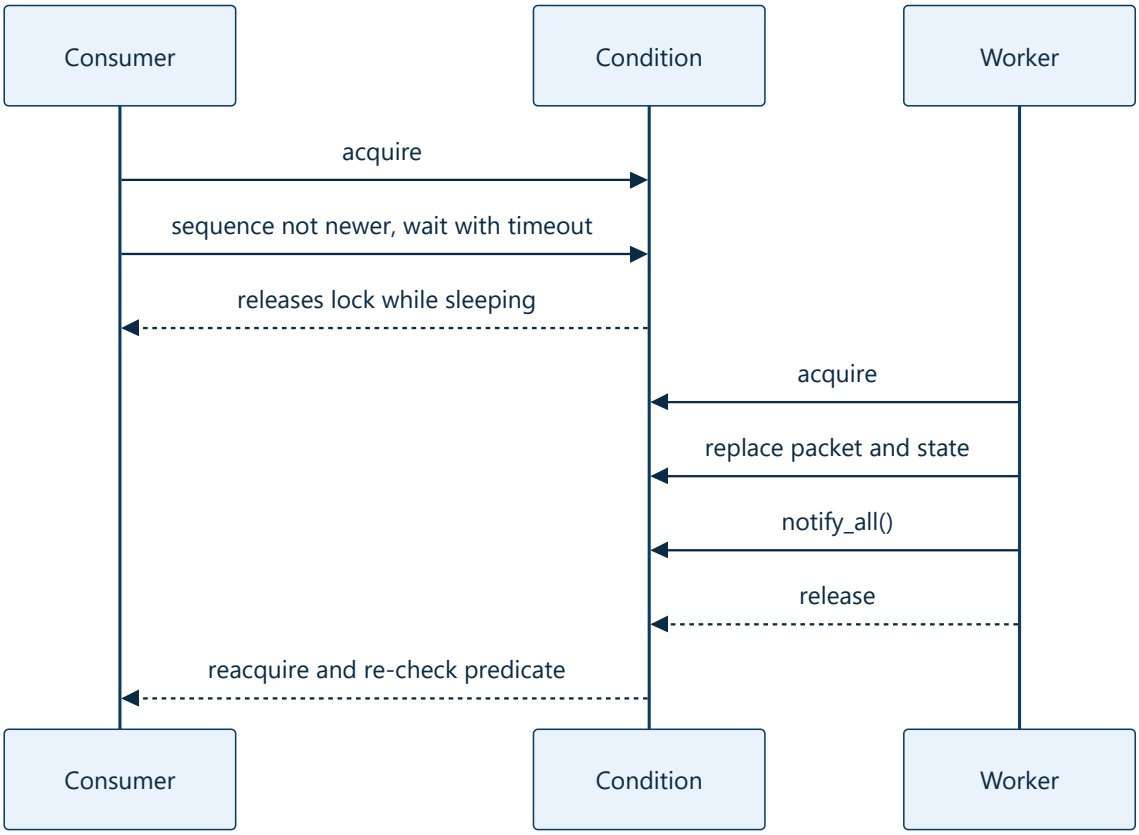
Condition-variable synchronization



TL;DR — One `threading.Condition` protects shared capture state and lets consumers sleep until a new frame or lifecycle change occurs. This avoids both unsafe races and wasteful polling.

Why a condition variable?

A mutex alone protects data but cannot efficiently express “wait until a newer sequence exists.” An event can signal something happened but does not naturally protect the packet and counters. A condition combines both.



The condition is created at `src/kardboard_vtuber/camera/stream.py:55`.

The predicate loop

Consumers use a loop because wakeups are notifications, not guarantees:

1. Inspect `_latest`.
2. Check whether it is newer than `after_sequence`.
3. Return it if valid.
4. Return `None` if the lifecycle is terminal.
5. Otherwise wait for notification or timeout.

Implementation: `stream.py:124–141`.

Shared state protected by the condition

- Latest packet.
- Lifecycle state.
- Sequence and diagnostic counters.
- Negotiated camera properties.
- Last error.

Why `notify_all()`?

The current CLI has one consumer, but lifecycle waiters and future tracker/composer consumers may coexist. Every waiter must re-check its own predicate after a publication or state change.

Lock granularity

The worker does **not** hold the condition while `capture.read()` blocks or while OpenCV transforms the frame. It acquires the lock only to publish compact state. That prevents slow I/O from blocking diagnostic reads and shutdown coordination.

Risk: frame ownership

`read(copy=True)` copies the NumPy array under the lock. The CLI uses `copy=False` because it consumes the frame immediately (`src/kardboard_vtuber/cli.py:74`). Future asynchronous consumers must either request a copy or guarantee ownership.

⬅️ Latest-frame slot · ➡️ Finite-state lifecycle

18

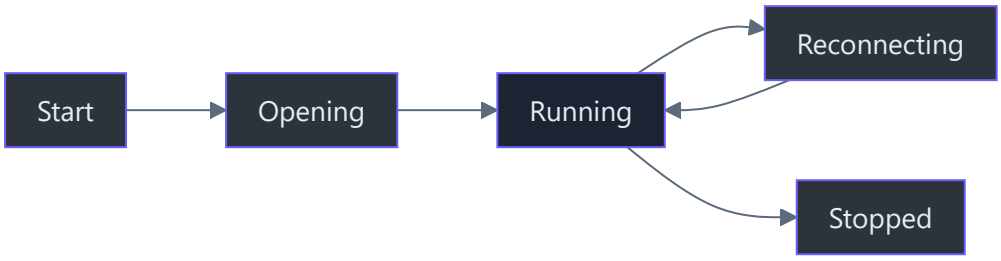
PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 18

Finite-state lifecycle and reconnect algorithm

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 18

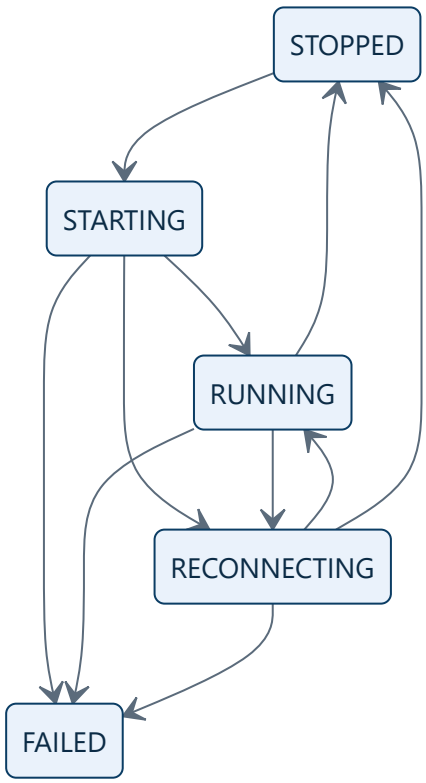
Finite-state lifecycle and reconnect algorithm



> **TL;DR** — The worker combines an explicit state machine with a consecutive-failure threshold. Short read glitches are retried cheaply; sustained failures release and reopen the source.

States are data

`CaptureState` has five values (`src/kardboard_vtuber/camera/models.py:51-58`). This is more expressive than `is_open: bool`, which cannot distinguish startup, retry, terminal failure, and clean shutdown.



Consecutive-failure threshold

```
on successful read:
    consecutive_failures = 0

on failed read:
    consecutive_failures += 1
    total_read_failures += 1
    if consecutive_failures ≥ threshold:
        release source
        state = RECONNECTING
        reconnects += 1
```

Implementation: `src/kardboard_vtuber/camera/stream.py:179-191`.

The distinction between consecutive and total failures matters:

- Consecutive failures drive recovery.
- Total failures explain long-run source quality.

Reopen loop

When no open capture exists, `_open()` creates a new adapter, reapplies requested properties, and reads negotiated properties (`stream.py:223-259`). Failure increments reconnects and waits the configured delay before another attempt.

Failure policy

Failure	Policy
One bad frame	Retry after 5 ms
Repeated bad frames	Release and reconnect
Source cannot open	Remain reconnecting
Unexpected Python/OpenCV exception	Enter `FAILED` and expose typed message
User shutdown	Stop retrying immediately

Why no exponential backoff yet?

The current sources are local cameras or a nearby phone, not a remote internet service. A fixed one-second retry is simple and responsive. Exponential backoff becomes useful if repeated network failures create noise or battery load.

⬅️ Condition synchronization · ➡️ Monotonic timing

19

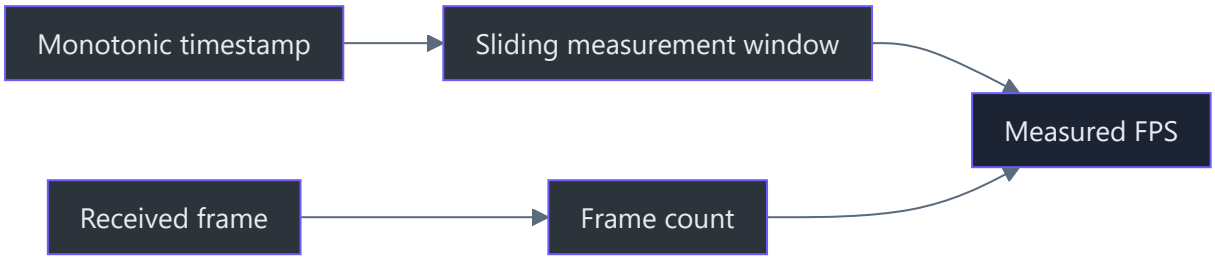
PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 19

Monotonic timing and FPS estimation

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 19

Monotonic timing and FPS estimation



> **TL;DR** — The subsystem uses monotonic time because durations must not jump when the wall clock changes. It measures received FPS over repeated windows of at least one second.

Two clocks, two jobs

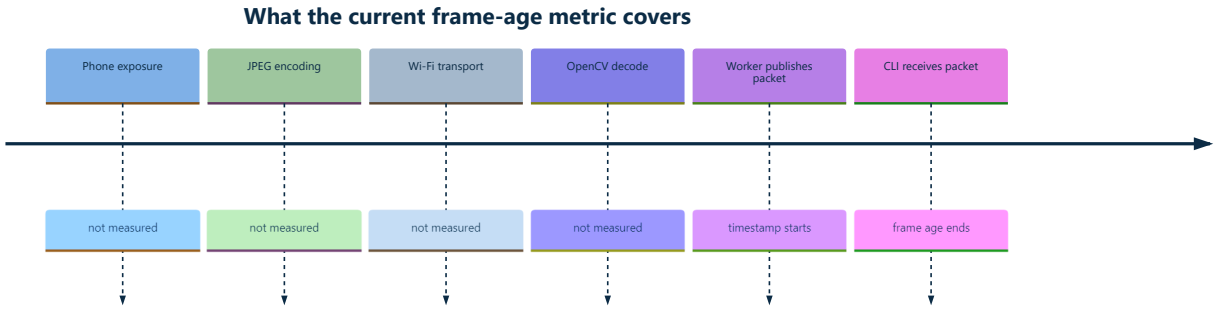
Clock	Suitable for	Unsuitable for
Wall clock	Human timestamps and logs	Frame age and timeout math
Monotonic clock	Durations, deadlines, FPS	Calendar timestamps

`FramePacket.captured_at_ns` is assigned with `time.monotonic_ns()` at `src/kardboard_vtuber/camera/stream.py:197`.

Frame-age calculation

```
frame_age_ms = (monotonic_now_ns - captured_at_ns) / 1,000,000
```

The preview computes this at `src/kardboard_vtuber/cli.py:88`.



Therefore a displayed 0.4 ms does not mean the phone-to-screen path is 0.4 ms.

FPS window

The worker counts published frames. Once at least one second has elapsed:

```
measured_fps = window_frame_count / elapsed_seconds
window_start = now
window_frame_count = 0
```

Implementation: `stream.py:281-288`.

Why not trust `CAP_PROP_FPS`?

That property is a negotiated or reported source value. It may be zero, stale, rounded, or simply different from delivered throughput. The project records both negotiated and measured FPS.

Timeout algorithm

`read()` computes a monotonic deadline and repeatedly calculates remaining time (`stream.py:123-141`). This prevents spurious wakeups from extending the requested timeout.

⬅️ Finite-state lifecycle · ➡️ Design principles

20

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 20

Asynchronous latest-result face inference

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 20

Asynchronous latest-result face inference

> **TL;DR** — Camera capture and face inference run in separate timing domains. The CLI submits current frames without waiting for inference and consumes the newest completed tracking result.

Algorithm

```
for each newest camera packet:
    downscale packet.frame
    convert BGR to RGB
    submit with strictly increasing timestamp
    continue preview loop

on MediaPipe callback:
    normalize result
    replace latest tracking state
```

Submission is implemented at `src/kardboard_vtuber/tracking/mediapipe_tracker.py:102-119`; callback publication is at `src/kardboard_vtuber/tracking/mediapipe_tracker.py:143-175`.

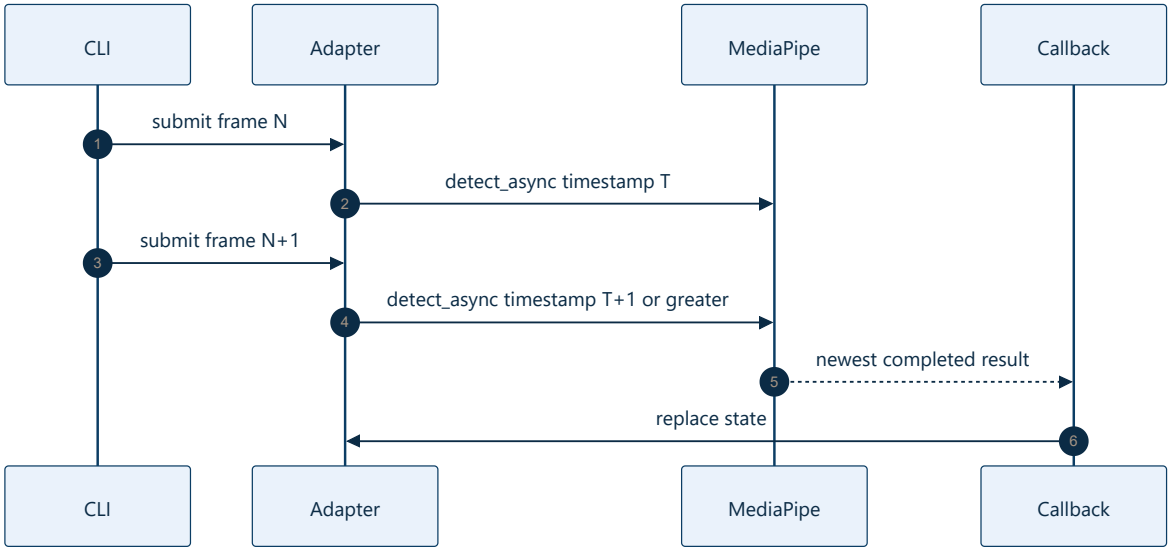


Timestamp monotonicity

MediaPipe live mode requires increasing millisecond timestamps. Camera timestamps are nanoseconds, so integer conversion can theoretically produce duplicates. The adapter enforces:

```
timestamp_ms = max(captured_ns // 1_000_000, last_timestamp_ms + 1)
```

This is protected by the tracker lock (`mediapipe_tracker.py:105-109`).



Complexity and memory

- Preprocessing: O(input pixels).
- Submission bookkeeping: O(1).
- Retained tracking memory: one state plus its landmarks.
- No application-level inference queue.

Why not `detect_for_video()`?

Synchronous video mode would block the preview loop on every inference. Live-stream mode lets MediaPipe manage inference asynchronously and may drop inputs when busy, matching the product's freshness-over-completeness principle.

References

- `src/kardboard_vtuber/camera/stream.py:114-141`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:44-175`
- `src/kardboard_vtuber/tracking/models.py:58-90`
- `src/kardboard_vtuber/cli.py:82-105`
- `pyproject.toml:16-23`

⬅️ Algorithm catalogue · ➡️ Blendshape normalization

21

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 21

Blendshape normalization

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 21

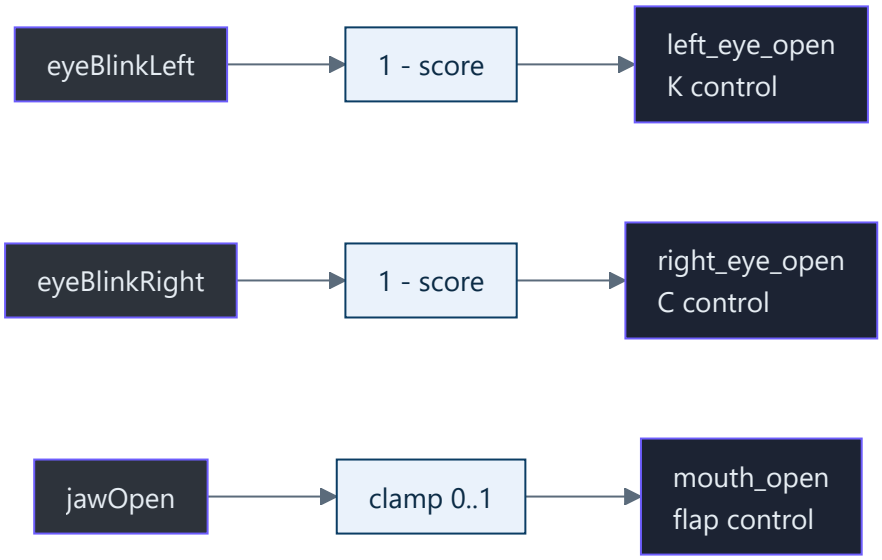
Blendshape normalization

> **TL;DR** — MediaPipe reports blink intensity, while the avatar needs eye openness. The adapter inverts blink scores and clamps every control to a finite `[0, 1]` range.

Mapping

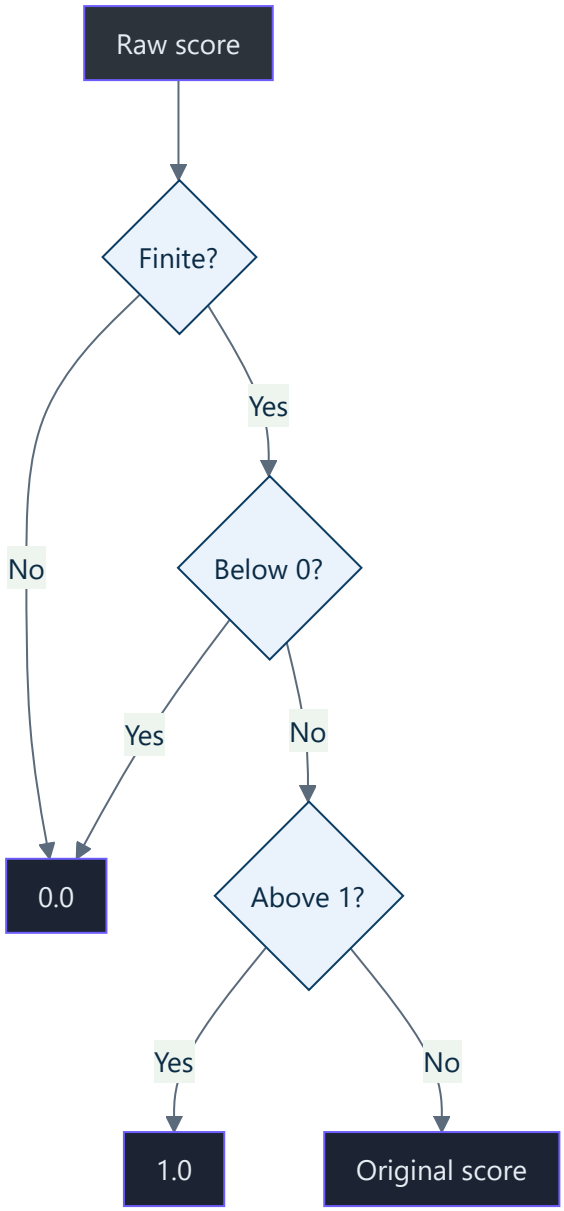
MediaPipe category	Project control	Formula
`eyeBlinkLeft`	`left_eye_open`	`1 - clamp(score)`
`eyeBlinkRight`	`right_eye_open`	`1 - clamp(score)`
`jawOpen`	`mouth_open`	`clamp(score)`

Implementation: `src/kardboard_vtuber/tracking/models.py:122-134`.



Defensive normalization

`_clamp01()` converts non-finite values to zero and bounds valid numbers (`src/kardboard_vtuber/tracking/models.py:148-151`).



Why calibration still matters

Raw scores vary by face, glasses, lighting, camera angle, and model behavior. Future calibration will map personal neutral/closed/open ranges into renderer controls. The current normalization is a safe common scale, not final animation tuning.

Test evidence

- Correct eye inversion and jaw extraction: `tests/test_tracking_models.py:40-66`
- Invalid and out-of-range score handling: `tests/test_tracking_models.py:82-101`

References

- `src/kardboard_vtuber/tracking/models.py:93-151`

- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:143-166`
- `tests/test_tracking_models.py:1`
- `assets/PNGTuberV1/model-manifest.json:116-149`
- `docs/01-foundations/visual-identity-and-source-avatar.md`

⬅️ Async inference · ➡️ Transformation matrix

22

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 22

Facial transformation matrix decomposition

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 22

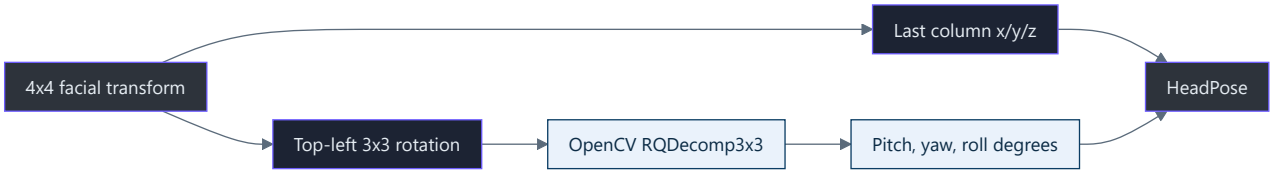
Facial transformation matrix decomposition

> **TL;DR** — MediaPipe emits a 4x4 transform. The project preserves translation and decomposes the rotation block into pitch, yaw, and roll for debugging. Rendering should later retain a matrix or quaternion to avoid Euler-angle artifacts.

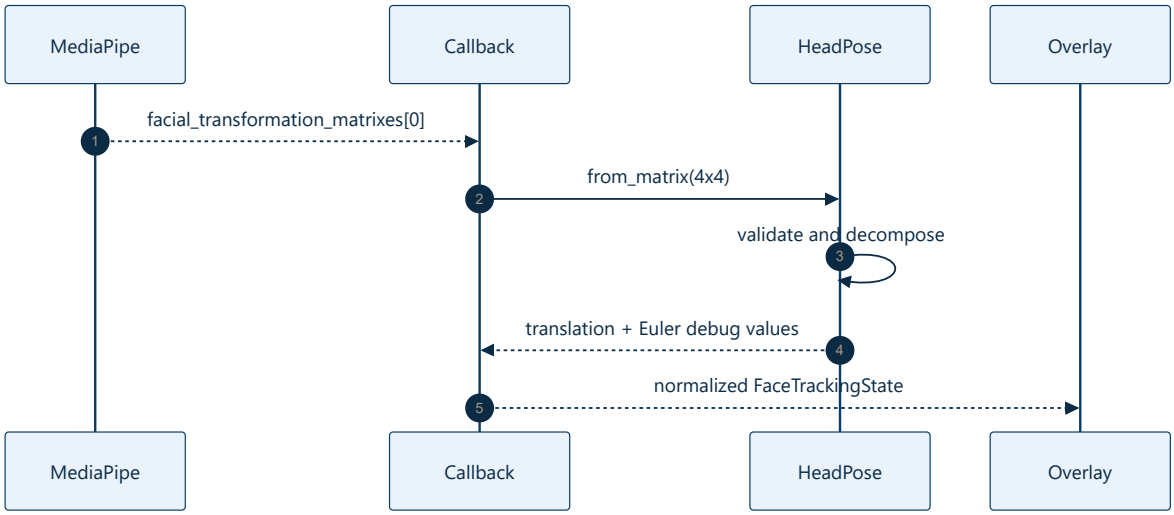
Matrix structure

```
[ r00 r01 r02 tx ]
[ r10 r11 r12 ty ]
[ r20 r21 r22 tz ]
[  0   0   0   1 ]
```

`HeadPose.from_matrix()` validates the exact 4x4 shape (`src/kardboard_vtuber/tracking/models.py:37-53`).



Data flow



Why Euler angles are not the final renderer representation

Euler angles are readable but suffer from axis-order ambiguity and gimbal-lock behavior. The PS1 renderer should smooth a quaternion or rotation matrix, then derive Euler values only for diagnostics.

Coordinate caution

The current values are reported in MediaPipe's transform convention. Rotation direction and translation scale require calibration against the final renderer. Mirroring happens before tracking (`src/kardboard_vtuber/camera/stream.py:192-197`), so signs must be validated visually.

Test evidence

- Identity transformation produces zero translation and rotation: `tests/test_tracking_models.py:24-33`
- Non-4x4 inputs fail explicitly: `tests/test_tracking_models.py:36-38`

References

- `src/kardboard_vtuber/tracking/models.py:25-53`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:143-166`
- `src/kardboard_vtuber/camera/stream.py:192-197`
- `src/kardboard_vtuber/cli.py:207-231`
- `tests/test_tracking_models.py:24-38`

⬅️ Blendshape normalization · 🏠 Algorithm catalogue

23

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 23

Facial action state machine

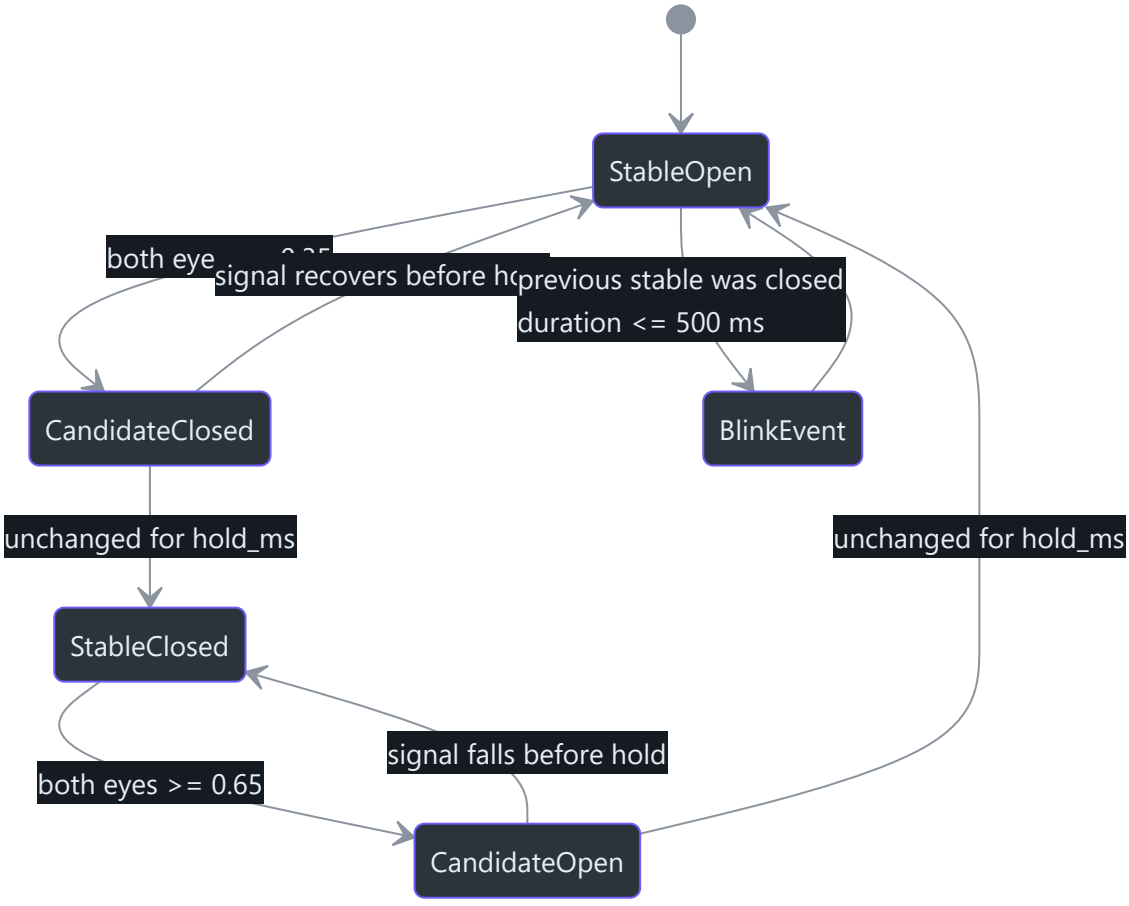
PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 23

Facial action state machine

> **Status: implemented and live-validated.**
>
> **TL;DR** — MediaPipe produces continuous values every frame, but logs and future avatar actions need discrete transitions. `FaceActionDetector` applies hysteresis, a stability hold, and timestamp ordering to emit low-noise events without logging every frame (`src/kardboard_vtuber/tracking/events.py:73-171`).

Why a state machine exists

An eye value moving from `0.72` to `0.69` is useful tracking data, but it is not a new user action. The detector therefore separates the latest measurement, a candidate state, and the last stable state. A candidate must remain unchanged for `hold_ms` before it becomes stable (`src/kardboard_vtuber/tracking/events.py:66-70`, `src/kardboard_vtuber/tracking/events.py:150-168`).

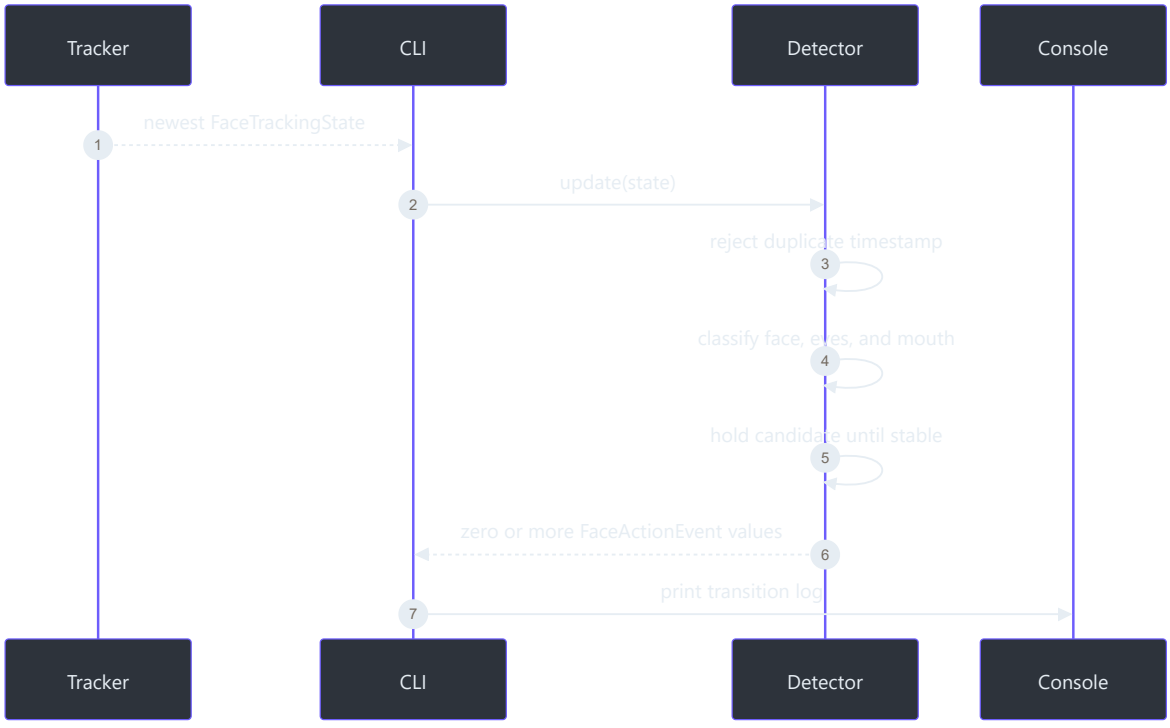


Event catalogue

Event	Classification rule	Purpose
`face_detected` / `face_lost`	Debounced `detected` transition	Tracking lifecycle
`eyes_open`	Both values at or above `0.65`	Neutral/open-eye state
`eyes_closed`	Both values at or below `0.35`	Closed-eye state
`left_wink`	Left at or below `0.65`, right open, difference at least `0.15`	Independent `K` eye control
`right_wink`	Right at or below `0.65`, left open, difference at least `0.15`	Independent `C` eye control
`blink`	Closed-to-open within 500 ms	Short bilateral closure
`mouth_open` / `mouth_closed`	Above `0.25` / below `0.12`	Cardboard flap control

The gap between each open and closed threshold is hysteresis: intermediate values preserve the current stable state rather than causing rapid toggling. Winks additionally use left/right asymmetry because spectacle reflections can keep a visibly closed eye well above the bilateral `eyes_closed` threshold (`src/kardboard_vtuber/tracking/events.py:24-47`, `src/kardboard_vtuber/tracking/events.py:134-166`).

Runtime data flow



The CLI updates the detector from the newest tracking snapshot and prints only returned events (`src/kardboard_vtuber/cli.py:94-128`). Each log contains the tracking timestamp and the three normalized expression values, with blink duration added when applicable (`src/kardboard_vtuber/tracking/events.py:44-63`).

Brief tracking loss

A single missed inference result must not erase stable expression state. Eye and mouth channels are reset only after `face_lost` itself survives the debounce hold. If detection returns before that point, no duplicate `eyes_open` or `mouth_closed` event is emitted (`src/kardboard_vtuber/tracking/events.py:94-105`, `tests/test_tracking_events.py:95-102`).

Spectacles

The live phone probe detected the user's face while spectacles were worn and produced changing eye values plus a complete blink event. A later screenshot showed a real wink at `0.79` versus `0.60`, which the original absolute `0.35` closure rule missed. The relative rule was then live-validated with repeated `left_wink` transitions at closed-eye values between approximately `0.43` and `0.60` and a `right_wink` transition at `0.75` versus `0.43`. Mouth hysteresis was also validated with repeated openings around `0.65-0.73` and closures around `0.02-0.04`. This proves basic operation for that setup, not universal glasses compatibility. Strong reflections, tinted lenses, frame occlusion, and off-axis head pose can still require calibration (`src/kardboard_vtuber/tracking/models.py:93-151`, `tests/test_tracking_events.py:48-72`).

Complexity

Each update performs a constant number of comparisons and state assignments: **O(1) time** and **O(1) additional space**. Duplicate or stale timestamps are ignored so asynchronous snapshots cannot replay an action (`src/kardboard_vtuber/tracking/events.py:84-88`).

References

- `src/kardboard_vtuber/tracking/events.py:11-171`
- `src/kardboard_vtuber/tracking/models.py:58-151`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:96-188`
- `src/kardboard_vtuber/cli.py:61-128`
- `tests/test_tracking_events.py:1-102`
- `tests/test_tracking_models.py:1-131`

⬅️ Blendshape normalization · 🏠 Algorithms and data structures

24

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 24

One Euro motion filtering

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 24

One Euro motion filtering

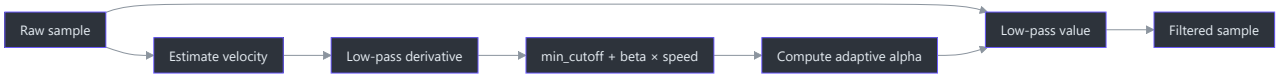
> **Status: implemented and offline/live validated.**
>
> **TL;DR** — The One Euro filter lowers cutoff during slow motion to suppress jitter and raises it when velocity increases to reduce lag. Raw state remains available for blink/wink classification; filtered state drives visual diagnostics and future rendering (`src/kardboard_vtuber/tracking/filters.py:16-175`, `src/kardboard_vtuber/tracking/mediapipe_tracker.py:88-190`).

Why fixed smoothing is insufficient

A strong fixed low-pass filter makes a stationary face look stable but delays intentional motion. A weak filter follows turns quickly but leaves visible landmark shimmer. One Euro filtering adapts its cutoff from the filtered derivative:

```
derivative = (value - previous_raw) / delta_time
cutoff = minimum_cutoff + beta * abs(filtered_derivative)
alpha = 1 / (1 + time_constant / delta_time)
filtered = alpha * value + (1 - alpha) * previous_filtered
```

`OneEuroFilter` enforces finite inputs and strictly increasing timestamps so timing defects fail explicitly (`src/kardboard_vtuber/tracking/filters.py:32-75`).



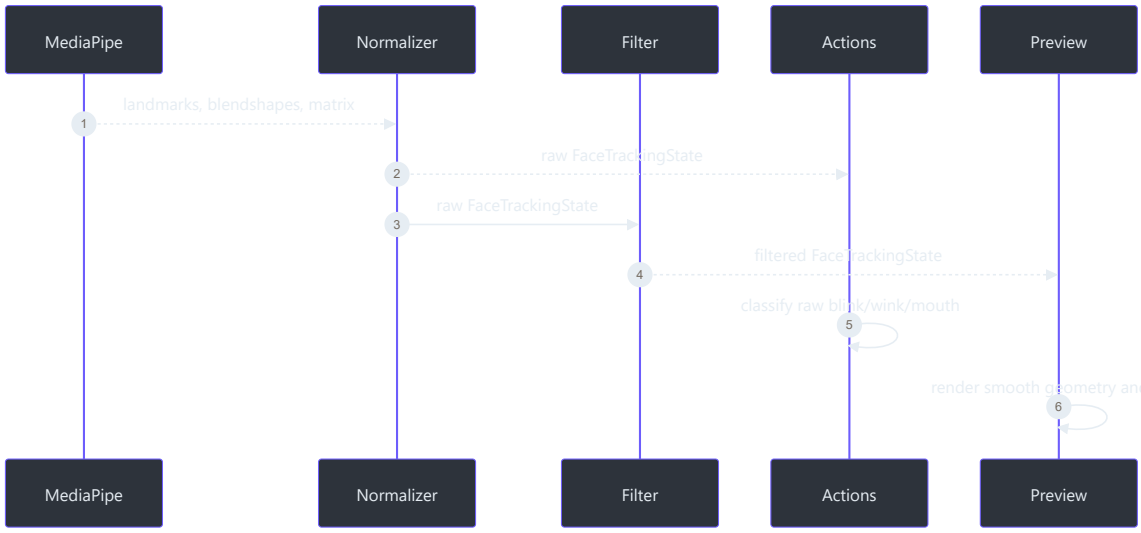
Face-level filter bank

`FaceMotionFilter` owns independent filters for all 478 landmark axes, face center/size, three expression controls, and six pose values. Different parameter groups avoid applying pose-scale tuning blindly to normalized eye values (`src/kardboard_vtuber/tracking/filters.py:78-168`).

Signal group	Default tuning	Intent
Landmarks	cutoff `1.2`, beta `0.02`	Stable mesh geometry
Bounds	cutoff `1.0`, beta `0.03`	Stable box placement and scale
Expressions	cutoff `3.0`, beta `0.2`	Preserve quick blinks and mouth motion
Pose	cutoff `1.0`, beta `0.02`	Reduce rotational shimmer

Filters reset when the face is lost or landmark count changes. Reacquisition starts directly from the new observation rather than blending from stale coordinates

(`src/kardboard_vtuber/tracking/filters.py:112-168`, `tests/test_tracking_filters.py:69-81`).



Raw and filtered state split

`TrackingSnapshot` exposes both `raw_state` and filtered `state` (`src/kardboard_vtuber/tracking/models.py:88-100`). The CLI deliberately feeds `raw_state` to `FaceActionDetector`, preventing smoothing from extending the effective eye debounce or swallowing normal blinks (`src/kardboard_vtuber/cli.py:130-144`, `src/kardboard_vtuber/tracking/events.py:84-181`).

Use `--no-motion-filter` to compare raw visual behavior without changing action semantics (`src/kardboard_vtuber/cli.py:76-81`, `src/kardboard_vtuber/cli.py:219-230`).

Measured evidence

A ten-second live neutral probe collected 299 detected samples:

Metric	Raw	Filtered
Center-X standard deviation	`0.002961`	`0.002880`
Yaw frame-step standard deviation	`0.110°`	`0.054°`

The recorded 45-second calibration clip was then replayed through the full pipeline: all 1,254 frames detected a face and emitted bilateral blink, both wink directions, and mouth transitions. The recording is a private session artifact and is intentionally not committed.

Complexity

Each scalar update is O(1). Filtering all landmarks is O(L), where L is 478, with O(L) persistent filter state. This remains bounded because the application tracks exactly one face.

References

- `src/kardboard_vtuber/tracking/filters.py:1-175`
- `src/kardboard_vtuber/tracking/models.py:55-137`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:43-199`
- `src/kardboard_vtuber/tracking/events.py:73-181`
- `src/kardboard_vtuber/cli.py:45-230`
- `tests/test_tracking_filters.py:1-92`

⬅️ Facial action state machine · 🏠 Algorithms and data structures

25

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 25

Damped spring integration

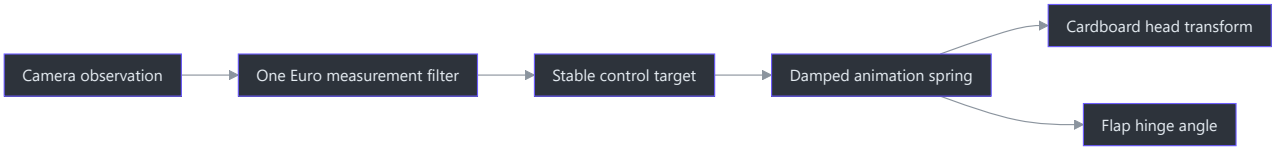
PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 25

Damped spring integration

> **Status: dynamics implemented, tested, and wired into all five textured flap hinges.**
>
> **TL;DR** — A damped harmonic oscillator gives the cardboard head and flaps controlled lag, settling, and optional overshoot. `DampedSpring` uses semi-implicit Euler integration with bounded internal steps so behavior remains similar across rendering frame rates (`src/kardboard_vtuber/motion/springs.py:10-85`).

Why a spring is separate from filtering

One Euro filtering estimates a cleaner measurement. A spring creates intentional animation. Combining these responsibilities would make tracking latency indistinguishable from artistic secondary motion (`src/kardboard_vtuber/tracking/filters.py:32-75` , `src/kardboard_vtuber/motion/springs.py:26-85`).



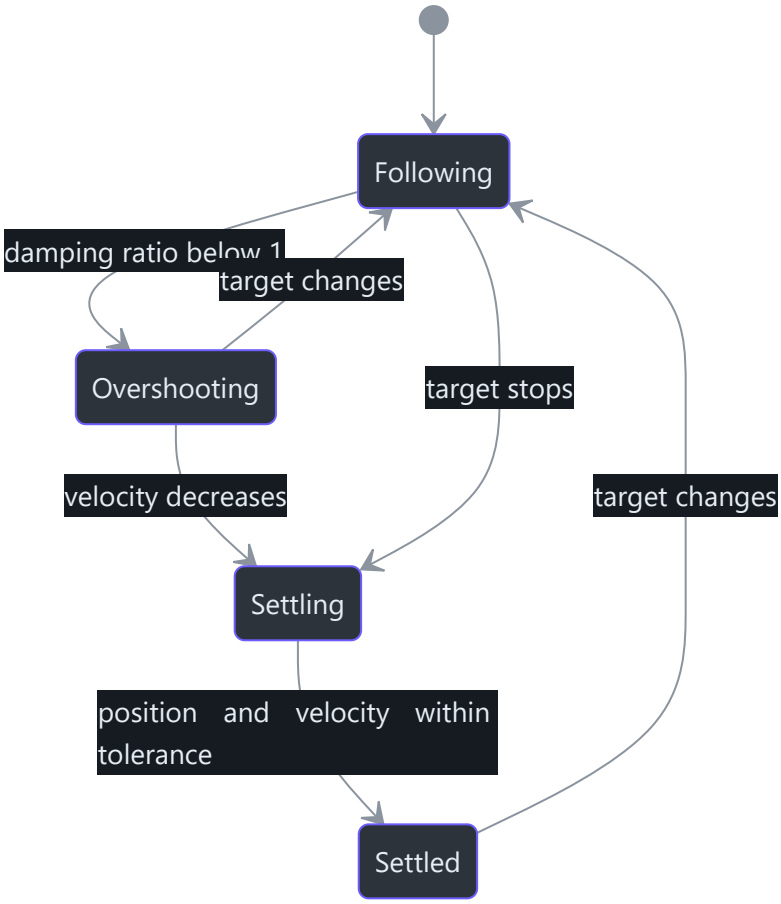
Physical model

For a unit mass:

```
acceleration = stiffness * (target - position) - damping * velocity
stiffness = angular_frequency²
damping = 2 * damping_ratio * angular_frequency
```

The implementation updates velocity before position, which is semi-implicit Euler. A frame delta is divided into steps no larger than `1/120` second to reduce frame-rate dependence and unstable large jumps (`src/kardboard_vtuber/motion/springs.py:49-67`).

Parameter	Meaning	Default
`frequency_hz`	How quickly the spring reacts	`4.0`
`damping_ratio`	`1` critical, `<1` overshoots, `>1` sluggish	`0.7`
`maximum_step_seconds`	Largest internal integration step	`1/120`



Current renderer use

- Two inward underside closure-panel springs respond to roll, yaw, and horizontal movement.
- One broad front underside flap responds to pitch and vertical movement.
- Two lower external side tabs use faster, lower-damping springs and amplified yaw targets.
- Sustained yaw drives the external tabs in opposite directions.
- Long frame gaps reset the five springs rather than integrating stale movement.

The shader rotates only vertices carrying hinge IDs `1..5`; the internal privacy volume keeps hinge ID `0` and never follows decorative flap motion (`src/kardboard_vtuber/renderer/textured_3d.py:15-89`, `src/kardboard_vtuber/renderer/textured_3d.py:329-472`, `tests/test_textured_3d_renderer.py:118-243`).

Validation

Primitive tests prove convergence, underdamped overshoot, frame-rate consistency, reset behavior, and explicit negative-time failure (`tests/test_motion_springs.py:1-71`). Renderer tests separately prove all hinge IDs exist and small yaw produces visible opposite-direction external-tab response (`tests/test_textured_3d_renderer.py:118-243`).

Complexity

Each internal step is O(1) time and O(1) space. For a frame delta `dt`, work is O(`ceil(dt / maximum_step_seconds)`), which is normally four steps at 30 FPS.

References

- `src/kardboard_vtuber/motion/springs.py:1-85`
- `src/kardboard_vtuber/motion/__init__.py:1-5`
- `src/kardboard_vtuber/tracking/filters.py:1-175`
- `src/kardboard_vtuber/tracking/models.py:55-100`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:88-199`
- `tests/test_motion_springs.py:1-71`
- `src/kardboard_vtuber/renderer/textured_3d.py:329-472`
- `tests/test_textured_3d_renderer.py:118-243`

⬅️ One Euro filtering · 🏠 Algorithms and data structures

26

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 26

Low-resolution overlay compositing

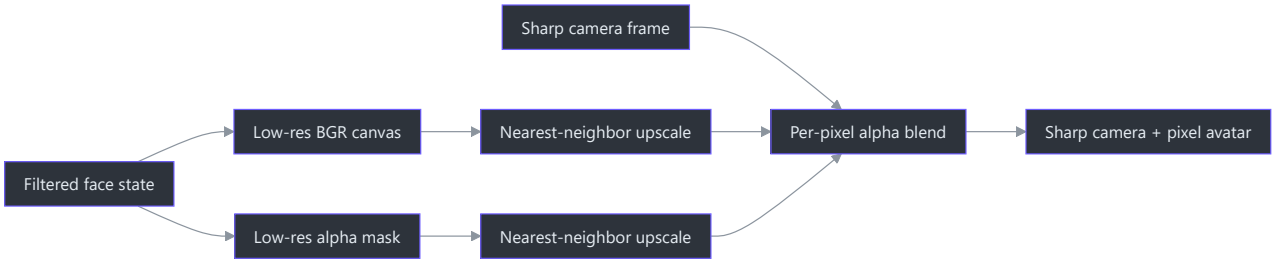
PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 26

Low-resolution overlay compositing

> **Status: implemented.**
>
> **TL;DR** — Avatar color and alpha are drawn on a small canvas, enlarged with nearest-neighbor interpolation, and blended over the original camera frame. Both renderer backends preserve a sharp camera image while pixelating only synthetic avatar output (`src/kardboard_vtuber/renderer/ps1_cardboard.py:45-165`, `src/kardboard_vtuber/renderer/textured_3d.py:180-279`).

Algorithm

```
low_width  = ceil(frame_width  / pixel_scale)
low_height = ceil(frame_height / pixel_scale)
draw avatar color and alpha at low resolution
upscale color and alpha with nearest-neighbor interpolation
output = avatar * alpha + camera * (1 - alpha)
```

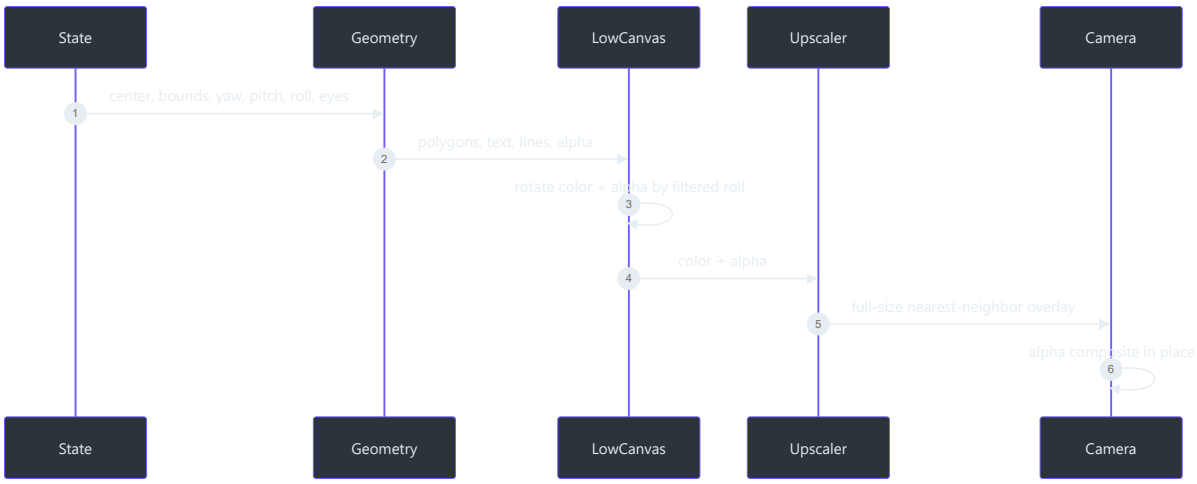


Why two buffers

The color canvas stores cardboard pixels; the alpha mask records coverage. Keeping them separate allows fully opaque panels, a transparent background, an intentional neck cutout, and text-shaped eyes without pixelating or recoloring the camera outside the avatar (`src/kardboard_vtuber/renderer/ps1_cardboard.py:51-59`, `src/kardboard_vtuber/renderer/ps1_cardboard.py:151-165`).

Geometry mapping

Face center and bounds are converted from normalized state into low-resolution coordinates, then expanded and shifted upward because facial landmarks do not include the hair/crown. Yaw compresses the front and exposes the opposite screen-side depth plane. Pitch relative to the calibrated `-10` degree baseline grows either the top plane when looking down or the split underside around the neck opening when looking up (`src/kardboard_vtuber/tracking/models.py:55-80`, `src/kardboard_vtuber/renderer/ps1_cardboard.py:66-136`).



Complexity

Low-resolution drawing is O((W/P) × (H/P)), where P is `pixel_scale`. The final resize and blend are O(W × H), required once per output pixel. Memory is one low-resolution color canvas, one low-resolution mask, and their temporary full-resolution upscales.

Validation

The procedural fallback blacks initial no-face frames, modifies only the tracked head region after acquisition, keeps visible `K C` ordering, preserves the camera only through its fallback-specific neck-safe V-shaped opening, keeps the lower face fully opaque, and freezes the last safely composited frame through tracking loss. Dedicated perspective tests verify that a rightward face turn exposes screen-left depth and that up/down pitch selects underside/top geometry while wink tests verify K/left and C/right anatomical closure and roll tests verify whole-shell rotation. The V apex is clamped below the tracked face bottom plus `16%` of tracked face height, and pitch regression tests verify that this chin/beard safety band stays opaque while the neck remains visible (`tests/test_ps1_cardboard_renderer.py:38-191`). Spring behavior is independently covered (`tests/test_motion_springs.py:18-71`), and CLI composition occurs before diagnostics (`src/kardboard_vtuber/cli.py:215-414`). The default textured renderer instead has a complete rectangular front face and an underside neck channel (`tests/test_textured_3d_renderer.py:351-409`).

References

- `src/kardboard_vtuber/renderer/ps1_cardboard.py:45-272`
- `src/kardboard_vtuber/tracking/models.py:55-100`
- `src/kardboard_vtuber/motion/springs.py:26-85`
- `src/kardboard_vtuber/cli.py:153-167`
- `tests/test_ps1_cardboard_renderer.py:1-86`
- `tests/test_motion_springs.py:1-71`

⬅️ Damped spring integration · 🏠 Algorithms and data structures

27

PART IV · ALGORITHMS AND DATA STRUCTURES

CHAPTER 27

Bounded studio mailbox

PART IV · ALGORITHMS AND DATA STRUCTURES Chapter 27

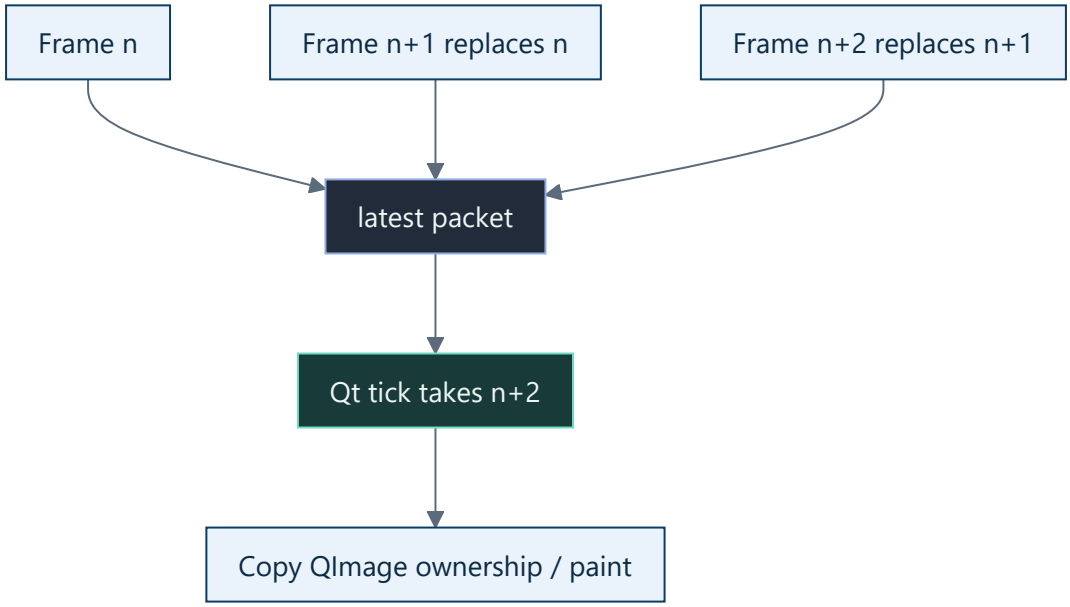
Bounded studio mailbox

> **TL;DR:** The UI needs the newest preview, not every historical frame. A one-slot mailbox applies the same latency-over-completeness idea as camera ingestion at a second boundary.

Problem

Suppose a producer renders 30 frames/s while a window temporarily paints at 10 frames/s. An ordinary queued “frame ready” signal can accumulate 20 stale frames each second. Moving rendering to a thread alone does not solve this backlog.

`RuntimeBridge.publish/take` stores one owned `PreviewPacket`. A publish replaces the waiting packet; a UI timer takes the latest reference and clears the slot. Image resizing/copying happens outside the short lock, so the widget does not hold a mutex during expensive pixel work.



Complexity and boundaries

Slot replacement is O(1); copying costs O(source pixels + bounded diagnostic/display pixels) when the full-resolution clean-window channel is present. A packet owns each channel separately: program, clean window, raw face, mesh and skeleton cannot contaminate one another. Waiting storage is one packet, not O(elapsed frames). Capture/virtual output keep independent buffers. Dropping a display packet does not drop live output or an offline encoded video frame.

Logs use a separate 500-entry deque. When it overwrites older entries, the consumer receives an explicit dropped-line count. The visible editor independently caps its text blocks at 1,500. This keeps a slow consumer from producing invisible, unbounded operational history.

Live tuning is another immutable snapshot behind the same small lock. Validation occurs before replacement, so a partially edited threshold tuple never reaches the worker.

Source: `src\kardboard_vtuber\runtime.py`, `RuntimeBridge`; UI consumer: `gui\window.py`, `StudioWindow._poll_runtime`; pixel ownership: `gui\widgets.py`, `VideoCanvas.set_frame`. Behavioral coverage: `tests\test_studio_runtime.py`, `tests\test_studio_gui.py`.

PART V

Design Principles

The engineering values behind the implementation: freshness, narrow boundaries, immutable observations, explicit failures, and fail-closed privacy.

28

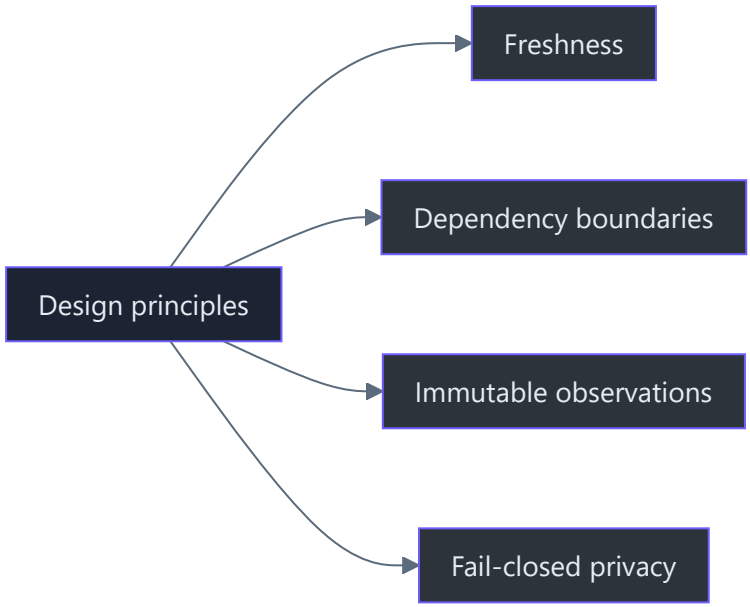
PART V · DESIGN PRINCIPLES

CHAPTER 28

Design principles

PART V · DESIGN PRINCIPLES Chapter 28

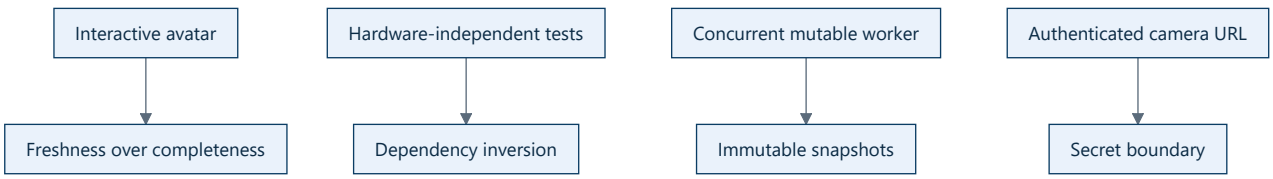
04 · Design principles



TL;DR — The implementation is governed by four deliberate principles: freshness over completeness, dependency inversion at hardware boundaries, immutable observation of mutable runtime state, and explicit security boundaries around camera sources.

Principle catalogue

Principle	Chapter	Concrete manifestation
Latency over completeness	Latency over completeness	Single latest-frame slot
Dependency inversion	Dependency inversion	`VideoCaptureLike` + injected factory
Immutable observations	Immutable snapshots	Frozen `CaptureSnapshot`
Security by boundary	Security and secret boundaries	URL redaction and ignored local config
Responsive control surfaces	Responsive control surfaces and progressive disclosure	Native widget/worker ownership, stable Stop placement, LIVE semantics and explicit raw consent



These are not slogans added after implementation. Each principle maps to code and tests.

⬅️ Algorithms · ➡️ Camera ingestion

29

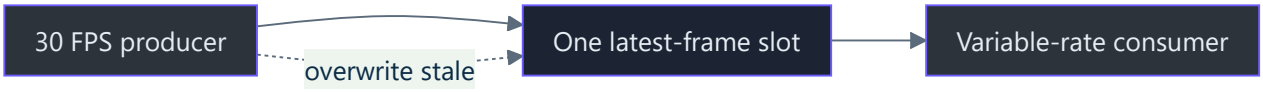
PART V · DESIGN PRINCIPLES

CHAPTER 29

Design principle: latency over completeness

PART V · DESIGN PRINCIPLES Chapter 29

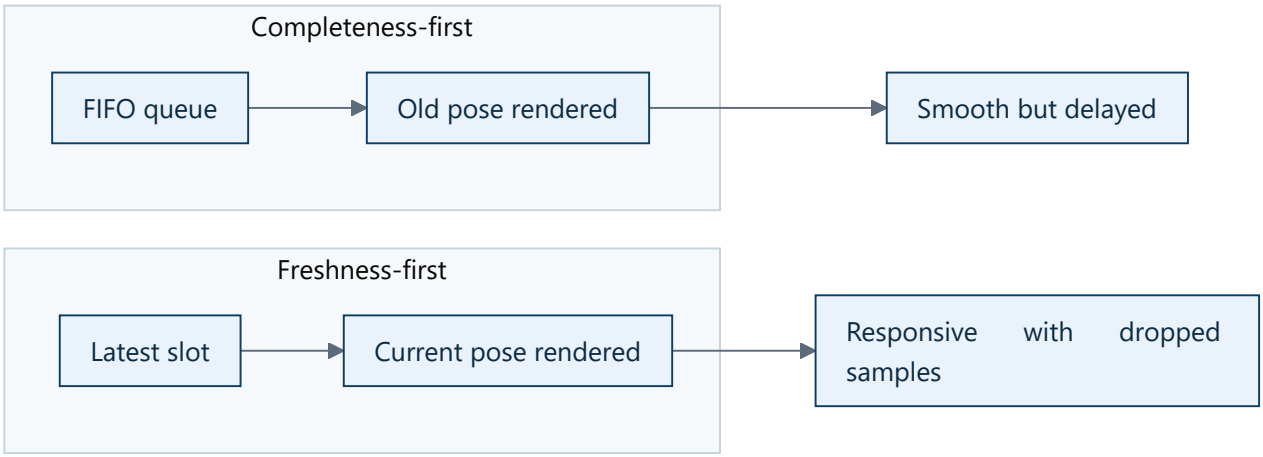
Design principle: latency over completeness



> **Rule:** In an interactive camera pipeline, process the newest useful observation rather than preserving every historical observation.

Why

Human perception is sensitive to motion lag. A complete but delayed head pose feels worse than a slightly lower effective frame rate that stays synchronized with the person.



Code evidence

- `_latest` stores one packet: `src/kardboard_vtuber/camera/stream.py:59`
- New packets overwrite it: `stream.py:198-204`
- Consumers request newer sequence values: `stream.py:114-141`
- Overwrite behavior is tested: `tests/test_camera_stream.py:67-80`

Limits of the principle

Use this principle for:

- face tracking;
- avatar pose;
- interactive preview;
- telemetry that describes current state.

Do not use it for:

- recordings;
- forensic evidence;
- offline model training;
- exact frame-by-frame export.

Interview explanation

“I treated frames as a sampled state stream rather than a work queue. That bounded memory to one frame and prevented unbounded latency when downstream processing became slower than capture.”

⬅️ Principles · ➡️ Dependency inversion

30

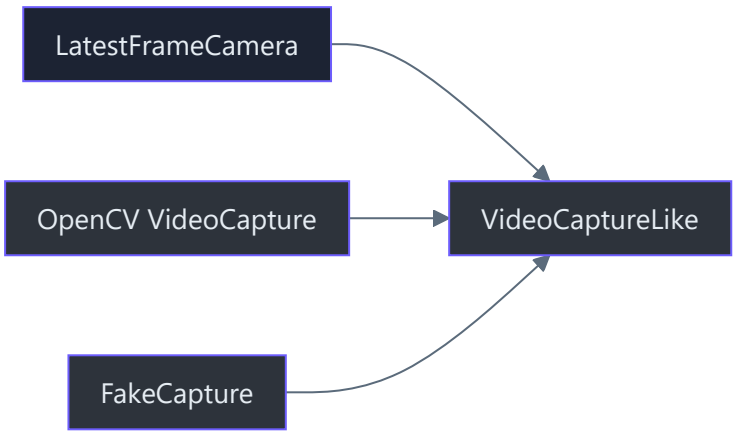
PART V · DESIGN PRINCIPLES

CHAPTER 30

Design principle: dependency inversion at the capture boundary

PART V · DESIGN PRINCIPLES Chapter 30

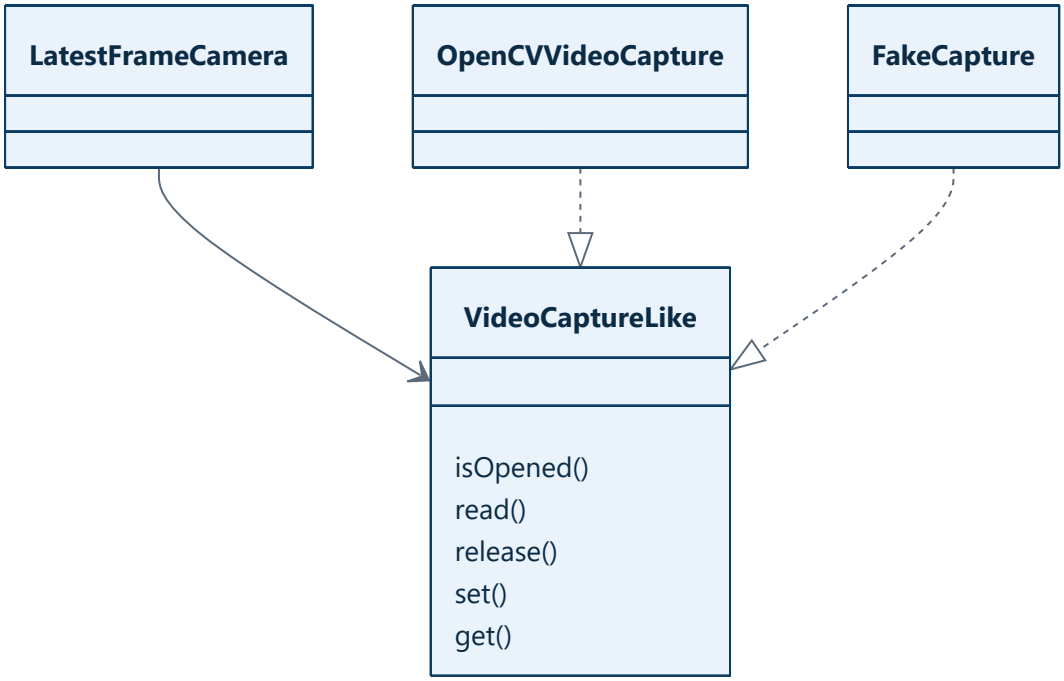
Design principle: dependency inversion at the capture boundary



> **Rule:** Core lifecycle logic should depend on the behavior it needs, not directly on a physical camera or a concrete OpenCV object.

The boundary

`VideoCaptureLike` declares five methods at `src/kardboard_vtuber/camera/stream.py:20-29`. Production adapts OpenCV through `_default_capture_factory()` at `stream.py:35-36`.



Python protocols use structural typing: the adapter does not need to inherit from the protocol. It only needs compatible methods.

Benefits

- Tests run without cameras or network access.
- Failure behavior can be simulated deterministically.
- OpenCV remains replaceable without rewriting the worker.
- The core class has a smaller contract than the full `cv2.VideoCapture` API.

Constructor injection

`LatestFrameCamera.__init__()` accepts `capture_factory` (`src/kardboard_vtuber/camera/stream.py:47-53`). Tests pass a lambda returning `FakeCapture` (`tests/test_camera_stream.py:49-53`).

Tradeoff

The protocol does not guarantee runtime correctness by itself. Tests and type checking verify the shape, while integration probes verify actual backend behavior.

Interview explanation

“I inverted the hardware dependency by defining the smallest capture protocol the worker needed and injecting a factory. That let unit tests exercise concurrency and buffering without flaky camera hardware.”

⬅️ Latency over completeness · ➡️ Immutable snapshots

31

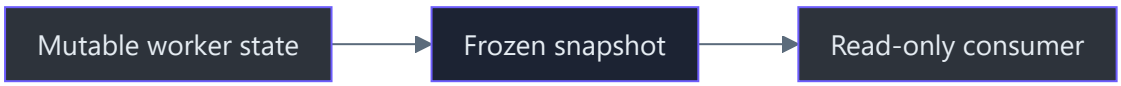
PART V · DESIGN PRINCIPLES

CHAPTER 31

Design principle: immutable observations of mutable state

PART V · DESIGN PRINCIPLES Chapter 31

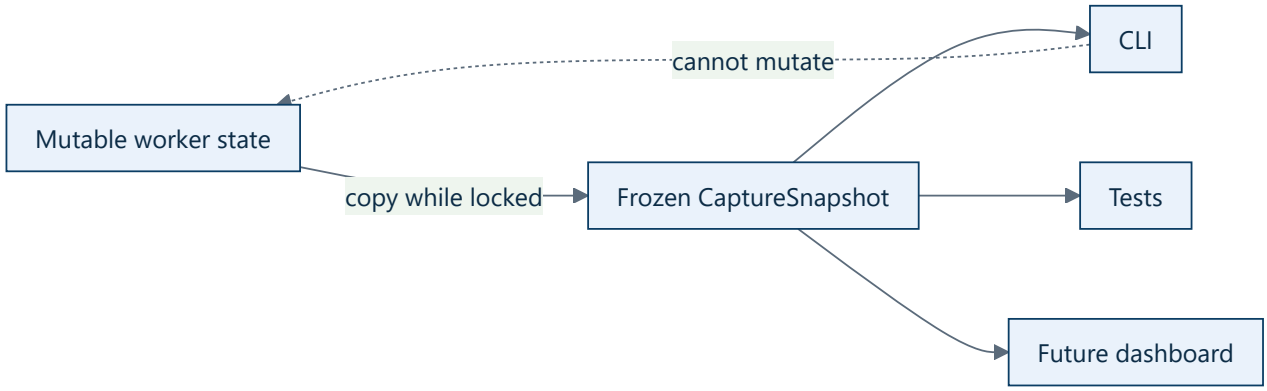
Design principle: immutable observations of mutable state



> **Rule:** The worker may own mutable runtime state, but readers receive immutable point-in-time values rather than references they can change.

The problem

The capture worker constantly changes counters, state, errors, and negotiated properties. Returning the internal object would let presentation code accidentally corrupt capture behavior.



`CaptureSnapshot` is a frozen, slotted dataclass at `src/kardboard_vtuber/camera/models.py:141-157`. `snapshot()` constructs it under the condition lock at `stream.py:143-158`.

Benefits

- Readers cannot mutate the worker.
- One snapshot remains internally consistent.
- Presentation logic is decoupled from synchronization details.
- Future JSON serialization or telemetry export has a stable schema.

Same idea in configuration

`CameraConfig`, `CameraSource`, and `FramePacket` are also frozen. A running camera cannot silently change configuration halfway through a frame. A future reconfiguration feature should construct a new config and perform an explicit transition.

Tradeoff

Snapshots allocate small Python objects. This cost is negligible compared with decoding and copying video frames, and the clarity is worth it.

⬅️ Dependency inversion · ➡️ Security boundaries

32

PART V · DESIGN PRINCIPLES

CHAPTER 32

Design principle: security and secret boundaries

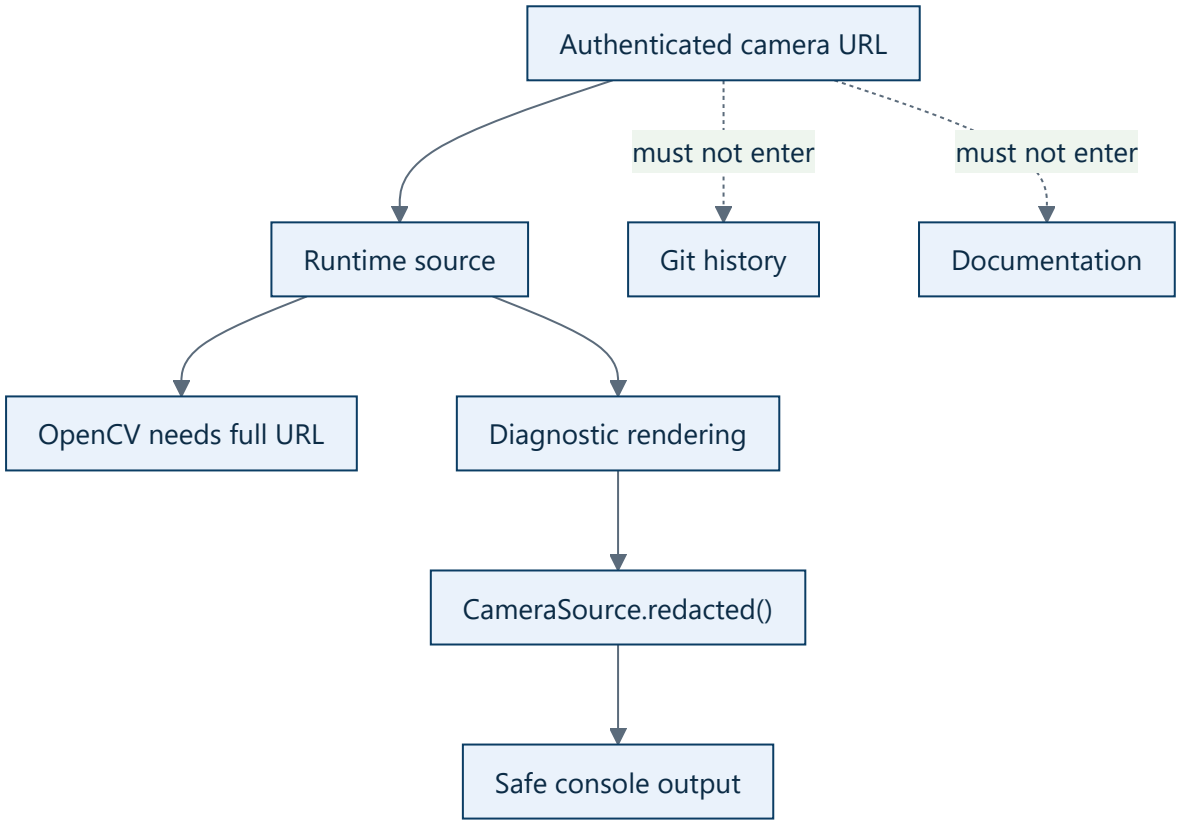
PART V · DESIGN PRINCIPLES Chapter 32

Design principle: security and secret boundaries



> **Rule:** A camera source may contain credentials, but logs, diagnostics, documentation, tests, and commits must not expose real credential values.

Threat model



Implemented controls

- `CameraSource.redacted()` replaces user information with `***` (`src/kardboard_vtuber/camera/models.py:80-87`).
- Snapshots use the redacted source (`stream.py:143-158`).
- `.env` , local config, logs, captures, and recordings are ignored (`.gitignore:1-35`).
- Documentation uses placeholders rather than actual credentials.

Important limitation

Passing a full authenticated URL on a command line can expose it in shell history or process inspection. Redacting application output does not erase that external exposure. A future configuration subsystem should support local secret storage or interactive credential input.

Operational rules

1. Use a camera-only password that is not reused elsewhere.
2. Do not commit the URL.
3. Prefer a private tethered network where practical.
4. Disable the server when not streaming.
5. Rotate credentials if they appear in screenshots or shared logs.

Test evidence

Credential redaction is verified by `test_camera_source_redacts_credentials()` at `tests/test_camera_models.py:20-24` using synthetic credentials only.

⬅️ Immutable snapshots · ➡️ Camera ingestion

33

PART V · DESIGN PRINCIPLES

CHAPTER 33

Responsive control surfaces and progressive disclosure

PART V · DESIGN PRINCIPLES Chapter 33

Responsive control surfaces and progressive disclosure

> **TL;DR:** Separate interaction ownership from expensive work, and expose advanced features without allowing them to obscure the selected safe operating mode.

Separation of responsibilities

The studio owns controls, validation presentation, tutorial and painting. The existing runtime owns inference/composition/output. One optional bridge is a narrower maintenance boundary than a second implementation of every privacy branch in Qt.

Qt widget access stays on the main thread. A session worker creates and closes the ModernGL context on its own thread. The UI never waits synchronously for shutdown; it shows Stopping until the owned worker actually finishes. Responsiveness is not a license to report false success.

Progressive disclosure is not feature removal

The page selector groups tasks: input, eyes/avatar, output/effects, experiments, recording, model setup and tutorial. Every CLI option remains reachable and a test checks that inventory. The default mode stays face-only; exposing hood/body controls does not mean they are validated replacements. Raw debugging has an explicit local privacy acknowledgement.

The Stop button stays in the same header position across pages because finding it should not depend on remembering which settings panel is open. Per-eye values sit alongside a large preview and persistent raw/filtered meters because calibration is an observe-adjust-compare task.

Numeric boxes accompany sliders for precise/repeatable calibration and keyboard access. Anatomical LEFT/K and RIGHT/C labels resolve mirror ambiguity. A visible LIVE label distinguishes edits that apply now from resource settings that require Stop. Invalid tuples keep the last valid snapshot; the interface reports the rejected edit instead of silently clamping unrelated controls.

Explicit consent and reversible state

Opening, loading a preset or reading help does not start capture. Presets do not persist a source credential or raw-face consent. A synthetic demo teaches operation without a camera, but its labels and unavailable real FPS values prevent it from masquerading as live evidence.

Theme is UI-only; it does not recolor output. The original KC-box icon identifies the application without requiring a network icon service or a large asset dependency.

See interface decisions for individual control-placement rationale and runtime architecture for the implementation boundaries.

PART VI

Camera Ingestion

How local devices and Android streams are opened, transformed, measured, reconnected, and delivered without building a latency-producing queue.

34

PART VI · CAMERA INGESTION

CHAPTER 34

Camera ingestion

PART VI · CAMERA INGESTION Chapter 34

05 · Camera ingestion



> **Status: implemented and verified.**
>
> **TL;DR** — The camera subsystem accepts local device indices and OpenCV-supported stream URLs, captures on a background thread, keeps only the latest frame, applies orientation transforms, reconnects after sustained failures, and reports negotiated and measured behavior.

Chapter map

- Runtime flow — one frame from source to preview
- Android IP Webcam — the verified phone setup
- Diagnostics and performance — metrics and their interpretation

Quick start

```
cd C:\devdesk\KardboardCode\KardboardCode-VTuber
.\.venv\Scripts\Activate.ps1
```

Local camera:

```
python -m kardboard_vtuber --source 0 --backend auto --mirror
```

Phone camera:

```
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror
```

Press `Q` or Escape to stop.

When `--source` resolves to a local video file, capture is paced at the file's negotiated FPS and stops at EOF rather than reconnecting. Use `--input-already-mirrored` for recordings captured from an already mirrored preview so anatomical eye mapping remains correct without flipping the pixels a second time.

Verified result



The displayed 0.4 ms frame age covered only post-decode time inside the Python application.

Supported backends

Value	OpenCV backend	Intended use
`auto`	`CAP_ANY`	Default; best verified local and phone result
`dshow`	`CAP_DSHOW`	Windows DirectShow fallback
`msmf`	`CAP_MSMF`	Windows Media Foundation fallback
`ffmpeg`	`CAP_FFMPEG`	Explicit network-stream fallback

Mappings: `src/kardboard_vtuber/camera/models.py:15-30`.

⬅️ Design principles · ➡️ Face tracking

35

PART VI · CAMERA INGESTION

CHAPTER 35

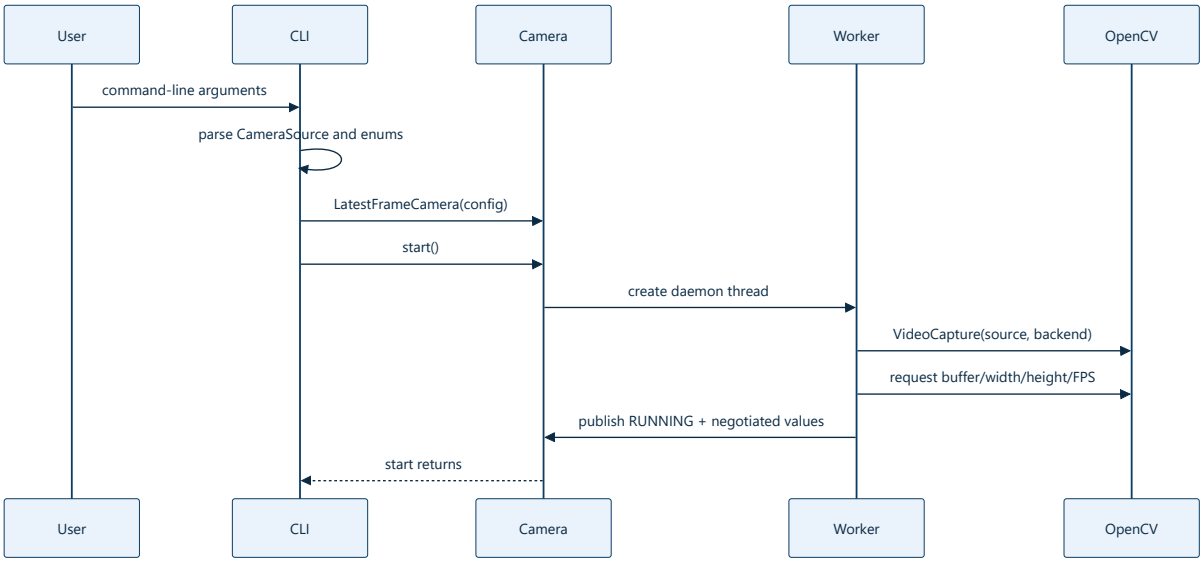
Camera runtime flow

PART VI · CAMERA INGESTION Chapter 35

Camera runtime flow

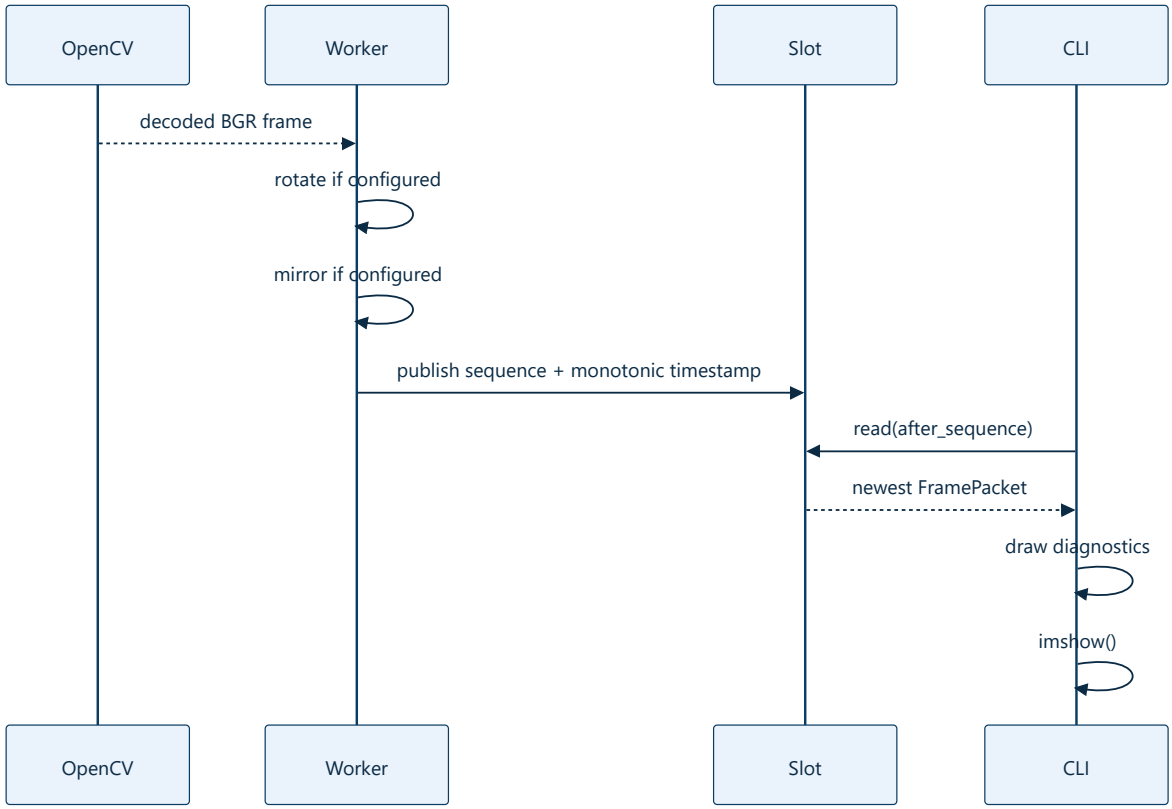
> **TL;DR** — The CLI constructs immutable configuration, starts the worker, waits for newer packets, renders diagnostics, and shuts down cleanly. Reconnection remains inside the camera class rather than leaking into the UI loop.

Startup sequence



CLI construction: `src/kardboard_vtuber/cli.py:54-70`.

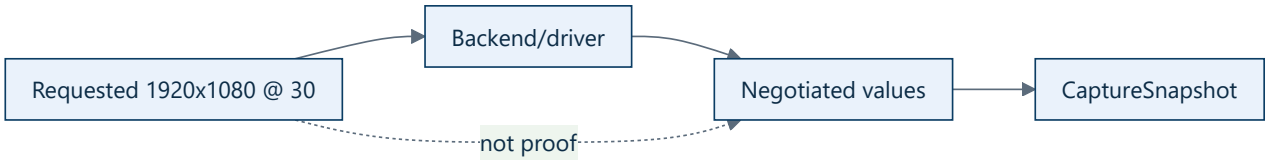
Per-frame sequence



Transformation order is rotation followed by mirroring (`src/kardboard_vtuber/camera/stream.py:192-197`). Mirroring after rotation means "horizontal" is relative to the final displayed orientation.

Requested versus negotiated properties

`_apply_requests()` asks the backend for buffer size, width, height, and FPS (`stream.py:261-268`). `_open()` then reads actual values back (`stream.py:246-248`).



Shutdown sequence

The CLI exits on duration, `Q`, Escape, Ctrl+C, or terminal error. Its `finally` block always calls `camera.stop()` and `cv2.destroyAllWindows()` (`src/kardboard_vtuber/cli.py:115-126`).

⬅️ Camera chapter · ➡️ Android IP Webcam

36

PART VI · CAMERA INGESTION

CHAPTER 36

Android IP Webcam integration

PART VI · CAMERA INGESTION Chapter 36

Android IP Webcam integration



> **Status: user-verified over Wi-Fi.**
>
> **TL;DR** — The Nothing Phone hosts an authenticated MJPEG stream. OpenCV reads the direct `/video` endpoint without installing a vendor camera client or virtual-camera driver on Windows.

Why this transport exists

The Nothing Phone (3a) build did not expose Android's native USB `Webcam` mode. The user also rejected third-party camera software on Windows. A phone-hosted network stream therefore provides the cleanest bridge:



Phone setup

1. Select the front camera in IP Webcam.
2. Configure a dedicated username and password.
3. Start the server.
4. Keep the phone and PC on the same private network.
5. Use the direct `/video` endpoint, not only the HTML control page.

Application command

```
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror
```

The current physical phone orientation requires `--rotate left`. `--mirror` produces the expected selfie behavior.

Verified observations

Observation	Result

Direct endpoint	`/video` returned multipart MJPEG
Source format reported	1920x1080 at 25 FPS
Post-restart measured throughput	Approximately 28-30 FPS
Final orientation	1080x1920 portrait
Read failures/reconnects during probe	None
Visual confirmation	User confirmed preview looked correct

An initial 3-5 FPS measurement disappeared after restarting the phone server. This demonstrates why one short benchmark should not be treated as a permanent transport limit.

Security

Real credentials and the private address are intentionally absent from this book. The application redacts credentials in diagnostics, but command-line history remains an external risk. Use a camera-specific password and stop the server when finished.

Troubleshooting

Symptom	Check
`could not open camera source`	Replace placeholders; confirm server is running
Browser works, OpenCV fails	Use direct `/video`; try `--backend ffmpeg`
Sideways output	Use `--rotate left`
Motion feels reversed	Add or remove `--mirror`
Low FPS	Restart server, lower resolution/quality, improve network
Reconnect loop	Keep app active and disable battery optimization

⬅️ Runtime flow · ➡️ Diagnostics

37

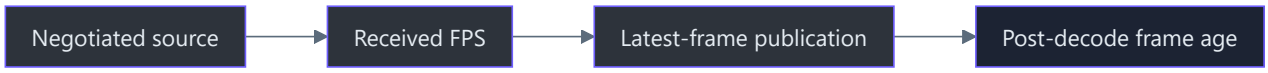
PART VI · CAMERA INGESTION

CHAPTER 37

Diagnostics and performance interpretation

PART VI · CAMERA INGESTION Chapter 37

Diagnostics and performance interpretation



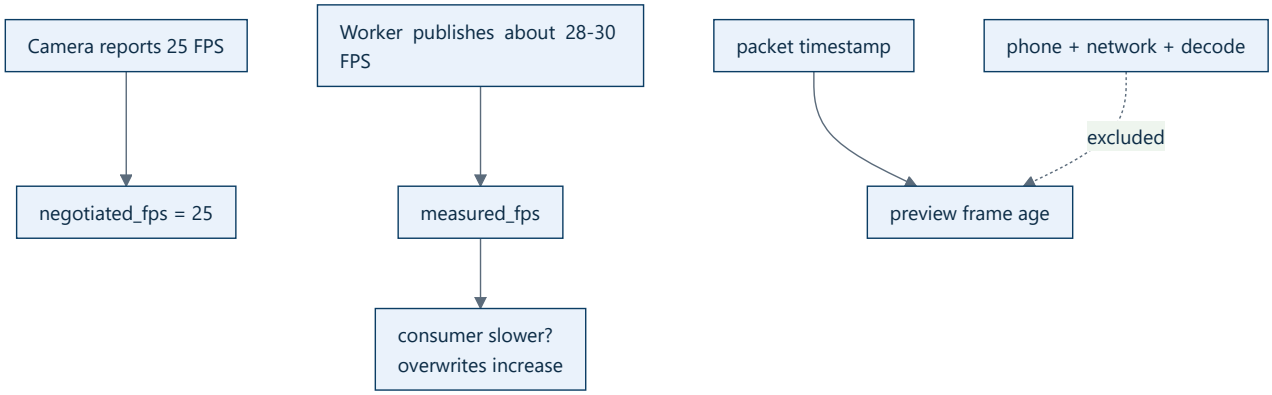
> **TL;DR** — Diagnostics distinguish what the source claims, what the worker actually receives, and how quickly the current packet reaches the preview after decode.

Snapshot fields

Field	Meaning
`state`	Current lifecycle
`source`	Redacted source
`backend`	Selected OpenCV backend
`negotiated_*`	Values reported by the opened source
`measured_fps`	Frames published per measured time window
`received_frames`	Total successful publications
`overwritten_frames`	Frames superseded in the latest slot
`read_failures`	Individual failed reads
`reconnects`	Source reopen attempts
`last_error`	Most recent open/read/worker error

Construction: `src/kardboard_vtuber/camera/stream.py:143-158`.

Metrics relationship



Interpreting overwrites

High overwrites with stable measured FPS are expected when the consumer asks only for the newest frame. Investigate only if visible motion is poor or a consumer was expected to inspect every frame.

Development-machine baselines

Source/backend	Result
Integrated camera, `auto`	640x480 at approximately 30 FPS
Integrated camera, `dshow`	Approximately 12 FPS
Integrated camera, `msmf`	Opened but did not continuously deliver in short probe
Phone MJPEG, `auto`, after restart	1080x1920 final portrait at approximately 28-30 FPS

Baselines are observations, not portable guarantees.

Benchmark protocol

1. Record source settings and backend.
2. Warm up for several seconds.
3. Record negotiated resolution/FPS.
4. Record measured FPS for at least 30 seconds.
5. Record read failures and reconnects.
6. Observe end-to-end motion delay separately from internal frame age.
7. Repeat after restart before concluding a transport limit.

⬅️ Android IP Webcam · 🏠 Camera chapter

PART VII

Face Tracking

MediaPipe integration, normalized project contracts, independent eye and mouth signals, calibrated pose, filtering, action events, and live validation.

38

PART VII · FACE TRACKING

CHAPTER 38

Face tracking

PART VII · FACE TRACKING Chapter 38

06 · Face tracking

> **Status: implemented and live-validated.**
>
> **TL;DR** — The tracker submits a downscaled copy of each latest camera frame to MediaPipe's asynchronous live-stream API. It retains one newest normalized result containing 478 landmarks, face bounds, independent eye openness, mouth openness, and a renderer-friendly head pose.

Why tracking is a separate subsystem

MediaPipe is an inference engine, not the application's domain model. The adapter converts MediaPipe results into frozen project contracts so future rendering can change independently (`src/kardboard_vtuber/tracking/models.py:1`, `src/kardboard_vtuber/tracking/mediapipe_tracker.py:1`).

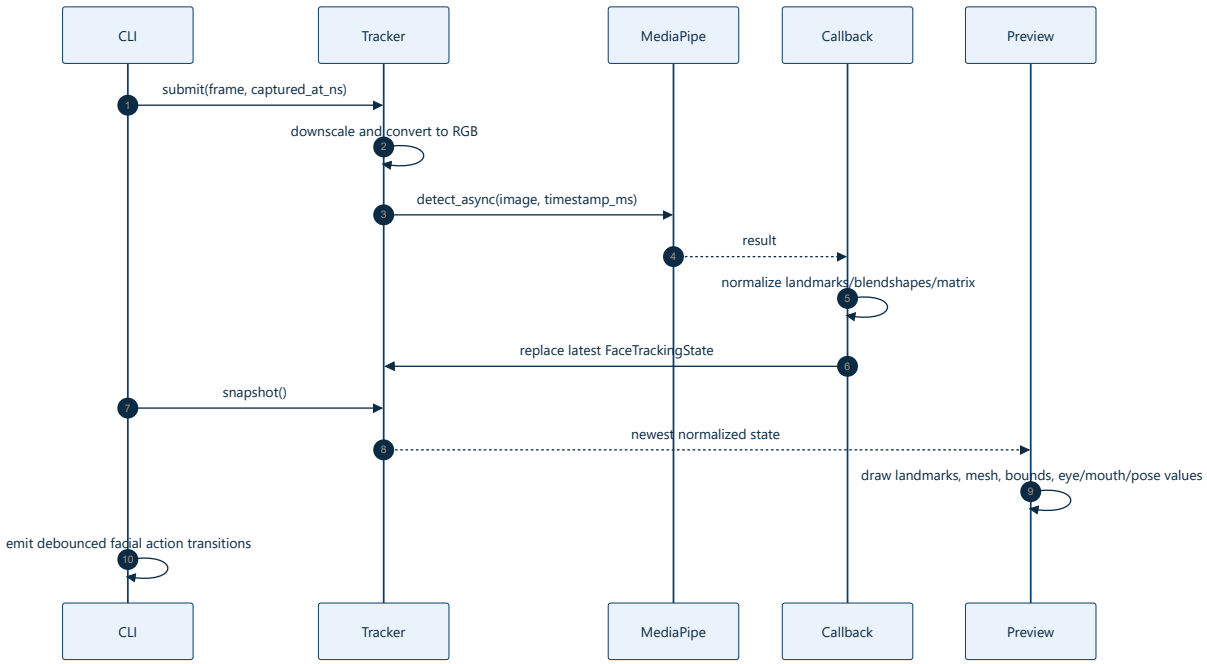


Implemented components

Component	Responsibility	Source
`FaceTrackingState`	Library-neutral face observation	`src/kardboard_vtuber/tracking/models.py:58`
`HeadPose`	Translation and Euler-angle view of 4x4 transform	`src/kardboard_vtuber/tracking/models.py:25`
`normalize_face()`	Landmark, bounds, blendshape, and pose conversion	`src/kardboard_vtuber/tracking/models.py:93`
`MediaPipeTrackerConfig`	Model path, input width, confidence thresholds	`src/kardboard_vtuber/tracking/mediapipe_tracker.py:23`
`MediaPipeFaceTracker`	Async task ownership and latest-result diagnostics	`src/kardboard_vtuber/tracking/mediapipe_tracker.py:44`
`FaceActionDetector`	Debounced blink, wink, eye, mouth, and face transitions	`src/kardboard_vtuber/tracking/events.py:73`
`draw_tracking_debug()`	Sparse landmarks, connected mesh inset, expressions, pose, and latest action	`src/kardboard_vtuber/tracking/mediapipe_tracker.py:191`
Model downloader	Official URL plus SHA-256 verification	`scripts/download_face_landmarker_model.py:1`

	cleanup	`src/kardboard_vtuber/cli.py:1`

Runtime sequence



Verified performance

The live Nothing Phone stream remained near 30 FPS while tracking at 640-pixel input width:

Signal	Observed
Camera receive rate	Approximately 29.5-31 FPS
Tracking result rate	Approximately 29-32 FPS
Pending or dropped count during probe	One current in-flight frame
Face detection	Continuous after startup
Tracking errors	None
Independent eye values	Present and changing
Mouth value	Present; near zero while mouth was closed
Spectacles	Face detected and a complete blink transition observed
Action transitions	Face detected, eyes closed/open, and blink emitted

This was a twelve-second probe, not a long-duration soak test.

Canonical regression recording

`scripts/record_guided_regression.py` records a clean 50-second camera video plus synchronized CSV telemetry. Its preview guides neutral pose, yaw, pitch, roll, blink, both winks, mouth movement, and combined motion. Preview instructions and debug graphics are never written into the saved MP4.

The CSV stores raw and filtered expressions/pose, filtered bounds, stage names, frame numbers, and action events. Generated recordings belong in the private session artifact directory and must not be committed.

Run it

```
.\.venv312\Scripts\Activate.ps1
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --track-face
```

Deep dives

- Native eye-calibration tutorial — independent anatomical thresholds, shared wink rules, raw/filtered meters and live updates
- Normalized face state
- Live debugging and validation
- Asynchronous live inference
- Blendshape normalization
- Transformation-matrix decomposition
- Facial action state machine

References

- `pyproject.toml:16-23` — tracking dependency
- `scripts/download_face_landmarker_model.py:1` — verified model acquisition
- `src/kardboard_vtuber/tracking/models.py:1` — normalized contracts
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:1` — MediaPipe adapter
- `src/kardboard_vtuber/cli.py:1` — runtime integration
- `tests/test_tracking_models.py:1` — normalization tests
- `src\kardboard_vtuber\tracking\eyes.py`, `EyeSensitivity` — shared per-eye rules
- `src\kardboard_vtuber\tracking\events.py`, `FaceActionDetector.set_thresholds` — live recalibration without a false blink spanning configurations

⬅️ Camera ingestion · ➡️ Quality and testing

39

PART VII · FACE TRACKING

CHAPTER 39

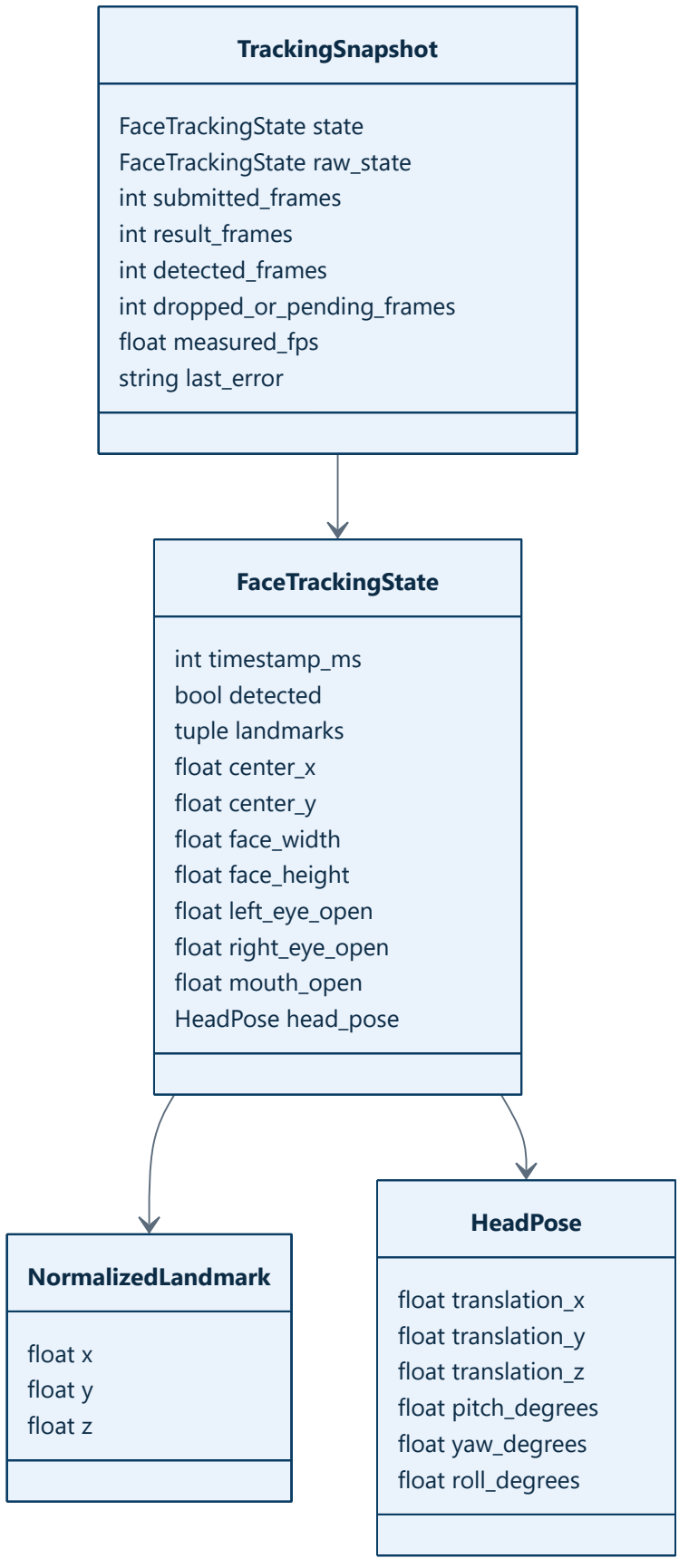
Normalized face state

PART VII · FACE TRACKING Chapter 39

Normalized face state

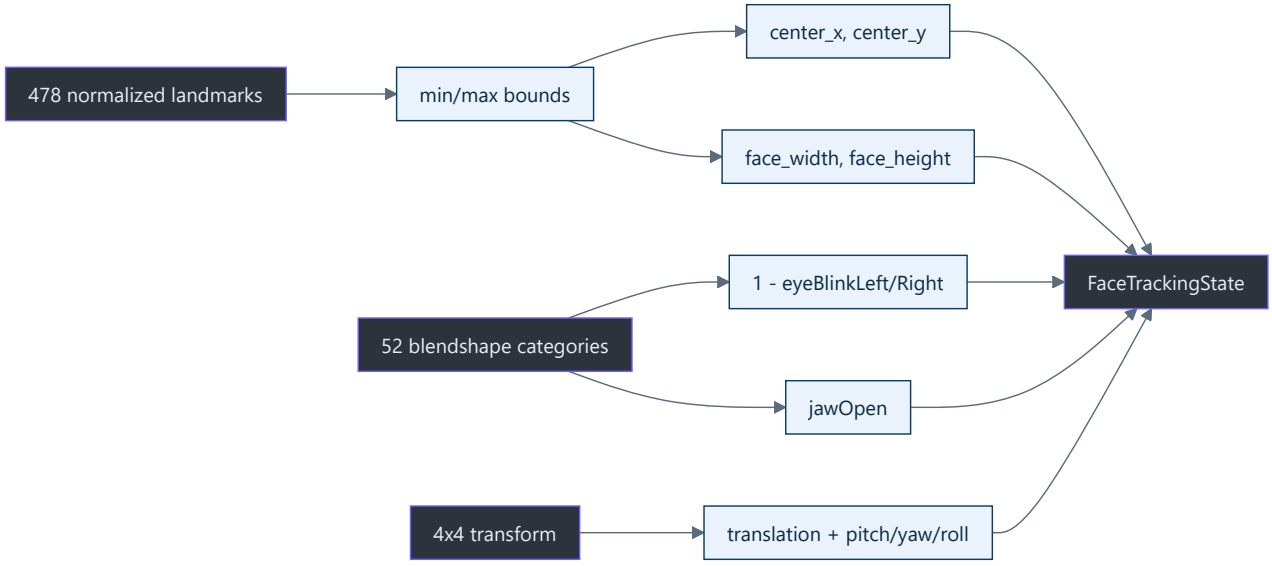
> **TL;DR** — `FaceTrackingState` is the anti-corruption layer between MediaPipe and the current renderer. It preserves only stable concepts the application owns.

Contract shape



Definitions live in `src/kardboard_vtuber/tracking/models.py:15-90`.

Data derivation



`normalize_face()` performs this conversion (`src/kardboard_vtuber/tracking/models.py:93-135`).

Invariants

- Eye and mouth controls are clamped to `[0, 1]` (`src/kardboard_vtuber/tracking/models.py:148-151`).
- Non-finite blendshape scores become zero.
- No-face observations use centered neutral defaults (`src/kardboard_vtuber/tracking/models.py:74-88`).
- All contracts are frozen and slotted.
- The result timestamp uses MediaPipe's millisecond timeline.

Why retain landmarks?

The opt-in debug overlay uses landmarks as spatial evidence. Current renderers use normalized bounds, center, expressions, and pose rather than consuming MediaPipe objects directly.

Raw and filtered states

`TrackingSnapshot` retains the raw normalized observation and the filtered renderer/debug state:

```
FaceTrackingState (raw normalized observation)
    → One Euro filter
    → FaceTrackingState (stable control targets)
    → AvatarControlState (renderer-specific values)
```

Action detection consumes `raw_state`; visual geometry and current renderer controls consume filtered `state`. This prevents quick blinks from being hidden by smoothing.

References

- `src/kardboard_vtuber/tracking/models.py:15-151`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:143-166`
- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:177-232`
- `src/kardboard_vtuber/cli.py:1`
- `tests/test_tracking_models.py:1`

⬅️ Face tracking · ➡️ Live debugging

40

PART VII · FACE TRACKING

CHAPTER 40

Live tracking debugging and validation

PART VII · FACE TRACKING Chapter 40

Live tracking debugging and validation

> **TL;DR** — A tracker is not proven by “face detected.” The preview exposes landmarks, bounds, a connected top-right face mesh, eye values, mouth value, pose, throughput, action transitions, submitted/results counts, and callback errors only when `--tracking-debug` is enabled.

Visual overlay

The normal preview is intentionally clean. `--tracking-debug` enables `draw_tracking_debug()`, which renders every eighth landmark, a face bounding box, independent eye openness, mouth openness, pitch/yaw/roll, and a black face-mesh inset in the top-right corner. The inset reserves its left side for a pose-driven XYZ axis gizmo: X is red, Y is green, and Z is blue. The arrows rotate from the same pitch/yaw/roll values shown in the text diagnostics (`src/kardboard_vtuber/cli.py:77-82`, `src/kardboard_vtuber/tracking/mediapipe_tracker.py:213-427`).

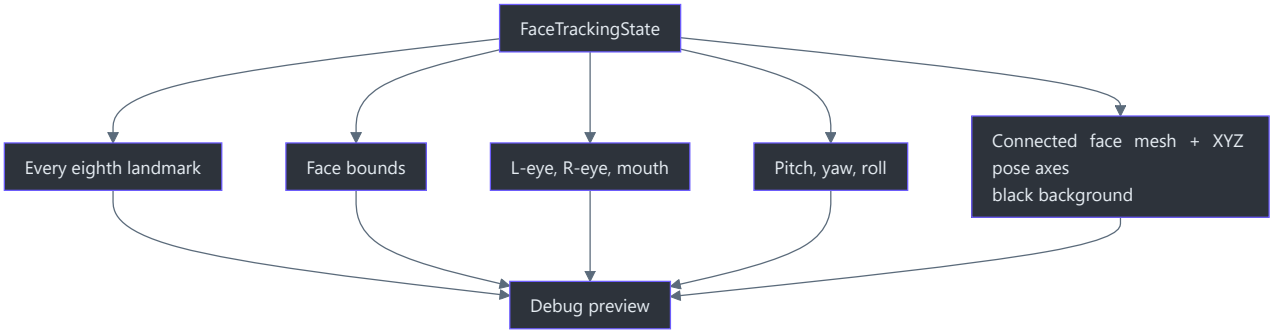


The real debug renderer operating on synthetic landmarks and the synthetic cardboard frame. No camera pixels or human face are present.

Before MediaPipe submission, the CLI applies a mild `+12` brightness offset only to internal tracking input. This improves feature contrast in dim input while the displayed and OBS-captured preview retains the original camera brightness. `--brightness 0` disables the lift; values up to `100` are accepted for local tuning (`src/kardboard_vtuber/cli.py`).

Film grain was removed after profiling showed its full-resolution Gaussian-noise allocation cost approximately `77 ms` per 1080x1920 frame. The fully opaque renderer also uses OpenCV's masked-copy path instead of allocating three full-frame floating-point blend buffers.

The latest primary transition is shown directly below the pose values as `ACTION = <ACTION NAME>`. The CLI retains the first event from a multi-event update so a meaningful transition such as `BLINK` is not immediately replaced by the accompanying `EYES OPEN` state (`src/kardboard_vtuber/cli.py:100-155`, `src/kardboard_vtuber/tracking/mediapipe_tracker.py:191-262`).



Face-mesh inset

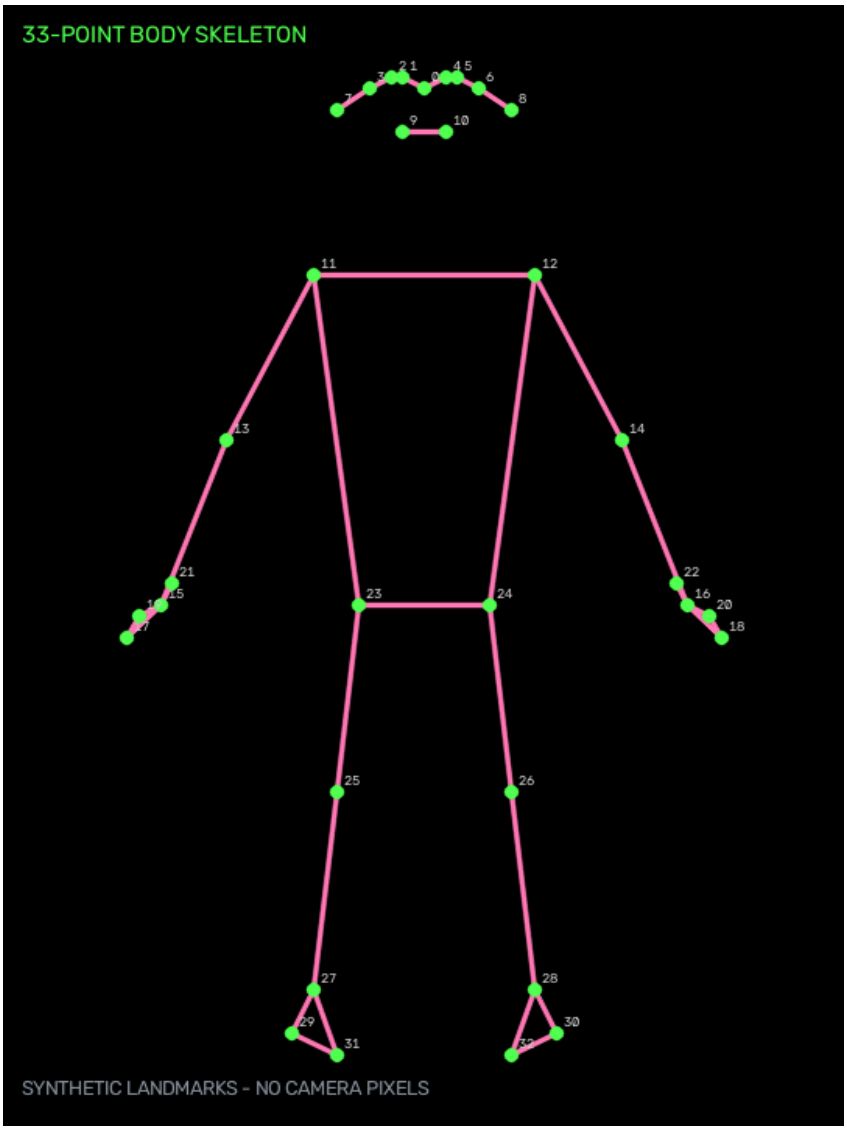
The inset first converts normalized X/Y deltas back into source-frame pixel proportions, then re-centers and scales them to fit a bounded top-right panel. The pixel conversion is essential for portrait video: applying one scale directly to normalized coordinates makes the face appear artificially wide and squashed. The corrected projection preserves the taller face and jaw contour. It connects the face oval, both eyes, both eyebrows, outer and inner lips, and nose paths, then adds sparse landmark points for additional motion feedback (`src/kardboard_vtuber/tracking/mediapipe_tracker.py:250-331`, `tests/test_tracking_models.py:132-171`).

The black background intentionally removes camera texture from this diagnostic region. That makes landmark motion, eye closure, mouth movement, and tracking loss easier to inspect. The inset shows `NO FACE` when no valid landmark set is available (`src/kardboard_vtuber/tracking/mediapipe_tracker.py:262-283`).

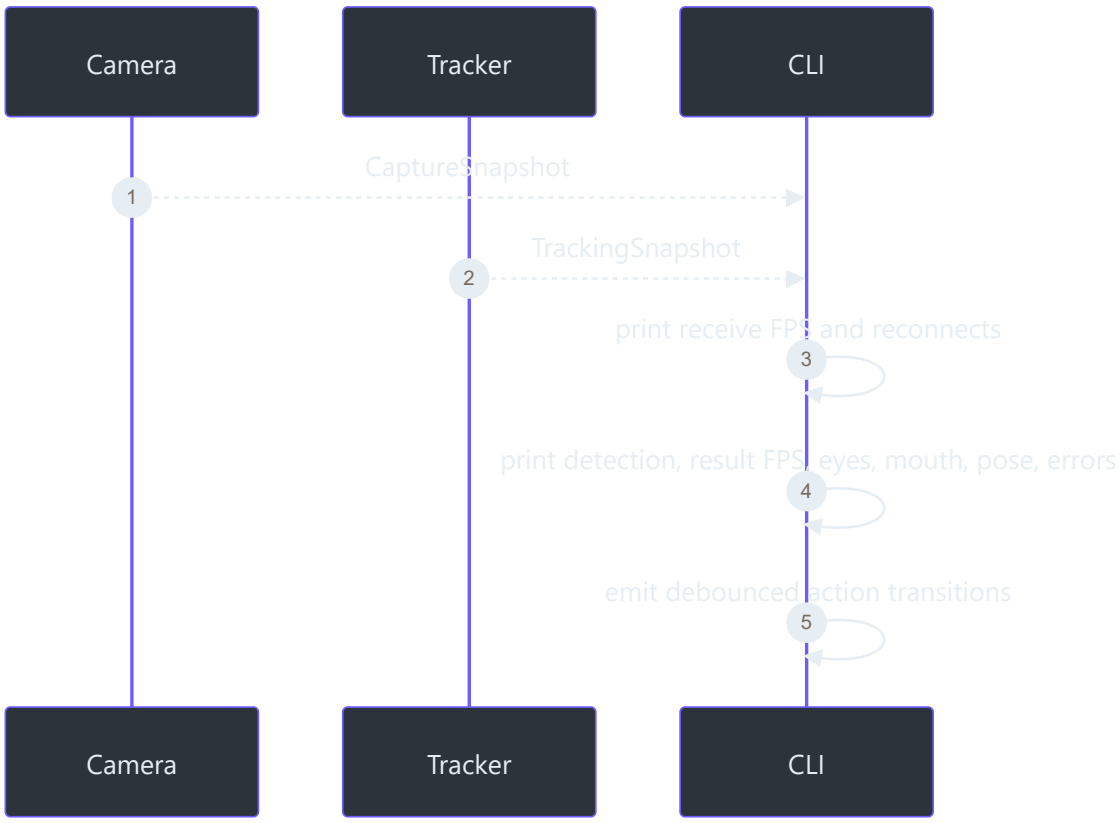
Console diagnostics

Every reporting interval, the CLI prints camera and tracker snapshots. These console diagnostics do not require the visual overlay (`src/kardboard_vtuber/cli.py:215-414`). Action messages are separate, transition-only logs generated by `FaceActionDetector` (`src/kardboard_vtuber/cli.py:122-128`, `src/kardboard_vtuber/tracking/events.py:73-171`).

Full-body skeleton window



`--full-body` opens this independent black diagnostic window. The documentation image calls the same `render_pose_skeleton_debug()` function with synthetic 33-point landmarks, preserving the real colors, connections, numbering, and layout without exposing a person (`src/kardboard_vtuber/tracking/full_body.py:176-245`, `scripts/generate_documentation_gallery.py:1`).



Manual validation script

1. Hold neutral expression and verify both eyes are near open.
2. Close only the left eye and verify one value falls.
3. Close only the right eye and verify the other value falls.
4. Open and close the mouth and verify `mouth` changes.
5. Rotate and tilt the head and verify pose signs change consistently.
6. Move closer/farther and verify bounds change.
7. Leave frame and verify `detected=False`.
8. Return and verify recovery without restarting.
9. Repeat with `--tracking-debug` and confirm the top-right mesh follows the face.
10. Repeat the eye tests with spectacles and watch for reflection-induced false closures.
11. Run without `--tracking-debug` and confirm no mesh, pose, action, FPS, or latency text appears.

Verified first probe

The latest twelve-second headless phone test used the same rotated and mirrored 1080p stream as the camera milestone. Camera and tracker remained around 30 FPS, face detection stayed active, one result was normally in flight, and no callback error was reported. With spectacles visible, the action detector emitted face detection, eyes closed, blink, and eyes open transitions. The inset's

pixel-level rendering is covered by an automated test, but its final placement still requires the user-visible preview rather than a headless probe (`tests/test_tracking_models.py:114-131`).

Completed expression and pose calibration

The guided live calibration was performed while the user wore spectacles:

Control	Observed evidence	Result
Left wink	Repeated events with the winked eye around `0.43-0.60`	Passed
Right wink	Event at left `0.75` , right `0.43`	Passed
Mouth open	Repeated events around `0.65-0.73`	Passed
Mouth closed	Repeated events around `0.02-0.04`	Passed
Yaw	Approximately `-48.8` to `+31.2` degrees	Responsive
Pitch	Approximately `-46.1` to `+4.1` degrees	Responsive
Roll	Approximately `-13.9` to `+12.6` degrees	Responsive
Throughput	Generally near 30 camera and tracking FPS	Passed

Very large head turns or tilts briefly produced `face_lost` , followed by automatic recovery. This is expected when eyes and central facial features become too oblique or leave the useful camera view; smoothing cannot reconstruct a genuinely missing observation.

Measurement cautions

- Tracking FPS is callback throughput, not model-only inference time.
- `submitted - results` combines current in-flight and inputs MediaPipe may drop.
- Eye openness is derived from blendshapes and still needs user calibration.
- Lens glare or thick frames can reduce eye/blink accuracy even when face detection remains stable.
- Wink classification therefore combines an open-eye requirement with a minimum left/right difference instead of requiring the winked eye to reach the bilateral closed threshold.
- Euler angles are a debug view; the textured renderer converts them into its model matrix.

References

- `src/kardboard_vtuber/cli.py:1`

- `src/kardboard_vtuber/tracking/mediapipe_tracker.py:44-232`
- `src/kardboard_vtuber/tracking/models.py:58-90`
- `src/kardboard_vtuber/tracking/events.py:1-171`
- `src/kardboard_vtuber/camera/stream.py:114-141`
- `tests/test_tracking_events.py:1-102`
- `tests/test_tracking_models.py:1`

⬅️ Normalized state · 🏠 Face tracking

PART VIII

Quality and Testing

Deterministic tests, GPU checks, hardware probes, privacy validation, measured performance, and the evidence required before behavior is called complete.

41

PART VIII · QUALITY AND TESTING

CHAPTER 41

Quality and testing

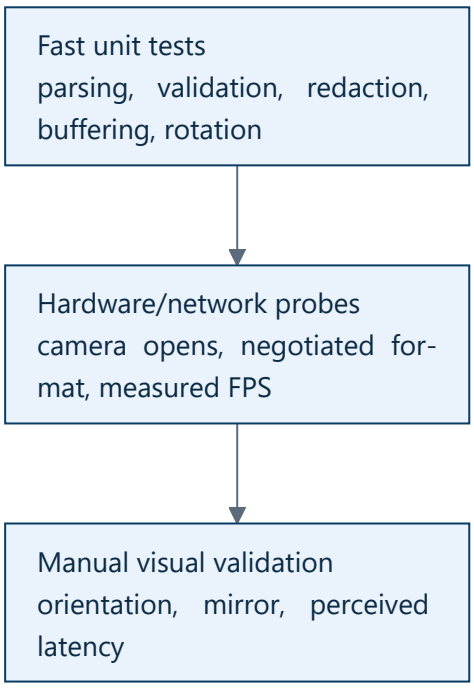
PART VIII · QUALITY AND TESTING Chapter 41

07 · Quality and testing

Status: 108 camera, tracking, motion, renderer, segmentation, body, occlusion, and CLI tests implemented and passing.

TL;DR — The project separates deterministic unit tests from hardware integration checks. Fakes prove deterministic contracts without hardware, offscreen rendering tests exercise GPU output, and real probes verify device behavior that mocks cannot establish.

Test pyramid



Unit-test inventory

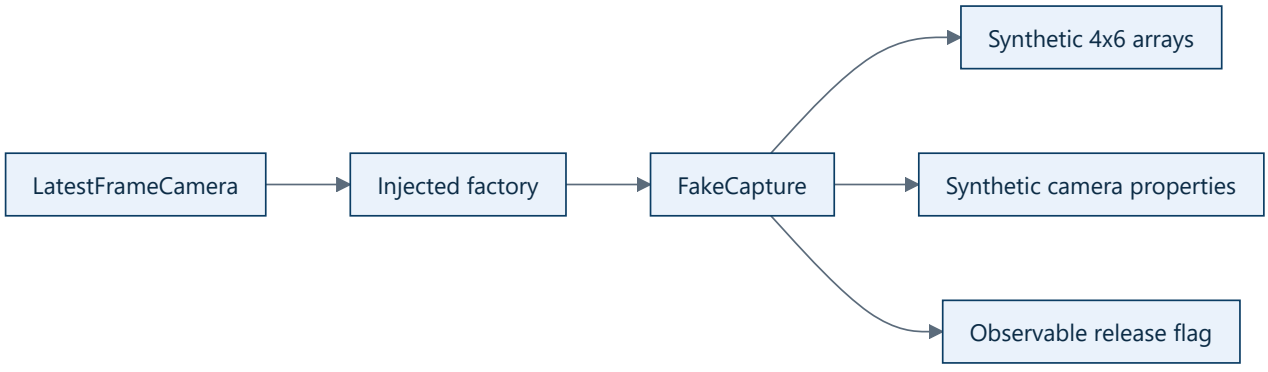
Test	Purpose	Source
Device-index parsing	`"2"` becomes integer `2`	`tests/test_camera_models.py:6-10`
URL preservation	Stream URL remains a string	`tests/test_camera_models.py:13-17`
Credential redaction	User info is hidden	`tests/test_camera_models.py:20-24`
Empty-source rejection	Invalid input fails early	`tests/test_camera_models.py:27-30`

		`-34`
Newer sequences	Consumer receives increasing sequence	`tests/test_camera_stream.py:47-64`
Frame overwrite	Producer can supersede unread frame	`tests/test_camera_stream.py:67-80`
Left rotation	Width/height swap correctly	`tests/test_camera_stream.py:83-98`
Identity head pose	Matrix decomposition has neutral output	`tests/test_tracking_models.py:25-33`
Matrix validation	Non-4x4 pose input fails explicitly	`tests/test_tracking_models.py:36-38`
Face normalization	Bounds, independent eyes, and mouth values	`tests/test_tracking_models.py:41-66`
No-face normalization	Neutral fallback has no landmarks	`tests/test_tracking_models.py:69-79`
Blendshape clamping	Invalid and out-of-range values are bounded	`tests/test_tracking_models.py:82-101`
Debug rendering	Overlay changes a blank frame	`tests/test_tracking_models.py:104-122`

Pytest reports fifteen tests because the empty-source test is parameterized with two inputs.

`FakeCapture`

`FakeCapture` at `tests/test_camera_stream.py:13-44` implements the same structural protocol used by production:



The fake sleeps briefly per read so the real worker thread can publish multiple frames.

What unit tests cannot prove

- Whether a Windows driver honors requested resolution.

- Whether FFmpeg can decode a particular phone stream.
- True network latency.
- Visual orientation and mirror preference.
- OBS capture behavior.

Those require integration or manual validation.

Verified commands

```
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe -m pytest
.\.venv\Scripts\python.exe -m kardboard_vtuber --help
```

Current verified result: Ruff passes and all fifteen tests pass in both the Python 3.12 tracking environment and Python 3.13 base environment.

Guided full-body regression recording

`scripts/record_guided_regression.py --full-body` records a private 93-second routine covering front calibration, complete clockwise and counterclockwise rotations, both profiles, two back holds, leaning, crouching, raised arms, head occlusion, and free motion. The ignored `recordings/` directory receives:

- the unmodified full-resolution MP4;
- synchronized face telemetry;
- synchronized 33-point pose telemetry encoded in the CSV `pose_landmarks` field.

The preview shows instructions over the camera and opens a separate synthetic body-skeleton window. Pose inference defaults to 320-pixel input at no more than 10 FPS so recording and face tracking retain most of the camera throughput.

```
.\.venv312\Scripts\python.exe scripts\record_guided_regression.py `
  --source "YOUR_CAMERA_URL" `
  --mirror `
  --full-body `
  --name "KardboardCode-full-body-rotation"
```

For an unguided recording with coloured clothing markers, add `--free-recording` and choose the duration:

```
.\.venv312\Scripts\python.exe scripts\record_guided_regression.py `
  --source "YOUR_CAMERA_URL" `
  --mirror `
  --full-body `
  --free-recording `
  --duration 90 `
  --name "KardboardCode-colour-marker-session"
```

Definition of done for a new behavior

1. Model/config contract updated.
2. Runtime behavior wired.
3. CLI or API surface updated where applicable.
4. Deterministic test added.
5. Matching book chapter updated.
6. Ruff and pytest pass.
7. Hardware behavior checked if the change touches a real backend.

⬅️ Face tracking · ➡️ Roadmap

42

PART VIII · QUALITY AND TESTING

CHAPTER 42

Validation evidence and known limits

PART VIII · QUALITY AND TESTING Chapter 42

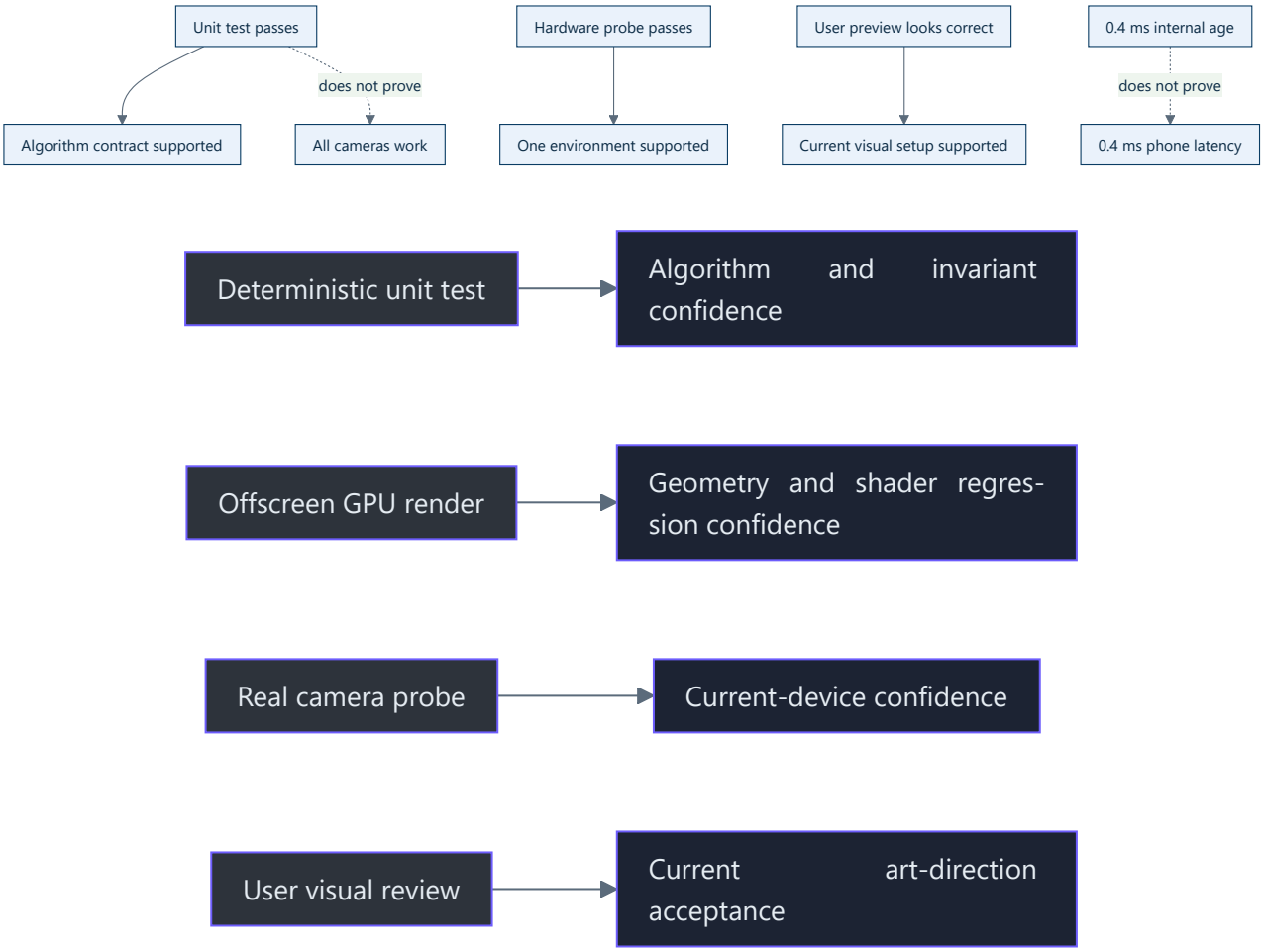
Validation evidence and known limits

> **TL;DR** — Validation evidence is recorded with scope. A passing fake-capture test proves algorithm behavior; a five-second device probe proves only that device/backend combination at that moment.

Evidence matrix

Claim	Evidence	Confidence
Source parsing works	Unit tests	High
Credentials are redacted in diagnostics	Unit test + runtime output	High
Latest-frame overwrite occurs	Threaded fake test	High
Left rotation swaps dimensions	Unit test	High
Integrated camera works with `auto`	Runtime probe	Machine-specific
Phone `/video` works	Authenticated runtime probe	Current phone/network
Final portrait is upright	Temporary frame inspection + user preview	High for current placement
Phone preview is about 28-30 FPS	Runtime output and user screenshot	Current setup
Textured shell has six coherent sides and complete front	Geometry and render regression tests	High
Five flap hinges respond independently	Spring and renderer tests	High
Green screen hides the room before a fresh mask	Unit tests + recorded-frame validation	High
Tracking diagnostics are hidden by default	CLI tests	High
Positive box depth is not artificially capped	CLI and renderer tests	High
End-to-end latency is sub-millisecond	**Not established**	Do not claim

Why evidence scope matters



Current gaps

- No long-duration soak test.
- No automated reconnect test with a failure-scripted fake.
- No end-to-end latency measurement using a visible timer or LED method.
- No CI workflow yet.
- No long-duration MediaPipe soak test yet.
- No committed pixel-for-pixel golden images; renderer assertions inspect geometry, masks, atlas regions, pixels, and measured motion instead.

Recommended next quality work

1. Add scripted read failures and assert reconnect transitions.
2. Add tests for right and 180-degree rotations plus rotate-then-mirror ordering.
3. Add a 30-minute phone-stream soak tool.
4. Add Markdown link and Mermaid validation.

5. Add CI for Ruff, pytest, and documentation checks.

Current automated coverage

The 2026-11-02 validated suite contains 266 tests spanning camera contracts and lifecycle, face-state normalization and actions, One Euro filtering, spring integration, procedural and textured renderers, experimental body tracking, hand occlusion, green-screen fail-closed behavior, virtual output, native widgets, calibration, safe presets, bounded frame/log ownership, recorder cancellation, branding assets, source-release integrity, frame-complete video export and CLI compatibility (`tests/`).

Desktop studio evidence - 2026-11-02

- Native camera selection is exercised through backend-specific metadata, strict inventory parsing, source/backend routing, mode switching, disappeared-device handling and a real asynchronous Windows inventory job. This machine returned Integrated Camera under MSMF and DirectShow and an OBS virtual-device entry. No camera streams were opened during these checks.
- The control-inventory test compares actual parser destinations against both declarative fields and constructed widgets: all 45 current camera options are represented.
- Live per-eye tests exercise both procedural and actual GPU expressions, retain safe loss holds, and prevent a fake blink across a configuration edit. Invalid edits retain the last valid tuple.
- Fake capture/tracker/output tests exercise the real CLI loop with GUI hooks, clean export before raw/debug overlays, stale-face freeze, no worker-thread signal/HighGUI calls, and partial-init cleanup. Cancellation during model initialization does not subsequently open capture or virtual output.
- Native widget tests cover camera-free startup, raw consent, source-free preset loading, tutorial navigation, live controls while running, heartbeat delivery, safe close/stop, detached QImage ownership, and an asynchronous offline model checksum verification.
- An actual offscreen GUI session replayed the private canonical recording with MediaPipe and the textured GPU renderer: 55 display packets, 54 effective detections, normal exit. No images were exported. These are integration counts, not a positioning/privacy accuracy score.
- `scripts\capture_studio_screenshots.py` captures real dark/light/compact/tutorial widgets and illustrative native-camera controls using only synthetic imagery. The screenshots are inspected before being linked from README.
- A six-second private-recording replay with the real GPU/MediaPipe pipeline delivered 117 GUI preview packets, 116 effective detections, and a successful live threshold edit. After a one-second warm-up following the first preview, 359 samples of a nominal 10 ms Qt heartbeat had a **31 ms 95th-percentile** interval and **47 ms maximum**. This measures service of the UI event loop

on this machine, not input-to-photon or phone latency. Face inference used 640-pixel input; higher-cost combinations still need their own qualification.

- An earlier separate screenshot run included initialization, theme swaps and widget/PNG capture and recorded a 375 ms maximum heartbeat interval. Do not present the steady replay measurement as a universal worst-case guarantee.

Fifteen-point Studio feedback - v1.1.0

- GUI tests verify independent program/raw/mesh buffers and the full-resolution clean-window channel. The actual shared live loop exports before local diagnostics, including stale-face hold. Experimental body-calibration masks also omit setup text from Studio/virtual clean output.
- Native checkbox tests inspect real rendered indicator pixels in both palettes, not just stylesheet text. Locked/live state badges, standard resolution routing, bottom full-command copy, video mode and a real asynchronous file-export worker are exercised.
- Eye regressions cover the reported 0.51/0.66 closed-eye score pattern after explicit OPEN/CLOSED calibration, both procedural and actual GPU rendering, retained geometry filtering and rejected overlapping calibration ranges.
- Genuinely encoded synthetic VFR files verify every frame, unchanged source dimensions, exact lossless processed RGB pixels, source timestamp spacing and copied PCM/AAC packet payloads. MP4 remains explicitly lossy. Tests reject high-bit-depth conversion, clean cancellation and preserve outputs created by a concurrent writer.
- Actual synchronous MediaPipe VIDEO mode plus the GPU processed a private 30-frame recorded clip at 1080x1920, ending with face detection present. Temporary input/output slices were removed. No live camera opened and no private imagery entered public screenshots.
- Eight refreshed native screenshots use only synthetic imagery/illustrative paths. The diagnostic panel uses a visibly synthetic checkerboard, not a human face. Capture-run heartbeat maxima include initialization, layout/theme changes, PNG grabs and concurrent checks; they are not steady-state latency measurements or universal UI guarantees.
- A separate four-second native demo run, after warm-up, recorded 161 nominal-10ms heartbeat intervals: p95 31ms, maximum 32ms. These are local UI service intervals on this machine, not end-to-end video latency or a real-camera/all-feature performance guarantee.
- README milestone cards are absent; transparent section/footer SVGs contain font outlines without raster embedding or a canvas background. Desktop/mobile browser checks load every image without horizontal overflow, with the restyled runtime hero immediately after the top banner.

First official release branding - 2026-11-02

- v1.0.0 adopts the supplied voxel KC box artwork; earlier v0.2.0 was unpublished local staging. The original 1345x1169 transparent PNG is retained byte-for-byte, not redrawn.
- Native Windows Qt decodes all seven derived ICO sizes from 16 through 256 pixels. Tests exercise transparent-color resampling, unchanged master bytes, repeatability and every

embedded PNG frame. No extra image-processing dependency was introduced.

- Refreshed screenshots show the new header identity on actual widgets. The README retains one tool screenshot above Current milestone, with the separate official logo above it.
- Release checks cover source/version alignment, privacy exclusions, ZIP contents, checksums and Git-stored byte stability. These checks do not establish live calling-app compatibility.

The full suite and Ruff are reproducible with the existing commands in the command reference. GUI tests skip only when the optional Qt dependency is not installed; install the GUI extra to include them. No live camera or real Discord/Teams/Meet call was used to qualify this GUI delivery.

🔙 Quality chapter · 🏠 Book home

PART IX

Roadmap

Completed milestones, remaining production-hardening work, acceptance gates, and the path from a working avatar to a dependable streaming tool.

43

PART IX · ROADMAP

CHAPTER 43

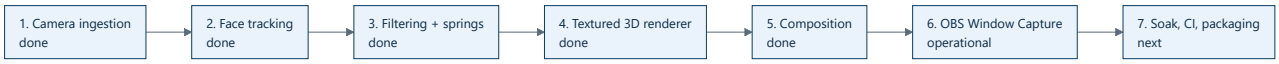
Roadmap: from working avatar to production hardening

PART IX · ROADMAP Chapter 43

08 · Roadmap: from working avatar to production hardening

> **TL;DR** — Camera ingestion, tracking, calibration, One Euro filtering, and spring dynamics are complete. The textured renderer, five-hinge physics, body mode, hand occlusion, and green-screen output are implemented. The remaining work is reliability, packaging, and optional transport.

Delivery sequence



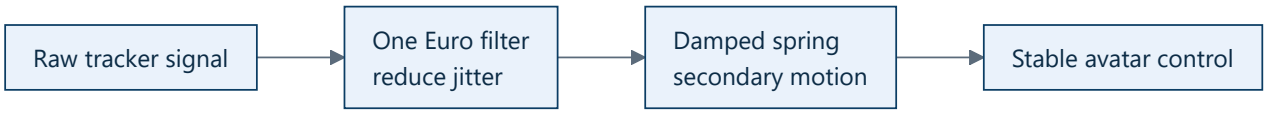
Milestone 2 · Face tracking — complete

Implemented responsibilities:

- Use MediaPipe Face Landmarker with one face.
- Process a downscaled latest frame asynchronously.
- Produce a library-neutral `NormalizedFaceState`.
- Extract head pose, left/right eye openness, and mouth openness.
- Emit live debug landmarks and diagnostics.

Personal expression calibration is complete. Tracking runs in `.venv312` because the optional dependency remains restricted below Python 3.13 (`pyproject.toml:21-23`).

Milestone 3 · Filtering and motion — complete



Implemented signal groups use separate tuning:

- Head translation and rotation.
- Left `K` eye.
- Right `C` eye.
- Mouth signal and action events; mouth-art motion remains deferred.
- Three underside hinges and two highly yaw-sensitive external side tabs.

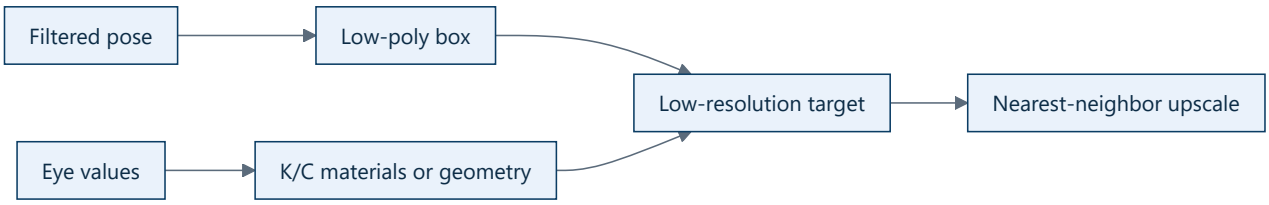
One Euro filtering is wired into tracking. Damped springs drive five renderer hinges. See One Euro motion filtering and Damped spring integration.

Milestone 4 · PS1 renderer — complete

The default renderer now:

- uses ModernGL and the Windows GPU for real depth-tested 3D rendering;
- generates a complete-front cubic box, neck-safe underside, flaps, corrugated edge layers, earcups, cushions, and headband;
- drives calibrated pitch/yaw/roll and anatomical K/C wink states;
- renders a transparent low-resolution framebuffer and composites it over the sharp camera;
- preserves black-before-acquisition and last-safe-frame tracking-loss behavior;
- includes aged asymmetric shipping labels, barcodes, package IDs, arrows, and top `FRAGILE`;
- exposes configurable perspective depth and five optional spring hinges;
- falls back to the procedural renderer if OpenGL initialization fails.

The current design is the accepted visual baseline. Future art changes should preserve its privacy volume, complete front face, neck channel, atlas assignments, and hinge identities.



Milestone 5 · Composition — complete

The high-resolution camera frame remains sharp while the avatar stays pixelated. Full-body pose appears only in a separate line-skeleton diagnostic window, hand/forearm masking can restore a bounded foreground, and person segmentation can replace the room with chroma green.

Milestone 6 · OBS — operational

Window Capture consumes the clean preview today. `--green-screen` provides a straightforward OBS chroma-key workflow. `--virtual-camera` can also send the composed output directly to the installed Windows OBS camera component for calling apps, without OBS running. Spout2 remains optional rather than required.

Remaining work

1. Add CI for Ruff, pytest, Markdown links, and Mermaid parsing.
2. Run long camera, tracking, segmentation, and OBS soak tests.
3. Package verified model-download and first-run setup.

4. Measure true end-to-end latency with an external visual clock.
5. Consider Spout2 only if Window Capture and chroma key become limiting.

Acceptance gates

Milestone	Gate
Tracking	Stable pose and independent eyes/mouth on recorded test clips
Filtering	Low jitter without visible lag
Renderer	Recognizable KardboardCode identity at target frame rate
Composition	Correct placement during translation and rotation
OBS	Stable capture at intended stream resolution

⬅️ Quality and testing · ➡️ Appendix

PART X

Desktop Studio

Native interaction, source-grounded toolkit and layout decisions, live eye calibration, bounded worker ownership, privacy-aware presets, recording, and a complete tutorial.

44

PART X · DESKTOP STUDIO

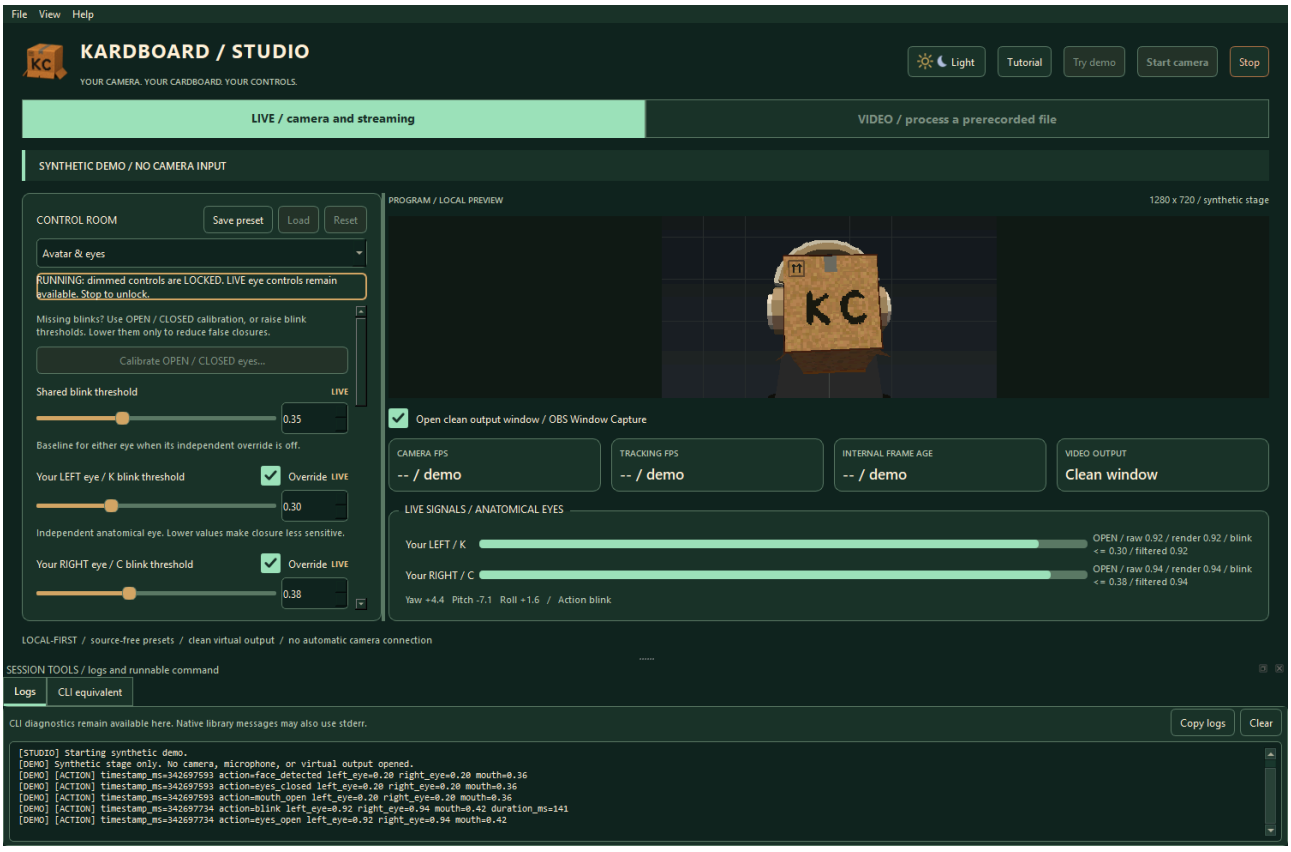
CHAPTER 44

Desktop Studio

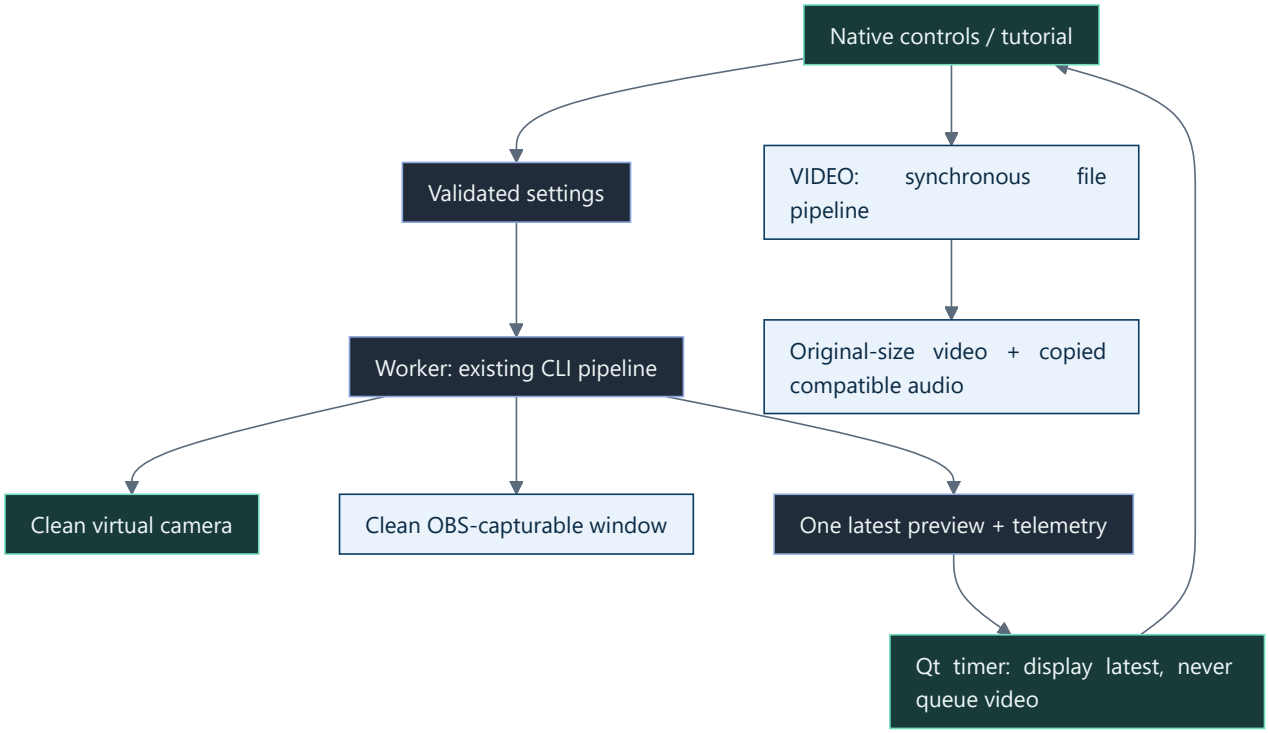
PART X · DESKTOP STUDIO Chapter 44

Desktop Studio

> **TL;DR:** The desktop studio is an optional native PySide6 interface to the same camera live pipeline used by the CLI, plus a frame-complete offline VIDEO workspace. This chapter covers both workflows, separate clean output/diagnostics, calibration, ownership and evidence.



Actual Qt widgets displaying the real textured renderer on a synthetic, camera-free stage. The independent thresholds shown are illustrative demo settings, not recommended personal calibration.



Reading order

- Research and design: five investigation lenses and toolkit decision.
- Interface specification: every control and its placement rationale.
- Runtime architecture: threading, ownership, errors, shutdown and calibration.
- Video processing: synchronous frames, lossless/lossy formats, timing/audio and safety.
- Windows installation: online MSI, private Python, integrity and resource ownership.
- Tutorial: both modes, clean window, calibration, private capture and troubleshooting.

Research findings describe the pre-GUI source; implementation/evidence sections are updated as work completes. Neither a synthetic screenshot nor a passing test establishes live camera privacy or calling-app compatibility.

45

PART X · DESKTOP STUDIO

CHAPTER 45

Desktop GUI research and design

PART X · DESKTOP STUDIO Chapter 45

Desktop GUI research and design

> Research date: 2026-11-02. Scope: a native, responsive, all-feature Python studio that retains the existing CLI and its privacy/output semantics. The user explicitly requested a tutorial. Confidence labels describe the evidence, not an assertion that the delivered GUI is fast.

Iteration 1 - Structural survey

HIGH, source-confirmed: `cli.build_parser` owns the operational flags (`src\kardboard_vtuber\cli.py:35-285` at research time). `cli.main` constructs tracking, renderer and output resources, then performs a continuous capture/composition loop (`cli.py:288-630`). The GUI must not run this loop on the widget thread.

HIGH: There are already two expression consumers: `FaceActionDetector._classify_eyes` (`tracking\events.py`) and each renderer's eye logic (`renderer\ps1_cardboard.py`, `renderer\textured_3d.py`). A slider connected only to the detector would change logs while leaving the visible K/C expression unchanged. This is the most important calibration trap.

HIGH: Model downloads and guided/free regression recording are separate scripts (`scripts\download_*_model.py`, `scripts\record_guided_regression.py:113-203`). An all-feature GUI must expose these too, rather than equating "all" with camera preview.

Next lens: trace pixel, control, and telemetry ownership through the loop.

Iteration 2 - Data flow and state

HIGH: The source camera and the adjusted tracking image are deliberately separate. `cli.main` makes the brightness-adjusted processing frame before tracker submission but starts composition from the original packet copy (`cli.py:431-465,500-548` at research time). The GUI's brightness slider must retain this distinction.

HIGH: Virtual-camera output precedes local diagnostics and raw-face preview (`cli.py:547-599`). `VirtualCameraOutput.send/repeat` owns the repeated output buffer (`virtual_camera.py:68-104`). A GUI cannot forward a widget screenshot to virtual output.

HIGH: `LatestFrameCamera.read` waits on a condition and returns the newest packet; `stop` joins the capture thread with a timeout (`camera\stream.py:99-143`). Closing a window cannot synchronously wait for this on the event thread.

Decision / engineering inference: use one immutable live-settings snapshot and one latest preview mailbox, plus a bounded log ring. The UI polls this mailbox at approximately 30 Hz. This intentionally drops superseded previews; the sender still receives its own full-size clean frames. Boundedness is established structurally; achieved responsiveness needs measurement.

Next lens: compare toolkit integration and dependency costs.

Iteration 3 - Toolkit and external contracts

Candidate	Verified capabilities	Project-specific tradeoff
PySide6 / Qt Widgets	Native widgets, layouts, accessibility/keyboard controls, timers, threads, image buffers, dialogs	Extra binary dependency; thread affinity must be respected; distribution licensing needs review
Tk/ttk	Standard Python GUI interface, themed controls, native dialogs, event-loop model	Viable smaller UI; richer split layouts, calibration controls, image display and styling require more project-owned work
Dear PyGui	GPU-driven UI and explicit render loop	Strong tool/debug UI; introduces a second graphics/render-loop abstraction beside ModernGL; not evidence of faster end-to-end video

Primary sources, read on the research date:

1. Qt QThread: a `QThread` object lives in its creator thread; `run()` executes separately. Queued slots on a subclass are not automatically worker-thread commands. Forced termination is discouraged.
2. Qt QTimer: timers belong to their event-loop thread; deadlines can slip under load. A nominal 33 ms timer does not guarantee 30 displayed frames per second.
3. Qt QImage: direct pixel access and image ownership differ from screen-oriented QPixmap.
4. Qt QPlainTextEdit: simplified layout and `maximumBlockCount` suit a bounded plain-text log.
5. Python Tkinter: Tcl/Tk event queues and threading rules still require work outside the event callback.
6. Dear PyGui render loop: expensive work in the render loop reduces UI frame rate.
7. Qt for Python licensing: community and commercial distributions have different licensing terms. Adding the community dependency is not a legal determination about a future packaged distribution.

Decision: optional `PySide6-Essentials` (QtCore/QtGui/QtWidgets and image/icon loading), not a browser server and not a mandatory CLI dependency. Widgets provide the sliders, numeric editing, splitter, logs, dialogs and tutorial without an HTML bridge or per-frame JavaScript encoding. This is a fit decision, not an unmeasured speed ranking.

Next lens: failure containment, interaction risks, and maintainability.

Iteration 4 - Patterns, risks, and anti-patterns

Avoid a second pipeline: reuse `cli.main` through optional runtime hooks. Keep ordinary CLI behavior when hooks are absent. Do not copy segmentation, hand restoration, freeze, fallback resolution or clean-export ordering into a separate Qt compositor.

Avoid one queued signal per frame: even a responsive producer can accumulate stale events in a slow UI. Publish only the latest resized preview. Copy the QImage before the NumPy owner can be replaced. Bound worker logs and displayed text independently.

Avoid implicit capture: opening the GUI, loading a preset or opening a tutorial must not start a camera. The demo must use synthetic imagery and say so visibly.

Avoid hidden live/restart mismatch: independent eye thresholds and internal brightness are live. Resource topology, source, resolution, models, mirroring, output format and tracking modes are restart settings. Keep live widgets enabled during work; lock or clearly mark restart-only settings instead of silently deferring them.

Avoid unsafe persistence: source URLs and raw-preview consent are session-only. Save only an allowlisted settings schema. Imported presets are data, not commands; do not evaluate arbitrary Python or spawn an imported command. Commands/logs redact URLs before display/export.

Keep known limitations visible: body/hood modes are experimental; no-face/no-marker does not prove rear orientation. Raw debug/recordings are privacy-breaking local operations. The existing fixed-FPS recorder's elapsed-time limitation is not resolved by adding a button.

Remaining qualification: native camera-driver stop latency, high-DPI appearance on other monitors, actual calling apps, and difficult distant-face lighting need owner-environment evidence.

Iteration 5 - Synthesis

Think of the studio as a control surface beside the existing machine, not a replacement machine. Qt owns interaction and display. The worker owns capture orchestration, inference consumers, the OpenGL context, composition, and camera delivery.

What that mental model hides: MediaPipe and OpenCV already have their own asynchronous/native work; a Python thread is not process-level crash isolation. A stuck native driver cannot be made safely cancellable by `QThread.terminate`. During shutdown the window must remain responsive and honestly show "Stopping" rather than destroy the thread or claim resources are released.

Key findings:

1. **HIGH:** tuning must reach both actions and rendering, not only logs (`FaceActionDetector`, `PS1CardboardRenderer`, `Textured3DCardboardRenderer`).
2. **HIGH:** export-before-debug is a required privacy boundary (`cli.main`, `VirtualCameraOutput`), independent of the UI layout.
3. **HIGH:** all-feature coverage includes the recorder/model scripts, not just parser flags.
4. **Design decision:** progressive disclosure by task keeps the default face-only workflow obvious without removing advanced flags requested by the user.
5. **Design decision:** a fixed-location Stop button, explicit raw consent, named anatomical eyes and nearby live meters reduce operational ambiguity.

6. **Unknown until measured:** actual UI heartbeat under inference, platform-specific widget sizing, and native-runtime cleanup. Tests/screenshots must record evidence, not aspirations.

Knowledge map after research

- **Explored:** parser, main loop, per-eye consumers, frame ownership, native toolkit contracts, recorder interface, capture stop, export ordering, documentation structure.
- **Partially explored:** native-driver cancellation and distribution packaging; no universal camera cancellation or installer behavior is claimed.
- **Unexplored / deliberately deferred:** alternate inference engines, new body-tracking fixes, a separately named Windows camera driver, and changes to recording timing.
- **Implementation direction:** one bounded runtime bridge, shared eye tuning, native widget controls, optional GUI extra, source-free presets, explicit private recording, and a synthetic demo/tutorial that works without a camera.

46

PART X · DESKTOP STUDIO

CHAPTER 46

Interface specification and placement rationale

PART X · DESKTOP STUDIO Chapter 46

Interface specification and placement rationale

> The first release exposes **all existing features**, including experimental flags and private recording, as requested. Face-only cardboard remains the default; visibility is not endorsement.

Spatial model

The header contains identity, theme, tutorial, stable Start/Stop actions and two prominent modes: **LIVE** for cameras/streaming and **VIDEO** for applying the avatar to an existing file. Settings sit left. The central program image stays clean; private raw face and synthetic face/body diagnostics occupy separate side panels. Bottom tabs contain **Logs** and the **CLI equivalent**, with one-click copying of the complete command.

Why these positions:

- **Start/Stop stay in the header:** changing settings pages must not hide the emergency stop. Stop is visually distinct from Start, never a destructive “kill thread” action.
- **Source is first:** most startup failures are source/model/output setup, not eye calibration.
- **Avatar/eyes sit next to the preview:** compare a left/right slider edit with the K/C result, using anatomical names rather than ambiguous mirrored screen-side labels.
- **Output is separate from input:** capture requests and virtual-camera dimensions are different contracts; changing one must not look like changing the other.
- **Experimental tracking gets its own page and warnings:** all features remain accessible without presenting hood/body estimates as the normal or privacy-certified path.
- **VIDEO is a front-level task:** input/output files, lossless/MP4 quality and progress are separate from live capture. The old raw recorder remains a consent-gated **Private capture** utility in Live mode, not the meaning of video processing.
- **Diagnostics never cover the program:** dedicated side panels preserve the complete central image. A separate clean output window supports OBS Window Capture without exposing Studio.
- **Model setup is discoverable:** missing files need actionable setup, not a cryptic traceback.
- **Tutorial is a persistent header action and a full page:** it is available before camera access, not a first-run modal that blocks the interface or disappears forever.
- **Logs remain selectable plain text:** no HTML interpretation, bounded history, explicit clearing and safe copying. Rich styling is reserved for status, not log formatting.

Complete runtime control inventory

Page	CLI fields / feature	Interaction

Camera	`source`	Computer-camera chooser with names/Refresh, or masked URL/file/manual-index entry
Camera	`backend`, `width`, `height`, `fps`	Choice and numeric requests; automatic values clearly labelled
Camera	`rotate`, `mirror`, `input_already_mirrored`	Rotation choice and separate checkboxes
Camera	`brightness`	Live internal-only slider with numeric value
Camera	`duration`, `preview_height`, `headless`	Optional duration, display-only height, preview suppression
Avatar / eyes	`render_cardboard`, `track_face`, `cardboard_renderer`, `physics`	Independent features and renderer selection
Avatar / eyes	`box_depth_offset`	Numeric perspective depth offset; no artificial positive cap
Avatar / eyes	`blink_threshold`, independent left/right overrides	Shared baseline plus live per-eye sliders/numeric editing
Avatar / eyes	`wink_threshold`, `eye_open_threshold`, `wink_min_difference`	Live calibrated controls with validation
Avatar / eyes	`action_hold_ms`, `eye_action_hold_ms`	Live debounce values
Avatar / eyes	`no_motion_filter`	Explicit smoothing toggle, restart-only
Output	`virtual_camera`, `virtual_camera_width`, `virtual_camera_height`, `virtual_camera_fps`	Enable, standard/monitor resolution preset and advertised FPS; width/height remain CLI fields
Output	`green_screen`, `segmentation_width`, `segmentation_model`	Feature toggle, input width, model file
Output	`hand_occlusion`, `hand_tracking_width`, `hand_model`	Anatomical foreground toggle, width, model file
Tracking labs	`full_body`, `pose_tracking_width`, `pose_model`	Separate skeleton diagnostics and pose configuration
Tracking labs	`body_head_fallback`, `hood_marker_tracking`	Explicit experimental toggles; reject their invalid combination
Tracking labs	`tracking_debug`, `debug_face_preview`	Local diagnostics and explicit raw-face consent

Models	`face_model`, optional model paths	Browse fields, existence indicators and checksum-verifying download actions

The new independent-eye CLI overrides preserve the original shared `--blink-threshold` default. They do not change anatomical K-left/C-right mapping.

Video processing and private capture

VIDEO offers a local input, a new output filename, lossless FFV1/MKV by default and explicitly lossy H.264/MP4 CRF 17. It processes every frame, retains source dimensions and timestamp spacing within container precision, and copies compatible audio streams. Live camera transforms, virtual output and experimental body/marker modes are visibly unavailable. Every processing setting locks during export for a deterministic result. See video processing.

Source/backend/rotation/mirror/brightness and face/pose model selections are reused from the current setup. The advanced private-capture page exposes `full_body`, `free_recording`, `duration`, `tracking_width`, `pose_tracking_width`, `pose_tracking_fps`, `output_dir`, `name`, `countdown`, and `preview_height`. These map to `scripts\record_guided_regression.py`, not to an invented output format.

Recording is mutually exclusive with the GUI's live/demo pipeline. Its raw preview remains the existing recorder window; the native studio remains available for logs and lifecycle control. The interface does not silently record the safe preview instead of the documented raw regression data.

State and validation

- Opening: no source connection, no output, no raw consent. Synthetic demo is explicit.
- Starting: disable duplicate starts, show initialization, retain Stop.
- Running: enable live tuning; restart-only settings show **LOCKED / STOP TO EDIT**, muted fields and a persistent explanation. Offline-only/live-only controls state their scope.
- Stopping: request cooperative shutdown; keep the window event loop running.
- Stopped/error: clear live video to an explicit idle/error state, release owned resources, enable setup.
- Invalid tuning: show the exact invariant; retain the last applied valid snapshot.
- Raw debug or unmasked-camera mode: acknowledgement before a real camera run, never stored in presets.
- Imported preset: source excluded; raw-preview consent always resets off.

Tutorial requirements

The in-app guide must explain first launch/model setup, source input, transformations, safe preview, anatomical eye calibration, live/restart badges, virtual-camera selection/single-producer cons-

traints, green screen/hands, experimental tracking, recording privacy, source-free presets, logs, and Stop. The same narrative must be readable in the book tutorial.

Calibration and capture routes

Guided calibration samples relaxed OPEN eyes, then BOTH CLOSED eyes after a countdown. Only numeric observations are retained. The complete compatible threshold tuple applies atomically; overlapping samples fail visibly. Higher blink thresholds help missed closures; lower thresholds reduce false closures. The readout shows raw score, effective rendered score and CLOSED/OPEN.

Device index 0 means the first device for an API. Media Foundation and DirectShow are capture backends, often exposing the same integrated camera, not an RGB/IR pair. No physical-device identity is inferred from equal names. The current automatic/manual route remains explicit.

Visual identity

Both themes follow the top banner: forest ink, mint, kraft and cream. The dark theme uses forest surfaces; the light theme uses warm paper. Checked boxes have an explicit contrasting tick; unchecked boxes are empty bordered squares, and disabled states remain distinguishable. Dark slider handles and filled tracks use cardboard brown (`#d5a568`) so frequently touched controls echo the box material. Mint remains reserved for checked/live/primary states rather than making every accent brown. Light sliders retain their deeper green for contrast on cream. Video pixels are never recolored by the UI. The theme button includes both a sun and a moon icon. The window/application/header icon uses the user's supplied voxel **KC cardboard-box artwork**. Its canonical PNG stays unchanged; optical bounds discard near-invisible outer noise in derived icons, which fill their small native slots. Windows controls the title-bar/taskbar slot size. Actual screenshots must be generated from the running widgets, not a design mockup.



Official tool identity for the first official release, v1.0.0. This supplied artwork replaces the earlier hand-authored SVG icon; it does not replace or modify the tracked 3D avatar.

47

PART X · DESKTOP STUDIO

CHAPTER 47

Desktop runtime architecture

PART X · DESKTOP STUDIO Chapter 47

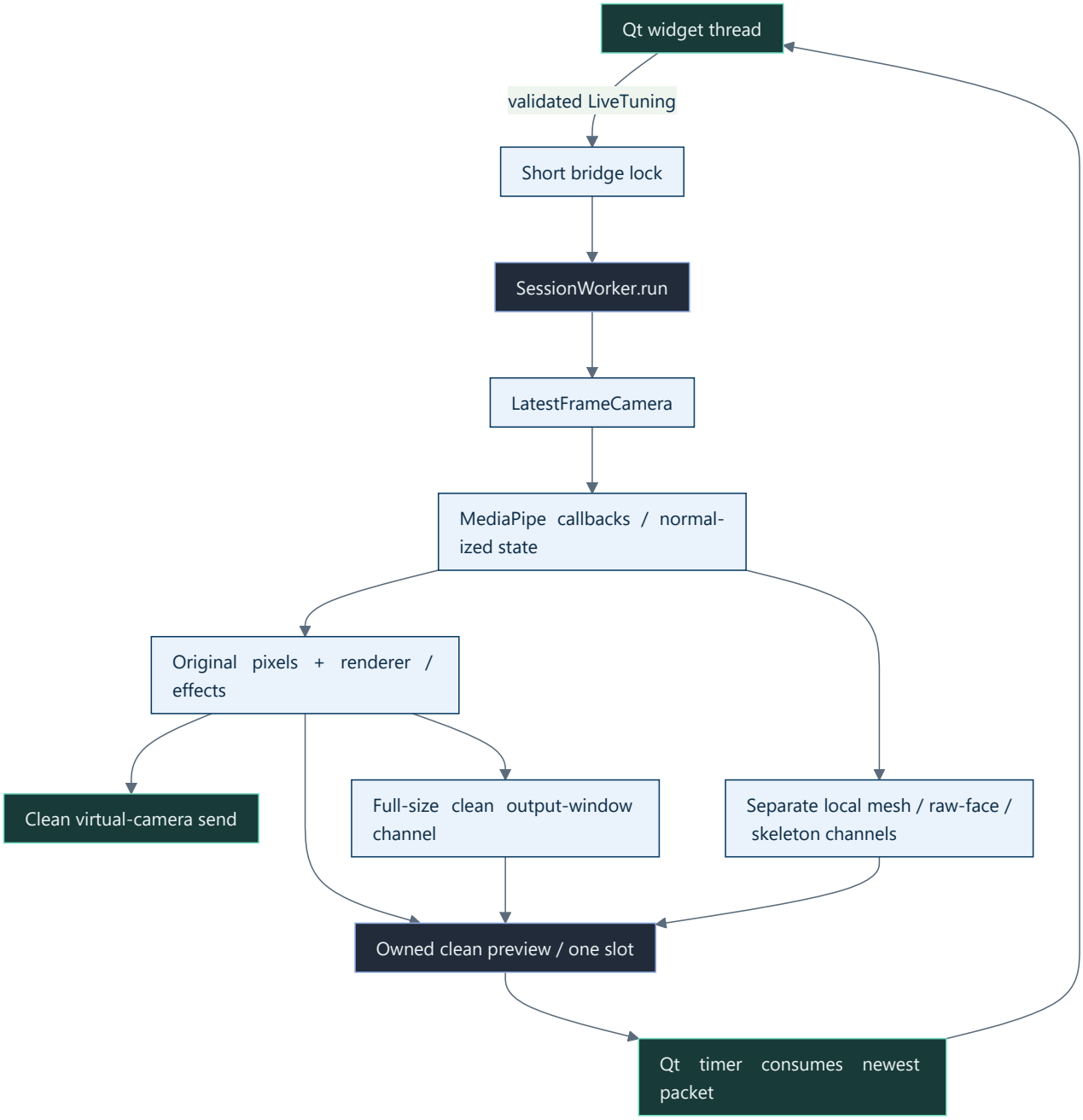
Desktop runtime architecture

> **TL;DR:** Qt owns widgets; a single session worker owns the existing pipeline and its native resources. A toolkit-neutral bridge transfers immutable tuning and one latest preview, not a growing stream of queued frame signals.

One pipeline, two hosts

`cli.main(argv, runtime=None)` remains the CLI host. With no bridge, it retains terminal reporting, SIGINT handling and OpenCV preview windows. `gui.worker.SessionWorker.run` calls the same function with a `RuntimeBridge`: no worker-thread signal-handler installation and no HighGUI windows. The GUI's `--headless` option suppresses local image transfer, not clean virtual output.

The shared `cli.validate_options` validates source, mode combinations, dimensions and tuning before native allocation. `gui.settings.namespace/arguments` reuse this contract rather than maintain separate numeric/mode rules. A coverage test compares parser destinations with every declarative GUI field, so adding a CLI feature without a native control fails visibly. GUI validation disables source-file inspection: model existence and recorded-source detection happen on the worker, not during slider callbacks. Editing controls does not repeatedly touch disk or query an unavailable network path.



Data contracts and ownership

`LiveTuning` contains an immutable `EyeSensitivity`, brightness and debounce intervals. The main thread validates the complete tuple before replacing it under a lock. The worker reads a snapshot between frames, applies eye rules to the action detector and renderer, and uses the chosen brightness for the next processing copy. Invalid edits remain visible but do not replace the last valid tuple.

`RuntimeTelemetry` separates raw/filtered face state, actual rendered eye state, effective detection, measured capture/tracker rates, internal frame age, output device and explicit synthetic/raw flags. The GUI must not infer safe masking solely from `face.detected`: an unmasked mode is deliberately available, and body/marker resolvers can have different state from the face tracker.

`RuntimeBridge.publish` bounds display copies to 1280x800 while preserving aspect ratio. The clean compositor/output stays full-size. A separate full-resolution owned frame feeds the controls-free output window, while mesh/raw-face/skeleton buffers feed local side panels. The mailbox owns its copies; `VideoCanvas.set_frame` also detaches the QImage from NumPy. Replacing a worker buffer therefore cannot change pixels while Qt is painting. No QPixmap or QWidget is touched by the worker.

There are at most one waiting preview packet and one packet currently being consumed. This bound excludes the camera's own latest slot, trackers, renderer targets and virtual output, which retain their existing ownership. Memory is bounded structurally, not claimed constant across different source resolutions.

Live eye calibration

`tracking.eyes.EyeSensitivity` is the shared source of blink/wink rules.

For anatomical eye `e`, a direct closure is `openness[e] ≤ blink_threshold[e]`. Relative wink closure also requires openness at or below the common wink threshold, the opposite eye at or above the common open threshold, and the required left/right asymmetry.

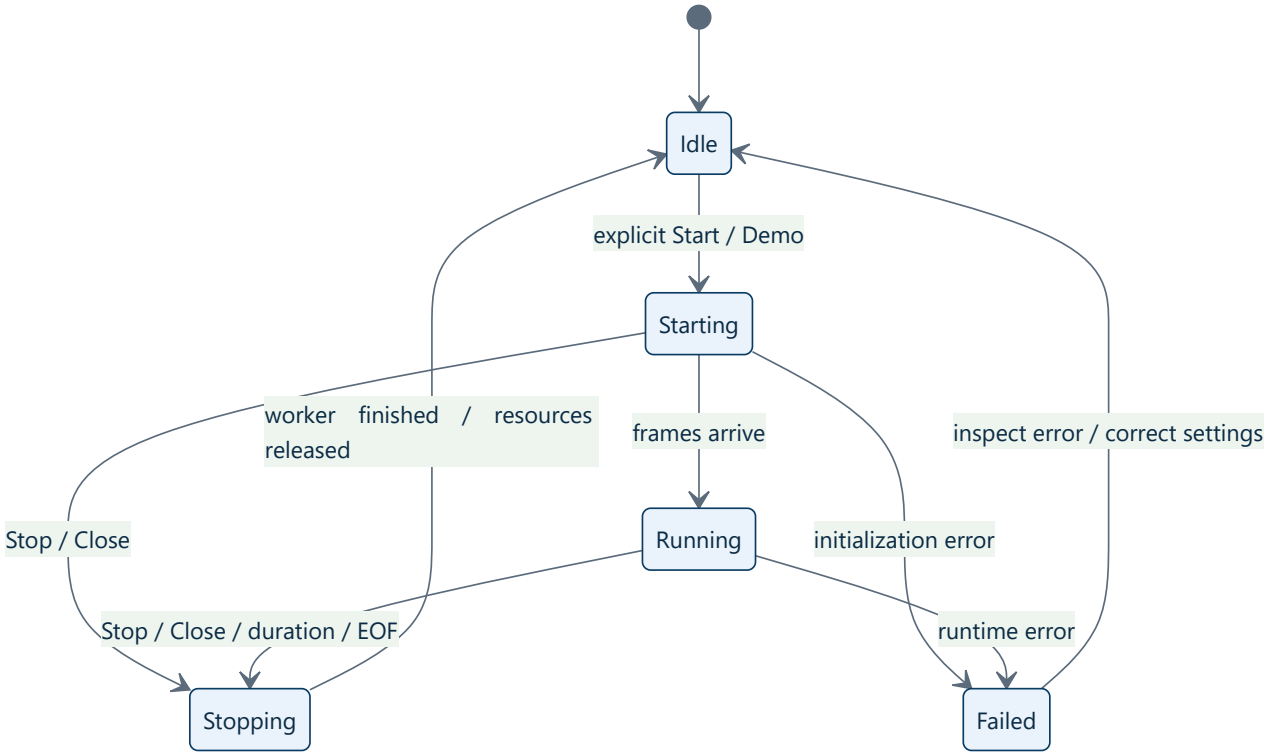
Both effective per-eye blink thresholds must be lower than the open and wink thresholds. Finite values lie in [0,1]. Existing `--blink-threshold` remains the shared fallback; optional `--left-blink-threshold` and `--right-blink-threshold` override only their respective eyes. An unused shared fallback does not override two explicit valid per-eye values.

`FaceActionDetector.set_thresholds` clears eye transition/blink timing when eye calibration changes, avoiding an artificial blink spanning two configurations. It does not reset an unchanged mouth channel. Both renderers accept the same sensitivity without rebuilding GPU resources or discarding the last-safe image. Texture regeneration remains conditional on the resulting open/closed expression changing.

Action logs and visual closure decisions use raw eye controls, while geometry/pose and mouth remain filtered. `render_eye_state` prevents the One Euro filter from smoothing away brief blinks. Both renderers still use the same `EyeSensitivity`. Telemetry carries the actual render state so fallback/held expressions are not misrepresented as raw face decisions.

`calibrate_eyes` uses at least 12 distinct observations for each OPEN/CLOSED phase. Conservative open/closed quantiles must have a useful gap. It derives both per-eye blink thresholds and the shared open/wink/asymmetry rules together. Overlapping ranges fail explicitly, retaining existing settings. `EyeCalibrationDialog` uses a countdown and timed sampling without saving face images.

Lifecycle and failure handling



The worker creates, uses and closes the standalone ModernGL context on the same thread. `ExitStack` registers resource cleanup immediately after each successful acquisition. If later initialization fails, already-created trackers are closed. Output closes before rendering and tracking resources; camera shutdown is also attempted. The outer Qt-thread exception boundary reports unexpected failures to the console, not a success-shaped empty preview.

Stop sets a thread event. Camera reads use short condition-wait timeouts in GUI mode so the loop can observe it without a per-frame queued Qt slot. A native camera open/read/release can still block internally. A cancellation received during model initialization is checked before acquiring output and again before starting capture, so cancelled startup does not open a new source. Closing during work keeps the window alive, requests Stop, and completes after `QThread.finished`; it does not call `terminate`, block the UI with `wait`, or destroy a running QThread. This is cooperative shutdown, not process/crash isolation.

Logs and subprocess tasks

Native machine camera selection

The Camera page explicitly separates **This computer's camera** from custom stream/file/index input. Named Windows discovery uses the optional `cv2-enumerate-cameras` metadata adapter in an owned subprocess, reusing the existing job lifecycle. Its Windows implementations query `MFEnumDeviceSources` or DirectShow's device enumerator/property bags; they do not call

`VideoCapture`, read frames, or activate a media source. No vendor camera client/driver is installed.

`camera.devices.discover_local_cameras` enumerates each supported API explicitly. It does not mix OpenCV's backend-encoded `CAP_ANY` indices with a different explicit backend. A named row contains a validated name, API and index; selecting it updates source/backend together. Device paths/serials are not returned or persisted. Listings are snapshots, so refresh/reselect after hardware changes; a missing previous selection is cleared rather than silently retargeted. Device `0` is a manual fallback, not an assertion that it is always the laptop camera.

The native selector starts no capture merely from switching mode, refreshing, or selecting a row. Custom URL/file input and its backend are retained only in session memory when switching back. Named discovery is currently Windows-specific; manual index capture remains available elsewhere.

Worker logs have a 500-entry ring; an overflow notice explicitly counts dropped older entries. The visible plain-text console retains at most 1,500 text blocks. Camera URLs, including query strings and authentication, are redacted before entering the GUI buffer/clipboard. Native library messages may still use the launching terminal's stderr; this is not a promise of universal third-party log interception.

Checksum-verifying model helpers and the existing recorder run as owned QProcesses. Arguments are constructed from typed controls without a shell. Only one session or job runs at a time. Model verification checks existing files; explicit force download can replace them after checksum verification. Model downloads are network operations only when the user requests them.

The recorder's hidden `--stop-file` is a cooperative cancellation contract: the header Stop creates a marker in an owned temporary directory; the recorder checks it in countdown/capture loops and still flushes its writer/CSV in `finally`. Its raw preview remains a separate recorder window. This UI addition deliberately does not alter the existing recording timing policy.

Persistence and distribution boundary

Presets contain a versioned allowlist of current CLI control values. Source and raw-face preview consent are excluded. Loading never starts capture and resets raw consent. Imported data cannot supply arbitrary command names or executable code. Model fields accept local paths, not stream URLs.

The GUI extra is optional. `python -m kardboard_vtuber` does not import Qt. `python -m kardboard_vtuber.gui` and the `kardboard-studio` GUI entry point launch the native app. The supplied canonical PNG and derived application icons are packaged under `gui\assets`. `scripts\build_branding.py` derives a transparent 256-pixel PNG and a Windows ICO containing 16/24/32/48/64/128/256-pixel images, using the existing OpenCV/NumPy dependencies. It trims only near-invisible outer pixels below alpha 16 from its optical bounds, preserves aspect ratio, uses no extra margin at 16/24/32px, and resamples premultiplied color to avoid colored fringes from invisible pixels. The master PNG is never rewritten. The same multi-resolution icon supplies the app-

lication, window/taskbar and header branding; the README embeds the unchanged master. This avoids separate, drifting logo designs.

Rebuild after an explicitly approved logo change with `python scripts\build_branding.py`. This is a branding-asset transformation, not a new avatar or installer. Model/recording helper actions currently require the source checkout's `scripts` directory; an installer is a separate deliverable. v1.0.0 is the first official version; v1.1.0 adds the feedback workspaces. Earlier v0.2.0 exports were local staging.

Offline processing host

VIDEO uses `VideoExportWorker` around the Qt-free `video_pipeline.render_avatar_video`, also used by `video_cli`. Unlike live capture, it never creates `LatestFrameCamera`. MediaPipe VIDEO mode performs synchronous face/hand/segmentation inference for each decoded frame. Progress signals are bounded in time; preview frames still use the latest-slot bridge.

`video.export_video` owns PyAV decode/encode/remux resources and preserves source PTS. FFV1/MKV keeps processed RGB pixels lossless; H.264/MP4 CRF 17 is explicitly lossy. Audio packets are copied through stream templates, not decoded/re-encoded. A same-filesystem temporary output is published with an atomic no-overwrite hard link only after successful close; cancellation/errors remove the owned temporary directory without touching input or existing outputs.

See video processing and its supported-input boundary.

48

PART X · DESKTOP STUDIO

CHAPTER 48

Studio tutorial

PART X · DESKTOP STUDIO Chapter 48

Studio tutorial

> **TL;DR:** Choose **LIVE** for cameras/streaming or **VIDEO** to process an existing recording. Calibrate actual OPEN/CLOSED eye scores. Diagnostics stay beside the program image, while the separate clean window supports OBS Window Capture. Nothing captures or records automatically.

1. Install and launch

From the project root:

```
.\.venv312\Scripts\python.exe -m pip install -e ".[dev,tracking,virtual-camera,gui]"
.\.venv312\Scripts\python.exe -m kardboard_vtuber.gui
```

Or use `.\.venv312\Scripts\kardboard-studio.exe` after editable installation. The GUI extra includes the video codec library; the ordinary camera CLI remains Qt-free. Use **Tutorial** in the header for the in-app walkthrough. It works before camera access.

2. Explore without a camera

Click **Try demo**, or launch:

```
.\.venv312\Scripts\python.exe -m kardboard_vtuber.gui --demo
```

This uses the real renderer and simulated signals on a labelled synthetic stage. It never opens a camera, runs live inference, records or publishes OBS Virtual Camera output. It can show the clean output window for layout review. Demo FPS values are unavailable, not invented measurements.

The front modes are **LIVE / camera and streaming** and **VIDEO / process a prerecorded file**. The left notebook contains Camera, Avatar & eyes, Output & effects, Tracking labs, Private capture, Models and Tutorial. Scroll within each page for all controls.

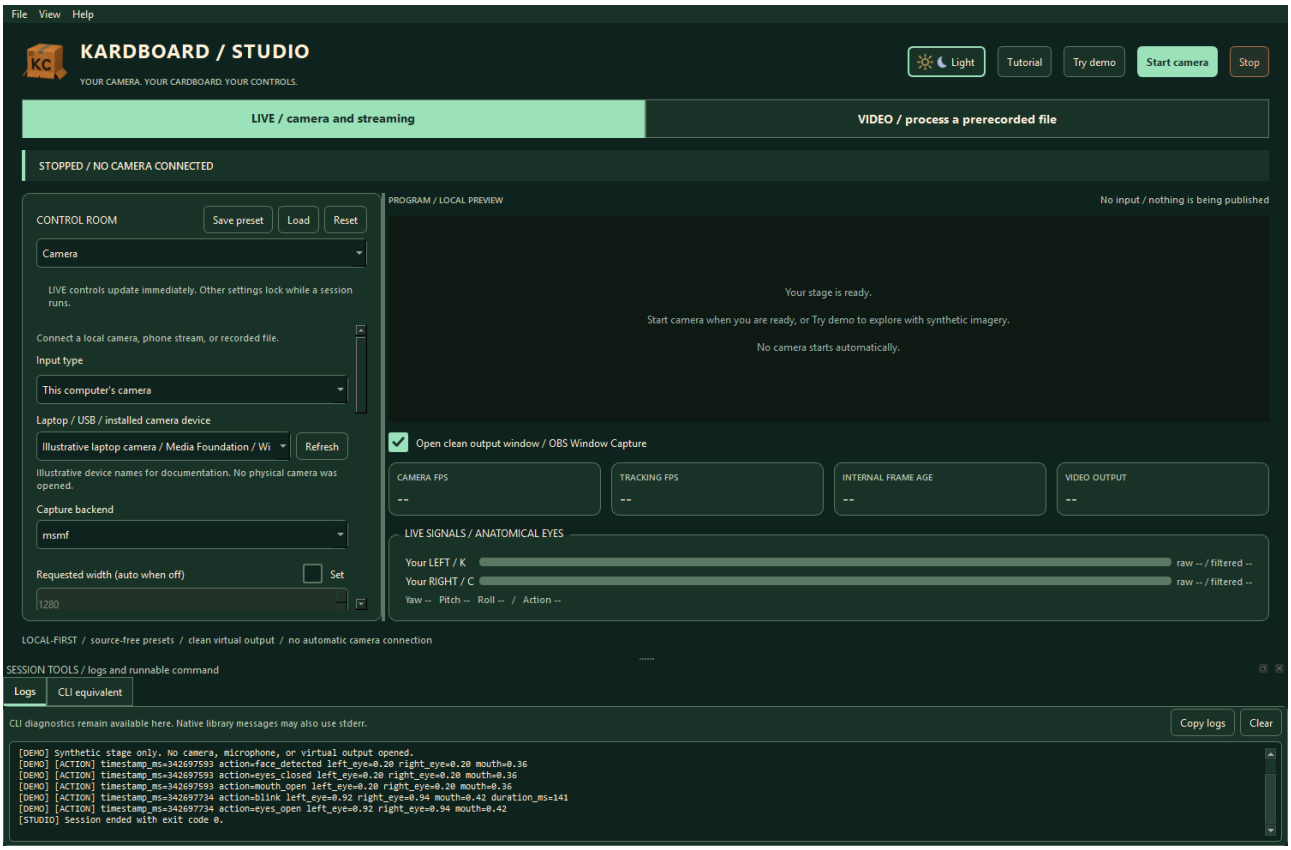
3. Verify models and enter a source

Stop other work. **Models → Verify / download** checks the expected hash or explicitly downloads the required official model. Force replacement is deliberate; “present” alone is not verification.

For laptop/USB input choose **Camera → This computer’s camera → Refresh → named device**. Listing metadata does not open a stream. Each named choice keeps its matching API/index. Windows startup may populate that metadata list, but capture starts only with **Start camera**.

Index `0` is the first device for an API, not an extra camera or a guaranteed laptop identity. Media Foundation and DirectShow are alternative Windows capture backends, often exposing the same

integrated camera, not RGB/IR modes. Equal names alone do not prove physical identity. Refresh/reselect after hardware changes; a missing previous named device is cleared.



Actual source controls with illustrative device names; no camera capture.

Choose **Phone stream / video file / manual source** for a phone URL, live-paced file replay or manual index. Sources/backend choices stay in session memory, not presets. Virtual devices can appear; never select OBS Virtual Camera as input while sending to that same output.

```
$env:KARDBOARD_CAMERA_URL = Read-Host "Paste your current camera stream URL"
.\.venv312\Scripts\python.exe -m kardboard_vtuber.gui
```

Set the variable in the same terminal before launch. Do not recover old credentials from history. Choose rotation/mirroring to fit the placement; already mirrored recordings have a separate flag. Tracking width changes analysis cost/detail, not final camera resolution.

4. Start and calibrate

Press **Start camera**. Initialization runs off the widget thread. Face-only cardboard begins black before acquisition and freezes the last-safe composite on face loss. This is a failure policy, not proof of perfect coverage for every detected angle.

In **Avatar & eyes**, use **Calibrate OPEN / CLOSED eyes** during a real live session:

1. Face the camera with both eyes relaxed and open; capture the open samples.
2. Begin the closed phase, close **both eyes** during its countdown and hold through capture.

3. The wizard derives compatible left/right/open/wink thresholds together. Overlapping ranges fail visibly; repeat rather than assuming a physically closed eye must have model score zero.

Only numeric observations are used; no face images are saved. K follows anatomical left and C anatomical right. The readout shows the actual rendered OPEN/CLOSED decision, raw/rendered scores and effective threshold. Geometry/pose remain smoothed; brief raw eye closures are not filtered away.

For manual tuning, **raise** blink thresholds for missed closures and **lower** them for false closures. Each must stay below both open/wink rules. An invalid edit displays **Not applied** and retains the last valid tuple. Shared blink is only the fallback for an eye whose Override is off. Relative wink rules also require the opposite eye to be open and sufficient asymmetry.



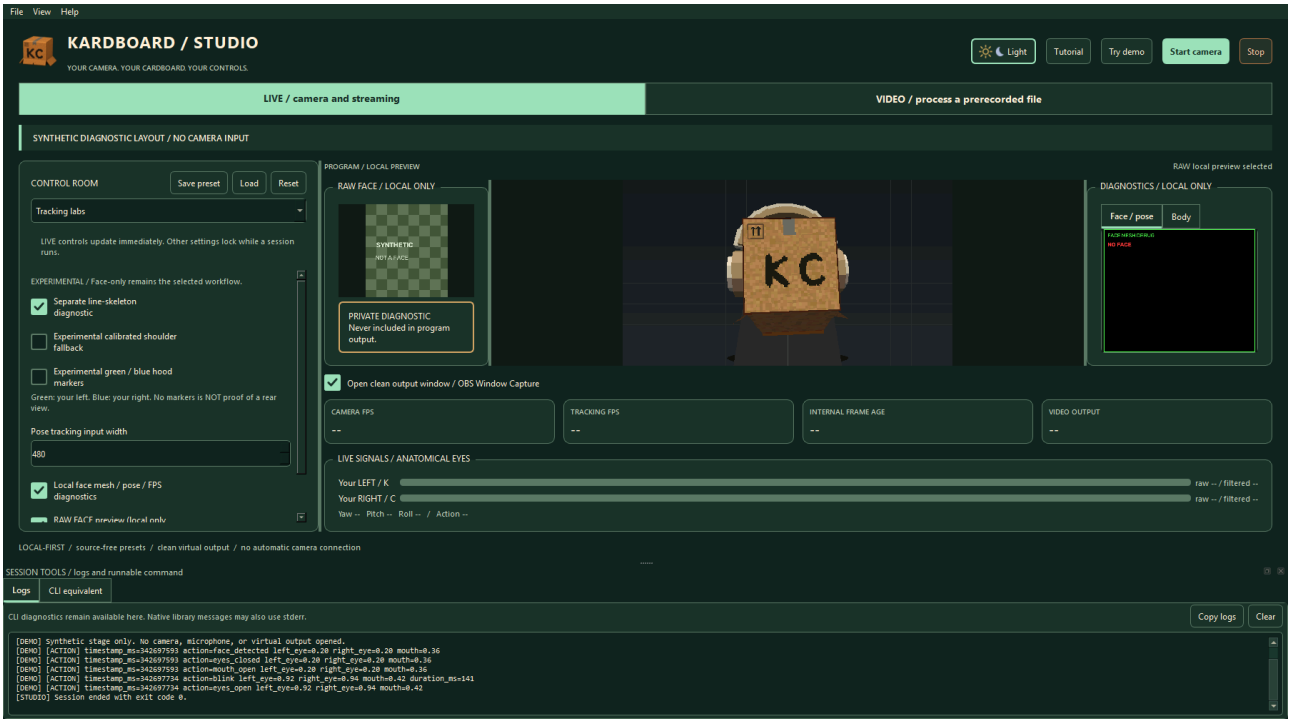
The in-app instructions explain timed sampling and effective closure decisions.

LIVE fields update during a live session. Other fields dim and show **LOCKED**, with a persistent Stop-to-unlock explanation. Video-mode Live-only controls are labelled. Offline processing locks every processing option until completion/cancellation. Internal brightness affects analysis only; width/brightness are not proven distance-tracking fixes.

5. Avatar, effects and separate diagnostics

Textured 3D is canonical; disable physics before selecting procedural 2D for comparison. Depth offset changes perspective placement and supports direct numeric entry without an artificial positive cap. Preview resizing does not crop or change source processing resolution.

The central program image stays unobstructed. Raw-face pixels occupy the local left panel; face mesh/XYZ and the Body skeleton use the right panel. They do not enter clean output. Skeleton diagnostics do not fill a full-body avatar into camera video.



Actual layout with synthetic placeholders, not a saved face image.

Green screen preserves the person over chroma green. Hand foreground restores conservative anatomical regions; RGB does not establish physical depth or recover arbitrary held objects. Inferred-depth foreground restoration was removed after privacy failures.

6. Use as a camera or a clean OBS window

OBS Virtual Camera

Install OBS Studio's Windows camera component once. Enable **Send clean video to OBS Virtual Camera** in Output & effects. Choose SD, HD, Full HD, QHD, 4K, portrait or current-display resolution plus advertised FPS. The display option reads the screen this Studio window is on; moving monitors does not change an already running output. Stop/restart to change topology.

Select **OBS Virtual Camera** in the calling application. OBS itself may stay closed, but only one producer can own it. No custom KardboardCode-named camera driver is installed. Video is aspect-

fitted with black bars; select microphone audio separately. Advertised FPS is not guaranteed delivered motion speed. Actual calling-app permissions still need qualification.

Separate clean window for OBS Window Capture

Open clean output window / OBS Window Capture is enabled by default for Live/demo. The stable **KardboardCode - Clean Output** window contains only clean program video. Select its client area in OBS Window Capture; it is distinct from OBS Virtual Camera. Closing it hides the window, not the live pipeline. Reopen it from View. Raw/unmasked modes do not populate this clean-output channel.



Actual controls-free synthetic output. No diagnostic overlays or private face panel.

Suppress preview retains processing/virtual delivery but suppresses local video transfer. Clean output is full-resolution before display scaling; Studio diagnostic copies are bounded.

7. Advanced tracking and raw debug

Tracking labs labels shoulder and hood modes experimental. Face-only remains the selected default. Shoulder fallback needs calibration. Hood mode uses green anatomical-left and blue anatomical-right squares; red/pink are unsupported. The fallback modes cannot be combined. Missing face/side markers does not confirm rear orientation.

Raw face preview and unmasked-camera operation require acknowledgement on each real start. Screen capture can still expose that private Studio panel. Presets never retain raw consent. Showing an available pose is not evidence that the estimate is visually accurate or private.

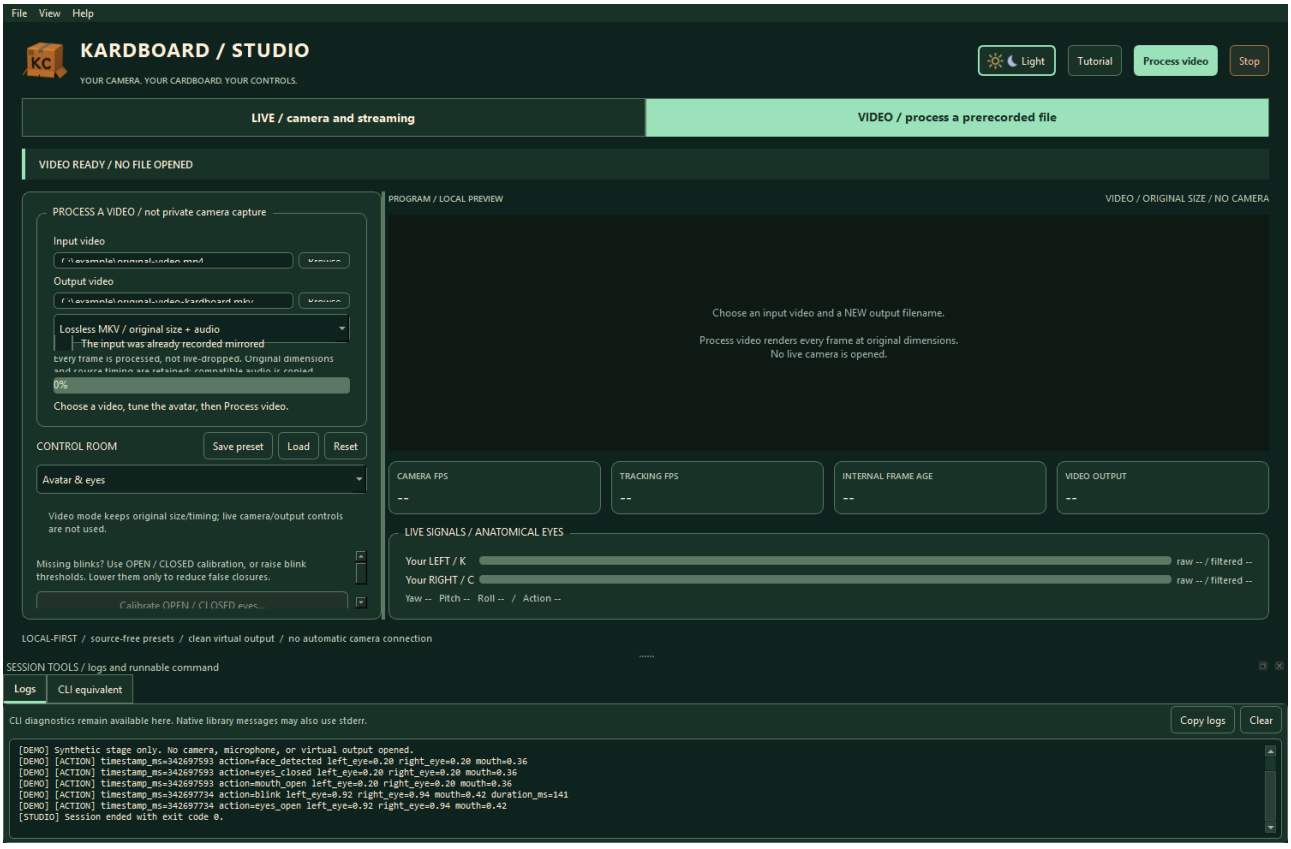
8. Process a prerecorded video

Choose front **VIDEO** mode. Browse an input and a NEW output filename. Default **lossless FFV1/MKV** avoids additional compression loss but can be large. **High-quality H.264/MP4 CRF 17** is smaller and explicitly lossy. The original/existing outputs are never overwritten.

Configure avatar/eyes, models and optional green-screen/hand effects, then **Process video**. Every frame is tracked/rendered synchronously at source dimensions and timestamp spacing within container precision. Compatible audio packets are copied. Source mirror/rotation, live capture/virtual-output dimensions and experimental body/marker modes do not apply. Mark already-mirrored input for correct anatomical semantics without flipping pixels again.

Supported input is opaque, progressive, square-pixel 8-bit SDR with valid timing. Unsupported HDR/alpha/rotation/interlacing or container/codecs fail explicitly, never through hidden quality conversion. Stop cancels and cleans owned temporary output. See video processing.

When processing succeeds, Studio automatically opens the completed file in your default media player. Failed or cancelled jobs do not open anything. If no application can be launched, the video remains saved and Studio displays its path and an actionable error. Your selected player must support the output format; lossless FFV1/MKV support varies between players.



Actual export controls with illustrative paths. No camera opens.

9. Advanced private capture and replay

This is a **LIVE-mode utility**, not VIDEO processing. Stop other work, open **Private capture** and choose guided face/full-body or free recording. Free duration applies only to free mode; guided stages have their own timing. Set countdown, pose width/rate, raw-preview height, destination folder and filename stem.

The existing recorder reuses source/transforms/model selections and saves **unmasked private MP4 + CSV**, not safe avatar output. Its own raw preview is separate. Stop requests a cooperative finish; native camera blocking may delay it. The fixed-FPS writer can compress playback duration when actual delivery is slower. That old capture limitation remains unresolved.

Keep raw footage private. Browse it as a live-paced replay for diagnostics, or process it in VIDEO.

10. Presets, command copying, logs and stopping

Presets remember validated controls, not sources or raw consent. Loading starts nothing. The bottom **CLI equivalent** tab copies the complete runnable command for the active mode. Live commands substitute same-terminal source setup instead of credentials; video commands include the chosen local input/output paths. The separate **Logs** tab is bounded and selectable. Native libraries can still write separate stderr.

Symptom	First checks
Empty/invalid source	Enter it here or set the variable before launching this process
Missing model	Select a valid file in Models and explicitly verify/download
Frozen output	Face visibility/framing/lighting; do not disable safe loss handling
Missed blink	Guided OPEN/CLOSED samples; raise thresholds, inspect effective decisions
Virtual camera unavailable	Installed component, competing producer, app permissions
Video export fails	Supported source, NEW output, correct extension and compatible audio/container
Dimmed controls	Stop first; only LIVE controls remain editable during live work
Slow shutdown	Owned native work is still pending; cleanup is cooperative

Stop remains in the header. Close during work requests shutdown, keeps the event loop alive and closes after the worker finishes; it never forcibly terminates a Qt thread.

49

PART X · DESKTOP STUDIO

CHAPTER 49

Prerecorded video processing

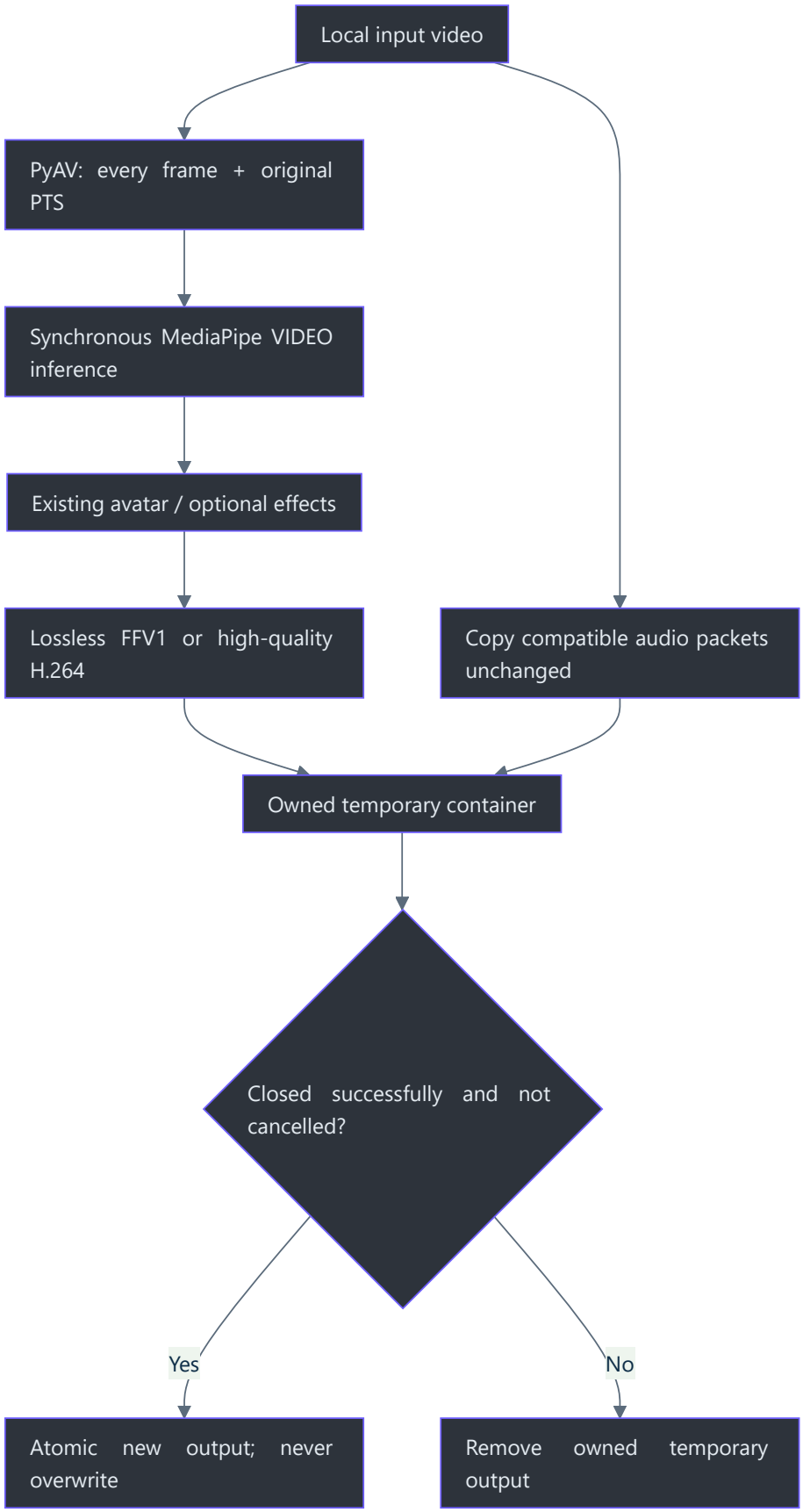
PART X · DESKTOP STUDIO Chapter 49

Prerecorded video processing

> **TL;DR:** Choose **VIDEO**, select an existing input and a new output filename, tune the avatar and process. This is not the old private camera recorder. No live camera opens. After a successful Studio export, the completed file opens in your default media player.

Why a separate mode exists

Live capture intentionally drops obsolete frames to stay current. Offline rendering must do the opposite: decode, track and render every frame, even when processing is slower than playback. It must not inherit camera mirroring, capture dimensions or an advertised virtual-camera FPS (`src/kardboard_vtuber/video.py:60-195`; `src/kardboard_vtuber/video_pipeline.py:43-140`).


Local input video
PyAV: every frame + original PTS
Synchronous MediaPipe VIDEO inference
Existing avatar / optional effects
Lossless FFV1 or high-quality H.264
Copy compatible audio packets unchanged
Owned temporary container
Closed successfully and not cancelled?
Yes
No
Atomic new output; never overwrite
Remove owned temporary output

Format contract

Choice	Video	Audio	Intended use
Lossless MKV, default	FFV1, opaque 8-bit RGB output without additional compression loss	Compatible streams copied	Quality-preserving master, potentially large
High-quality MP4	H.264, CRF 17, medium preset, YUV420	Compatible streams copied	Smaller, broadly usable delivery; explicitly lossy

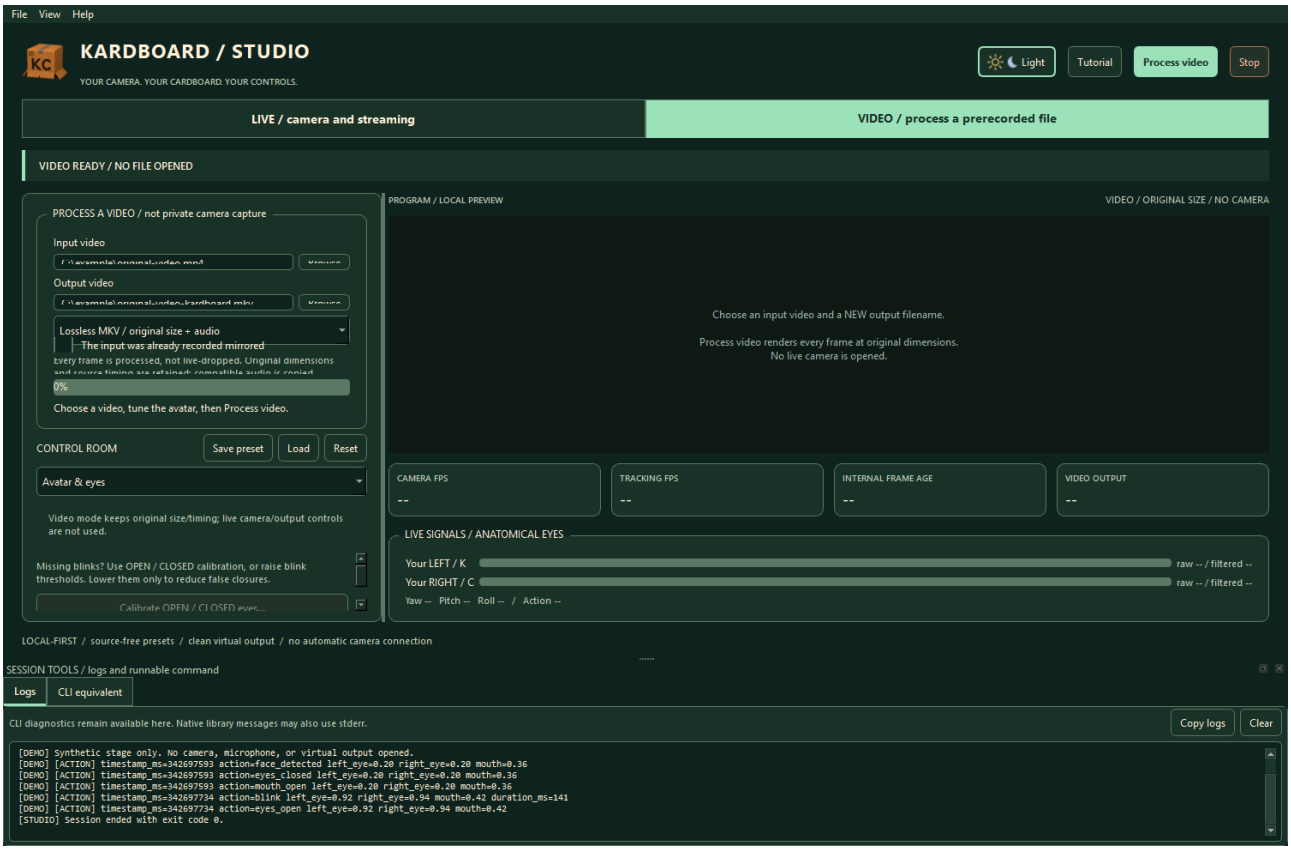
The avatar changes pixels, so output cannot be byte-identical to the source file. Lossless refers to the **processed RGB frames**, not to retaining the original compressed video bitstream. Source dimensions are not resized. Presentation timestamps preserve source spacing to the output container's precision; Matroska commonly quantizes timestamps to milliseconds. No live-rate frame dropping, artificial fixed-FPS acceleration or silent audio transcoding occurs (`video.py:70-195`).

Supported input is one opaque, progressive, square-pixel, 8-bit SDR video stream with valid timestamps and usable rate metadata. HDR/high-bit-depth, alpha, interlacing, rotation metadata, changing dimensions and invalid timestamps fail explicitly. MP4 additionally needs even source dimensions and compatible audio codecs. Additional subtitle/data/attachment streams fail explicitly instead of being silently discarded. Use MKV when an audio codec is not supported by MP4. Unsupported inputs are not silently downconverted.

Studio operation

1. Stop other work and choose the front **VIDEO** mode.
2. Browse an input and a **new** output filename.
3. Choose MKV or MP4; keep the extension matching the selected format.
4. If the file was already mirrored, mark that fact so anatomical eye semantics are correct. The raster itself is not mirrored again.
5. Configure avatar/eyes, models and optional green-screen/hand effects.
6. Press **Process video**. All processing settings lock; progress reports frame count, source seconds and original dimensions. Stop cancels cooperatively.
7. Once export and worker cleanup finish successfully, Studio opens the actual saved output using the system's default application for its file type. Failed/cancelled jobs open nothing. If the OS cannot launch a player, Studio keeps **VIDEO SAVED** and shows an explicit error with the saved path; the export is not discarded or retried.

The selected player must support the exported codec/container, especially lossless FFV1/MKV. Accepting a launch request does not guarantee codec support. No particular player is installed or hardcoded. Command-line video exports remain noninteractive and Qt-free.



Actual widgets with illustrative paths. Capturing this documentation image did not open a video or camera.

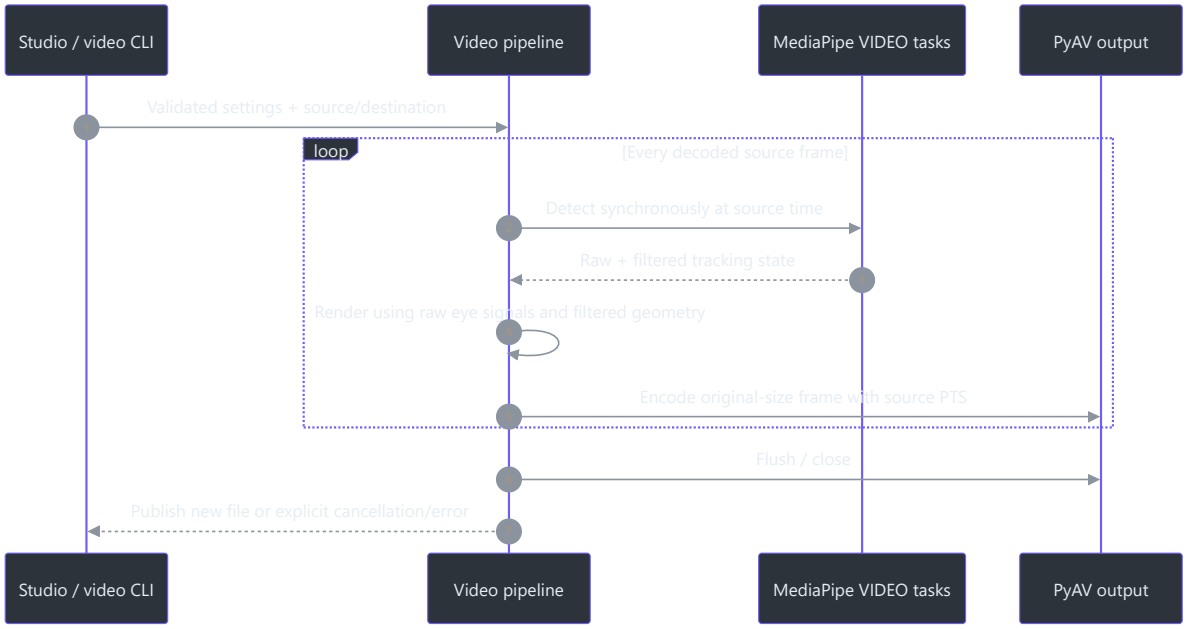
Live transforms, virtual-camera output and experimental body/hood modes are not used in this mode. Those controls are labelled Live-only. The existing RAW recorder remains an advanced **Private capture** utility under LIVE; it is not the export workflow (`src/kardboard_vtuber/gui/window.py`, `_change_work_mode`, `start_video`, `_set_busy`).

Command-line equivalent

The bottom CLI tab copies a runnable command for the active mode. For example:

```
.\.venv312\Scripts\python.exe -m kardboard_vtuber.video_cli `
  --input-video='C:\videos\input.mp4' `
  --output-video='C:\videos\input-kardboard.mkv' `
  --video-quality lossless `
  --physics
```

Replace the illustrative file paths with your own local files. The input/output path is never a camera endpoint. Ctrl+C requests cancellation. Qt is not imported by the video CLI (`src/kardboard_vtuber/video_cli.py:1-90`).



Ownership and validation

The worker owns tracker/GPU creation, use and close on one thread. ExitStack releases partial initialization too. Preview transfer is throttled and bounded independently from encoding: a slow GUI cannot cause frames to disappear from the output.

Tests use genuinely encoded synthetic VFR video/audio, compare decoded lossless pixels and timestamps, compare audio packet payloads and verify cancellation/no-overwrite behavior. A private 30-frame recorded-footage run also exercised actual synchronous MediaPipe and GPU rendering at 1080x1920, with face detection present. That is integration evidence, not exhaustive visual/privacy or every-codec certification (`tests/test_video_export.py`; `tests/test_studio_gui.py`).

References

- `video.py` — format checks, frame loop, remux, temporary ownership and publication.
- `video_pipeline.py` — shared tracking/render pipeline and effect ownership.
- `gui/video_worker.py` — Qt thread boundary, progress and explicit error reporting.
- `gui/window.py`, `_worker_finished` — main-thread default-player dispatch for the successful worker result, independently of editable output fields.
- `video_cli.py` — runnable command-line host without Qt.
- `tracking/mediapipe_tracker.py` — synchronous VIDEO versus asynchronous LIVE_STREAM.
- `tracking/green_screen.py`, `tracking/hand_occlusion.py` — synchronous optional effects.

50

PART X · DESKTOP STUDIO

CHAPTER 50

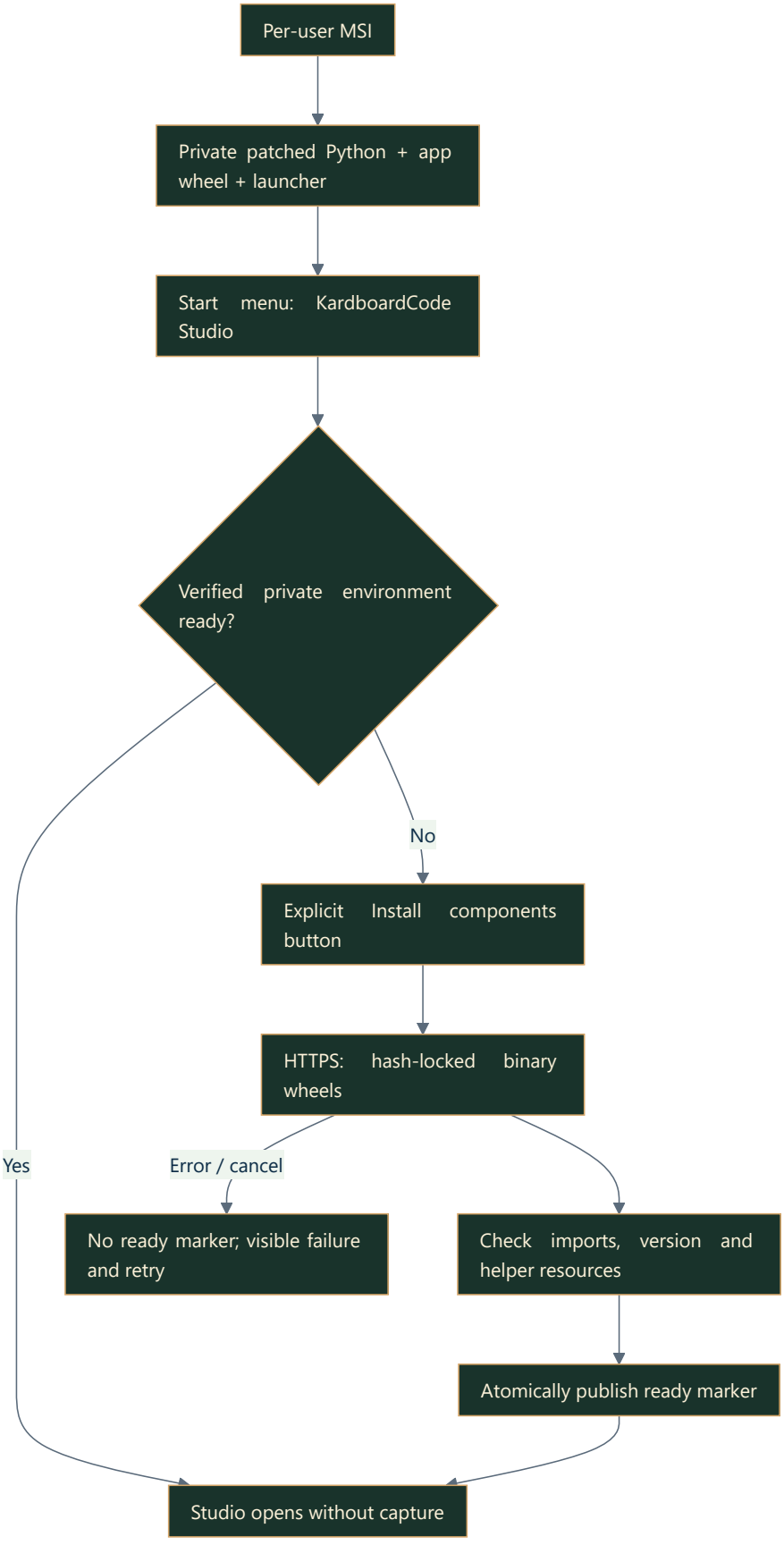
Windows installation

PART X · DESKTOP STUDIO Chapter 50

Windows installation

TL;DR: Install the per-user Windows x64 MSI, open Studio from Start, and explicitly install components in the first-launch window. No manual Python/pip or system PATH edits. This is an online, unsigned package; models and OBS's virtual-camera component remain separate.

From setup to Studio



Download the MSI from *release*, install, and open **KardboardCode Studio** from Windows Start. Choose **Install components** and keep its progress/log window open. Once setup succeeds, subsequent starts go directly to Studio. **Try demo** is camera-free. Use **Models** to explicitly download the model for each feature you choose.

The intended target is Windows 10/11 x64, with a working OpenGL 3.3 graphics driver for the textured avatar. The MSI does not install GPU or camera drivers. OBS Studio's registered virtual-camera component is required only for that output. OBS Window Capture of the separate clean output window is a different workflow.

Unsigned: compare the published SHA-256 files and obtain the package from the trusted owner. A checksum detects changed bytes, not a publisher's identity. If organizational application control blocks it, use the approved signing/IT process; do not turn off protections.

Why an online MSI rather than a frozen executable?

The user explicitly selected the smaller online bootstrap instead of a large offline bundle. The MSI contains patched CPython, the application's wheel, a stdlib/Tk setup window and the dependency lock. Qt, MediaPipe, OpenCV, codecs and other upstream components download privately on first launch. No existing Python installation is required or modified.

Normal private Python preserves the existing CLI, `QProcess` helpers and shared runtime instead of adding a frozen-program dispatch protocol. Python 3.12 is deliberate: it is the established tracking-compatible interpreter. The Windows builder also uses 3.12's `msilib`; that deprecated module is build-only, not an application-runtime dependency.

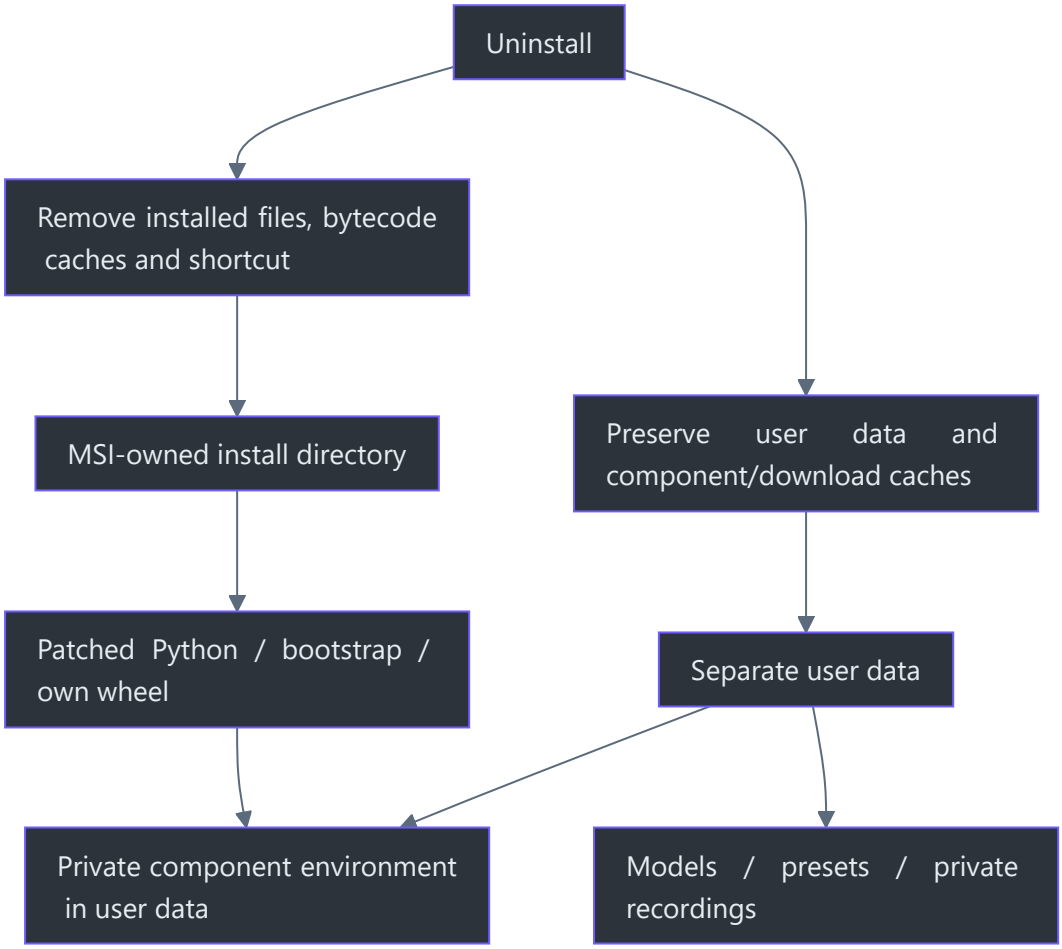
The stdlib/Tk wizard can run before Qt exists. Work happens off its main thread; a queue feeds bounded visible logs and progress. Cancel terminates only the owned component process. A per-environment file lock prevents concurrent setup. Readiness is published only after successful hash-verified installation and import/resource checks; errors are never labelled success.

Files and ownership

Location	Ownership
`%LOCALAPPDATA%\Programs\KardboardCode\Studio`	MSI-owned private Python, bootstrap, own wheel and icon
`%LOCALAPPDATA%\KardboardCode\Studio\models`	Explicitly downloaded user models
`%LOCALAPPDATA%\KardboardCode\Studio\presets`	Source-free saved presets
`%LOCALAPPDATA%\KardboardCode\Studio\recordings`	Private raw-capture default
`%LOCALAPPDATA%\KardboardCode\Studio\runtimes`	Version/input-specific private component environments
`%LOCALAPPDATA%\KardboardCode\Studio\download-cache`	Reusable package cache

Installed code never writes models, presets or recordings into its resource directory. `app_paths.py` detects source versus installed use once; normal source checkout paths remain compatible. The wheel includes all four model helpers, camera metadata listing and the private recorder. Installed copied CLI commands name the private interpreter, not a nonexistent checkout.

Windows Settings can uninstall Studio. It removes MSI-owned files and the Start shortcut, but preserves per-user data, environments and cache. This is deliberate; reinstalling is not permission to erase private recordings or downloaded models. Existing running source/development instances are not automatically terminated.



Integrity and network boundaries

`python-runtime.json` pins the patched standalone runtime URL and SHA-256. `requirements-lock.json` records exact Windows wheel filenames/versions/hashes; `requirements-windows.txt` feeds pip's `--require-hashes --only-binary=:all:`. Lock generation compares each acquired wheel against authoritative PyPI release metadata. The MSI payload manifest separately hashes its own application wheel and requirements file.

The first-launch process accepts existing verified HTTPS package-index and certificate/proxy settings. It copies only network settings into a cleaned child environment; `PIP_TARGET`, `PIP_USER`, prefix overrides and other unrelated pip configuration cannot redirect installation. Plain-HTTP ind-

exes and `trusted-host` TLS bypasses are rejected. Private feed configuration is not shipped, and network URLs are redacted from setup logs.

If component downloads fail, inspect the visible error and retry after correcting connectivity or the approved certificate/mirror configuration. A partially prepared environment is never marked ready. Cancelled downloads may retain reusable cache files. Do not disable certificate validation or change machine security policy to make setup succeed.

Qt wheels contain unusually long nested resource paths. Setup passes a canonical Windows extended-length prefix to pip, scoped to its private environment, rather than asking users to change the machine-wide long-path registry setting. Retry reinstalls the exact locked wheels so a partially copied dependency cannot be mistaken for a complete installation.

Build and release contracts

The *release handoff* contains separate source and installer manifests and checksum files. The source ZIP never embeds generated installer/archive artifacts.

Build from the repository with Windows x64 Python 3.12:

```
.\.venv312\Scripts\python.exe scripts\build_windows_installer.py
.\.venv312\Scripts\python.exe scripts\build_windows_installer.py --check
```

Only intentionally use `--update-lock` after resolving the declared runtime extras in the build environment. It verifies upstream wheel hashes before publishing new lock data. The normal build uses the committed lock; no private package source is written into it.

Commit public inputs, then regenerate the source export and finalize installer provenance. `--refresh-provenance` only updates the external receipt when MSI bytes and the runtime-input fingerprint still match; it cannot bless a stale installer. Runtime/source receipt commits are separate from manual publication. These scripts never push, tag or create a remote release.

Source map and evidence boundary

The real per-user MSI lifecycle was exercised with private Python 3.12.16: strict online component setup, all six packaged helpers, installed camera/GUI/video CLI help, native pythonw Studio, metadata-only camera discovery, checksum-only model verification, a textured synthetic demo and clean output, saved presets, and a four-frame synchronous MediaPipe/GPU video export. The default-player request was intercepted during testing, not sent to a real player. No live camera was opened. Uninstall removed the complete owned installation tree and Start shortcut while preserving both a user sentinel and saved preset.

The long-path wheel regression verifies real installation beyond 260 characters without changing machine settings. MSI generation explicitly owns its runtime bytecode directories so empty caches cannot strand parent folders after uninstall. Artifact-level regressions also check that Escape on the Welcome dialog binds Cancel, not Install.

- `scripts/build_windows_installer.py`: runtime acquisition, upstream lock, wheel/image/MSI and receipts.
- `installer/windows/bootstrap.py`: manifest checks, network-only configuration, private setup and launch.
- `src/kardboard_vtuber/app_paths.py`: read-only installed resources versus writable user data.
- `src/kardboard_vtuber/gui/window.py`: installed helper jobs and visible filesystem errors.
- `tests/test_windows_setup.py`, `tests/test_app_paths.py`: integrity/cancellation/network/path regressions.
- *Workstream 12*: actual lifecycle evidence and remaining limits.

An installed synthetic demo establishes packaging/resource operation, not real-camera accuracy, privacy for every pose, calling-app compatibility or owner usability approval.

REFERENCE

Appendices

Commands, vocabulary, repository navigation, and the source-of-truth hierarchy for operating and extending the project.

A

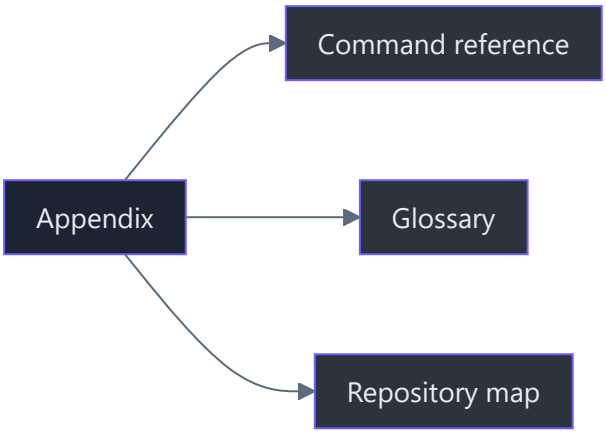
APPENDICES

APPENDIX A

Appendix

APPENDICES Appendix A

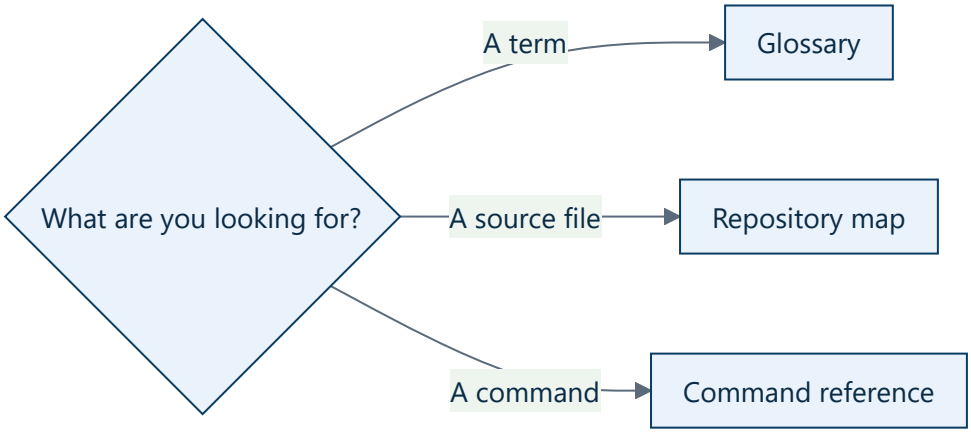
99 · Appendix



> **TL;DR** — Quick references for commands, vocabulary, repository navigation, and evidence.

Contents

- Glossary — terms used throughout the book
- Repository map — every important file and responsibility
- Command reference — setup, run, test, and troubleshooting commands



Source-of-truth hierarchy

1. Current code and tests.
2. Measured runtime evidence.
3. This book.
4. Planned architecture, explicitly labeled planned.
5. Historical PNGTuber V1 assets and manifest.

If documentation conflicts with implemented code, the code is authoritative and the documentation must be corrected.

⬅️ Roadmap · 🏠 Book home

B

APPENDICES

APPENDIX B

Glossary

APPENDICES Appendix B

Glossary

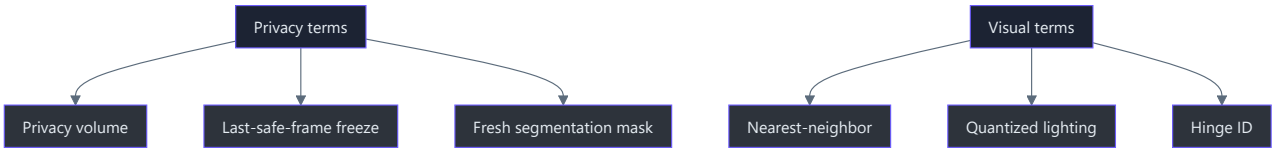
> Terms are defined in the context of KardboardCode-VTuber.



Term	Definition
Backend	OpenCV implementation selected to open a source, such as automatic, DirectShow, Media Foundation, or FFmpeg
BGR	OpenCV's default blue-green-red channel ordering
Capture worker	Background thread that owns `VideoCapture` and publishes frames
Condition variable	Lock plus waiting/notification mechanism used for shared state
Consecutive failure	Uninterrupted sequence of failed reads used to trigger reconnect
Consumer	Code that reads the latest packet, currently the CLI preview
Device index	Integer identifying a local camera, commonly `0`
Damped spring	Implemented bounded-step motion model used by the five textured flap hinges
Dependency inversion	Depending on a small protocol rather than concrete camera hardware
Dithering	Patterned color approximation; the current GPU style instead uses quantized lighting and 5-bit color
Dropped frame	Frame intentionally or unintentionally not processed by a consumer
End-to-end latency	Time from physical capture to visible output
FFmpeg	OpenCV backend useful for network video decoding
Finite-state machine	Explicit set of lifecycle states and allowed transitions
Frame age	Current post-decode time since the worker published a packet
FramePacket	Sequence, monotonic timestamp, and BGR frame
FPS	Frames per second

GIL	Python Global Interpreter Lock; native OpenCV work can execute outside normal Python bytecode
Head pose	Tracked head translation and orientation
IP Webcam	Android application hosting the phone camera as a network stream
JPEG	Compressed image format used by MJPEG
Latest-frame slot	Single replaceable packet used instead of a FIFO queue
MediaPipe	Implemented face-landmark, blendshape, and transformation-matrix inference engine
Mirroring	Horizontal flip that produces selfie-style interaction
MJPEG	Stream of JPEG images delivered as multipart HTTP
Monotonic clock	Clock guaranteed not to move backward, used for durations
ModernGL	Python OpenGL wrapper used by the default offscreen textured renderer
Negotiated format	Width, height, and FPS reported after opening a source
Nearest-neighbor	Pixel-preserving upscale used by both renderer paths
NumPy array	In-memory representation of an OpenCV frame
OBS	Streaming/recording application that captures the preview through Window Capture and optional chroma key
One Euro filter	Implemented adaptive low-pass filter for tracking jitter reduction
OpenCV	Current video capture, transform, diagnostics, and preview library
Overwritten frame	Packet replaced before a consumer read it
Producer	Capture thread that publishes packets
Protocol	Structural interface describing required capture methods
PS1 style	Deliberately low-poly, low-resolution, quantized visual language
Quaternion	Rotation representation emitted by tracking math; the current renderer consumes derived Euler pose through a model matrix
Green screen	Fail-closed person-mask compositor that replaces non-person pixels with pure green

	flap rotations
Privacy volume	Static opaque internal head-covering geometry behind the decorative shell and moving flaps
Reconnect	Release and reopen cycle after sustained failures
Requested format	Width, height, or FPS asked of a source but not guaranteed
Rotation	90-degree left/right or 180-degree frame transform
Sequence number	Strictly increasing packet identity
Snapshot	Immutable point-in-time diagnostic record
Source	Local device index or network URL
Spout2	Future same-PC GPU texture-sharing option for OBS
Stale frame	Frame whose pose no longer represents the user's current movement
USB tethering	Private phone-to-PC IP network over USB
VideoCapture	OpenCV object that opens and reads video sources
Window Capture	Initial OBS method for ingesting the application preview



⬅️ Appendix · 🏠 Book home

APPENDICES

APPENDIX C

Repository map

APPENDICES Appendix C

Repository map

> **TL;DR** — Camera, tracking, motion, rendering, composition, and presentation are separated into focused packages, with deterministic proof in `tests/` and verified model downloaders in `scripts/`.

```
KardboardCode-VTuber/
├── assets/PNGTuberV1/       Preserved original avatar and behavior manifest
├── docs/                    This engineering book
│   └── images/              Face-free synthetic documentation renders
├── models/                  Downloaded ML models at runtime; ignored except .gitkeep
├── scripts/                 Verified model downloaders and regression tooling
├── src/kardboard_vtuber/
│   ├── camera/models.py     Typed contracts, enums, validation, redaction
│   ├── camera/stream.py     Thread, latest slot, lifecycle, reconnects, metrics
│   ├── tracking/            Face, pose, hand, person-mask, filtering, events
│   ├── motion/springs.py    Bounded damped-spring primitive
│   ├── gui/                 Optional native studio, controls, presets, tutorial and icon
│   ├── runtime.py           Toolkit-neutral bounded preview/log bridge and live tuning
│   ├── renderer/            Textured GPU, procedural fallback, and body renderers
│   ├── cli.py               Arguments, orchestration, composition, preview, shutdown
│   └── __main__.py          python -m entry
├── tests/                   Deterministic unit and offscreen render tests
├── pyproject.toml           Packaging, dependencies, Ruff, pytest
└── README.md                Project landing page
```

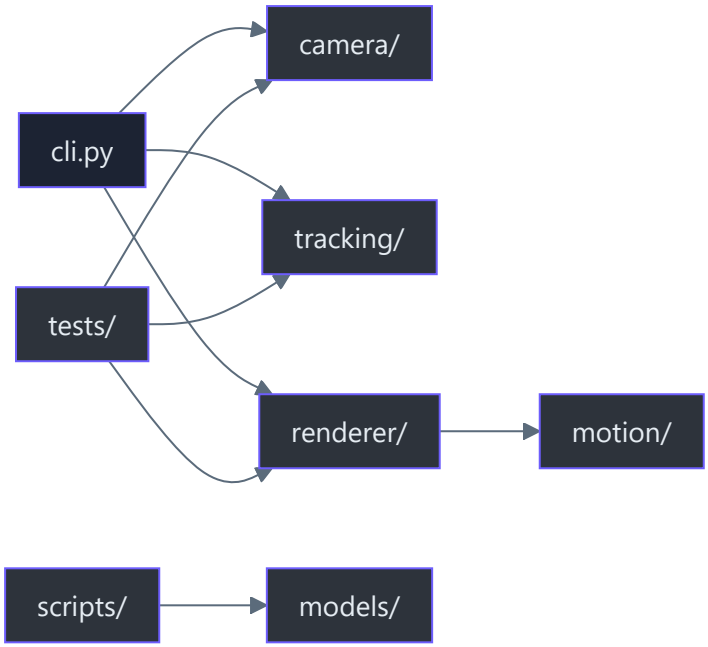
Key-file reference

File	Responsibility
`src/kardboard_vtuber/camera/models.py:15-48`	Backend and rotation translation
`src/kardboard_vtuber/camera/models.py:62-87`	Source parsing and credential redaction
`src/kardboard_vtuber/camera/models.py:90-121`	Configuration and invariants
`src/kardboard_vtuber/camera/models.py:124-157`	Frame and diagnostic contracts
`src/kardboard_vtuber/camera/stream.py:39-73`	Worker-owned runtime state
`src/kardboard_vtuber/camera/stream.py:78-141`	Public lifecycle/read API
`src/kardboard_vtuber/camera/stream.py:167-220`	Capture loop
`src/kardboard_vtuber/camera/stream.py:223-288`	Open, configure, reconnect, FPS
`src/kardboard_vtuber/tracking/models.py:15-151`	Library-neutral face state
`src/kardboard_vtuber/tracking/green_screen.py:16-152`	Async segmentation and fail-closed chroma composition
`src/kardboard_vtuber/motion/springs.py:26-85`	Bounded damped spring

`src/kardboard_vtuber/renderer/textured_3d.py:1111-1414`	Default mesh, shader, atlas, decals, privacy, and hinges
`src/kardboard_vtuber/renderer/ps1_cardboard.py:35-370`	Procedural privacy-safe fallback
`src\kardboard_vtuber\cli.py`, `build_parser`	Command-line interface
`src\kardboard_vtuber\cli.py`, `main` / `validate_options`	Shared runtime orchestration, validation and cleanup
`scripts/generate_documentation_gallery.py:1`	Face-free angle, scenario, physics, surface, tracking, skeleton, and mesh-debug galleries
`scripts/generate_readme_animation.py:1`	Face-free animated GIF driven by private numeric tracking telemetry
`tests/test_camera_models.py:6-34`	Model tests
`tests/test_camera_stream.py:13-98`	Fake adapter and worker tests
`tests/test_textured_3d_renderer.py:1-409`	GPU geometry, decals, privacy, depth, and physics
`tests/test_green_screen.py:1-47`	Segmentation composition and stale-mask behavior
`assets/PNGTuberV1/model-manifest.json:1`	Original avatar state/layer semantics
`src\kardboard_vtuber\gui\settings.py`, `PAGES`	Complete runtime control inventory and source-free validated presets
`src\kardboard_vtuber\gui\window.py`, `StudioWindow`	Native controls, preview, recording/model jobs, logs and lifecycle
`src\kardboard_vtuber\gui\worker.py`, `SessionWorker`	Shared pipeline worker and explicitly synthetic demo
`src\kardboard_vtuber\gui\tutorial.py`, `LESSONS`	Built-in nine-step walkthrough
`src\kardboard_vtuber\tracking\eyes.py`, `EyeSensitivity`	Shared independent-eye rules for actions and both renderers
`scripts\capture_studio_screenshots.py`	Real widget screenshots with synthetic-only data
`TODO\00_INDEX.todo`	Numbered studio workstreams and completion evidence

Change-impact guide





⬅️ Appendix · ➡️ Command reference

D

APPENDICES

APPENDIX D

Command reference

APPENDICES Appendix D

Command reference

Native desktop studio

From `C:\devdesk\homedesk\KardboardCode\KardboardCode-VTuber`:

```
.\.venv312\Scripts\python.exe -m pip install -e ".[dev,tracking,virtual-camera,gui]"
.\.venv312\Scripts\python.exe -m kardboard_vtuber.gui
```

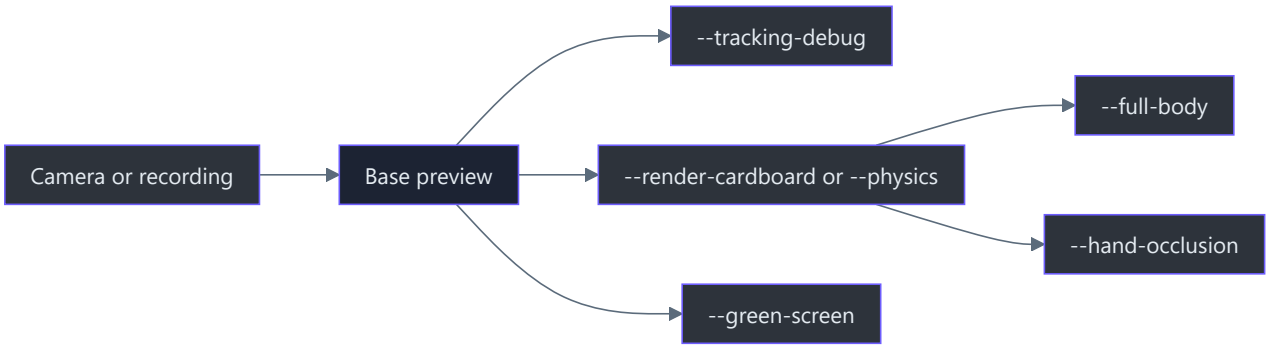
Use `--demo` for the synthetic, camera-free demonstration or `--preset "C:\path\studio.json"` to load source-free controls without starting capture. The GUI entry point is `.\.venv312\Scripts\kardboard-studio.exe`; existing valid camera commands remain supported, including zero-duration CLI probes. The new per-eye overrides are opt-in.

The native interface includes all camera flags below, model verification/download, guided/free private recording, and a built-in **Tutorial**. Read the complete walkthrough.

Independent eye overrides are also available directly:

```
$env:KARDBOARD_CAMERA_URL = Read-Host "Paste your current camera source"
.\.venv312\Scripts\python.exe -m kardboard_vtuber `
  --source $env:KARDBOARD_CAMERA_URL `
  --physics --mirror `
  --left-blink-threshold 0.30 `
  --right-blink-threshold 0.40
```

Each override falls back to `--blink-threshold` when omitted. Effective blink values must be lower than both `--eye-open-threshold` and `--wink-threshold`. In the GUI these values, relative wink tuning, action hold times and brightness are live. Other settings unlock after Stop.



Environment

```
cd C:\devdesk\homedesk\KardboardCode\KardboardCode-VTuber
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
```

Local camera

```
python -m kardboard_vtuber --source 0 --backend auto --mirror
```

Tracker inputs receive a mild brightness lift of `12`; the displayed and OBS-captured preview retains the original camera brightness. Override internal processing with `--brightness 0..100`, for example `--brightness 20` in a darker room.

Android IP Webcam

```
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror
```

Headless benchmark

```
python -m kardboard_vtuber `
  --source "http://USERNAME:PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --headless `
  --duration 30
```

Direct virtual camera for calling apps (Windows)

Install OBS Studio once for its camera component. The OBS application can remain closed:

```
.\.venv312\Scripts\python.exe -m pip install -e ".[tracking,virtual-camera]"
$env:KARDBOARD_CAMERA_URL = Read-Host "Paste your full camera stream URL"
.\.venv312\Scripts\python.exe -m kardboard_vtuber `
  --source $env:KARDBOARD_CAMERA_URL `
  --mirror `
  --physics `
  --tracking-width 1280 `
  --brightness 30 `
  --virtual-camera
```

Set `KARDBOARD_CAMERA_URL` in the same PowerShell terminal before running; session assignments do not carry over to new terminals. Then select **OBS Virtual Camera** in Discord, Teams, Meet, or another camera-capable app. Stop any other producer of that device, including OBS's **Start Virtual Camera** output. There is only one OBS camera instance; this mode does not install or rename a driver. It uses pyvirtualcam to send composed frames directly.

`--virtual-camera` enables cardboard rendering and face tracking even without `--physics`. `--headless` suppresses local windows, not rendering or camera delivery. Only the clean composite is sent; all local debug overlays and the opt-in raw-face preview are excluded. In face-only mode, startup is black and lost or stale face tracking freezes the last safe frame. Capture interruptions repeat that composite, never the unprocessed input. `--brightness` does not brighten exported video.

Option	Default	Meaning
`--virtual-camera-width`	`1280`	Fixed output width, positive even number
`--virtual-camera-height`	`720`	Fixed output height, positive even number
`--virtual-camera-fps`	`30`	Advertised FPS (1-60); actual motion follows source/processing speed

Portrait or differently shaped input is fitted with black bars. Changes to capture resolution do not change the camera's output format mid-call. `--preview-height` affects only the local window. Microphone audio remains a separate device. If the device is missing, check OBS's camera-component installation and restart the calling app; app permissions or managed-device policies may also restrict camera access.

Recorded calibration video

Local video files are automatically paced at their recorded FPS and stop cleanly at EOF. The guided recording is already upright and mirrored:

```
python -m kardboard_vtuber `
  --source "C:\path\to\KardboardCode-tracking-calibration-guided-45s.mp4" `
  --input-already-mirrored `
  --render-cardboard `
  --cardboard-renderer textured-3d
```

Do not add `--rotate left` or `--mirror` for this recording. Use `--cardboard-renderer procedural-2d` to compare the preserved original prototype.

Record a new canonical regression video

```
python scripts\record_guided_regression.py `
  --source "http://YOUR_USERNAME:YOUR_PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --brightness 12 `
  --preview-height 720 `
  --output-dir "C:\path\to\private-regression-output" `
  --name "KardboardCode-canonical-regression"
```

Follow the 12 on-screen stages. The clean MP4 and synchronized CSV receive the same timestamp.

Optional raw-face debug panel

The standard preview is clean by default. Add `--tracking-debug` to show the synthetic face-mesh inset, pose axes, action state, tracked pose values, FPS, resolution, and frame-age text. This does not expose raw face pixels.

The raw face panel is disabled by default because it reveals the user's face. Enable it only for local debugging:

```
python -m kardboard_vtuber `
  --source "http://YOUR_USERNAME:YOUR_PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --render-cardboard `
  --debug-face-preview `
  --preview-height 900
```

`--preview-height` changes only the displayed window size. Capture, tracking, and rendering remain at the source resolution and retain the full camera frame.

Tune blink and wink sensitivity

The default calibrated thresholds are:

Option	Default	Meaning
`--blink-threshold`	`0.35`	Both eyes at or below this value count as closed
`--wink-threshold`	`0.70`	The winked eye must be at or below this value
`--eye-open-threshold`	`0.65`	The opposite eye must be at or above this value
`--wink-min-difference`	`0.15`	Minimum openness difference between the eyes

For fewer false blinks and winks, start with:

```
--blink-threshold 0.20 `
--wink-threshold 0.50 `
--eye-open-threshold 0.72 `
--wink-min-difference 0.25
```

Lower closed-eye thresholds are less sensitive. Higher open-eye and difference requirements make winks stricter. Values must remain between `0` and `1`; the blink threshold must remain below both the open-eye and wink thresholds. The same settings drive action logs and the visible K/C state. If brief noise still survives, also raise `--eye-action-hold-ms` from `40` to `60` or `80`.

Full-body avatar and skeleton

Download and verify the official MediaPipe Pose Landmarker Lite model:

```
python scripts\download_pose_landmarker_model.py
```

Then enable the feature:

```
python -m kardboard_vtuber `
  --source "******PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --full-body `
  --preview-height 900
```

`--full-body` automatically enables face tracking and cardboard rendering. A separate `KardboardCode Full Body Skeleton` window shows the 33 numbered pose landmarks and line connections on black. The skeleton is not filled or composited into the camera preview. The tracker supports one person and can only track body parts visible in the camera frame.

To keep only the cardboard head moving when the face turns away, use:

```
python -m kardboard_vtuber `
  --source "YOUR_CAMERA_URL" `
  --rotate left `
  --mirror `
  --body-head-fallback
```

`--body-head-fallback` automatically enables cardboard, face, and pose tracking but does not render the synthetic body or open the skeleton window. While the face is visible, it learns the relationship between the face and shoulders. During face loss, it uses confident shoulder landmarks to estimate head position, size, roll, and continuing yaw with neutral expressions. If the initial calibration has not completed or the shoulders are also lost, rendering falls back to the existing frozen privacy-safe frame. To protect preview performance, fallback-only pose tracking runs at no more than 256-pixel input and 8 FPS; `--full-body` retains the requested pose resolution and unrestricted submission rate.

Begin the session facing the camera normally for about one second. The fallback deliberately refuses to invent a generic head size until it has collected enough paired face-and-shoulder observations. During that startup period, the preview keeps the lower body visible, blacks out everything above the detected shoulders, and displays calibration progress. If body pose is not available either, the entire camera image remains opaque behind the instructions.

For the colour-square hood:

```
python -m kardboard_vtuber `
  --source $env:KARDBOARD_CAMERA_URL `
  --mirror `
  --physics `
  --hood-marker-tracking `
  --tracking-debug
```

The marker tracker recognizes green on the wearer's anatomical left and blue on the anatomical right. Red and pink are ignored. Face plus body tracking selects front view; body tracking without face tracking or either side marker selects rear view. Side markers preserve the latest face/body-derived head dimensions to prevent the cardboard box from enlarging during profile turns. `--tracking-debug` outlines accepted squares and prints marker state.

Enable flap hinge physics

```
python -m kardboard_vtuber `
  --source "YOUR_CAMERA_URL" `
  --backend auto `
  --rotate left `
  --mirror `
  --physics `
  --preview-height 900
```

`--physics` automatically enables face tracking and the textured 3D cardboard renderer. Both inward underside flaps and the broad front underside flap receive separate bounded damped-spring hinges driven by sustained tracked head turns, tilt, and movement. The two external side tabs use wider, highly yaw-sensitive hinges for obvious left-right secondary motion. The internal opaque privacy volume remains static.

The textured model is moved slightly backward in perspective by default. Add `--box-depth-offset 0` to restore the previous Z position. Positive offsets have no upper cap; large values can make the projected box too small to preserve head coverage.

Green-screen the camera background

Download and verify the official MediaPipe Selfie Segmenter model:

```
python scripts\download_selfie_segmenter_model.py
```

Then add `--green-screen` to the normal camera command. Person pixels remain visible and all background pixels become pure chroma green for OBS. The compositor fails closed to a fully green frame before the first mask or whenever the latest mask becomes stale.

Privacy-safe hand occlusion

Download the verified official MediaPipe model once:

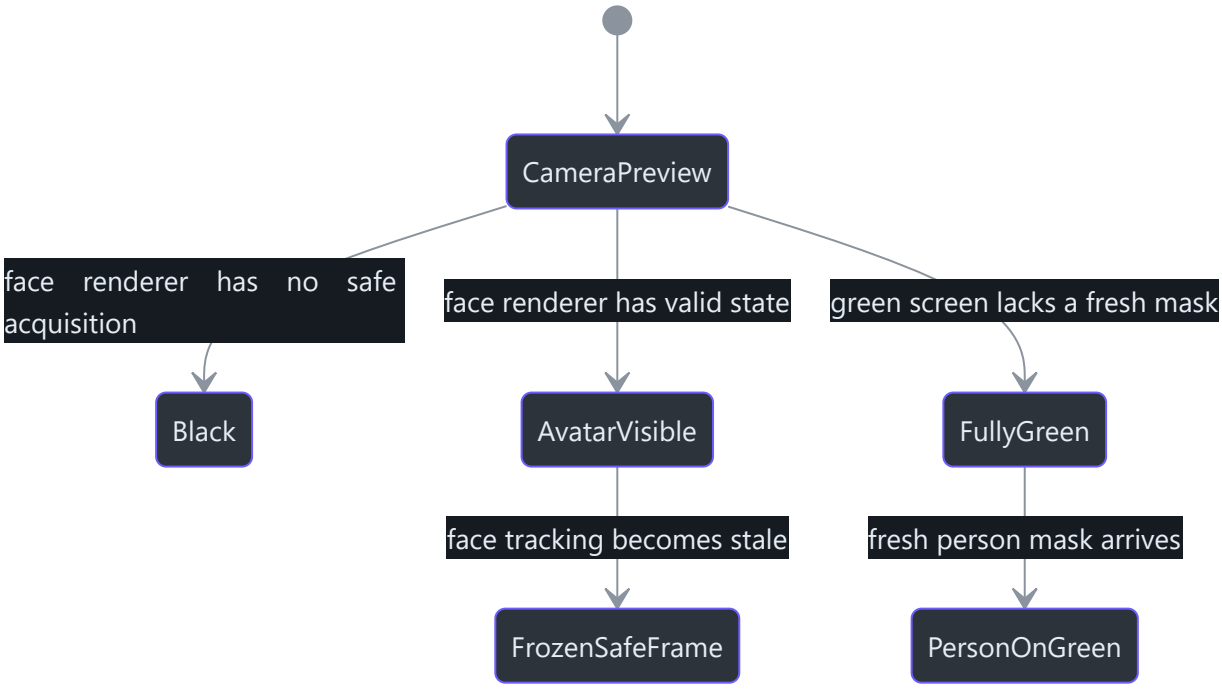
```
python scripts\download_hand_landmarker_model.py
```

Then add `--hand-occlusion` to the textured renderer command:

```
python -m kardboard_vtuber `
  --source "http://YOUR_USERNAME:YOUR_PASSWORD@PHONE_IP:8080/video" `
  --backend auto `
  --rotate left `
  --mirror `
  --render-cardboard `
  --hand-occlusion
```

The hand tracker runs asynchronously at a 320-pixel input width and restores detected hand and forearm pixels over the rendered avatar. This provides convincing foreground interaction when a hand is in front, but a monocular RGB camera cannot prove depth; a hand physically behind the avatar may also be treated as foreground.

This mode intentionally supports only the anatomical hand and forearm mask. Arbitrary held-object occlusion is not supported by the RGB-only camera pipeline. Inferred monocular depth was tested and removed because it could classify face pixels as foreground and violate fail-closed privacy. Reliable general-object occlusion requires an aligned hardware/AR depth stream.



Quality checks

```
python -m ruff check .
python -m pytest
git --no-pager diff --check
```

Regenerate the face-free documentation gallery

```
python scripts\generate_documentation_gallery.py
```

This recreates the expanded angle, scenario, performance, flap-motion, surface-tour, tracking debug, body-skeleton, and render-mesh sheets. Runtime debug functions receive synthetic landmarks, and every other image uses the runtime mesh on a solid dark background. The script never reads a camera frame.

Regenerate the animated README demo

```
python scripts\generate_readme_animation.py `
  --telemetry "C:\path\to\private-regression.csv"
```

Only numeric tracking telemetry is read. The generated `docs/images/kardboardcode-live-demo.gif` contains synthetic renderer output and text, never camera frames.

Regenerate the real-life black-hoodie demo

```
python scripts\generate_real_life_animation.py `
  --video "C:\path\to\private-black-hoodie-recording.mp4" `
  --telemetry "C:\path\to\matching-private-telemetry.csv"
```

The private recording and telemetry remain outside the repository. The generator samples every recorded calibration stage, re-detects the face bounds, applies the synchronized filtered pose and expression values, verifies that the rendered avatar covers at least 98% of face landmarks, and adds an opaque head-region backing before writing `docs/images/kardboardcode-real-life-demo.gif`. The output contains the approved body and background camera pixels, but no visible face or hairline.

Add `--green-screen` to process the same private source through person segmentation and write `docs/images/kardboardcode-green-screen-demo.gif`:

```
python scripts\generate_real_life_animation.py `
  --video "C:\path\to\private-black-hoodie-recording.mp4" `
  --telemetry "C:\path\to\matching-private-telemetry.csv" `
  --green-screen
```

Composition uses the runtime threshold, morphological cleanup, and exact BGR chroma color `(0, 255, 0)` before GIF palette encoding.

CLI help

```
python -m kardboard_vtuber --help
```

Backend fallback order

1. `auto`
2. `ffmpeg` for network streams
3. `dshow` for local Windows cameras
4. `msmf` for local Windows cameras

Never commit a command containing real credentials.

`Ctrl+C` requests an orderly shutdown. The capture loop exits and closes the renderer, MediaPipe trackers, camera stream, and OpenCV windows before the process returns.

⬅️ Repository map · 🏠 Book home